```
/**
  ******************************************************************************
  * @file    stm32f10x.h
  * @author  MCD Application Team
  * @version V3.5.0
  * @date    11-March-2011
  * @brief   CMSIS Cortex-M3 Device Peripheral Access Layer Header File.
  *          This file contains all the peripheral register's definitions, bits
  *          definitions and memory mapping for STM32F10x Connectivity line,
  *          High density, High density value line, Medium density,
  *          Medium density Value line, Low density, Low density Value line
  *          and XL-density devices.
  *
  *          The file is the unique include file that the application programmer
  *          is using in the C source code, usually in main.c. This file contains:
  *           - Configuration section that allows to select:
  *              - The device used in the target application
  *              - To use or not the peripheral’s drivers in application code(i.e.
  *                code will be based on direct access to peripheral’s registers
  *                rather than drivers API), this option is controlled by
  *                "#define USE_STDPERIPH_DRIVER"
  *              - To change few application-specific parameters such as the HSE
  *                crystal frequency
  *           - Data structures and the address mapping for all peripherals
  *           - Peripheral's registers declarations and bits definition
  *           - Macros to access peripheral’s registers hardware
  *
  ******************************************************************************
  * @attention
  *
  * THE PRESENT FIRMWARE WHICH IS FOR GUIDANCE ONLY AIMS AT PROVIDING CUSTOMERS
  * WITH CODING INFORMATION REGARDING THEIR PRODUCTS IN ORDER FOR THEM TO SAVE
  * TIME. AS A RESULT, STMICROELECTRONICS SHALL NOT BE HELD LIABLE FOR ANY
  * DIRECT, INDIRECT OR CONSEQUENTIAL DAMAGES WITH RESPECT TO ANY CLAIMS ARISING
  * FROM THE CONTENT OF SUCH FIRMWARE AND/OR THE USE MADE BY CUSTOMERS OF THE
  * CODING INFORMATION CONTAINED HEREIN IN CONNECTION WITH THEIR PRODUCTS.
  *
  * <h2><center>&copy; COPYRIGHT 2011 STMicroelectronics</center></h2>
  ******************************************************************************
  */

/** @addtogroup CMSIS
  * @{
  */

/** @addtogroup stm32f10x
  * @{
  */

#ifndef __STM32F10x_H
#define __STM32F10x_H

#ifdef __cplusplus
 extern "C" {
#endif

/** @addtogroup Library_configuration_section
  * @{
  */

/* Uncomment the line below according to the target STM32 device used in your
   application
  */

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD) && !defined (STM32F10X_HD_VL) && !defined (STM32F10X_XL) && !defined (STM32F10X_CL)
  /* #define STM32F10X_LD */     /*!< STM32F10X_LD: STM32 Low density devices */
  /* #define STM32F10X_LD_VL */  /*!< STM32F10X_LD_VL: STM32 Low density Value Line devices */
  /* #define STM32F10X_MD */     /*!< STM32F10X_MD: STM32 Medium density devices */
  /* #define STM32F10X_MD_VL */  /*!< STM32F10X_MD_VL: STM32 Medium density Value Line devices */
  /* #define STM32F10X_HD */     /*!< STM32F10X_HD: STM32 High density devices */
  /* #define STM32F10X_HD_VL */  /*!< STM32F10X_HD_VL: STM32 High density value line devices */
```

```
  /* #define STM32F10X_XL */     /*!< STM32F10X_XL: STM32 XL-density devices */
  /* #define STM32F10X_CL */     /*!< STM32F10X_CL: STM32 Connectivity line devices */
#endif
/*  Tip: To avoid modifying this file each time you need to switch between these
        devices, you can define the device in your toolchain compiler preprocessor.

 - Low-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers
   where the Flash memory density ranges between 16 and 32 Kbytes.
 - Low-density value line devices are STM32F100xx microcontrollers where the Flash
   memory density ranges between 16 and 32 Kbytes.
 - Medium-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers
   where the Flash memory density ranges between 64 and 128 Kbytes.
 - Medium-density value line devices are STM32F100xx microcontrollers where the
   Flash memory density ranges between 64 and 128 Kbytes.
 - High-density devices are STM32F101xx and STM32F103xx microcontrollers where
   the Flash memory density ranges between 256 and 512 Kbytes.
 - High-density value line devices are STM32F100xx microcontrollers where the
   Flash memory density ranges between 256 and 512 Kbytes.
 - XL-density devices are STM32F101xx and STM32F103xx microcontrollers where
   the Flash memory density ranges between 512 and 1024 Kbytes.
 - Connectivity line devices are STM32F105xx and STM32F107xx microcontrollers.
  */

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD) && !defined (STM32F10X_HD_VL) && !defined (STM32F10X_XL) && !defined (STM32F10X_CL)
 #error "Please select first the target STM32F10x device used in your application (in stm32f10x.h file)"
#endif

#if !defined  USE_STDPERIPH_DRIVER
/**
 * @brief Comment the line below if you will not use the peripherals drivers.
   In this case, these drivers will not be included and the application code will
   be based on direct access to peripherals registers
   */
  /*#define USE_STDPERIPH_DRIVER*/
#endif

/**
 * @brief In the following line adjust the value of External High Speed oscillator (HSE)
   used in your application

   Tip: To avoid modifying this file each time you need to use different HSE, you
        can define the HSE value in your toolchain compiler preprocessor.
  */
#if !defined  HSE_VALUE
 #ifdef STM32F10X_CL
  #define HSE_VALUE    ((uint32_t)25000000) /*!< Value of the External oscillator in Hz */
 #else
  #define HSE_VALUE    ((uint32_t)8000000) /*!< Value of the External oscillator in Hz */
 #endif /* STM32F10X_CL */
#endif /* HSE_VALUE */


/**
 * @brief In the following line adjust the External High Speed oscillator (HSE) Startup
   Timeout value
   */
#define HSE_STARTUP_TIMEOUT   ((uint16_t)0x0500) /*!< Time out for HSE start up */

#define HSI_VALUE    ((uint32_t)8000000) /*!< Value of the Internal oscillator in Hz*/

/**
 * @brief STM32F10x Standard Peripheral Library version number
   */
#define __STM32F10X_STDPERIPH_VERSION_MAIN   (0x03) /*!< [31:24] main version */
#define __STM32F10X_STDPERIPH_VERSION_SUB1   (0x05) /*!< [23:16] sub1 version */
#define __STM32F10X_STDPERIPH_VERSION_SUB2   (0x00) /*!< [15:8]  sub2 version */
#define __STM32F10X_STDPERIPH_VERSION_RC     (0x00) /*!< [7:0]  release candidate */
#define __STM32F10X_STDPERIPH_VERSION       ( (__STM32F10X_STDPERIPH_VERSION_MAIN << 24)\
                                             |(__STM32F10X_STDPERIPH_VERSION_SUB1 << 16)\
                                             |(__STM32F10X_STDPERIPH_VERSION_SUB2 << 8)\
```

```
                                          |(__STM32F10X_STDPERIPH_VERSION_RC))

  /**
    * @}
    */

  /** @addtogroup Configuration_section_for_CMSIS
    * @{
    */

  /**
   * @brief Configuration of the Cortex-M3 Processor and Core Peripherals
   */
  #ifdef STM32F10X_XL
   #define __MPU_PRESENT             1 /*!< STM32 XL-density devices provide an MPU */
  #else
   #define __MPU_PRESENT             0 /*!< Other STM32 devices does not provide an MPU */
  #endif /* STM32F10X_XL */
  #define __NVIC_PRIO_BITS          4 /*!< STM32 uses 4 Bits for the Priority Levels    */
  #define __Vendor_SysTickConfig    0 /*!< Set to 1 if different SysTick Config is used */

  /**
   * @brief STM32F10x Interrupt Number Definition, according to the selected device
   *        in @ref Library_configuration_section
   */
  typedef enum IRQn
  {
  /******  Cortex-M3 Processor Exceptions Numbers ***************************************************/
    NonMaskableInt_IRQn         = -14,    /*!< 2 Non Maskable Interrupt                             */
    MemoryManagement_IRQn       = -12,    /*!< 4 Cortex-M3 Memory Management Interrupt              */
    BusFault_IRQn               = -11,    /*!< 5 Cortex-M3 Bus Fault Interrupt                      */
    UsageFault_IRQn             = -10,    /*!< 6 Cortex-M3 Usage Fault Interrupt                    */
    SVCall_IRQn                 = -5,     /*!< 11 Cortex-M3 SV Call Interrupt                       */
    DebugMonitor_IRQn           = -4,     /*!< 12 Cortex-M3 Debug Monitor Interrupt                 */
    PendSV_IRQn                 = -2,     /*!< 14 Cortex-M3 Pend SV Interrupt                       */
    SysTick_IRQn                = -1,     /*!< 15 Cortex-M3 System Tick Interrupt                   */

  /******  STM32 specific Interrupt Numbers *********************************************************/
    WWDG_IRQn                   = 0,      /*!< Window WatchDog Interrupt                            */
    PVD_IRQn                    = 1,      /*!< PVD through EXTI Line detection Interrupt            */
    TAMPER_IRQn                 = 2,      /*!< Tamper Interrupt                                     */
    RTC_IRQn                    = 3,      /*!< RTC global Interrupt                                 */
    FLASH_IRQn                  = 4,      /*!< FLASH global Interrupt                               */
    RCC_IRQn                    = 5,      /*!< RCC global Interrupt                                 */
    EXTI0_IRQn                  = 6,      /*!< EXTI Line0 Interrupt                                 */
    EXTI1_IRQn                  = 7,      /*!< EXTI Line1 Interrupt                                 */
    EXTI2_IRQn                  = 8,      /*!< EXTI Line2 Interrupt                                 */
    EXTI3_IRQn                  = 9,      /*!< EXTI Line3 Interrupt                                 */
    EXTI4_IRQn                  = 10,     /*!< EXTI Line4 Interrupt                                 */
    DMA1_Channel1_IRQn          = 11,     /*!< DMA1 Channel 1 global Interrupt                      */
    DMA1_Channel2_IRQn          = 12,     /*!< DMA1 Channel 2 global Interrupt                      */
    DMA1_Channel3_IRQn          = 13,     /*!< DMA1 Channel 3 global Interrupt                      */
    DMA1_Channel4_IRQn          = 14,     /*!< DMA1 Channel 4 global Interrupt                      */
    DMA1_Channel5_IRQn          = 15,     /*!< DMA1 Channel 5 global Interrupt                      */
    DMA1_Channel6_IRQn          = 16,     /*!< DMA1 Channel 6 global Interrupt                      */
    DMA1_Channel7_IRQn          = 17,     /*!< DMA1 Channel 7 global Interrupt                      */

  #ifdef STM32F10X_LD
    ADC1_2_IRQn                 = 18,     /*!< ADC1 and ADC2 global Interrupt                       */
    USB_HP_CAN1_TX_IRQn         = 19,     /*!< USB Device High Priority or CAN1 TX Interrupts       */
    USB_LP_CAN1_RX0_IRQn        = 20,     /*!< USB Device Low Priority or CAN1 RX0 Interrupts       */
    CAN1_RX1_IRQn               = 21,     /*!< CAN1 RX1 Interrupt                                   */
    CAN1_SCE_IRQn               = 22,     /*!< CAN1 SCE Interrupt                                   */
    EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                        */
    TIM1_BRK_IRQn               = 24,     /*!< TIM1 Break Interrupt                                 */
    TIM1_UP_IRQn                = 25,     /*!< TIM1 Update Interrupt                                */
    TIM1_TRG_COM_IRQn           = 26,     /*!< TIM1 Trigger and Commutation Interrupt               */
    TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                       */
    TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                */
    TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                */
    I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                 */
    I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                 */
    SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                */
```

```
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                                 */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                                 */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                         */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                   */
  USBWakeUp_IRQn              = 42      /*!< USB Device WakeUp from suspend through EXTI Line Interrupt
*/
#endif /* STM32F10X_LD */

#ifdef STM32F10X_LD_VL
  ADC1_IRQn                   = 18,     /*!< ADC1 global Interrupt                                   */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                           */
  TIM1_BRK_TIM15_IRQn         = 24,     /*!< TIM1 Break and TIM15 Interrupts                         */
  TIM1_UP_TIM16_IRQn          = 25,     /*!< TIM1 Update and TIM16 Interrupts                        */
  TIM1_TRG_COM_TIM17_IRQn     = 26,     /*!< TIM1 Trigger and Commutation and TIM17 Interrupt        */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                          */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                   */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                   */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                    */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                    */
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                   */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                                 */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                                 */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                         */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                   */
  CEC_IRQn                    = 42,     /*!< HDMI-CEC Interrupt                                      */
  TIM6_DAC_IRQn               = 54,     /*!< TIM6 and DAC underrun Interrupt                         */
  TIM7_IRQn                   = 55      /*!< TIM7 Interrupt
*/
#endif /* STM32F10X_LD_VL */

#ifdef STM32F10X_MD
  ADC1_2_IRQn                 = 18,     /*!< ADC1 and ADC2 global Interrupt                          */
  USB_HP_CAN1_TX_IRQn         = 19,     /*!< USB Device High Priority or CAN1 TX Interrupts          */
  USB_LP_CAN1_RX0_IRQn        = 20,     /*!< USB Device Low Priority or CAN1 RX0 Interrupts          */
  CAN1_RX1_IRQn               = 21,     /*!< CAN1 RX1 Interrupt                                      */
  CAN1_SCE_IRQn               = 22,     /*!< CAN1 SCE Interrupt                                      */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                           */
  TIM1_BRK_IRQn               = 24,     /*!< TIM1 Break Interrupt                                    */
  TIM1_UP_IRQn                = 25,     /*!< TIM1 Update Interrupt                                   */
  TIM1_TRG_COM_IRQn           = 26,     /*!< TIM1 Trigger and Commutation Interrupt                  */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                          */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                   */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                   */
  TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                   */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                    */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                    */
  I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                    */
  I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                    */
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                   */
  SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                   */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                                 */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                                 */
  USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                                 */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                         */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                   */
  USBWakeUp_IRQn              = 42      /*!< USB Device WakeUp from suspend through EXTI Line Interrupt
*/
#endif /* STM32F10X_MD */

#ifdef STM32F10X_MD_VL
  ADC1_IRQn                   = 18,     /*!< ADC1 global Interrupt                                   */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                           */
  TIM1_BRK_TIM15_IRQn         = 24,     /*!< TIM1 Break and TIM15 Interrupts                         */
  TIM1_UP_TIM16_IRQn          = 25,     /*!< TIM1 Update and TIM16 Interrupts                        */
  TIM1_TRG_COM_TIM17_IRQn     = 26,     /*!< TIM1 Trigger and Commutation and TIM17 Interrupt        */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                          */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                   */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                   */
  TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                   */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                    */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                    */
  I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                    */
  I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                    */
```

```
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                */
  SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                              */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                              */
  USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                              */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                      */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                */
  CEC_IRQn                    = 42,     /*!< HDMI-CEC Interrupt                                   */
  TIM6_DAC_IRQn               = 54,     /*!< TIM6 and DAC underrun Interrupt                      */
  TIM7_IRQn                   = 55      /*!< TIM7 Interrupt
*/
#endif /* STM32F10X_MD_VL */

#ifdef STM32F10X_HD
  ADC1_2_IRQn                 = 18,     /*!< ADC1 and ADC2 global Interrupt                       */
  USB_HP_CAN1_TX_IRQn         = 19,     /*!< USB Device High Priority or CAN1 TX Interrupts       */
  USB_LP_CAN1_RX0_IRQn        = 20,     /*!< USB Device Low Priority or CAN1 RX0 Interrupts       */
  CAN1_RX1_IRQn               = 21,     /*!< CAN1 RX1 Interrupt                                   */
  CAN1_SCE_IRQn               = 22,     /*!< CAN1 SCE Interrupt                                   */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                        */
  TIM1_BRK_IRQn               = 24,     /*!< TIM1 Break Interrupt                                 */
  TIM1_UP_IRQn                = 25,     /*!< TIM1 Update Interrupt                                */
  TIM1_TRG_COM_IRQn           = 26,     /*!< TIM1 Trigger and Commutation Interrupt               */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                       */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                */
  TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                 */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                 */
  I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                 */
  I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                 */
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                */
  SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                              */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                              */
  USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                              */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                      */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                */
  USBWakeUp_IRQn              = 42,     /*!< USB Device WakeUp from suspend through EXTI Line Interrupt
*/
  TIM8_BRK_IRQn               = 43,     /*!< TIM8 Break Interrupt                                 */
  TIM8_UP_IRQn                = 44,     /*!< TIM8 Update Interrupt                                */
  TIM8_TRG_COM_IRQn           = 45,     /*!< TIM8 Trigger and Commutation Interrupt               */
  TIM8_CC_IRQn                = 46,     /*!< TIM8 Capture Compare Interrupt                       */
  ADC3_IRQn                   = 47,     /*!< ADC3 global Interrupt                                */
  FSMC_IRQn                   = 48,     /*!< FSMC global Interrupt                                */
  SDIO_IRQn                   = 49,     /*!< SDIO global Interrupt                                */
  TIM5_IRQn                   = 50,     /*!< TIM5 global Interrupt                                */
  SPI3_IRQn                   = 51,     /*!< SPI3 global Interrupt                                */
  UART4_IRQn                  = 52,     /*!< UART4 global Interrupt                               */
  UART5_IRQn                  = 53,     /*!< UART5 global Interrupt                               */
  TIM6_IRQn                   = 54,     /*!< TIM6 global Interrupt                                */
  TIM7_IRQn                   = 55,     /*!< TIM7 global Interrupt                                */
  DMA2_Channel1_IRQn          = 56,     /*!< DMA2 Channel 1 global Interrupt                      */
  DMA2_Channel2_IRQn          = 57,     /*!< DMA2 Channel 2 global Interrupt                      */
  DMA2_Channel3_IRQn          = 58,     /*!< DMA2 Channel 3 global Interrupt                      */
  DMA2_Channel4_5_IRQn        = 59      /*!< DMA2 Channel 4 and Channel 5 global Interrupt        */
#endif /* STM32F10X_HD */

#ifdef STM32F10X_HD_VL
  ADC1_IRQn                   = 18,     /*!< ADC1 global Interrupt                                */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                        */
  TIM1_BRK_TIM15_IRQn         = 24,     /*!< TIM1 Break and TIM15 Interrupts                      */
  TIM1_UP_TIM16_IRQn          = 25,     /*!< TIM1 Update and TIM16 Interrupts                     */
  TIM1_TRG_COM_TIM17_IRQn     = 26,     /*!< TIM1 Trigger and Commutation and TIM17 Interrupt     */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                       */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                */
  TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                 */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                 */
  I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                 */
  I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                 */
```

```
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                  */
  SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                  */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                                */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                                */
  USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                                */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                        */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                  */
  CEC_IRQn                    = 42,     /*!< HDMI-CEC Interrupt                                     */
  TIM12_IRQn                  = 43,     /*!< TIM12 global Interrupt                                 */
  TIM13_IRQn                  = 44,     /*!< TIM13 global Interrupt                                 */
  TIM14_IRQn                  = 45,     /*!< TIM14 global Interrupt                                 */
  TIM5_IRQn                   = 50,     /*!< TIM5 global Interrupt                                  */
  SPI3_IRQn                   = 51,     /*!< SPI3 global Interrupt                                  */
  UART4_IRQn                  = 52,     /*!< UART4 global Interrupt                                 */
  UART5_IRQn                  = 53,     /*!< UART5 global Interrupt                                 */
  TIM6_DAC_IRQn               = 54,     /*!< TIM6 and DAC underrun Interrupt                        */
  TIM7_IRQn                   = 55,     /*!< TIM7 Interrupt                                         */
  DMA2_Channel1_IRQn          = 56,     /*!< DMA2 Channel 1 global Interrupt                        */
  DMA2_Channel2_IRQn          = 57,     /*!< DMA2 Channel 2 global Interrupt                        */
  DMA2_Channel3_IRQn          = 58,     /*!< DMA2 Channel 3 global Interrupt                        */
  DMA2_Channel4_5_IRQn        = 59,     /*!< DMA2 Channel 4 and Channel 5 global Interrupt          */
  DMA2_Channel5_IRQn          = 60      /*!< DMA2 Channel 5 global Interrupt (DMA2 Channel 5 is
                                             mapped at position 60 only if the MISC_REMAP bit in
                                             the AFIO_MAPR2 register is set)
  */
#endif /* STM32F10X_HD_VL */

#ifdef STM32F10X_XL
  ADC1_2_IRQn                 = 18,     /*!< ADC1 and ADC2 global Interrupt                         */
  USB_HP_CAN1_TX_IRQn         = 19,     /*!< USB Device High Priority or CAN1 TX Interrupts         */
  USB_LP_CAN1_RX0_IRQn        = 20,     /*!< USB Device Low Priority or CAN1 RX0 Interrupts         */
  CAN1_RX1_IRQn               = 21,     /*!< CAN1 RX1 Interrupt                                     */
  CAN1_SCE_IRQn               = 22,     /*!< CAN1 SCE Interrupt                                     */
  EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                          */
  TIM1_BRK_TIM9_IRQn          = 24,     /*!< TIM1 Break Interrupt and TIM9 global Interrupt         */
  TIM1_UP_TIM10_IRQn          = 25,     /*!< TIM1 Update Interrupt and TIM10 global Interrupt       */
  TIM1_TRG_COM_TIM11_IRQn     = 26,     /*!< TIM1 Trigger and Commutation Interrupt and TIM11 global
interrupt */
  TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                         */
  TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                  */
  TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                  */
  TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                  */
  I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                   */
  I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                   */
  I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                   */
  I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                   */
  SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                  */
  SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                  */
  USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                                */
  USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                                */
  USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                                */
  EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                        */
  RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                  */
  USBWakeUp_IRQn              = 42,     /*!< USB Device WakeUp from suspend through EXTI Line Interrupt
*/
  TIM8_BRK_TIM12_IRQn         = 43,     /*!< TIM8 Break Interrupt and TIM12 global Interrupt        */
  TIM8_UP_TIM13_IRQn          = 44,     /*!< TIM8 Update Interrupt and TIM13 global Interrupt       */
  TIM8_TRG_COM_TIM14_IRQn     = 45,     /*!< TIM8 Trigger and Commutation Interrupt and TIM14 global
interrupt */
  TIM8_CC_IRQn                = 46,     /*!< TIM8 Capture Compare Interrupt                         */
  ADC3_IRQn                   = 47,     /*!< ADC3 global Interrupt                                  */
  FSMC_IRQn                   = 48,     /*!< FSMC global Interrupt                                  */
  SDIO_IRQn                   = 49,     /*!< SDIO global Interrupt                                  */
  TIM5_IRQn                   = 50,     /*!< TIM5 global Interrupt                                  */
  SPI3_IRQn                   = 51,     /*!< SPI3 global Interrupt                                  */
  UART4_IRQn                  = 52,     /*!< UART4 global Interrupt                                 */
  UART5_IRQn                  = 53,     /*!< UART5 global Interrupt                                 */
  TIM6_IRQn                   = 54,     /*!< TIM6 global Interrupt                                  */
  TIM7_IRQn                   = 55,     /*!< TIM7 global Interrupt                                  */
  DMA2_Channel1_IRQn          = 56,     /*!< DMA2 Channel 1 global Interrupt                        */
  DMA2_Channel2_IRQn          = 57,     /*!< DMA2 Channel 2 global Interrupt                        */
  DMA2_Channel3_IRQn          = 58,     /*!< DMA2 Channel 3 global Interrupt                        */
  DMA2_Channel4_5_IRQn        = 59      /*!< DMA2 Channel 4 and Channel 5 global Interrupt          */
```

```
425   #endif /* STM32F10X_XL */
426
427   #ifdef STM32F10X_CL
428     ADC1_2_IRQn                 = 18,     /*!< ADC1 and ADC2 global Interrupt                        */
429     CAN1_TX_IRQn                = 19,     /*!< USB Device High Priority or CAN1 TX Interrupts        */
430     CAN1_RX0_IRQn               = 20,     /*!< USB Device Low Priority or CAN1 RX0 Interrupts        */
431     CAN1_RX1_IRQn               = 21,     /*!< CAN1 RX1 Interrupt                                    */
432     CAN1_SCE_IRQn               = 22,     /*!< CAN1 SCE Interrupt                                    */
433     EXTI9_5_IRQn                = 23,     /*!< External Line[9:5] Interrupts                         */
434     TIM1_BRK_IRQn               = 24,     /*!< TIM1 Break Interrupt                                  */
435     TIM1_UP_IRQn                = 25,     /*!< TIM1 Update Interrupt                                 */
436     TIM1_TRG_COM_IRQn           = 26,     /*!< TIM1 Trigger and Commutation Interrupt                */
437     TIM1_CC_IRQn                = 27,     /*!< TIM1 Capture Compare Interrupt                        */
438     TIM2_IRQn                   = 28,     /*!< TIM2 global Interrupt                                 */
439     TIM3_IRQn                   = 29,     /*!< TIM3 global Interrupt                                 */
440     TIM4_IRQn                   = 30,     /*!< TIM4 global Interrupt                                 */
441     I2C1_EV_IRQn                = 31,     /*!< I2C1 Event Interrupt                                  */
442     I2C1_ER_IRQn                = 32,     /*!< I2C1 Error Interrupt                                  */
443     I2C2_EV_IRQn                = 33,     /*!< I2C2 Event Interrupt                                  */
444     I2C2_ER_IRQn                = 34,     /*!< I2C2 Error Interrupt                                  */
445     SPI1_IRQn                   = 35,     /*!< SPI1 global Interrupt                                 */
446     SPI2_IRQn                   = 36,     /*!< SPI2 global Interrupt                                 */
447     USART1_IRQn                 = 37,     /*!< USART1 global Interrupt                               */
448     USART2_IRQn                 = 38,     /*!< USART2 global Interrupt                               */
449     USART3_IRQn                 = 39,     /*!< USART3 global Interrupt                               */
450     EXTI15_10_IRQn              = 40,     /*!< External Line[15:10] Interrupts                       */
451     RTCAlarm_IRQn               = 41,     /*!< RTC Alarm through EXTI Line Interrupt                 */
452     OTG_FS_WKUP_IRQn            = 42,     /*!< USB OTG FS WakeUp from suspend through EXTI Line Interrupt
    */
453     TIM5_IRQn                   = 50,     /*!< TIM5 global Interrupt                                 */
454     SPI3_IRQn                   = 51,     /*!< SPI3 global Interrupt                                 */
455     UART4_IRQn                  = 52,     /*!< UART4 global Interrupt                                */
456     UART5_IRQn                  = 53,     /*!< UART5 global Interrupt                                */
457     TIM6_IRQn                   = 54,     /*!< TIM6 global Interrupt                                 */
458     TIM7_IRQn                   = 55,     /*!< TIM7 global Interrupt                                 */
459     DMA2_Channel1_IRQn          = 56,     /*!< DMA2 Channel 1 global Interrupt                       */
460     DMA2_Channel2_IRQn          = 57,     /*!< DMA2 Channel 2 global Interrupt                       */
461     DMA2_Channel3_IRQn          = 58,     /*!< DMA2 Channel 3 global Interrupt                       */
462     DMA2_Channel4_IRQn          = 59,     /*!< DMA2 Channel 4 global Interrupt                       */
463     DMA2_Channel5_IRQn          = 60,     /*!< DMA2 Channel 5 global Interrupt                       */
464     ETH_IRQn                    = 61,     /*!< Ethernet global Interrupt                             */
465     ETH_WKUP_IRQn               = 62,     /*!< Ethernet Wakeup through EXTI line Interrupt           */
466     CAN2_TX_IRQn                = 63,     /*!< CAN2 TX Interrupt                                     */
467     CAN2_RX0_IRQn               = 64,     /*!< CAN2 RX0 Interrupt                                    */
468     CAN2_RX1_IRQn               = 65,     /*!< CAN2 RX1 Interrupt                                    */
469     CAN2_SCE_IRQn               = 66,     /*!< CAN2 SCE Interrupt                                    */
470     OTG_FS_IRQn                 = 67      /*!< USB OTG FS global Interrupt                           */
471   #endif /* STM32F10X_CL */
472   } IRQn_Type;
473
474   /**
475     * @}
476     */
477
478   #include "core_cm3.h"
479   #include "system_stm32f10x.h"
480   #include <stdint.h>
481
482   /** @addtogroup Exported_types
483     * @{
484     */
485
486   /*!< STM32F10x Standard Peripheral Library old types (maintained for legacy purpose) */
487   typedef int32_t  s32;
488   typedef int16_t s16;
489   typedef int8_t  s8;
490
491   typedef const int32_t sc32;  /*!< Read Only */
492   typedef const int16_t sc16;  /*!< Read Only */
493   typedef const int8_t sc8;   /*!< Read Only */
494
495   typedef __IO int32_t  vs32;
496   typedef __IO int16_t  vs16;
```

```
typedef __IO int8_t   vs8;

typedef __I int32_t vsc32;  /*!< Read Only */
typedef __I int16_t vsc16;  /*!< Read Only */
typedef __I int8_t vsc8;   /*!< Read Only */

typedef uint32_t  u32;
typedef uint16_t u16;
typedef uint8_t  u8;

typedef const uint32_t uc32;  /*!< Read Only */
typedef const uint16_t uc16;  /*!< Read Only */
typedef const uint8_t uc8;   /*!< Read Only */

typedef __IO uint32_t  vu32;
typedef __IO uint16_t vu16;
typedef __IO uint8_t  vu8;

typedef __I uint32_t vuc32;  /*!< Read Only */
typedef __I uint16_t vuc16;  /*!< Read Only */
typedef __I uint8_t vuc8;   /*!< Read Only */

typedef enum {RESET = 0, SET = !RESET} FlagStatus, ITStatus;

typedef enum {DISABLE = 0, ENABLE = !DISABLE} FunctionalState;
#define IS_FUNCTIONAL_STATE(STATE) (((STATE) == DISABLE) || ((STATE) == ENABLE))

typedef enum {ERROR = 0, SUCCESS = !ERROR} ErrorStatus;

/*!< STM32F10x Standard Peripheral Library old definitions (maintained for legacy purpose) */
#define HSEStartUp_TimeOut   HSE_STARTUP_TIMEOUT
#define HSE_Value            HSE_VALUE
#define HSI_Value            HSI_VALUE
/**
  * @}
  */

/** @addtogroup Peripheral_registers_structures
  * @{
  */

/**
  * @brief Analog to Digital Converter
  */

typedef struct
{
  __IO uint32_t SR;
  __IO uint32_t CR1;
  __IO uint32_t CR2;
  __IO uint32_t SMPR1;
  __IO uint32_t SMPR2;
  __IO uint32_t JOFR1;
  __IO uint32_t JOFR2;
  __IO uint32_t JOFR3;
  __IO uint32_t JOFR4;
  __IO uint32_t HTR;
  __IO uint32_t LTR;
  __IO uint32_t SQR1;
  __IO uint32_t SQR2;
  __IO uint32_t SQR3;
  __IO uint32_t JSQR;
  __IO uint32_t JDR1;
  __IO uint32_t JDR2;
  __IO uint32_t JDR3;
  __IO uint32_t JDR4;
  __IO uint32_t DR;
} ADC_TypeDef;

/**
  * @brief Backup Registers
  */

```

```
570   typedef struct
571   {
572     uint32_t  RESERVED0;
573     __IO uint16_t DR1;
574     uint16_t  RESERVED1;
575     __IO uint16_t DR2;
576     uint16_t  RESERVED2;
577     __IO uint16_t DR3;
578     uint16_t  RESERVED3;
579     __IO uint16_t DR4;
580     uint16_t  RESERVED4;
581     __IO uint16_t DR5;
582     uint16_t  RESERVED5;
583     __IO uint16_t DR6;
584     uint16_t  RESERVED6;
585     __IO uint16_t DR7;
586     uint16_t  RESERVED7;
587     __IO uint16_t DR8;
588     uint16_t  RESERVED8;
589     __IO uint16_t DR9;
590     uint16_t  RESERVED9;
591     __IO uint16_t DR10;
592     uint16_t  RESERVED10;
593     __IO uint16_t RTCCR;
594     uint16_t  RESERVED11;
595     __IO uint16_t CR;
596     uint16_t  RESERVED12;
597     __IO uint16_t CSR;
598     uint16_t  RESERVED13[5];
599     __IO uint16_t DR11;
600     uint16_t  RESERVED14;
601     __IO uint16_t DR12;
602     uint16_t  RESERVED15;
603     __IO uint16_t DR13;
604     uint16_t  RESERVED16;
605     __IO uint16_t DR14;
606     uint16_t  RESERVED17;
607     __IO uint16_t DR15;
608     uint16_t  RESERVED18;
609     __IO uint16_t DR16;
610     uint16_t  RESERVED19;
611     __IO uint16_t DR17;
612     uint16_t  RESERVED20;
613     __IO uint16_t DR18;
614     uint16_t  RESERVED21;
615     __IO uint16_t DR19;
616     uint16_t  RESERVED22;
617     __IO uint16_t DR20;
618     uint16_t  RESERVED23;
619     __IO uint16_t DR21;
620     uint16_t  RESERVED24;
621     __IO uint16_t DR22;
622     uint16_t  RESERVED25;
623     __IO uint16_t DR23;
624     uint16_t  RESERVED26;
625     __IO uint16_t DR24;
626     uint16_t  RESERVED27;
627     __IO uint16_t DR25;
628     uint16_t  RESERVED28;
629     __IO uint16_t DR26;
630     uint16_t  RESERVED29;
631     __IO uint16_t DR27;
632     uint16_t  RESERVED30;
633     __IO uint16_t DR28;
634     uint16_t  RESERVED31;
635     __IO uint16_t DR29;
636     uint16_t  RESERVED32;
637     __IO uint16_t DR30;
638     uint16_t  RESERVED33;
639     __IO uint16_t DR31;
640     uint16_t  RESERVED34;
641     __IO uint16_t DR32;
642     uint16_t  RESERVED35;
```

```
  __IO uint16_t DR33;
  uint16_t  RESERVED36;
  __IO uint16_t DR34;
  uint16_t  RESERVED37;
  __IO uint16_t DR35;
  uint16_t  RESERVED38;
  __IO uint16_t DR36;
  uint16_t  RESERVED39;
  __IO uint16_t DR37;
  uint16_t  RESERVED40;
  __IO uint16_t DR38;
  uint16_t  RESERVED41;
  __IO uint16_t DR39;
  uint16_t  RESERVED42;
  __IO uint16_t DR40;
  uint16_t  RESERVED43;
  __IO uint16_t DR41;
  uint16_t  RESERVED44;
  __IO uint16_t DR42;
  uint16_t  RESERVED45;
} BKP_TypeDef;

/**
  * @brief Controller Area Network TxMailBox
  */

typedef struct
{
  __IO uint32_t TIR;
  __IO uint32_t TDTR;
  __IO uint32_t TDLR;
  __IO uint32_t TDHR;
} CAN_TxMailBox_TypeDef;

/**
  * @brief Controller Area Network FIFOMailBox
  */

typedef struct
{
  __IO uint32_t RIR;
  __IO uint32_t RDTR;
  __IO uint32_t RDLR;
  __IO uint32_t RDHR;
} CAN_FIFOMailBox_TypeDef;

/**
  * @brief Controller Area Network FilterRegister
  */

typedef struct
{
  __IO uint32_t FR1;
  __IO uint32_t FR2;
} CAN_FilterRegister_TypeDef;

/**
  * @brief Controller Area Network
  */

typedef struct
{
  __IO uint32_t MCR;
  __IO uint32_t MSR;
  __IO uint32_t TSR;
  __IO uint32_t RF0R;
  __IO uint32_t RF1R;
  __IO uint32_t IER;
  __IO uint32_t ESR;
  __IO uint32_t BTR;
  uint32_t  RESERVED0[88];
  CAN_TxMailBox_TypeDef sTxMailBox[3];
  CAN_FIFOMailBox_TypeDef sFIFOMailBox[2];
```

```
  uint32_t  RESERVED1[12];
  __IO uint32_t FMR;
  __IO uint32_t FM1R;
  uint32_t  RESERVED2;
  __IO uint32_t FS1R;
  uint32_t  RESERVED3;
  __IO uint32_t FFA1R;
  uint32_t  RESERVED4;
  __IO uint32_t FA1R;
  uint32_t  RESERVED5[8];
#ifndef STM32F10X_CL
  CAN_FilterRegister_TypeDef sFilterRegister[14];
#else
  CAN_FilterRegister_TypeDef sFilterRegister[28];
#endif /* STM32F10X_CL */
} CAN_TypeDef;

/**
  * @brief Consumer Electronics Control (CEC)
  */
typedef struct
{
  __IO uint32_t CFGR;
  __IO uint32_t OAR;
  __IO uint32_t PRES;
  __IO uint32_t ESR;
  __IO uint32_t CSR;
  __IO uint32_t TXD;
  __IO uint32_t RXD;
} CEC_TypeDef;

/**
  * @brief CRC calculation unit
  */

typedef struct
{
  __IO uint32_t DR;
  __IO uint8_t  IDR;
  uint8_t   RESERVED0;
  uint16_t  RESERVED1;
  __IO uint32_t CR;
} CRC_TypeDef;

/**
  * @brief Digital to Analog Converter
  */

typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t SWTRIGR;
  __IO uint32_t DHR12R1;
  __IO uint32_t DHR12L1;
  __IO uint32_t DHR8R1;
  __IO uint32_t DHR12R2;
  __IO uint32_t DHR12L2;
  __IO uint32_t DHR8R2;
  __IO uint32_t DHR12RD;
  __IO uint32_t DHR12LD;
  __IO uint32_t DHR8RD;
  __IO uint32_t DOR1;
  __IO uint32_t DOR2;
#if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
  __IO uint32_t SR;
#endif
} DAC_TypeDef;

/**
  * @brief Debug MCU
  */

typedef struct
```

```
  {
    __IO uint32_t IDCODE;
    __IO uint32_t CR;
  }DBGMCU_TypeDef;

  /**
    * @brief DMA Controller
    */

  typedef struct
  {
    __IO uint32_t CCR;
    __IO uint32_t CNDTR;
    __IO uint32_t CPAR;
    __IO uint32_t CMAR;
  } DMA_Channel_TypeDef;

  typedef struct
  {
    __IO uint32_t ISR;
    __IO uint32_t IFCR;
  } DMA_TypeDef;

  /**
    * @brief Ethernet MAC
    */

  typedef struct
  {
    __IO uint32_t MACCR;
    __IO uint32_t MACFFR;
    __IO uint32_t MACHTHR;
    __IO uint32_t MACHTLR;
    __IO uint32_t MACMIIAR;
    __IO uint32_t MACMIIDR;
    __IO uint32_t MACFCR;
    __IO uint32_t MACVLANTR;             /*    8 */
         uint32_t RESERVED0[2];
    __IO uint32_t MACRWUFFR;             /*   11 */
    __IO uint32_t MACPMTCSR;
         uint32_t RESERVED1[2];
    __IO uint32_t MACSR;                 /*   15 */
    __IO uint32_t MACIMR;
    __IO uint32_t MACA0HR;
    __IO uint32_t MACA0LR;
    __IO uint32_t MACA1HR;
    __IO uint32_t MACA1LR;
    __IO uint32_t MACA2HR;
    __IO uint32_t MACA2LR;
    __IO uint32_t MACA3HR;
    __IO uint32_t MACA3LR;               /*   24 */
         uint32_t RESERVED2[40];
    __IO uint32_t MMCCR;                 /*   65 */
    __IO uint32_t MMCRIR;
    __IO uint32_t MMCTIR;
    __IO uint32_t MMCRIMR;
    __IO uint32_t MMCTIMR;               /*   69 */
         uint32_t RESERVED3[14];
    __IO uint32_t MMCTGFSCCR;            /*   84 */
    __IO uint32_t MMCTGFMSCCR;
         uint32_t RESERVED4[5];
    __IO uint32_t MMCTGFCR;
         uint32_t RESERVED5[10];
    __IO uint32_t MMCRFCECR;
    __IO uint32_t MMCRFAECR;
         uint32_t RESERVED6[10];
    __IO uint32_t MMCRGUFCR;
         uint32_t RESERVED7[334];
    __IO uint32_t PTPTSCR;
    __IO uint32_t PTPSSIR;
    __IO uint32_t PTPTSHR;
    __IO uint32_t PTPTSLR;
    __IO uint32_t PTPTSHUR;
```

```
  862     __IO uint32_t PTPTSLUR;
  863     __IO uint32_t PTPTSAR;
  864     __IO uint32_t PTPTTHR;
  865     __IO uint32_t PTPTTLR;
  866          uint32_t RESERVED8[567];
  867     __IO uint32_t DMABMR;
  868     __IO uint32_t DMATPDR;
  869     __IO uint32_t DMARPDR;
  870     __IO uint32_t DMARDLAR;
  871     __IO uint32_t DMATDLAR;
  872     __IO uint32_t DMASR;
  873     __IO uint32_t DMAOMR;
  874     __IO uint32_t DMAIER;
  875     __IO uint32_t DMAMFBOCR;
  876          uint32_t RESERVED9[9];
  877     __IO uint32_t DMACHTDR;
  878     __IO uint32_t DMACHRDR;
  879     __IO uint32_t DMACHTBAR;
  880     __IO uint32_t DMACHRBAR;
  881   } ETH_TypeDef;
  882
  883   /**
  884     * @brief External Interrupt/Event Controller
  885     */
  886
  887   typedef struct
  888   {
  889     __IO uint32_t IMR;
  890     __IO uint32_t EMR;
  891     __IO uint32_t RTSR;
  892     __IO uint32_t FTSR;
  893     __IO uint32_t SWIER;
  894     __IO uint32_t PR;
  895   } EXTI_TypeDef;
  896
  897   /**
  898     * @brief FLASH Registers
  899     */
  900
  901   typedef struct
  902   {
  903     __IO uint32_t ACR;
  904     __IO uint32_t KEYR;
  905     __IO uint32_t OPTKEYR;
  906     __IO uint32_t SR;
  907     __IO uint32_t CR;
  908     __IO uint32_t AR;
  909     __IO uint32_t RESERVED;
  910     __IO uint32_t OBR;
  911     __IO uint32_t WRPR;
  912   #ifdef STM32F10X_XL
  913     uint32_t RESERVED1[8];
  914     __IO uint32_t KEYR2;
  915     uint32_t RESERVED2;
  916     __IO uint32_t SR2;
  917     __IO uint32_t CR2;
  918     __IO uint32_t AR2;
  919   #endif /* STM32F10X_XL */
  920   } FLASH_TypeDef;
  921
  922   /**
  923     * @brief Option Bytes Registers
  924     */
  925
  926   typedef struct
  927   {
  928     __IO uint16_t RDP;
  929     __IO uint16_t USER;
  930     __IO uint16_t Data0;
  931     __IO uint16_t Data1;
  932     __IO uint16_t WRP0;
  933     __IO uint16_t WRP1;
  934     __IO uint16_t WRP2;
```

```
  __IO uint16_t WRP3;
} OB_TypeDef;

/**
  * @brief Flexible Static Memory Controller
  */

typedef struct
{
  __IO uint32_t BTCR[8];
} FSMC_Bank1_TypeDef;

/**
  * @brief Flexible Static Memory Controller Bank1E
  */

typedef struct
{
  __IO uint32_t BWTR[7];
} FSMC_Bank1E_TypeDef;

/**
  * @brief Flexible Static Memory Controller Bank2
  */

typedef struct
{
  __IO uint32_t PCR2;
  __IO uint32_t SR2;
  __IO uint32_t PMEM2;
  __IO uint32_t PATT2;
  uint32_t  RESERVED0;
  __IO uint32_t ECCR2;
} FSMC_Bank2_TypeDef;

/**
  * @brief Flexible Static Memory Controller Bank3
  */

typedef struct
{
  __IO uint32_t PCR3;
  __IO uint32_t SR3;
  __IO uint32_t PMEM3;
  __IO uint32_t PATT3;
  uint32_t  RESERVED0;
  __IO uint32_t ECCR3;
} FSMC_Bank3_TypeDef;

/**
  * @brief Flexible Static Memory Controller Bank4
  */

typedef struct
{
  __IO uint32_t PCR4;
  __IO uint32_t SR4;
  __IO uint32_t PMEM4;
  __IO uint32_t PATT4;
  __IO uint32_t PIO4;
} FSMC_Bank4_TypeDef;

/**
  * @brief General Purpose I/O
  */

typedef struct
{
  __IO uint32_t CRL;
  __IO uint32_t CRH;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
```

```
  __IO uint32_t BRR;
  __IO uint32_t LCKR;
} GPIO_TypeDef;

/**
  * @brief Alternate Function I/O
  */

typedef struct
{
  __IO uint32_t EVCR;
  __IO uint32_t MAPR;
  __IO uint32_t EXTICR[4];
  uint32_t RESERVED0;
  __IO uint32_t MAPR2;
} AFIO_TypeDef;
/**
  * @brief Inter Integrated Circuit Interface
  */

typedef struct
{
  __IO uint16_t CR1;
  uint16_t  RESERVED0;
  __IO uint16_t CR2;
  uint16_t  RESERVED1;
  __IO uint16_t OAR1;
  uint16_t  RESERVED2;
  __IO uint16_t OAR2;
  uint16_t  RESERVED3;
  __IO uint16_t DR;
  uint16_t  RESERVED4;
  __IO uint16_t SR1;
  uint16_t  RESERVED5;
  __IO uint16_t SR2;
  uint16_t  RESERVED6;
  __IO uint16_t CCR;
  uint16_t  RESERVED7;
  __IO uint16_t TRISE;
  uint16_t  RESERVED8;
} I2C_TypeDef;

/**
  * @brief Independent WATCHDOG
  */

typedef struct
{
  __IO uint32_t KR;
  __IO uint32_t PR;
  __IO uint32_t RLR;
  __IO uint32_t SR;
} IWDG_TypeDef;

/**
  * @brief Power Control
  */

typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t CSR;
} PWR_TypeDef;

/**
  * @brief Reset and Clock Control
  */

typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t CFGR;
  __IO uint32_t CIR;
```

```
  __IO uint32_t APB2RSTR;
  __IO uint32_t APB1RSTR;
  __IO uint32_t AHBENR;
  __IO uint32_t APB2ENR;
  __IO uint32_t APB1ENR;
  __IO uint32_t BDCR;
  __IO uint32_t CSR;

#ifdef STM32F10X_CL
  __IO uint32_t AHBRSTR;
  __IO uint32_t CFGR2;
#endif /* STM32F10X_CL */

#if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
  uint32_t RESERVED0;
  __IO uint32_t CFGR2;
#endif /* STM32F10X_LD_VL || STM32F10X_MD_VL || STM32F10X_HD_VL */
} RCC_TypeDef;

/**
  * @brief Real-Time Clock
  */

typedef struct
{
  __IO uint16_t CRH;
  uint16_t  RESERVED0;
  __IO uint16_t CRL;
  uint16_t  RESERVED1;
  __IO uint16_t PRLH;
  uint16_t  RESERVED2;
  __IO uint16_t PRLL;
  uint16_t  RESERVED3;
  __IO uint16_t DIVH;
  uint16_t  RESERVED4;
  __IO uint16_t DIVL;
  uint16_t  RESERVED5;
  __IO uint16_t CNTH;
  uint16_t  RESERVED6;
  __IO uint16_t CNTL;
  uint16_t  RESERVED7;
  __IO uint16_t ALRH;
  uint16_t  RESERVED8;
  __IO uint16_t ALRL;
  uint16_t  RESERVED9;
} RTC_TypeDef;

/**
  * @brief SD host Interface
  */

typedef struct
{
  __IO uint32_t POWER;
  __IO uint32_t CLKCR;
  __IO uint32_t ARG;
  __IO uint32_t CMD;
  __I uint32_t RESPCMD;
  __I uint32_t RESP1;
  __I uint32_t RESP2;
  __I uint32_t RESP3;
  __I uint32_t RESP4;
  __IO uint32_t DTIMER;
  __IO uint32_t DLEN;
  __IO uint32_t DCTRL;
  __I uint32_t DCOUNT;
  __I uint32_t STA;
  __IO uint32_t ICR;
  __IO uint32_t MASK;
  uint32_t  RESERVED0[2];
  __I uint32_t FIFOCNT;
  uint32_t  RESERVED1[13];
  __IO uint32_t FIFO;
```

```
} SDIO_TypeDef;

/**
  * @brief Serial Peripheral Interface
  */

typedef struct
{
  __IO uint16_t CR1;
  uint16_t  RESERVED0;
  __IO uint16_t CR2;
  uint16_t  RESERVED1;
  __IO uint16_t SR;
  uint16_t  RESERVED2;
  __IO uint16_t DR;
  uint16_t  RESERVED3;
  __IO uint16_t CRCPR;
  uint16_t  RESERVED4;
  __IO uint16_t RXCRCR;
  uint16_t  RESERVED5;
  __IO uint16_t TXCRCR;
  uint16_t  RESERVED6;
  __IO uint16_t I2SCFGR;
  uint16_t  RESERVED7;
  __IO uint16_t I2SPR;
  uint16_t  RESERVED8;
} SPI_TypeDef;

/**
  * @brief TIM
  */

typedef struct
{
  __IO uint16_t CR1;
  uint16_t  RESERVED0;
  __IO uint16_t CR2;
  uint16_t  RESERVED1;
  __IO uint16_t SMCR;
  uint16_t  RESERVED2;
  __IO uint16_t DIER;
  uint16_t  RESERVED3;
  __IO uint16_t SR;
  uint16_t  RESERVED4;
  __IO uint16_t EGR;
  uint16_t  RESERVED5;
  __IO uint16_t CCMR1;
  uint16_t  RESERVED6;
  __IO uint16_t CCMR2;
  uint16_t  RESERVED7;
  __IO uint16_t CCER;
  uint16_t  RESERVED8;
  __IO uint16_t CNT;
  uint16_t  RESERVED9;
  __IO uint16_t PSC;
  uint16_t  RESERVED10;
  __IO uint16_t ARR;
  uint16_t  RESERVED11;
  __IO uint16_t RCR;
  uint16_t  RESERVED12;
  __IO uint16_t CCR1;
  uint16_t  RESERVED13;
  __IO uint16_t CCR2;
  uint16_t  RESERVED14;
  __IO uint16_t CCR3;
  uint16_t  RESERVED15;
  __IO uint16_t CCR4;
  uint16_t  RESERVED16;
  __IO uint16_t BDTR;
  uint16_t  RESERVED17;
  __IO uint16_t DCR;
  uint16_t  RESERVED18;
  __IO uint16_t DMAR;
```

```
  uint16_t  RESERVED19;
} TIM_TypeDef;

/**
  * @brief Universal Synchronous Asynchronous Receiver Transmitter
  */

typedef struct
{
  __IO uint16_t SR;
  uint16_t  RESERVED0;
  __IO uint16_t DR;
  uint16_t  RESERVED1;
  __IO uint16_t BRR;
  uint16_t  RESERVED2;
  __IO uint16_t CR1;
  uint16_t  RESERVED3;
  __IO uint16_t CR2;
  uint16_t  RESERVED4;
  __IO uint16_t CR3;
  uint16_t  RESERVED5;
  __IO uint16_t GTPR;
  uint16_t  RESERVED6;
} USART_TypeDef;

/**
  * @brief Window WATCHDOG
  */

typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t CFR;
  __IO uint32_t SR;
} WWDG_TypeDef;

/**
  * @}
  */

/** @addtogroup Peripheral_memory_map
  * @{
  */


#define FLASH_BASE            ((uint32_t)0x08000000) /*!< FLASH base address in the alias region */
#define SRAM_BASE             ((uint32_t)0x20000000) /*!< SRAM base address in the alias region */
#define PERIPH_BASE           ((uint32_t)0x40000000) /*!< Peripheral base address in the alias region */

#define SRAM_BB_BASE          ((uint32_t)0x22000000) /*!< SRAM base address in the bit-band region */
#define PERIPH_BB_BASE        ((uint32_t)0x42000000) /*!< Peripheral base address in the bit-band region */

#define FSMC_R_BASE           ((uint32_t)0xA0000000) /*!< FSMC registers base address */

/*!< Peripheral memory map */
#define APB1PERIPH_BASE       PERIPH_BASE
#define APB2PERIPH_BASE       (PERIPH_BASE + 0x10000)
#define AHBPERIPH_BASE        (PERIPH_BASE + 0x20000)

#define TIM2_BASE             (APB1PERIPH_BASE + 0x0000)
#define TIM3_BASE             (APB1PERIPH_BASE + 0x0400)
#define TIM4_BASE             (APB1PERIPH_BASE + 0x0800)
#define TIM5_BASE             (APB1PERIPH_BASE + 0x0C00)
#define TIM6_BASE             (APB1PERIPH_BASE + 0x1000)
#define TIM7_BASE             (APB1PERIPH_BASE + 0x1400)
#define TIM12_BASE            (APB1PERIPH_BASE + 0x1800)
#define TIM13_BASE            (APB1PERIPH_BASE + 0x1C00)
#define TIM14_BASE            (APB1PERIPH_BASE + 0x2000)
#define RTC_BASE              (APB1PERIPH_BASE + 0x2800)
#define WWDG_BASE             (APB1PERIPH_BASE + 0x2C00)
#define IWDG_BASE             (APB1PERIPH_BASE + 0x3000)
#define SPI2_BASE             (APB1PERIPH_BASE + 0x3800)
```

```
#define SPI3_BASE             (APB1PERIPH_BASE + 0x3C00)
#define USART2_BASE           (APB1PERIPH_BASE + 0x4400)
#define USART3_BASE           (APB1PERIPH_BASE + 0x4800)
#define UART4_BASE            (APB1PERIPH_BASE + 0x4C00)
#define UART5_BASE            (APB1PERIPH_BASE + 0x5000)
#define I2C1_BASE             (APB1PERIPH_BASE + 0x5400)
#define I2C2_BASE             (APB1PERIPH_BASE + 0x5800)
#define CAN1_BASE             (APB1PERIPH_BASE + 0x6400)
#define CAN2_BASE             (APB1PERIPH_BASE + 0x6800)
#define BKP_BASE              (APB1PERIPH_BASE + 0x6C00)
#define PWR_BASE              (APB1PERIPH_BASE + 0x7000)
#define DAC_BASE              (APB1PERIPH_BASE + 0x7400)
#define CEC_BASE              (APB1PERIPH_BASE + 0x7800)

#define AFIO_BASE             (APB2PERIPH_BASE + 0x0000)
#define EXTI_BASE             (APB2PERIPH_BASE + 0x0400)
#define GPIOA_BASE            (APB2PERIPH_BASE + 0x0800)
#define GPIOB_BASE            (APB2PERIPH_BASE + 0x0C00)
#define GPIOC_BASE            (APB2PERIPH_BASE + 0x1000)
#define GPIOD_BASE            (APB2PERIPH_BASE + 0x1400)
#define GPIOE_BASE            (APB2PERIPH_BASE + 0x1800)
#define GPIOF_BASE            (APB2PERIPH_BASE + 0x1C00)
#define GPIOG_BASE            (APB2PERIPH_BASE + 0x2000)
#define ADC1_BASE             (APB2PERIPH_BASE + 0x2400)
#define ADC2_BASE             (APB2PERIPH_BASE + 0x2800)
#define TIM1_BASE             (APB2PERIPH_BASE + 0x2C00)
#define SPI1_BASE             (APB2PERIPH_BASE + 0x3000)
#define TIM8_BASE             (APB2PERIPH_BASE + 0x3400)
#define USART1_BASE           (APB2PERIPH_BASE + 0x3800)
#define ADC3_BASE             (APB2PERIPH_BASE + 0x3C00)
#define TIM15_BASE            (APB2PERIPH_BASE + 0x4000)
#define TIM16_BASE            (APB2PERIPH_BASE + 0x4400)
#define TIM17_BASE            (APB2PERIPH_BASE + 0x4800)
#define TIM9_BASE             (APB2PERIPH_BASE + 0x4C00)
#define TIM10_BASE            (APB2PERIPH_BASE + 0x5000)
#define TIM11_BASE            (APB2PERIPH_BASE + 0x5400)

#define SDIO_BASE             (PERIPH_BASE + 0x18000)

#define DMA1_BASE             (AHBPERIPH_BASE + 0x0000)
#define DMA1_Channel1_BASE    (AHBPERIPH_BASE + 0x0008)
#define DMA1_Channel2_BASE    (AHBPERIPH_BASE + 0x001C)
#define DMA1_Channel3_BASE    (AHBPERIPH_BASE + 0x0030)
#define DMA1_Channel4_BASE    (AHBPERIPH_BASE + 0x0044)
#define DMA1_Channel5_BASE    (AHBPERIPH_BASE + 0x0058)
#define DMA1_Channel6_BASE    (AHBPERIPH_BASE + 0x006C)
#define DMA1_Channel7_BASE    (AHBPERIPH_BASE + 0x0080)
#define DMA2_BASE             (AHBPERIPH_BASE + 0x0400)
#define DMA2_Channel1_BASE    (AHBPERIPH_BASE + 0x0408)
#define DMA2_Channel2_BASE    (AHBPERIPH_BASE + 0x041C)
#define DMA2_Channel3_BASE    (AHBPERIPH_BASE + 0x0430)
#define DMA2_Channel4_BASE    (AHBPERIPH_BASE + 0x0444)
#define DMA2_Channel5_BASE    (AHBPERIPH_BASE + 0x0458)
#define RCC_BASE              (AHBPERIPH_BASE + 0x1000)
#define CRC_BASE              (AHBPERIPH_BASE + 0x3000)

#define FLASH_R_BASE          (AHBPERIPH_BASE + 0x2000) /*!< Flash registers base address */
#define OB_BASE               ((uint32_t)0x1FFFF800)    /*!< Flash Option Bytes base address */

#define ETH_BASE              (AHBPERIPH_BASE + 0x8000)
#define ETH_MAC_BASE          (ETH_BASE)
#define ETH_MMC_BASE          (ETH_BASE + 0x0100)
#define ETH_PTP_BASE          (ETH_BASE + 0x0700)
#define ETH_DMA_BASE          (ETH_BASE + 0x1000)

#define FSMC_Bank1_R_BASE     (FSMC_R_BASE + 0x0000) /*!< FSMC Bank1 registers base address */
#define FSMC_Bank1E_R_BASE    (FSMC_R_BASE + 0x0104) /*!< FSMC Bank1E registers base address */
#define FSMC_Bank2_R_BASE     (FSMC_R_BASE + 0x0060) /*!< FSMC Bank2 registers base address */
#define FSMC_Bank3_R_BASE     (FSMC_R_BASE + 0x0080) /*!< FSMC Bank3 registers base address */
#define FSMC_Bank4_R_BASE     (FSMC_R_BASE + 0x00A0) /*!< FSMC Bank4 registers base address */

#define DBGMCU_BASE          ((uint32_t)0xE0042000) /*!< Debug MCU registers base address */

```

```
  /**
    * @}
    */

  /** @addtogroup Peripheral_declaration
    * @{
    */

  #define TIM2                ((TIM_TypeDef *) TIM2_BASE)
  #define TIM3                ((TIM_TypeDef *) TIM3_BASE)
  #define TIM4                ((TIM_TypeDef *) TIM4_BASE)
  #define TIM5                ((TIM_TypeDef *) TIM5_BASE)
  #define TIM6                ((TIM_TypeDef *) TIM6_BASE)
  #define TIM7                ((TIM_TypeDef *) TIM7_BASE)
  #define TIM12               ((TIM_TypeDef *) TIM12_BASE)
  #define TIM13               ((TIM_TypeDef *) TIM13_BASE)
  #define TIM14               ((TIM_TypeDef *) TIM14_BASE)
  #define RTC                 ((RTC_TypeDef *) RTC_BASE)
  #define WWDG                ((WWDG_TypeDef *) WWDG_BASE)
  #define IWDG                ((IWDG_TypeDef *) IWDG_BASE)
  #define SPI2                ((SPI_TypeDef *) SPI2_BASE)
  #define SPI3                ((SPI_TypeDef *) SPI3_BASE)
  #define USART2              ((USART_TypeDef *) USART2_BASE)
  #define USART3              ((USART_TypeDef *) USART3_BASE)
  #define UART4               ((USART_TypeDef *) UART4_BASE)
  #define UART5               ((USART_TypeDef *) UART5_BASE)
  #define I2C1                ((I2C_TypeDef *) I2C1_BASE)
  #define I2C2                ((I2C_TypeDef *) I2C2_BASE)
  #define CAN1                ((CAN_TypeDef *) CAN1_BASE)
  #define CAN2                ((CAN_TypeDef *) CAN2_BASE)
  #define BKP                 ((BKP_TypeDef *) BKP_BASE)
  #define PWR                 ((PWR_TypeDef *) PWR_BASE)
  #define DAC                 ((DAC_TypeDef *) DAC_BASE)
  #define CEC                 ((CEC_TypeDef *) CEC_BASE)
  #define AFIO                ((AFIO_TypeDef *) AFIO_BASE)
  #define EXTI                ((EXTI_TypeDef *) EXTI_BASE)
  #define GPIOA               ((GPIO_TypeDef *) GPIOA_BASE)
  #define GPIOB               ((GPIO_TypeDef *) GPIOB_BASE)
  #define GPIOC               ((GPIO_TypeDef *) GPIOC_BASE)
  #define GPIOD               ((GPIO_TypeDef *) GPIOD_BASE)
  #define GPIOE               ((GPIO_TypeDef *) GPIOE_BASE)
  #define GPIOF               ((GPIO_TypeDef *) GPIOF_BASE)
  #define GPIOG               ((GPIO_TypeDef *) GPIOG_BASE)
  #define ADC1                ((ADC_TypeDef *) ADC1_BASE)
  #define ADC2                ((ADC_TypeDef *) ADC2_BASE)
  #define TIM1                ((TIM_TypeDef *) TIM1_BASE)
  #define SPI1                ((SPI_TypeDef *) SPI1_BASE)
  #define TIM8                ((TIM_TypeDef *) TIM8_BASE)
  #define USART1              ((USART_TypeDef *) USART1_BASE)
  #define ADC3                ((ADC_TypeDef *) ADC3_BASE)
  #define TIM15               ((TIM_TypeDef *) TIM15_BASE)
  #define TIM16               ((TIM_TypeDef *) TIM16_BASE)
  #define TIM17               ((TIM_TypeDef *) TIM17_BASE)
  #define TIM9                ((TIM_TypeDef *) TIM9_BASE)
  #define TIM10               ((TIM_TypeDef *) TIM10_BASE)
  #define TIM11               ((TIM_TypeDef *) TIM11_BASE)
  #define SDIO                ((SDIO_TypeDef *) SDIO_BASE)
  #define DMA1                ((DMA_TypeDef *) DMA1_BASE)
  #define DMA2                ((DMA_TypeDef *) DMA2_BASE)
  #define DMA1_Channel1       ((DMA_Channel_TypeDef *) DMA1_Channel1_BASE)
  #define DMA1_Channel2       ((DMA_Channel_TypeDef *) DMA1_Channel2_BASE)
  #define DMA1_Channel3       ((DMA_Channel_TypeDef *) DMA1_Channel3_BASE)
  #define DMA1_Channel4       ((DMA_Channel_TypeDef *) DMA1_Channel4_BASE)
  #define DMA1_Channel5       ((DMA_Channel_TypeDef *) DMA1_Channel5_BASE)
  #define DMA1_Channel6       ((DMA_Channel_TypeDef *) DMA1_Channel6_BASE)
  #define DMA1_Channel7       ((DMA_Channel_TypeDef *) DMA1_Channel7_BASE)
  #define DMA2_Channel1       ((DMA_Channel_TypeDef *) DMA2_Channel1_BASE)
  #define DMA2_Channel2       ((DMA_Channel_TypeDef *) DMA2_Channel2_BASE)
  #define DMA2_Channel3       ((DMA_Channel_TypeDef *) DMA2_Channel3_BASE)
  #define DMA2_Channel4       ((DMA_Channel_TypeDef *) DMA2_Channel4_BASE)
  #define DMA2_Channel5       ((DMA_Channel_TypeDef *) DMA2_Channel5_BASE)
  #define RCC                 ((RCC_TypeDef *) RCC_BASE)
  #define CRC                 ((CRC_TypeDef *) CRC_BASE)
```

```
#define FLASH               ((FLASH_TypeDef *) FLASH_R_BASE)
#define OB                  ((OB_TypeDef *) OB_BASE)
#define ETH                 ((ETH_TypeDef *) ETH_BASE)
#define FSMC_Bank1          ((FSMC_Bank1_TypeDef *) FSMC_Bank1_R_BASE)
#define FSMC_Bank1E         ((FSMC_Bank1E_TypeDef *) FSMC_Bank1E_R_BASE)
#define FSMC_Bank2          ((FSMC_Bank2_TypeDef *) FSMC_Bank2_R_BASE)
#define FSMC_Bank3          ((FSMC_Bank3_TypeDef *) FSMC_Bank3_R_BASE)
#define FSMC_Bank4          ((FSMC_Bank4_TypeDef *) FSMC_Bank4_R_BASE)
#define DBGMCU              ((DBGMCU_TypeDef *) DBGMCU_BASE)

/**
  * @}
  */

/** @addtogroup Exported_constants
  * @{
  */

  /** @addtogroup Peripheral_Registers_Bits_Definition
  * @{
  */

/******************************************************************************/
/*                        Peripheral Registers_Bits_Definition                */
/******************************************************************************/

/******************************************************************************/
/*                                                                            */
/*                         CRC calculation unit                               */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for CRC_DR register  *******************/
#define  CRC_DR_DR                           ((uint32_t)0xFFFFFFFF) /*!< Data register bits */


/*******************  Bit definition for CRC_IDR register  ******************/
#define  CRC_IDR_IDR                         ((uint8_t)0xFF)        /*!< General-purpose 8-bit data register bits */


/********************  Bit definition for CRC_CR register  ******************/
#define  CRC_CR_RESET                        ((uint8_t)0x01)        /*!< RESET bit */

/******************************************************************************/
/*                                                                            */
/*                             Power Control                                  */
/*                                                                            */
/******************************************************************************/

/********************  Bit definition for PWR_CR register  ******************/
#define  PWR_CR_LPDS                         ((uint16_t)0x0001)     /*!< Low-Power Deepsleep */
#define  PWR_CR_PDDS                         ((uint16_t)0x0002)     /*!< Power Down Deepsleep */
#define  PWR_CR_CWUF                         ((uint16_t)0x0004)     /*!< Clear Wakeup Flag */
#define  PWR_CR_CSBF                         ((uint16_t)0x0008)     /*!< Clear Standby Flag */
#define  PWR_CR_PVDE                         ((uint16_t)0x0010)     /*!< Power Voltage Detector Enable */

#define  PWR_CR_PLS                          ((uint16_t)0x00E0)     /*!< PLS[2:0] bits (PVD Level Selection) */
#define  PWR_CR_PLS_0                        ((uint16_t)0x0020)     /*!< Bit 0 */
#define  PWR_CR_PLS_1                        ((uint16_t)0x0040)     /*!< Bit 1 */
#define  PWR_CR_PLS_2                        ((uint16_t)0x0080)     /*!< Bit 2 */

/*!< PVD level configuration */
#define  PWR_CR_PLS_2V2                      ((uint16_t)0x0000)     /*!< PVD level 2.2V */
#define  PWR_CR_PLS_2V3                      ((uint16_t)0x0020)     /*!< PVD level 2.3V */
#define  PWR_CR_PLS_2V4                      ((uint16_t)0x0040)     /*!< PVD level 2.4V */
#define  PWR_CR_PLS_2V5                      ((uint16_t)0x0060)     /*!< PVD level 2.5V */
#define  PWR_CR_PLS_2V6                      ((uint16_t)0x0080)     /*!< PVD level 2.6V */
#define  PWR_CR_PLS_2V7                      ((uint16_t)0x00A0)     /*!< PVD level 2.7V */
#define  PWR_CR_PLS_2V8                      ((uint16_t)0x00C0)     /*!< PVD level 2.8V */
#define  PWR_CR_PLS_2V9                      ((uint16_t)0x00E0)     /*!< PVD level 2.9V */

```

```
#define  PWR_CR_DBP                          ((uint16_t)0x0100)     /*!< Disable Backup Domain write protection */


/********************  Bit definition for PWR_CSR register  ********************/
#define  PWR_CSR_WUF                         ((uint16_t)0x0001)     /*!< Wakeup Flag */
#define  PWR_CSR_SBF                         ((uint16_t)0x0002)     /*!< Standby Flag */
#define  PWR_CSR_PVDO                        ((uint16_t)0x0004)     /*!< PVD Output */
#define  PWR_CSR_EWUP                        ((uint16_t)0x0100)     /*!< Enable WKUP pin */

/******************************************************************************/
/*                                                                            */
/*                            Backup registers                                */
/*                                                                            */
/******************************************************************************/

/********************  Bit definition for BKP_DR1 register  ********************/
#define  BKP_DR1_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR2 register  ********************/
#define  BKP_DR2_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR3 register  ********************/
#define  BKP_DR3_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR4 register  ********************/
#define  BKP_DR4_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR5 register  ********************/
#define  BKP_DR5_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR6 register  ********************/
#define  BKP_DR6_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR7 register  ********************/
#define  BKP_DR7_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR8 register  ********************/
#define  BKP_DR8_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR9 register  ********************/
#define  BKP_DR9_D                           ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR10 register  ********************/
#define  BKP_DR10_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR11 register  ********************/
#define  BKP_DR11_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR12 register  ********************/
#define  BKP_DR12_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR13 register  ********************/
#define  BKP_DR13_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR14 register  ********************/
#define  BKP_DR14_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR15 register  ********************/
#define  BKP_DR15_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR16 register  ********************/
#define  BKP_DR16_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR17 register  ********************/
#define  BKP_DR17_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR18 register  ********************/
#define  BKP_DR18_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/********************  Bit definition for BKP_DR19 register  ********************/
#define  BKP_DR19_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

```

```
/*******************  Bit definition for BKP_DR20 register  *******************/
#define  BKP_DR20_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR21 register  *******************/
#define  BKP_DR21_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR22 register  *******************/
#define  BKP_DR22_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR23 register  *******************/
#define  BKP_DR23_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR24 register  *******************/
#define  BKP_DR24_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR25 register  *******************/
#define  BKP_DR25_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR26 register  *******************/
#define  BKP_DR26_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR27 register  *******************/
#define  BKP_DR27_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR28 register  *******************/
#define  BKP_DR28_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR29 register  *******************/
#define  BKP_DR29_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR30 register  *******************/
#define  BKP_DR30_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR31 register  *******************/
#define  BKP_DR31_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR32 register  *******************/
#define  BKP_DR32_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR33 register  *******************/
#define  BKP_DR33_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR34 register  *******************/
#define  BKP_DR34_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR35 register  *******************/
#define  BKP_DR35_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR36 register  *******************/
#define  BKP_DR36_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR37 register  *******************/
#define  BKP_DR37_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR38 register  *******************/
#define  BKP_DR38_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR39 register  *******************/
#define  BKP_DR39_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR40 register  *******************/
#define  BKP_DR40_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR41 register  *******************/
#define  BKP_DR41_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/*******************  Bit definition for BKP_DR42 register  *******************/
#define  BKP_DR42_D                          ((uint16_t)0xFFFF)     /*!< Backup data */

/******************  Bit definition for BKP_RTCCR register  *******************/
#define  BKP_RTCCR_CAL                       ((uint16_t)0x007F)     /*!< Calibration value */
#define  BKP_RTCCR_CCO                       ((uint16_t)0x0080)     /*!< Calibration Clock Output */
#define  BKP_RTCCR_ASOE                      ((uint16_t)0x0100)     /*!< Alarm or Second Output Enable */
```

```
1661   #define  BKP_RTCCR_ASOS                      ((uint16_t)0x0200)     /*!< Alarm or Second Output
       Selection */
1662
1663   /********************  Bit definition for BKP_CR register  ********************/
1664   #define  BKP_CR_TPE                          ((uint8_t)0x01)        /*!< TAMPER pin enable */
1665   #define  BKP_CR_TPAL                         ((uint8_t)0x02)        /*!< TAMPER pin active level */
1666
1667   /********************  Bit definition for BKP_CSR register  ********************/
1668   #define  BKP_CSR_CTE                         ((uint16_t)0x0001)     /*!< Clear Tamper event */
1669   #define  BKP_CSR_CTI                         ((uint16_t)0x0002)     /*!< Clear Tamper Interrupt */
1670   #define  BKP_CSR_TPIE                        ((uint16_t)0x0004)     /*!< TAMPER Pin interrupt enable */
1671   #define  BKP_CSR_TEF                         ((uint16_t)0x0100)     /*!< Tamper Event Flag */
1672   #define  BKP_CSR_TIF                         ((uint16_t)0x0200)     /*!< Tamper Interrupt Flag */
1673
1674   /******************************************************************************/
1675   /*                                                                            */
1676   /*                         Reset and Clock Control                            */
1677   /*                                                                            */
1678   /******************************************************************************/
1679
1680   /********************  Bit definition for RCC_CR register  ********************/
1681   #define  RCC_CR_HSION                        ((uint32_t)0x00000001)        /*!< Internal High Speed
       clock enable */
1682   #define  RCC_CR_HSIRDY                       ((uint32_t)0x00000002)        /*!< Internal High Speed
       clock ready flag */
1683   #define  RCC_CR_HSITRIM                      ((uint32_t)0x000000F8)        /*!< Internal High Speed
       clock trimming */
1684   #define  RCC_CR_HSICAL                       ((uint32_t)0x0000FF00)        /*!< Internal High Speed
       clock Calibration */
1685   #define  RCC_CR_HSEON                        ((uint32_t)0x00010000)        /*!< External High Speed
       clock enable */
1686   #define  RCC_CR_HSERDY                       ((uint32_t)0x00020000)        /*!< External High Speed
       clock ready flag */
1687   #define  RCC_CR_HSEBYP                       ((uint32_t)0x00040000)        /*!< External High Speed
       clock Bypass */
1688   #define  RCC_CR_CSSON                        ((uint32_t)0x00080000)        /*!< Clock Security System
       enable */
1689   #define  RCC_CR_PLLON                        ((uint32_t)0x01000000)        /*!< PLL enable */
1690   #define  RCC_CR_PLLRDY                       ((uint32_t)0x02000000)        /*!< PLL clock ready flag */
1691
1692   #ifdef STM32F10X_CL
1693    #define  RCC_CR_PLL2ON                       ((uint32_t)0x04000000)        /*!< PLL2 enable */
1694    #define  RCC_CR_PLL2RDY                      ((uint32_t)0x08000000)        /*!< PLL2 clock ready flag */
1695    #define  RCC_CR_PLL3ON                       ((uint32_t)0x10000000)        /*!< PLL3 enable */
1696    #define  RCC_CR_PLL3RDY                      ((uint32_t)0x20000000)        /*!< PLL3 clock ready flag */
1697   #endif /* STM32F10X_CL */
1698
1699   /*******************  Bit definition for RCC_CFGR register  *******************/
1700   /*!< SW configuration */
1701   #define  RCC_CFGR_SW                         ((uint32_t)0x00000003)        /*!< SW[1:0] bits (System
       clock Switch) */
1702   #define  RCC_CFGR_SW_0                       ((uint32_t)0x00000001)        /*!< Bit 0 */
1703   #define  RCC_CFGR_SW_1                       ((uint32_t)0x00000002)        /*!< Bit 1 */
1704
1705   #define  RCC_CFGR_SW_HSI                     ((uint32_t)0x00000000)        /*!< HSI selected as system
       clock */
1706   #define  RCC_CFGR_SW_HSE                     ((uint32_t)0x00000001)        /*!< HSE selected as system
       clock */
1707   #define  RCC_CFGR_SW_PLL                     ((uint32_t)0x00000002)        /*!< PLL selected as system
       clock */
1708
1709   /*!< SWS configuration */
1710   #define  RCC_CFGR_SWS                        ((uint32_t)0x0000000C)        /*!< SWS[1:0] bits (System
       Clock Switch Status) */
1711   #define  RCC_CFGR_SWS_0                      ((uint32_t)0x00000004)        /*!< Bit 0 */
1712   #define  RCC_CFGR_SWS_1                      ((uint32_t)0x00000008)        /*!< Bit 1 */
1713
1714   #define  RCC_CFGR_SWS_HSI                    ((uint32_t)0x00000000)        /*!< HSI oscillator used as
       system clock */
1715   #define  RCC_CFGR_SWS_HSE                    ((uint32_t)0x00000004)        /*!< HSE oscillator used as
       system clock */
1716   #define  RCC_CFGR_SWS_PLL                    ((uint32_t)0x00000008)        /*!< PLL used as system
       clock */
```

```
1717
1718   /*!< HPRE configuration */
1719   #define  RCC_CFGR_HPRE                       ((uint32_t)0x000000F0)        /*!< HPRE[3:0] bits (AHB
       prescaler) */
1720   #define  RCC_CFGR_HPRE_0                     ((uint32_t)0x00000010)        /*!< Bit 0 */
1721   #define  RCC_CFGR_HPRE_1                     ((uint32_t)0x00000020)        /*!< Bit 1 */
1722   #define  RCC_CFGR_HPRE_2                     ((uint32_t)0x00000040)        /*!< Bit 2 */
1723   #define  RCC_CFGR_HPRE_3                     ((uint32_t)0x00000080)        /*!< Bit 3 */
1724
1725   #define  RCC_CFGR_HPRE_DIV1                  ((uint32_t)0x00000000)        /*!< SYSCLK not divided */
1726   #define  RCC_CFGR_HPRE_DIV2                  ((uint32_t)0x00000080)        /*!< SYSCLK divided by 2 */
1727   #define  RCC_CFGR_HPRE_DIV4                  ((uint32_t)0x00000090)        /*!< SYSCLK divided by 4 */
1728   #define  RCC_CFGR_HPRE_DIV8                  ((uint32_t)0x000000A0)        /*!< SYSCLK divided by 8 */
1729   #define  RCC_CFGR_HPRE_DIV16                 ((uint32_t)0x000000B0)        /*!< SYSCLK divided by 16 */
1730   #define  RCC_CFGR_HPRE_DIV64                 ((uint32_t)0x000000C0)        /*!< SYSCLK divided by 64 */
1731   #define  RCC_CFGR_HPRE_DIV128                ((uint32_t)0x000000D0)        /*!< SYSCLK divided by 128 */
1732   #define  RCC_CFGR_HPRE_DIV256                ((uint32_t)0x000000E0)        /*!< SYSCLK divided by 256 */
1733   #define  RCC_CFGR_HPRE_DIV512                ((uint32_t)0x000000F0)        /*!< SYSCLK divided by 512 */
1734
1735   /*!< PPRE1 configuration */
1736   #define  RCC_CFGR_PPRE1                      ((uint32_t)0x00000700)        /*!< PRE1[2:0] bits (APB1
       prescaler) */
1737   #define  RCC_CFGR_PPRE1_0                    ((uint32_t)0x00000100)        /*!< Bit 0 */
1738   #define  RCC_CFGR_PPRE1_1                    ((uint32_t)0x00000200)        /*!< Bit 1 */
1739   #define  RCC_CFGR_PPRE1_2                    ((uint32_t)0x00000400)        /*!< Bit 2 */
1740
1741   #define  RCC_CFGR_PPRE1_DIV1                 ((uint32_t)0x00000000)        /*!< HCLK not divided */
1742   #define  RCC_CFGR_PPRE1_DIV2                 ((uint32_t)0x00000400)        /*!< HCLK divided by 2 */
1743   #define  RCC_CFGR_PPRE1_DIV4                 ((uint32_t)0x00000500)        /*!< HCLK divided by 4 */
1744   #define  RCC_CFGR_PPRE1_DIV8                 ((uint32_t)0x00000600)        /*!< HCLK divided by 8 */
1745   #define  RCC_CFGR_PPRE1_DIV16                ((uint32_t)0x00000700)        /*!< HCLK divided by 16 */
1746
1747   /*!< PPRE2 configuration */
1748   #define  RCC_CFGR_PPRE2                      ((uint32_t)0x00003800)        /*!< PRE2[2:0] bits (APB2
       prescaler) */
1749   #define  RCC_CFGR_PPRE2_0                    ((uint32_t)0x00000800)        /*!< Bit 0 */
1750   #define  RCC_CFGR_PPRE2_1                    ((uint32_t)0x00001000)        /*!< Bit 1 */
1751   #define  RCC_CFGR_PPRE2_2                    ((uint32_t)0x00002000)        /*!< Bit 2 */
1752
1753   #define  RCC_CFGR_PPRE2_DIV1                 ((uint32_t)0x00000000)        /*!< HCLK not divided */
1754   #define  RCC_CFGR_PPRE2_DIV2                 ((uint32_t)0x00002000)        /*!< HCLK divided by 2 */
1755   #define  RCC_CFGR_PPRE2_DIV4                 ((uint32_t)0x00002800)        /*!< HCLK divided by 4 */
1756   #define  RCC_CFGR_PPRE2_DIV8                 ((uint32_t)0x00003000)        /*!< HCLK divided by 8 */
1757   #define  RCC_CFGR_PPRE2_DIV16                ((uint32_t)0x00003800)        /*!< HCLK divided by 16 */
1758
1759   /*!< ADCPPRE configuration */
1760   #define  RCC_CFGR_ADCPRE                     ((uint32_t)0x0000C000)        /*!< ADCPRE[1:0] bits (ADC
       prescaler) */
1761   #define  RCC_CFGR_ADCPRE_0                   ((uint32_t)0x00004000)        /*!< Bit 0 */
1762   #define  RCC_CFGR_ADCPRE_1                   ((uint32_t)0x00008000)        /*!< Bit 1 */
1763
1764   #define  RCC_CFGR_ADCPRE_DIV2                ((uint32_t)0x00000000)        /*!< PCLK2 divided by 2 */
1765   #define  RCC_CFGR_ADCPRE_DIV4                ((uint32_t)0x00004000)        /*!< PCLK2 divided by 4 */
1766   #define  RCC_CFGR_ADCPRE_DIV6                ((uint32_t)0x00008000)        /*!< PCLK2 divided by 6 */
1767   #define  RCC_CFGR_ADCPRE_DIV8                ((uint32_t)0x0000C000)        /*!< PCLK2 divided by 8 */
1768
1769   #define  RCC_CFGR_PLLSRC                     ((uint32_t)0x00010000)        /*!< PLL entry clock source */
1770
1771   #define  RCC_CFGR_PLLXTPRE                   ((uint32_t)0x00020000)        /*!< HSE divider for PLL
       entry */
1772
1773   /*!< PLLMUL configuration */
1774   #define  RCC_CFGR_PLLMULL                    ((uint32_t)0x003C0000)        /*!< PLLMUL[3:0] bits (PLL
       multiplication factor) */
1775   #define  RCC_CFGR_PLLMULL_0                  ((uint32_t)0x00040000)        /*!< Bit 0 */
1776   #define  RCC_CFGR_PLLMULL_1                  ((uint32_t)0x00080000)        /*!< Bit 1 */
1777   #define  RCC_CFGR_PLLMULL_2                  ((uint32_t)0x00100000)        /*!< Bit 2 */
1778   #define  RCC_CFGR_PLLMULL_3                  ((uint32_t)0x00200000)        /*!< Bit 3 */
1779
1780   #ifdef STM32F10X_CL
1781    #define  RCC_CFGR_PLLSRC_HSI_Div2            ((uint32_t)0x00000000)        /*!< HSI clock divided by 2
       selected as PLL entry clock source */
1782    #define  RCC_CFGR_PLLSRC_PREDIV1             ((uint32_t)0x00010000)        /*!< PREDIV1 clock selected
```

```
      as PLL entry clock source */

  #define  RCC_CFGR_PLLXTPRE_PREDIV1          ((uint32_t)0x00000000)        /*!< PREDIV1 clock not divided for PLL entry */
  #define  RCC_CFGR_PLLXTPRE_PREDIV1_Div2     ((uint32_t)0x00020000)        /*!< PREDIV1 clock divided by 2 for PLL entry */

  #define  RCC_CFGR_PLLMULL4                  ((uint32_t)0x00080000)        /*!< PLL input clock * 4 */
  #define  RCC_CFGR_PLLMULL5                  ((uint32_t)0x000C0000)        /*!< PLL input clock * 5 */
  #define  RCC_CFGR_PLLMULL6                  ((uint32_t)0x00100000)        /*!< PLL input clock * 6 */
  #define  RCC_CFGR_PLLMULL7                  ((uint32_t)0x00140000)        /*!< PLL input clock * 7 */
  #define  RCC_CFGR_PLLMULL8                  ((uint32_t)0x00180000)        /*!< PLL input clock * 8 */
  #define  RCC_CFGR_PLLMULL9                  ((uint32_t)0x001C0000)        /*!< PLL input clock * 9 */
  #define  RCC_CFGR_PLLMULL6_5                ((uint32_t)0x00340000)        /*!< PLL input clock * 6.5 */

  #define  RCC_CFGR_OTGFSPRE                  ((uint32_t)0x00400000)        /*!< USB OTG FS prescaler */

/*!< MCO configuration */
  #define  RCC_CFGR_MCO                       ((uint32_t)0x0F000000)        /*!< MCO[3:0] bits (Microcontroller Clock Output) */
  #define  RCC_CFGR_MCO_0                     ((uint32_t)0x01000000)        /*!< Bit 0 */
  #define  RCC_CFGR_MCO_1                     ((uint32_t)0x02000000)        /*!< Bit 1 */
  #define  RCC_CFGR_MCO_2                     ((uint32_t)0x04000000)        /*!< Bit 2 */
  #define  RCC_CFGR_MCO_3                     ((uint32_t)0x08000000)        /*!< Bit 3 */

  #define  RCC_CFGR_MCO_NOCLOCK               ((uint32_t)0x00000000)        /*!< No clock */
  #define  RCC_CFGR_MCO_SYSCLK                ((uint32_t)0x04000000)        /*!< System clock selected as MCO source */
  #define  RCC_CFGR_MCO_HSI                   ((uint32_t)0x05000000)        /*!< HSI clock selected as MCO source */
  #define  RCC_CFGR_MCO_HSE                   ((uint32_t)0x06000000)        /*!< HSE clock selected as MCO source */
  #define  RCC_CFGR_MCO_PLLCLK_Div2           ((uint32_t)0x07000000)        /*!< PLL clock divided by 2 selected as MCO source */
  #define  RCC_CFGR_MCO_PLL2CLK               ((uint32_t)0x08000000)        /*!< PLL2 clock selected as MCO source*/
  #define  RCC_CFGR_MCO_PLL3CLK_Div2          ((uint32_t)0x09000000)        /*!< PLL3 clock divided by 2 selected as MCO source*/
  #define  RCC_CFGR_MCO_Ext_HSE               ((uint32_t)0x0A000000)        /*!< XT1 external 3-25 MHz oscillator clock selected as MCO source */
  #define  RCC_CFGR_MCO_PLL3CLK               ((uint32_t)0x0B000000)        /*!< PLL3 clock selected as MCO source */
#elif defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
  #define  RCC_CFGR_PLLSRC_HSI_Div2           ((uint32_t)0x00000000)        /*!< HSI clock divided by 2 selected as PLL entry clock source */
  #define  RCC_CFGR_PLLSRC_PREDIV1            ((uint32_t)0x00010000)        /*!< PREDIV1 clock selected as PLL entry clock source */

  #define  RCC_CFGR_PLLXTPRE_PREDIV1          ((uint32_t)0x00000000)        /*!< PREDIV1 clock not divided for PLL entry */
  #define  RCC_CFGR_PLLXTPRE_PREDIV1_Div2     ((uint32_t)0x00020000)        /*!< PREDIV1 clock divided by 2 for PLL entry */

  #define  RCC_CFGR_PLLMULL2                  ((uint32_t)0x00000000)        /*!< PLL input clock*2 */
  #define  RCC_CFGR_PLLMULL3                  ((uint32_t)0x00040000)        /*!< PLL input clock*3 */
  #define  RCC_CFGR_PLLMULL4                  ((uint32_t)0x00080000)        /*!< PLL input clock*4 */
  #define  RCC_CFGR_PLLMULL5                  ((uint32_t)0x000C0000)        /*!< PLL input clock*5 */
  #define  RCC_CFGR_PLLMULL6                  ((uint32_t)0x00100000)        /*!< PLL input clock*6 */
  #define  RCC_CFGR_PLLMULL7                  ((uint32_t)0x00140000)        /*!< PLL input clock*7 */
  #define  RCC_CFGR_PLLMULL8                  ((uint32_t)0x00180000)        /*!< PLL input clock*8 */
  #define  RCC_CFGR_PLLMULL9                  ((uint32_t)0x001C0000)        /*!< PLL input clock*9 */
  #define  RCC_CFGR_PLLMULL10                 ((uint32_t)0x00200000)        /*!< PLL input clock10 */
  #define  RCC_CFGR_PLLMULL11                 ((uint32_t)0x00240000)        /*!< PLL input clock*11 */
  #define  RCC_CFGR_PLLMULL12                 ((uint32_t)0x00280000)        /*!< PLL input clock*12 */
  #define  RCC_CFGR_PLLMULL13                 ((uint32_t)0x002C0000)        /*!< PLL input clock*13 */
  #define  RCC_CFGR_PLLMULL14                 ((uint32_t)0x00300000)        /*!< PLL input clock*14 */
  #define  RCC_CFGR_PLLMULL15                 ((uint32_t)0x00340000)        /*!< PLL input clock*15 */
  #define  RCC_CFGR_PLLMULL16                 ((uint32_t)0x00380000)        /*!< PLL input clock*16 */

/*!< MCO configuration */
  #define  RCC_CFGR_MCO                       ((uint32_t)0x07000000)        /*!< MCO[2:0] bits (Microcontroller Clock Output) */
  #define  RCC_CFGR_MCO_0                     ((uint32_t)0x01000000)        /*!< Bit 0 */
```

```
 #define  RCC_CFGR_MCO_1                      ((uint32_t)0x02000000)        /*!< Bit 1 */
 #define  RCC_CFGR_MCO_2                      ((uint32_t)0x04000000)        /*!< Bit 2 */

 #define  RCC_CFGR_MCO_NOCLOCK                ((uint32_t)0x00000000)        /*!< No clock */
 #define  RCC_CFGR_MCO_SYSCLK                 ((uint32_t)0x04000000)        /*!< System clock selected as MCO source */
 #define  RCC_CFGR_MCO_HSI                    ((uint32_t)0x05000000)        /*!< HSI clock selected as MCO source */
 #define  RCC_CFGR_MCO_HSE                    ((uint32_t)0x06000000)        /*!< HSE clock selected as MCO source  */
 #define  RCC_CFGR_MCO_PLL                    ((uint32_t)0x07000000)        /*!< PLL clock divided by 2 selected as MCO source */
#else
 #define  RCC_CFGR_PLLSRC_HSI_Div2            ((uint32_t)0x00000000)        /*!< HSI clock divided by 2 selected as PLL entry clock source */
 #define  RCC_CFGR_PLLSRC_HSE                 ((uint32_t)0x00010000)        /*!< HSE clock selected as PLL entry clock source */

 #define  RCC_CFGR_PLLXTPRE_HSE               ((uint32_t)0x00000000)        /*!< HSE clock not divided for PLL entry */
 #define  RCC_CFGR_PLLXTPRE_HSE_Div2          ((uint32_t)0x00020000)        /*!< HSE clock divided by 2 for PLL entry */

 #define  RCC_CFGR_PLLMULL2                   ((uint32_t)0x00000000)        /*!< PLL input clock*2 */
 #define  RCC_CFGR_PLLMULL3                   ((uint32_t)0x00040000)        /*!< PLL input clock*3 */
 #define  RCC_CFGR_PLLMULL4                   ((uint32_t)0x00080000)        /*!< PLL input clock*4 */
 #define  RCC_CFGR_PLLMULL5                   ((uint32_t)0x000C0000)        /*!< PLL input clock*5 */
 #define  RCC_CFGR_PLLMULL6                   ((uint32_t)0x00100000)        /*!< PLL input clock*6 */
 #define  RCC_CFGR_PLLMULL7                   ((uint32_t)0x00140000)        /*!< PLL input clock*7 */
 #define  RCC_CFGR_PLLMULL8                   ((uint32_t)0x00180000)        /*!< PLL input clock*8 */
 #define  RCC_CFGR_PLLMULL9                   ((uint32_t)0x001C0000)        /*!< PLL input clock*9 */
 #define  RCC_CFGR_PLLMULL10                  ((uint32_t)0x00200000)        /*!< PLL input clock10 */
 #define  RCC_CFGR_PLLMULL11                  ((uint32_t)0x00240000)        /*!< PLL input clock*11 */
 #define  RCC_CFGR_PLLMULL12                  ((uint32_t)0x00280000)        /*!< PLL input clock*12 */
 #define  RCC_CFGR_PLLMULL13                  ((uint32_t)0x002C0000)        /*!< PLL input clock*13 */
 #define  RCC_CFGR_PLLMULL14                  ((uint32_t)0x00300000)        /*!< PLL input clock*14 */
 #define  RCC_CFGR_PLLMULL15                  ((uint32_t)0x00340000)        /*!< PLL input clock*15 */
 #define  RCC_CFGR_PLLMULL16                  ((uint32_t)0x00380000)        /*!< PLL input clock*16 */
 #define  RCC_CFGR_USBPRE                     ((uint32_t)0x00400000)        /*!< USB Device prescaler */

/*!< MCO configuration */
 #define  RCC_CFGR_MCO                        ((uint32_t)0x07000000)        /*!< MCO[2:0] bits (Microcontroller Clock Output) */
 #define  RCC_CFGR_MCO_0                      ((uint32_t)0x01000000)        /*!< Bit 0 */
 #define  RCC_CFGR_MCO_1                      ((uint32_t)0x02000000)        /*!< Bit 1 */
 #define  RCC_CFGR_MCO_2                      ((uint32_t)0x04000000)        /*!< Bit 2 */

 #define  RCC_CFGR_MCO_NOCLOCK                ((uint32_t)0x00000000)        /*!< No clock */
 #define  RCC_CFGR_MCO_SYSCLK                 ((uint32_t)0x04000000)        /*!< System clock selected as MCO source */
 #define  RCC_CFGR_MCO_HSI                    ((uint32_t)0x05000000)        /*!< HSI clock selected as MCO source */
 #define  RCC_CFGR_MCO_HSE                    ((uint32_t)0x06000000)        /*!< HSE clock selected as MCO source  */
 #define  RCC_CFGR_MCO_PLL                    ((uint32_t)0x07000000)        /*!< PLL clock divided by 2 selected as MCO source */
#endif /* STM32F10X_CL */

/*!<******************  Bit definition for RCC_CIR register  ********************/
#define  RCC_CIR_LSIRDYF                     ((uint32_t)0x00000001)        /*!< LSI Ready Interrupt flag */
#define  RCC_CIR_LSERDYF                     ((uint32_t)0x00000002)        /*!< LSE Ready Interrupt flag */
#define  RCC_CIR_HSIRDYF                     ((uint32_t)0x00000004)        /*!< HSI Ready Interrupt flag */
#define  RCC_CIR_HSERDYF                     ((uint32_t)0x00000008)        /*!< HSE Ready Interrupt flag */
#define  RCC_CIR_PLLRDYF                     ((uint32_t)0x00000010)        /*!< PLL Ready Interrupt flag */
#define  RCC_CIR_CSSF                        ((uint32_t)0x00000080)        /*!< Clock Security System Interrupt flag */
#define  RCC_CIR_LSIRDYIE                    ((uint32_t)0x00000100)        /*!< LSI Ready Interrupt Enable */
```

```
#define  RCC_CIR_LSERDYIE                    ((uint32_t)0x00000200)        /*!< LSE Ready Interrupt Enable */
#define  RCC_CIR_HSIRDYIE                    ((uint32_t)0x00000400)        /*!< HSI Ready Interrupt Enable */
#define  RCC_CIR_HSERDYIE                    ((uint32_t)0x00000800)        /*!< HSE Ready Interrupt Enable */
#define  RCC_CIR_PLLRDYIE                    ((uint32_t)0x00001000)        /*!< PLL Ready Interrupt Enable */
#define  RCC_CIR_LSIRDYC                     ((uint32_t)0x00010000)        /*!< LSI Ready Interrupt Clear */
#define  RCC_CIR_LSERDYC                     ((uint32_t)0x00020000)        /*!< LSE Ready Interrupt Clear */
#define  RCC_CIR_HSIRDYC                     ((uint32_t)0x00040000)        /*!< HSI Ready Interrupt Clear */
#define  RCC_CIR_HSERDYC                     ((uint32_t)0x00080000)        /*!< HSE Ready Interrupt Clear */
#define  RCC_CIR_PLLRDYC                     ((uint32_t)0x00100000)        /*!< PLL Ready Interrupt Clear */
#define  RCC_CIR_CSSC                        ((uint32_t)0x00800000)        /*!< Clock Security System Interrupt Clear */

#ifdef STM32F10X_CL
 #define  RCC_CIR_PLL2RDYF                    ((uint32_t)0x00000020)        /*!< PLL2 Ready Interrupt flag */
 #define  RCC_CIR_PLL3RDYF                    ((uint32_t)0x00000040)        /*!< PLL3 Ready Interrupt flag */
 #define  RCC_CIR_PLL2RDYIE                   ((uint32_t)0x00002000)        /*!< PLL2 Ready Interrupt Enable */
 #define  RCC_CIR_PLL3RDYIE                   ((uint32_t)0x00004000)        /*!< PLL3 Ready Interrupt Enable */
 #define  RCC_CIR_PLL2RDYC                    ((uint32_t)0x00200000)        /*!< PLL2 Ready Interrupt Clear */
 #define  RCC_CIR_PLL3RDYC                    ((uint32_t)0x00400000)        /*!< PLL3 Ready Interrupt Clear */
#endif /* STM32F10X_CL */

/*****************  Bit definition for RCC_APB2RSTR register  *****************/
#define  RCC_APB2RSTR_AFIORST                ((uint32_t)0x00000001)        /*!< Alternate Function I/O reset */
#define  RCC_APB2RSTR_IOPARST                ((uint32_t)0x00000004)        /*!< I/O port A reset */
#define  RCC_APB2RSTR_IOPBRST                ((uint32_t)0x00000008)        /*!< I/O port B reset */
#define  RCC_APB2RSTR_IOPCRST                ((uint32_t)0x00000010)        /*!< I/O port C reset */
#define  RCC_APB2RSTR_IOPDRST                ((uint32_t)0x00000020)        /*!< I/O port D reset */
#define  RCC_APB2RSTR_ADC1RST                ((uint32_t)0x00000200)        /*!< ADC 1 interface reset */

#if !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD_VL)
#define  RCC_APB2RSTR_ADC2RST                ((uint32_t)0x00000400)        /*!< ADC 2 interface reset */
#endif

#define  RCC_APB2RSTR_TIM1RST                ((uint32_t)0x00000800)        /*!< TIM1 Timer reset */
#define  RCC_APB2RSTR_SPI1RST                ((uint32_t)0x00001000)        /*!< SPI 1 reset */
#define  RCC_APB2RSTR_USART1RST              ((uint32_t)0x00004000)        /*!< USART1 reset */

#if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
#define  RCC_APB2RSTR_TIM15RST               ((uint32_t)0x00010000)        /*!< TIM15 Timer reset */
#define  RCC_APB2RSTR_TIM16RST               ((uint32_t)0x00020000)        /*!< TIM16 Timer reset */
#define  RCC_APB2RSTR_TIM17RST               ((uint32_t)0x00040000)        /*!< TIM17 Timer reset */
#endif

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL)
 #define  RCC_APB2RSTR_IOPERST                ((uint32_t)0x00000040)        /*!< I/O port E reset */
#endif /* STM32F10X_LD && STM32F10X_LD_VL */

#if defined (STM32F10X_HD) || defined (STM32F10X_XL)
 #define  RCC_APB2RSTR_IOPFRST                ((uint32_t)0x00000080)        /*!< I/O port F reset */
 #define  RCC_APB2RSTR_IOPGRST                ((uint32_t)0x00000100)        /*!< I/O port G reset */
 #define  RCC_APB2RSTR_TIM8RST                ((uint32_t)0x00002000)        /*!< TIM8 Timer reset */
 #define  RCC_APB2RSTR_ADC3RST                ((uint32_t)0x00008000)        /*!< ADC3 interface reset */
#endif

#if defined (STM32F10X_HD_VL)
 #define  RCC_APB2RSTR_IOPFRST                ((uint32_t)0x00000080)        /*!< I/O port F reset */
 #define  RCC_APB2RSTR_IOPGRST                ((uint32_t)0x00000100)        /*!< I/O port G reset */
```

```
#endif

#ifdef STM32F10X_XL
 #define  RCC_APB2RSTR_TIM9RST               ((uint32_t)0x00080000)         /*!< TIM9 Timer reset */
 #define  RCC_APB2RSTR_TIM10RST              ((uint32_t)0x00100000)         /*!< TIM10 Timer reset */
 #define  RCC_APB2RSTR_TIM11RST              ((uint32_t)0x00200000)         /*!< TIM11 Timer reset */
#endif /* STM32F10X_XL */

/*****************  Bit definition for RCC_APB1RSTR register  *****************/
#define  RCC_APB1RSTR_TIM2RST                ((uint32_t)0x00000001)        /*!< Timer 2 reset */
#define  RCC_APB1RSTR_TIM3RST                ((uint32_t)0x00000002)        /*!< Timer 3 reset */
#define  RCC_APB1RSTR_WWDGRST                ((uint32_t)0x00000800)        /*!< Window Watchdog reset */
#define  RCC_APB1RSTR_USART2RST              ((uint32_t)0x00020000)        /*!< USART 2 reset */
#define  RCC_APB1RSTR_I2C1RST                ((uint32_t)0x00200000)        /*!< I2C 1 reset */

#if !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD_VL)
#define  RCC_APB1RSTR_CAN1RST                ((uint32_t)0x02000000)        /*!< CAN1 reset */
#endif

#define  RCC_APB1RSTR_BKPRST                 ((uint32_t)0x08000000)        /*!< Backup interface reset */
#define  RCC_APB1RSTR_PWRRST                 ((uint32_t)0x10000000)        /*!< Power interface reset */

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL)
 #define  RCC_APB1RSTR_TIM4RST               ((uint32_t)0x00000004)        /*!< Timer 4 reset */
 #define  RCC_APB1RSTR_SPI2RST               ((uint32_t)0x00004000)        /*!< SPI 2 reset */
 #define  RCC_APB1RSTR_USART3RST             ((uint32_t)0x00040000)        /*!< USART 3 reset */
 #define  RCC_APB1RSTR_I2C2RST               ((uint32_t)0x00400000)        /*!< I2C 2 reset */
#endif /* STM32F10X_LD && STM32F10X_LD_VL */

#if defined (STM32F10X_HD) || defined (STM32F10X_MD) || defined (STM32F10X_LD) || defined  (STM32F10X_XL)
 #define  RCC_APB1RSTR_USBRST                ((uint32_t)0x00800000)        /*!< USB Device reset */
#endif

#if defined (STM32F10X_HD) || defined  (STM32F10X_CL) || defined  (STM32F10X_XL)
 #define  RCC_APB1RSTR_TIM5RST                ((uint32_t)0x00000008)        /*!< Timer 5 reset */
 #define  RCC_APB1RSTR_TIM6RST                ((uint32_t)0x00000010)        /*!< Timer 6 reset */
 #define  RCC_APB1RSTR_TIM7RST                ((uint32_t)0x00000020)        /*!< Timer 7 reset */
 #define  RCC_APB1RSTR_SPI3RST                ((uint32_t)0x00008000)        /*!< SPI 3 reset */
 #define  RCC_APB1RSTR_UART4RST               ((uint32_t)0x00080000)        /*!< UART 4 reset */
 #define  RCC_APB1RSTR_UART5RST               ((uint32_t)0x00100000)        /*!< UART 5 reset */
 #define  RCC_APB1RSTR_DACRST                 ((uint32_t)0x20000000)        /*!< DAC interface reset */
#endif

#if defined (STM32F10X_LD_VL) || defined  (STM32F10X_MD_VL) || defined  (STM32F10X_HD_VL)
 #define  RCC_APB1RSTR_TIM6RST                ((uint32_t)0x00000010)        /*!< Timer 6 reset */
 #define  RCC_APB1RSTR_TIM7RST                ((uint32_t)0x00000020)        /*!< Timer 7 reset */
 #define  RCC_APB1RSTR_DACRST                 ((uint32_t)0x20000000)        /*!< DAC interface reset */
 #define  RCC_APB1RSTR_CECRST                 ((uint32_t)0x40000000)        /*!< CEC interface reset */
#endif

#if defined  (STM32F10X_HD_VL)
 #define  RCC_APB1RSTR_TIM5RST                ((uint32_t)0x00000008)        /*!< Timer 5 reset */
 #define  RCC_APB1RSTR_TIM12RST               ((uint32_t)0x00000040)        /*!< TIM12 Timer reset */
 #define  RCC_APB1RSTR_TIM13RST               ((uint32_t)0x00000080)        /*!< TIM13 Timer reset */
 #define  RCC_APB1RSTR_TIM14RST               ((uint32_t)0x00000100)        /*!< TIM14 Timer reset */
 #define  RCC_APB1RSTR_SPI3RST                ((uint32_t)0x00008000)        /*!< SPI 3 reset */
 #define  RCC_APB1RSTR_UART4RST               ((uint32_t)0x00080000)        /*!< UART 4 reset */
 #define  RCC_APB1RSTR_UART5RST               ((uint32_t)0x00100000)        /*!< UART 5 reset */
#endif

#ifdef STM32F10X_CL
 #define  RCC_APB1RSTR_CAN2RST                ((uint32_t)0x04000000)        /*!< CAN2 reset */
#endif /* STM32F10X_CL */

#ifdef STM32F10X_XL
 #define  RCC_APB1RSTR_TIM12RST               ((uint32_t)0x00000040)         /*!< TIM12 Timer reset */
 #define  RCC_APB1RSTR_TIM13RST               ((uint32_t)0x00000080)         /*!< TIM13 Timer reset */
 #define  RCC_APB1RSTR_TIM14RST               ((uint32_t)0x00000100)         /*!< TIM14 Timer reset */
#endif /* STM32F10X_XL */

/******************  Bit definition for RCC_AHBENR register  ******************/
#define  RCC_AHBENR_DMA1EN                   ((uint16_t)0x0001)            /*!< DMA1 clock enable */
#define  RCC_AHBENR_SRAMEN                   ((uint16_t)0x0004)            /*!< SRAM interface clock
```

```
enable */
#define  RCC_AHBENR_FLITFEN                  ((uint16_t)0x0010)            /*!< FLITF clock enable */
#define  RCC_AHBENR_CRCEN                    ((uint16_t)0x0040)            /*!< CRC clock enable */

#if defined (STM32F10X_HD) || defined  (STM32F10X_CL) || defined  (STM32F10X_HD_VL)
 #define  RCC_AHBENR_DMA2EN                  ((uint16_t)0x0002)            /*!< DMA2 clock enable */
#endif

#if defined (STM32F10X_HD) || defined (STM32F10X_XL)
 #define  RCC_AHBENR_FSMCEN                  ((uint16_t)0x0100)            /*!< FSMC clock enable */
 #define  RCC_AHBENR_SDIOEN                  ((uint16_t)0x0400)            /*!< SDIO clock enable */
#endif

#if defined (STM32F10X_HD_VL)
 #define  RCC_AHBENR_FSMCEN                  ((uint16_t)0x0100)            /*!< FSMC clock enable */
#endif

#ifdef STM32F10X_CL
 #define  RCC_AHBENR_OTGFSEN                  ((uint32_t)0x00001000)         /*!< USB OTG FS clock
enable */
 #define  RCC_AHBENR_ETHMACEN                 ((uint32_t)0x00004000)         /*!< ETHERNET MAC clock
enable */
 #define  RCC_AHBENR_ETHMACTXEN               ((uint32_t)0x00008000)         /*!< ETHERNET MAC Tx clock
enable */
 #define  RCC_AHBENR_ETHMACRXEN               ((uint32_t)0x00010000)         /*!< ETHERNET MAC Rx clock
enable */
#endif /* STM32F10X_CL */

/******************  Bit definition for RCC_APB2ENR register  *****************/
#define  RCC_APB2ENR_AFIOEN                  ((uint32_t)0x00000001)         /*!< Alternate Function I/O
clock enable */
#define  RCC_APB2ENR_IOPAEN                  ((uint32_t)0x00000004)         /*!< I/O port A clock
enable */
#define  RCC_APB2ENR_IOPBEN                  ((uint32_t)0x00000008)         /*!< I/O port B clock
enable */
#define  RCC_APB2ENR_IOPCEN                  ((uint32_t)0x00000010)         /*!< I/O port C clock
enable */
#define  RCC_APB2ENR_IOPDEN                  ((uint32_t)0x00000020)         /*!< I/O port D clock
enable */
#define  RCC_APB2ENR_ADC1EN                  ((uint32_t)0x00000200)         /*!< ADC 1 interface clock
enable */

#if !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD_VL)
#define  RCC_APB2ENR_ADC2EN                  ((uint32_t)0x00000400)         /*!< ADC 2 interface clock
enable */
#endif

#define  RCC_APB2ENR_TIM1EN                  ((uint32_t)0x00000800)         /*!< TIM1 Timer clock
enable */
#define  RCC_APB2ENR_SPI1EN                  ((uint32_t)0x00001000)         /*!< SPI 1 clock enable */
#define  RCC_APB2ENR_USART1EN                ((uint32_t)0x00004000)         /*!< USART1 clock enable */

#if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
#define  RCC_APB2ENR_TIM15EN                 ((uint32_t)0x00010000)         /*!< TIM15 Timer clock
enable */
#define  RCC_APB2ENR_TIM16EN                 ((uint32_t)0x00020000)         /*!< TIM16 Timer clock
enable */
#define  RCC_APB2ENR_TIM17EN                 ((uint32_t)0x00040000)         /*!< TIM17 Timer clock
enable */
#endif

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL)
 #define  RCC_APB2ENR_IOPEEN                  ((uint32_t)0x00000040)         /*!< I/O port E clock
enable */
#endif /* STM32F10X_LD && STM32F10X_LD_VL */

#if defined (STM32F10X_HD) || defined (STM32F10X_XL)
 #define  RCC_APB2ENR_IOPFEN                  ((uint32_t)0x00000080)         /*!< I/O port F clock
enable */
 #define  RCC_APB2ENR_IOPGEN                  ((uint32_t)0x00000100)         /*!< I/O port G clock
enable */
 #define  RCC_APB2ENR_TIM8EN                  ((uint32_t)0x00002000)         /*!< TIM8 Timer clock
enable */
```

```
 #define  RCC_APB2ENR_ADC3EN                 ((uint32_t)0x00008000)         /*!< DMA1 clock enable */
#endif

#if defined (STM32F10X_HD_VL)
 #define  RCC_APB2ENR_IOPFEN                 ((uint32_t)0x00000080)         /*!< I/O port F clock enable */
 #define  RCC_APB2ENR_IOPGEN                 ((uint32_t)0x00000100)         /*!< I/O port G clock enable */
#endif

#ifdef STM32F10X_XL
 #define  RCC_APB2ENR_TIM9EN                 ((uint32_t)0x00080000)         /*!< TIM9 Timer clock enable  */
 #define  RCC_APB2ENR_TIM10EN                ((uint32_t)0x00100000)         /*!< TIM10 Timer clock enable  */
 #define  RCC_APB2ENR_TIM11EN                ((uint32_t)0x00200000)         /*!< TIM11 Timer clock enable */
#endif

/*****************  Bit definition for RCC_APB1ENR register  ******************/
#define  RCC_APB1ENR_TIM2EN                  ((uint32_t)0x00000001)        /*!< Timer 2 clock enabled*/
#define  RCC_APB1ENR_TIM3EN                  ((uint32_t)0x00000002)        /*!< Timer 3 clock enable */
#define  RCC_APB1ENR_WWDGEN                  ((uint32_t)0x00000800)        /*!< Window Watchdog clock enable */
#define  RCC_APB1ENR_USART2EN                ((uint32_t)0x00020000)        /*!< USART 2 clock enable */
#define  RCC_APB1ENR_I2C1EN                  ((uint32_t)0x00200000)        /*!< I2C 1 clock enable */

#if !defined (STM32F10X_LD_VL) && !defined (STM32F10X_MD_VL) && !defined (STM32F10X_HD_VL)
#define  RCC_APB1ENR_CAN1EN                  ((uint32_t)0x02000000)        /*!< CAN1 clock enable */
#endif

#define  RCC_APB1ENR_BKPEN                   ((uint32_t)0x08000000)        /*!< Backup interface clock enable */
#define  RCC_APB1ENR_PWREN                   ((uint32_t)0x10000000)        /*!< Power interface clock enable */

#if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL)
 #define  RCC_APB1ENR_TIM4EN                 ((uint32_t)0x00000004)        /*!< Timer 4 clock enable */
 #define  RCC_APB1ENR_SPI2EN                 ((uint32_t)0x00004000)        /*!< SPI 2 clock enable */
 #define  RCC_APB1ENR_USART3EN               ((uint32_t)0x00040000)        /*!< USART 3 clock enable */
 #define  RCC_APB1ENR_I2C2EN                 ((uint32_t)0x00400000)        /*!< I2C 2 clock enable */
#endif /* STM32F10X_LD && STM32F10X_LD_VL */

#if defined (STM32F10X_HD) || defined (STM32F10X_MD) || defined  (STM32F10X_LD)
 #define  RCC_APB1ENR_USBEN                  ((uint32_t)0x00800000)        /*!< USB Device clock enable */
#endif

#if defined (STM32F10X_HD) || defined  (STM32F10X_CL)
 #define  RCC_APB1ENR_TIM5EN                 ((uint32_t)0x00000008)        /*!< Timer 5 clock enable */
 #define  RCC_APB1ENR_TIM6EN                 ((uint32_t)0x00000010)        /*!< Timer 6 clock enable */
 #define  RCC_APB1ENR_TIM7EN                 ((uint32_t)0x00000020)        /*!< Timer 7 clock enable */
 #define  RCC_APB1ENR_SPI3EN                 ((uint32_t)0x00008000)        /*!< SPI 3 clock enable */
 #define  RCC_APB1ENR_UART4EN                ((uint32_t)0x00080000)        /*!< UART 4 clock enable */
 #define  RCC_APB1ENR_UART5EN                ((uint32_t)0x00100000)        /*!< UART 5 clock enable */
 #define  RCC_APB1ENR_DACEN                  ((uint32_t)0x20000000)        /*!< DAC interface clock enable */
#endif

#if defined (STM32F10X_LD_VL) || defined  (STM32F10X_MD_VL) || defined  (STM32F10X_HD_VL)
 #define  RCC_APB1ENR_TIM6EN                 ((uint32_t)0x00000010)        /*!< Timer 6 clock enable */
 #define  RCC_APB1ENR_TIM7EN                 ((uint32_t)0x00000020)        /*!< Timer 7 clock enable */
 #define  RCC_APB1ENR_DACEN                  ((uint32_t)0x20000000)        /*!< DAC interface clock enable */
 #define  RCC_APB1ENR_CECEN                  ((uint32_t)0x40000000)        /*!< CEC interface clock enable */
#endif

#ifdef STM32F10X_HD_VL
 #define  RCC_APB1ENR_TIM5EN                 ((uint32_t)0x00000008)        /*!< Timer 5 clock enable */
 #define  RCC_APB1ENR_TIM12EN                ((uint32_t)0x00000040)         /*!< TIM12 Timer clock enable  */
 #define  RCC_APB1ENR_TIM13EN                ((uint32_t)0x00000080)         /*!< TIM13 Timer clock
```

```
enable  */
 #define  RCC_APB1ENR_TIM14EN                ((uint32_t)0x00000100)         /*!< TIM14 Timer clock enable */
 #define  RCC_APB1ENR_SPI3EN                 ((uint32_t)0x00008000)        /*!< SPI 3 clock enable */
 #define  RCC_APB1ENR_UART4EN                ((uint32_t)0x00080000)        /*!< UART 4 clock enable */
 #define  RCC_APB1ENR_UART5EN                ((uint32_t)0x00100000)        /*!< UART 5 clock enable */
#endif /* STM32F10X_HD_VL */

#ifdef STM32F10X_CL
 #define  RCC_APB1ENR_CAN2EN                  ((uint32_t)0x04000000)        /*!< CAN2 clock enable */
#endif /* STM32F10X_CL */

#ifdef STM32F10X_XL
 #define  RCC_APB1ENR_TIM12EN                ((uint32_t)0x00000040)         /*!< TIM12 Timer clock enable  */
 #define  RCC_APB1ENR_TIM13EN                ((uint32_t)0x00000080)         /*!< TIM13 Timer clock enable  */
 #define  RCC_APB1ENR_TIM14EN                ((uint32_t)0x00000100)         /*!< TIM14 Timer clock enable */
#endif /* STM32F10X_XL */

/*******************  Bit definition for RCC_BDCR register  *******************/
#define  RCC_BDCR_LSEON                      ((uint32_t)0x00000001)        /*!< External Low Speed oscillator enable */
#define  RCC_BDCR_LSERDY                     ((uint32_t)0x00000002)        /*!< External Low Speed oscillator Ready */
#define  RCC_BDCR_LSEBYP                     ((uint32_t)0x00000004)        /*!< External Low Speed oscillator Bypass */

#define  RCC_BDCR_RTCSEL                     ((uint32_t)0x00000300)        /*!< RTCSEL[1:0] bits (RTC clock source selection) */
#define  RCC_BDCR_RTCSEL_0                   ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  RCC_BDCR_RTCSEL_1                   ((uint32_t)0x00000200)        /*!< Bit 1 */

/*!< RTC congiguration */
#define  RCC_BDCR_RTCSEL_NOCLOCK             ((uint32_t)0x00000000)        /*!< No clock */
#define  RCC_BDCR_RTCSEL_LSE                 ((uint32_t)0x00000100)        /*!< LSE oscillator clock used as RTC clock */
#define  RCC_BDCR_RTCSEL_LSI                 ((uint32_t)0x00000200)        /*!< LSI oscillator clock used as RTC clock */
#define  RCC_BDCR_RTCSEL_HSE                 ((uint32_t)0x00000300)        /*!< HSE oscillator clock divided by 128 used as RTC clock */

#define  RCC_BDCR_RTCEN                      ((uint32_t)0x00008000)        /*!< RTC clock enable */
#define  RCC_BDCR_BDRST                      ((uint32_t)0x00010000)        /*!< Backup domain software reset  */

/*******************  Bit definition for RCC_CSR register  *******************/
#define  RCC_CSR_LSION                       ((uint32_t)0x00000001)        /*!< Internal Low Speed oscillator enable */
#define  RCC_CSR_LSIRDY                      ((uint32_t)0x00000002)        /*!< Internal Low Speed oscillator Ready */
#define  RCC_CSR_RMVF                        ((uint32_t)0x01000000)        /*!< Remove reset flag */
#define  RCC_CSR_PINRSTF                     ((uint32_t)0x04000000)        /*!< PIN reset flag */
#define  RCC_CSR_PORRSTF                     ((uint32_t)0x08000000)        /*!< POR/PDR reset flag */
#define  RCC_CSR_SFTRSTF                     ((uint32_t)0x10000000)        /*!< Software Reset flag */
#define  RCC_CSR_IWDGRSTF                    ((uint32_t)0x20000000)        /*!< Independent Watchdog reset flag */
#define  RCC_CSR_WWDGRSTF                    ((uint32_t)0x40000000)        /*!< Window watchdog reset flag */
#define  RCC_CSR_LPWRRSTF                    ((uint32_t)0x80000000)        /*!< Low-Power reset flag */

#ifdef STM32F10X_CL
/*******************  Bit definition for RCC_AHBRSTR register  ****************/
 #define  RCC_AHBRSTR_OTGFSRST                ((uint32_t)0x00001000)         /*!< USB OTG FS reset */
 #define  RCC_AHBRSTR_ETHMACRST               ((uint32_t)0x00004000)         /*!< ETHERNET MAC reset */

/*******************  Bit definition for RCC_CFGR2 register  ******************/
/*!< PREDIV1 configuration */
 #define  RCC_CFGR2_PREDIV1                   ((uint32_t)0x0000000F)        /*!< PREDIV1[3:0] bits */
 #define  RCC_CFGR2_PREDIV1_0                 ((uint32_t)0x00000001)        /*!< Bit 0 */
 #define  RCC_CFGR2_PREDIV1_1                 ((uint32_t)0x00000002)        /*!< Bit 1 */
 #define  RCC_CFGR2_PREDIV1_2                 ((uint32_t)0x00000004)        /*!< Bit 2 */
```

```
 #define  RCC_CFGR2_PREDIV1_3                ((uint32_t)0x00000008)        /*!< Bit 3 */

 #define  RCC_CFGR2_PREDIV1_DIV1             ((uint32_t)0x00000000)        /*!< PREDIV1 input clock not divided */
 #define  RCC_CFGR2_PREDIV1_DIV2             ((uint32_t)0x00000001)        /*!< PREDIV1 input clock divided by 2 */
 #define  RCC_CFGR2_PREDIV1_DIV3             ((uint32_t)0x00000002)        /*!< PREDIV1 input clock divided by 3 */
 #define  RCC_CFGR2_PREDIV1_DIV4             ((uint32_t)0x00000003)        /*!< PREDIV1 input clock divided by 4 */
 #define  RCC_CFGR2_PREDIV1_DIV5             ((uint32_t)0x00000004)        /*!< PREDIV1 input clock divided by 5 */
 #define  RCC_CFGR2_PREDIV1_DIV6             ((uint32_t)0x00000005)        /*!< PREDIV1 input clock divided by 6 */
 #define  RCC_CFGR2_PREDIV1_DIV7             ((uint32_t)0x00000006)        /*!< PREDIV1 input clock divided by 7 */
 #define  RCC_CFGR2_PREDIV1_DIV8             ((uint32_t)0x00000007)        /*!< PREDIV1 input clock divided by 8 */
 #define  RCC_CFGR2_PREDIV1_DIV9             ((uint32_t)0x00000008)        /*!< PREDIV1 input clock divided by 9 */
 #define  RCC_CFGR2_PREDIV1_DIV10            ((uint32_t)0x00000009)        /*!< PREDIV1 input clock divided by 10 */
 #define  RCC_CFGR2_PREDIV1_DIV11            ((uint32_t)0x0000000A)        /*!< PREDIV1 input clock divided by 11 */
 #define  RCC_CFGR2_PREDIV1_DIV12            ((uint32_t)0x0000000B)        /*!< PREDIV1 input clock divided by 12 */
 #define  RCC_CFGR2_PREDIV1_DIV13            ((uint32_t)0x0000000C)        /*!< PREDIV1 input clock divided by 13 */
 #define  RCC_CFGR2_PREDIV1_DIV14            ((uint32_t)0x0000000D)        /*!< PREDIV1 input clock divided by 14 */
 #define  RCC_CFGR2_PREDIV1_DIV15            ((uint32_t)0x0000000E)        /*!< PREDIV1 input clock divided by 15 */
 #define  RCC_CFGR2_PREDIV1_DIV16            ((uint32_t)0x0000000F)        /*!< PREDIV1 input clock divided by 16 */

 /*!< PREDIV2 configuration */
 #define  RCC_CFGR2_PREDIV2                  ((uint32_t)0x000000F0)        /*!< PREDIV2[3:0] bits */
 #define  RCC_CFGR2_PREDIV2_0                ((uint32_t)0x00000010)        /*!< Bit 0 */
 #define  RCC_CFGR2_PREDIV2_1                ((uint32_t)0x00000020)        /*!< Bit 1 */
 #define  RCC_CFGR2_PREDIV2_2                ((uint32_t)0x00000040)        /*!< Bit 2 */
 #define  RCC_CFGR2_PREDIV2_3                ((uint32_t)0x00000080)        /*!< Bit 3 */

 #define  RCC_CFGR2_PREDIV2_DIV1             ((uint32_t)0x00000000)        /*!< PREDIV2 input clock not divided */
 #define  RCC_CFGR2_PREDIV2_DIV2             ((uint32_t)0x00000010)        /*!< PREDIV2 input clock divided by 2 */
 #define  RCC_CFGR2_PREDIV2_DIV3             ((uint32_t)0x00000020)        /*!< PREDIV2 input clock divided by 3 */
 #define  RCC_CFGR2_PREDIV2_DIV4             ((uint32_t)0x00000030)        /*!< PREDIV2 input clock divided by 4 */
 #define  RCC_CFGR2_PREDIV2_DIV5             ((uint32_t)0x00000040)        /*!< PREDIV2 input clock divided by 5 */
 #define  RCC_CFGR2_PREDIV2_DIV6             ((uint32_t)0x00000050)        /*!< PREDIV2 input clock divided by 6 */
 #define  RCC_CFGR2_PREDIV2_DIV7             ((uint32_t)0x00000060)        /*!< PREDIV2 input clock divided by 7 */
 #define  RCC_CFGR2_PREDIV2_DIV8             ((uint32_t)0x00000070)        /*!< PREDIV2 input clock divided by 8 */
 #define  RCC_CFGR2_PREDIV2_DIV9             ((uint32_t)0x00000080)        /*!< PREDIV2 input clock divided by 9 */
 #define  RCC_CFGR2_PREDIV2_DIV10            ((uint32_t)0x00000090)        /*!< PREDIV2 input clock divided by 10 */
 #define  RCC_CFGR2_PREDIV2_DIV11            ((uint32_t)0x000000A0)        /*!< PREDIV2 input clock divided by 11 */
 #define  RCC_CFGR2_PREDIV2_DIV12            ((uint32_t)0x000000B0)        /*!< PREDIV2 input clock divided by 12 */
 #define  RCC_CFGR2_PREDIV2_DIV13            ((uint32_t)0x000000C0)        /*!< PREDIV2 input clock divided by 13 */
 #define  RCC_CFGR2_PREDIV2_DIV14            ((uint32_t)0x000000D0)        /*!< PREDIV2 input clock divided by 14 */
 #define  RCC_CFGR2_PREDIV2_DIV15            ((uint32_t)0x000000E0)        /*!< PREDIV2 input clock divided by 15 */
 #define  RCC_CFGR2_PREDIV2_DIV16            ((uint32_t)0x000000F0)        /*!< PREDIV2 input clock
```

```
      divided by 16 */

  /*!< PLL2MUL configuration */
   #define  RCC_CFGR2_PLL2MUL                  ((uint32_t)0x00000F00)        /*!< PLL2MUL[3:0] bits */
   #define  RCC_CFGR2_PLL2MUL_0                ((uint32_t)0x00000100)        /*!< Bit 0 */
   #define  RCC_CFGR2_PLL2MUL_1                ((uint32_t)0x00000200)        /*!< Bit 1 */
   #define  RCC_CFGR2_PLL2MUL_2                ((uint32_t)0x00000400)        /*!< Bit 2 */
   #define  RCC_CFGR2_PLL2MUL_3                ((uint32_t)0x00000800)        /*!< Bit 3 */

   #define  RCC_CFGR2_PLL2MUL8                 ((uint32_t)0x00000600)        /*!< PLL2 input clock * 8 */
   #define  RCC_CFGR2_PLL2MUL9                 ((uint32_t)0x00000700)        /*!< PLL2 input clock * 9 */
   #define  RCC_CFGR2_PLL2MUL10                ((uint32_t)0x00000800)        /*!< PLL2 input clock * 10 */
   #define  RCC_CFGR2_PLL2MUL11                ((uint32_t)0x00000900)        /*!< PLL2 input clock * 11 */
   #define  RCC_CFGR2_PLL2MUL12                ((uint32_t)0x00000A00)        /*!< PLL2 input clock * 12 */
   #define  RCC_CFGR2_PLL2MUL13                ((uint32_t)0x00000B00)        /*!< PLL2 input clock * 13 */
   #define  RCC_CFGR2_PLL2MUL14                ((uint32_t)0x00000C00)        /*!< PLL2 input clock * 14 */
   #define  RCC_CFGR2_PLL2MUL16                ((uint32_t)0x00000E00)        /*!< PLL2 input clock * 16 */
   #define  RCC_CFGR2_PLL2MUL20                ((uint32_t)0x00000F00)        /*!< PLL2 input clock * 20 */

  /*!< PLL3MUL configuration */
   #define  RCC_CFGR2_PLL3MUL                  ((uint32_t)0x0000F000)        /*!< PLL3MUL[3:0] bits */
   #define  RCC_CFGR2_PLL3MUL_0                ((uint32_t)0x00001000)        /*!< Bit 0 */
   #define  RCC_CFGR2_PLL3MUL_1                ((uint32_t)0x00002000)        /*!< Bit 1 */
   #define  RCC_CFGR2_PLL3MUL_2                ((uint32_t)0x00004000)        /*!< Bit 2 */
   #define  RCC_CFGR2_PLL3MUL_3                ((uint32_t)0x00008000)        /*!< Bit 3 */

   #define  RCC_CFGR2_PLL3MUL8                 ((uint32_t)0x00006000)        /*!< PLL3 input clock * 8 */
   #define  RCC_CFGR2_PLL3MUL9                 ((uint32_t)0x00007000)        /*!< PLL3 input clock * 9 */
   #define  RCC_CFGR2_PLL3MUL10                ((uint32_t)0x00008000)        /*!< PLL3 input clock * 10 */
   #define  RCC_CFGR2_PLL3MUL11                ((uint32_t)0x00009000)        /*!< PLL3 input clock * 11 */
   #define  RCC_CFGR2_PLL3MUL12                ((uint32_t)0x0000A000)        /*!< PLL3 input clock * 12 */
   #define  RCC_CFGR2_PLL3MUL13                ((uint32_t)0x0000B000)        /*!< PLL3 input clock * 13 */
   #define  RCC_CFGR2_PLL3MUL14                ((uint32_t)0x0000C000)        /*!< PLL3 input clock * 14 */
   #define  RCC_CFGR2_PLL3MUL16                ((uint32_t)0x0000E000)        /*!< PLL3 input clock * 16 */
   #define  RCC_CFGR2_PLL3MUL20                ((uint32_t)0x0000F000)        /*!< PLL3 input clock * 20 */

   #define  RCC_CFGR2_PREDIV1SRC               ((uint32_t)0x00010000)        /*!< PREDIV1 entry clock
      source */
   #define  RCC_CFGR2_PREDIV1SRC_PLL2          ((uint32_t)0x00010000)        /*!< PLL2 selected as
      PREDIV1 entry clock source */
   #define  RCC_CFGR2_PREDIV1SRC_HSE           ((uint32_t)0x00000000)        /*!< HSE selected as PREDIV1
      entry clock source */
   #define  RCC_CFGR2_I2S2SRC                  ((uint32_t)0x00020000)        /*!< I2S2 entry clock source
      */
   #define  RCC_CFGR2_I2S3SRC                  ((uint32_t)0x00040000)        /*!< I2S3 clock source */
  #endif /* STM32F10X_CL */

  #if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
  /*******************  Bit definition for RCC_CFGR2 register  ******************/
  /*!< PREDIV1 configuration */
   #define  RCC_CFGR2_PREDIV1                  ((uint32_t)0x0000000F)        /*!< PREDIV1[3:0] bits */
   #define  RCC_CFGR2_PREDIV1_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
   #define  RCC_CFGR2_PREDIV1_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
   #define  RCC_CFGR2_PREDIV1_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
   #define  RCC_CFGR2_PREDIV1_3                ((uint32_t)0x00000008)        /*!< Bit 3 */

   #define  RCC_CFGR2_PREDIV1_DIV1             ((uint32_t)0x00000000)        /*!< PREDIV1 input clock not
      divided */
   #define  RCC_CFGR2_PREDIV1_DIV2             ((uint32_t)0x00000001)        /*!< PREDIV1 input clock
      divided by 2 */
   #define  RCC_CFGR2_PREDIV1_DIV3             ((uint32_t)0x00000002)        /*!< PREDIV1 input clock
      divided by 3 */
   #define  RCC_CFGR2_PREDIV1_DIV4             ((uint32_t)0x00000003)        /*!< PREDIV1 input clock
      divided by 4 */
   #define  RCC_CFGR2_PREDIV1_DIV5             ((uint32_t)0x00000004)        /*!< PREDIV1 input clock
      divided by 5 */
   #define  RCC_CFGR2_PREDIV1_DIV6             ((uint32_t)0x00000005)        /*!< PREDIV1 input clock
      divided by 6 */
   #define  RCC_CFGR2_PREDIV1_DIV7             ((uint32_t)0x00000006)        /*!< PREDIV1 input clock
      divided by 7 */
   #define  RCC_CFGR2_PREDIV1_DIV8             ((uint32_t)0x00000007)        /*!< PREDIV1 input clock
      divided by 8 */
   #define  RCC_CFGR2_PREDIV1_DIV9             ((uint32_t)0x00000008)        /*!< PREDIV1 input clock
```

```
divided by 9 */
 #define  RCC_CFGR2_PREDIV1_DIV10            ((uint32_t)0x00000009)        /*!< PREDIV1 input clock divided by 10 */
 #define  RCC_CFGR2_PREDIV1_DIV11            ((uint32_t)0x0000000A)        /*!< PREDIV1 input clock divided by 11 */
 #define  RCC_CFGR2_PREDIV1_DIV12            ((uint32_t)0x0000000B)        /*!< PREDIV1 input clock divided by 12 */
 #define  RCC_CFGR2_PREDIV1_DIV13            ((uint32_t)0x0000000C)        /*!< PREDIV1 input clock divided by 13 */
 #define  RCC_CFGR2_PREDIV1_DIV14            ((uint32_t)0x0000000D)        /*!< PREDIV1 input clock divided by 14 */
 #define  RCC_CFGR2_PREDIV1_DIV15            ((uint32_t)0x0000000E)        /*!< PREDIV1 input clock divided by 15 */
 #define  RCC_CFGR2_PREDIV1_DIV16            ((uint32_t)0x0000000F)        /*!< PREDIV1 input clock divided by 16 */
#endif

/******************************************************************************/
/*                                                                            */
/*                 General Purpose and Alternate Function I/O                 */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for GPIO_CRL register  *******************/
#define  GPIO_CRL_MODE                      ((uint32_t)0x33333333)        /*!< Port x mode bits */

#define  GPIO_CRL_MODE0                     ((uint32_t)0x00000003)        /*!< MODE0[1:0] bits (Port x mode bits, pin 0) */
#define  GPIO_CRL_MODE0_0                   ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  GPIO_CRL_MODE0_1                   ((uint32_t)0x00000002)        /*!< Bit 1 */

#define  GPIO_CRL_MODE1                     ((uint32_t)0x00000030)        /*!< MODE1[1:0] bits (Port x mode bits, pin 1) */
#define  GPIO_CRL_MODE1_0                   ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  GPIO_CRL_MODE1_1                   ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  GPIO_CRL_MODE2                     ((uint32_t)0x00000300)        /*!< MODE2[1:0] bits (Port x mode bits, pin 2) */
#define  GPIO_CRL_MODE2_0                   ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  GPIO_CRL_MODE2_1                   ((uint32_t)0x00000200)        /*!< Bit 1 */

#define  GPIO_CRL_MODE3                     ((uint32_t)0x00003000)        /*!< MODE3[1:0] bits (Port x mode bits, pin 3) */
#define  GPIO_CRL_MODE3_0                   ((uint32_t)0x00001000)        /*!< Bit 0 */
#define  GPIO_CRL_MODE3_1                   ((uint32_t)0x00002000)        /*!< Bit 1 */

#define  GPIO_CRL_MODE4                     ((uint32_t)0x00030000)        /*!< MODE4[1:0] bits (Port x mode bits, pin 4) */
#define  GPIO_CRL_MODE4_0                   ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  GPIO_CRL_MODE4_1                   ((uint32_t)0x00020000)        /*!< Bit 1 */

#define  GPIO_CRL_MODE5                     ((uint32_t)0x00300000)        /*!< MODE5[1:0] bits (Port x mode bits, pin 5) */
#define  GPIO_CRL_MODE5_0                   ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  GPIO_CRL_MODE5_1                   ((uint32_t)0x00200000)        /*!< Bit 1 */

#define  GPIO_CRL_MODE6                     ((uint32_t)0x03000000)        /*!< MODE6[1:0] bits (Port x mode bits, pin 6) */
#define  GPIO_CRL_MODE6_0                   ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  GPIO_CRL_MODE6_1                   ((uint32_t)0x02000000)        /*!< Bit 1 */

#define  GPIO_CRL_MODE7                     ((uint32_t)0x30000000)        /*!< MODE7[1:0] bits (Port x mode bits, pin 7) */
#define  GPIO_CRL_MODE7_0                   ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  GPIO_CRL_MODE7_1                   ((uint32_t)0x20000000)        /*!< Bit 1 */

#define  GPIO_CRL_CNF                       ((uint32_t)0xCCCCCCCC)        /*!< Port x configuration bits */

#define  GPIO_CRL_CNF0                      ((uint32_t)0x0000000C)        /*!< CNF0[1:0] bits (Port x configuration bits, pin 0) */
#define  GPIO_CRL_CNF0_0                    ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  GPIO_CRL_CNF0_1                    ((uint32_t)0x00000008)        /*!< Bit 1 */
```

```
#define  GPIO_CRL_CNF1                        ((uint32_t)0x000000C0)        /*!< CNF1[1:0] bits (Port x configuration bits, pin 1) */
#define  GPIO_CRL_CNF1_0                      ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  GPIO_CRL_CNF1_1                      ((uint32_t)0x00000080)        /*!< Bit 1 */

#define  GPIO_CRL_CNF2                        ((uint32_t)0x00000C00)        /*!< CNF2[1:0] bits (Port x configuration bits, pin 2) */
#define  GPIO_CRL_CNF2_0                      ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  GPIO_CRL_CNF2_1                      ((uint32_t)0x00000800)        /*!< Bit 1 */

#define  GPIO_CRL_CNF3                        ((uint32_t)0x0000C000)        /*!< CNF3[1:0] bits (Port x configuration bits, pin 3) */
#define  GPIO_CRL_CNF3_0                      ((uint32_t)0x00004000)        /*!< Bit 0 */
#define  GPIO_CRL_CNF3_1                      ((uint32_t)0x00008000)        /*!< Bit 1 */

#define  GPIO_CRL_CNF4                        ((uint32_t)0x000C0000)        /*!< CNF4[1:0] bits (Port x configuration bits, pin 4) */
#define  GPIO_CRL_CNF4_0                      ((uint32_t)0x00040000)        /*!< Bit 0 */
#define  GPIO_CRL_CNF4_1                      ((uint32_t)0x00080000)        /*!< Bit 1 */

#define  GPIO_CRL_CNF5                        ((uint32_t)0x00C00000)        /*!< CNF5[1:0] bits (Port x configuration bits, pin 5) */
#define  GPIO_CRL_CNF5_0                      ((uint32_t)0x00400000)        /*!< Bit 0 */
#define  GPIO_CRL_CNF5_1                      ((uint32_t)0x00800000)        /*!< Bit 1 */

#define  GPIO_CRL_CNF6                        ((uint32_t)0x0C000000)        /*!< CNF6[1:0] bits (Port x configuration bits, pin 6) */
#define  GPIO_CRL_CNF6_0                      ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  GPIO_CRL_CNF6_1                      ((uint32_t)0x08000000)        /*!< Bit 1 */

#define  GPIO_CRL_CNF7                        ((uint32_t)0xC0000000)        /*!< CNF7[1:0] bits (Port x configuration bits, pin 7) */
#define  GPIO_CRL_CNF7_0                      ((uint32_t)0x40000000)        /*!< Bit 0 */
#define  GPIO_CRL_CNF7_1                      ((uint32_t)0x80000000)        /*!< Bit 1 */

/*******************  Bit definition for GPIO_CRH register  *******************/
#define  GPIO_CRH_MODE                        ((uint32_t)0x33333333)        /*!< Port x mode bits */

#define  GPIO_CRH_MODE8                       ((uint32_t)0x00000003)        /*!< MODE8[1:0] bits (Port x mode bits, pin 8) */
#define  GPIO_CRH_MODE8_0                     ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  GPIO_CRH_MODE8_1                     ((uint32_t)0x00000002)        /*!< Bit 1 */

#define  GPIO_CRH_MODE9                       ((uint32_t)0x00000030)        /*!< MODE9[1:0] bits (Port x mode bits, pin 9) */
#define  GPIO_CRH_MODE9_0                     ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  GPIO_CRH_MODE9_1                     ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  GPIO_CRH_MODE10                      ((uint32_t)0x00000300)        /*!< MODE10[1:0] bits (Port x mode bits, pin 10) */
#define  GPIO_CRH_MODE10_0                    ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  GPIO_CRH_MODE10_1                    ((uint32_t)0x00000200)        /*!< Bit 1 */

#define  GPIO_CRH_MODE11                      ((uint32_t)0x00003000)        /*!< MODE11[1:0] bits (Port x mode bits, pin 11) */
#define  GPIO_CRH_MODE11_0                    ((uint32_t)0x00001000)        /*!< Bit 0 */
#define  GPIO_CRH_MODE11_1                    ((uint32_t)0x00002000)        /*!< Bit 1 */

#define  GPIO_CRH_MODE12                      ((uint32_t)0x00030000)        /*!< MODE12[1:0] bits (Port x mode bits, pin 12) */
#define  GPIO_CRH_MODE12_0                    ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  GPIO_CRH_MODE12_1                    ((uint32_t)0x00020000)        /*!< Bit 1 */

#define  GPIO_CRH_MODE13                      ((uint32_t)0x00300000)        /*!< MODE13[1:0] bits (Port x mode bits, pin 13) */
#define  GPIO_CRH_MODE13_0                    ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  GPIO_CRH_MODE13_1                    ((uint32_t)0x00200000)        /*!< Bit 1 */

#define  GPIO_CRH_MODE14                      ((uint32_t)0x03000000)        /*!< MODE14[1:0] bits (Port x mode bits, pin 14) */
#define  GPIO_CRH_MODE14_0                    ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  GPIO_CRH_MODE14_1                    ((uint32_t)0x02000000)        /*!< Bit 1 */
```

```
#define  GPIO_CRH_MODE15                    ((uint32_t)0x30000000)        /*!< MODE15[1:0] bits (Port x mode bits, pin 15) */
#define  GPIO_CRH_MODE15_0                  ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  GPIO_CRH_MODE15_1                  ((uint32_t)0x20000000)        /*!< Bit 1 */

#define  GPIO_CRH_CNF                       ((uint32_t)0xCCCCCCCC)        /*!< Port x configuration bits */

#define  GPIO_CRH_CNF8                      ((uint32_t)0x0000000C)        /*!< CNF8[1:0] bits (Port x configuration bits, pin 8) */
#define  GPIO_CRH_CNF8_0                    ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  GPIO_CRH_CNF8_1                    ((uint32_t)0x00000008)        /*!< Bit 1 */

#define  GPIO_CRH_CNF9                      ((uint32_t)0x000000C0)        /*!< CNF9[1:0] bits (Port x configuration bits, pin 9) */
#define  GPIO_CRH_CNF9_0                    ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  GPIO_CRH_CNF9_1                    ((uint32_t)0x00000080)        /*!< Bit 1 */

#define  GPIO_CRH_CNF10                     ((uint32_t)0x00000C00)        /*!< CNF10[1:0] bits (Port x configuration bits, pin 10) */
#define  GPIO_CRH_CNF10_0                   ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  GPIO_CRH_CNF10_1                   ((uint32_t)0x00000800)        /*!< Bit 1 */

#define  GPIO_CRH_CNF11                     ((uint32_t)0x0000C000)        /*!< CNF11[1:0] bits (Port x configuration bits, pin 11) */
#define  GPIO_CRH_CNF11_0                   ((uint32_t)0x00004000)        /*!< Bit 0 */
#define  GPIO_CRH_CNF11_1                   ((uint32_t)0x00008000)        /*!< Bit 1 */

#define  GPIO_CRH_CNF12                     ((uint32_t)0x000C0000)        /*!< CNF12[1:0] bits (Port x configuration bits, pin 12) */
#define  GPIO_CRH_CNF12_0                   ((uint32_t)0x00040000)        /*!< Bit 0 */
#define  GPIO_CRH_CNF12_1                   ((uint32_t)0x00080000)        /*!< Bit 1 */

#define  GPIO_CRH_CNF13                     ((uint32_t)0x00C00000)        /*!< CNF13[1:0] bits (Port x configuration bits, pin 13) */
#define  GPIO_CRH_CNF13_0                   ((uint32_t)0x00400000)        /*!< Bit 0 */
#define  GPIO_CRH_CNF13_1                   ((uint32_t)0x00800000)        /*!< Bit 1 */

#define  GPIO_CRH_CNF14                     ((uint32_t)0x0C000000)        /*!< CNF14[1:0] bits (Port x configuration bits, pin 14) */
#define  GPIO_CRH_CNF14_0                   ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  GPIO_CRH_CNF14_1                   ((uint32_t)0x08000000)        /*!< Bit 1 */

#define  GPIO_CRH_CNF15                     ((uint32_t)0xC0000000)        /*!< CNF15[1:0] bits (Port x configuration bits, pin 15) */
#define  GPIO_CRH_CNF15_0                   ((uint32_t)0x40000000)        /*!< Bit 0 */
#define  GPIO_CRH_CNF15_1                   ((uint32_t)0x80000000)        /*!< Bit 1 */

/*!<******************  Bit definition for GPIO_IDR register  *******************/
#define GPIO_IDR_IDR0                       ((uint16_t)0x0001)            /*!< Port input data, bit 0 */
#define GPIO_IDR_IDR1                       ((uint16_t)0x0002)            /*!< Port input data, bit 1 */
#define GPIO_IDR_IDR2                       ((uint16_t)0x0004)            /*!< Port input data, bit 2 */
#define GPIO_IDR_IDR3                       ((uint16_t)0x0008)            /*!< Port input data, bit 3 */
#define GPIO_IDR_IDR4                       ((uint16_t)0x0010)            /*!< Port input data, bit 4 */
#define GPIO_IDR_IDR5                       ((uint16_t)0x0020)            /*!< Port input data, bit 5 */
#define GPIO_IDR_IDR6                       ((uint16_t)0x0040)            /*!< Port input data, bit 6 */
#define GPIO_IDR_IDR7                       ((uint16_t)0x0080)            /*!< Port input data, bit 7 */
#define GPIO_IDR_IDR8                       ((uint16_t)0x0100)            /*!< Port input data, bit 8 */
#define GPIO_IDR_IDR9                       ((uint16_t)0x0200)            /*!< Port input data, bit 9 */
#define GPIO_IDR_IDR10                      ((uint16_t)0x0400)            /*!< Port input data, bit 10 */
#define GPIO_IDR_IDR11                      ((uint16_t)0x0800)            /*!< Port input data, bit 11 */
#define GPIO_IDR_IDR12                      ((uint16_t)0x1000)            /*!< Port input data, bit 12 */
#define GPIO_IDR_IDR13                      ((uint16_t)0x2000)            /*!< Port input data, bit 13 */
#define GPIO_IDR_IDR14                      ((uint16_t)0x4000)            /*!< Port input data, bit 14 */
#define GPIO_IDR_IDR15                      ((uint16_t)0x8000)            /*!< Port input data, bit 15 */
```

```
/*******************  Bit definition for GPIO_ODR register  *******************/
#define GPIO_ODR_ODR0                       ((uint16_t)0x0001)            /*!< Port output data, bit 0 */
#define GPIO_ODR_ODR1                       ((uint16_t)0x0002)            /*!< Port output data, bit 1 */
#define GPIO_ODR_ODR2                       ((uint16_t)0x0004)            /*!< Port output data, bit 2 */
#define GPIO_ODR_ODR3                       ((uint16_t)0x0008)            /*!< Port output data, bit 3 */
#define GPIO_ODR_ODR4                       ((uint16_t)0x0010)            /*!< Port output data, bit 4 */
#define GPIO_ODR_ODR5                       ((uint16_t)0x0020)            /*!< Port output data, bit 5 */
#define GPIO_ODR_ODR6                       ((uint16_t)0x0040)            /*!< Port output data, bit 6 */
#define GPIO_ODR_ODR7                       ((uint16_t)0x0080)            /*!< Port output data, bit 7 */
#define GPIO_ODR_ODR8                       ((uint16_t)0x0100)            /*!< Port output data, bit 8 */
#define GPIO_ODR_ODR9                       ((uint16_t)0x0200)            /*!< Port output data, bit 9 */
#define GPIO_ODR_ODR10                      ((uint16_t)0x0400)            /*!< Port output data, bit 10 */
#define GPIO_ODR_ODR11                      ((uint16_t)0x0800)            /*!< Port output data, bit 11 */
#define GPIO_ODR_ODR12                      ((uint16_t)0x1000)            /*!< Port output data, bit 12 */
#define GPIO_ODR_ODR13                      ((uint16_t)0x2000)            /*!< Port output data, bit 13 */
#define GPIO_ODR_ODR14                      ((uint16_t)0x4000)            /*!< Port output data, bit 14 */
#define GPIO_ODR_ODR15                      ((uint16_t)0x8000)            /*!< Port output data, bit 15 */

/*******************  Bit definition for GPIO_BSRR register  *******************/
#define GPIO_BSRR_BS0                       ((uint32_t)0x00000001)        /*!< Port x Set bit 0 */
#define GPIO_BSRR_BS1                       ((uint32_t)0x00000002)        /*!< Port x Set bit 1 */
#define GPIO_BSRR_BS2                       ((uint32_t)0x00000004)        /*!< Port x Set bit 2 */
#define GPIO_BSRR_BS3                       ((uint32_t)0x00000008)        /*!< Port x Set bit 3 */
#define GPIO_BSRR_BS4                       ((uint32_t)0x00000010)        /*!< Port x Set bit 4 */
#define GPIO_BSRR_BS5                       ((uint32_t)0x00000020)        /*!< Port x Set bit 5 */
#define GPIO_BSRR_BS6                       ((uint32_t)0x00000040)        /*!< Port x Set bit 6 */
#define GPIO_BSRR_BS7                       ((uint32_t)0x00000080)        /*!< Port x Set bit 7 */
#define GPIO_BSRR_BS8                       ((uint32_t)0x00000100)        /*!< Port x Set bit 8 */
#define GPIO_BSRR_BS9                       ((uint32_t)0x00000200)        /*!< Port x Set bit 9 */
#define GPIO_BSRR_BS10                      ((uint32_t)0x00000400)        /*!< Port x Set bit 10 */
#define GPIO_BSRR_BS11                      ((uint32_t)0x00000800)        /*!< Port x Set bit 11 */
#define GPIO_BSRR_BS12                      ((uint32_t)0x00001000)        /*!< Port x Set bit 12 */
#define GPIO_BSRR_BS13                      ((uint32_t)0x00002000)        /*!< Port x Set bit 13 */
#define GPIO_BSRR_BS14                      ((uint32_t)0x00004000)        /*!< Port x Set bit 14 */
#define GPIO_BSRR_BS15                      ((uint32_t)0x00008000)        /*!< Port x Set bit 15 */

#define GPIO_BSRR_BR0                       ((uint32_t)0x00010000)        /*!< Port x Reset bit 0 */
#define GPIO_BSRR_BR1                       ((uint32_t)0x00020000)        /*!< Port x Reset bit 1 */
#define GPIO_BSRR_BR2                       ((uint32_t)0x00040000)        /*!< Port x Reset bit 2 */
#define GPIO_BSRR_BR3                       ((uint32_t)0x00080000)        /*!< Port x Reset bit 3 */
#define GPIO_BSRR_BR4                       ((uint32_t)0x00100000)        /*!< Port x Reset bit 4 */
#define GPIO_BSRR_BR5                       ((uint32_t)0x00200000)        /*!< Port x Reset bit 5 */
#define GPIO_BSRR_BR6                       ((uint32_t)0x00400000)        /*!< Port x Reset bit 6 */
#define GPIO_BSRR_BR7                       ((uint32_t)0x00800000)        /*!< Port x Reset bit 7 */
#define GPIO_BSRR_BR8                       ((uint32_t)0x01000000)        /*!< Port x Reset bit 8 */
#define GPIO_BSRR_BR9                       ((uint32_t)0x02000000)        /*!< Port x Reset bit 9 */
#define GPIO_BSRR_BR10                      ((uint32_t)0x04000000)        /*!< Port x Reset bit 10 */
#define GPIO_BSRR_BR11                      ((uint32_t)0x08000000)        /*!< Port x Reset bit 11 */
#define GPIO_BSRR_BR12                      ((uint32_t)0x10000000)        /*!< Port x Reset bit 12 */
#define GPIO_BSRR_BR13                      ((uint32_t)0x20000000)        /*!< Port x Reset bit 13 */
#define GPIO_BSRR_BR14                      ((uint32_t)0x40000000)        /*!< Port x Reset bit 14 */
#define GPIO_BSRR_BR15                      ((uint32_t)0x80000000)        /*!< Port x Reset bit 15 */

/*******************  Bit definition for GPIO_BRR register  *******************/
#define GPIO_BRR_BR0                        ((uint16_t)0x0001)            /*!< Port x Reset bit 0 */
#define GPIO_BRR_BR1                        ((uint16_t)0x0002)            /*!< Port x Reset bit 1 */
#define GPIO_BRR_BR2                        ((uint16_t)0x0004)            /*!< Port x Reset bit 2 */
```

```
#define GPIO_BRR_BR3                        ((uint16_t)0x0008)            /*!< Port x Reset bit 3 */
#define GPIO_BRR_BR4                        ((uint16_t)0x0010)            /*!< Port x Reset bit 4 */
#define GPIO_BRR_BR5                        ((uint16_t)0x0020)            /*!< Port x Reset bit 5 */
#define GPIO_BRR_BR6                        ((uint16_t)0x0040)            /*!< Port x Reset bit 6 */
#define GPIO_BRR_BR7                        ((uint16_t)0x0080)            /*!< Port x Reset bit 7 */
#define GPIO_BRR_BR8                        ((uint16_t)0x0100)            /*!< Port x Reset bit 8 */
#define GPIO_BRR_BR9                        ((uint16_t)0x0200)            /*!< Port x Reset bit 9 */
#define GPIO_BRR_BR10                       ((uint16_t)0x0400)            /*!< Port x Reset bit 10 */
#define GPIO_BRR_BR11                       ((uint16_t)0x0800)            /*!< Port x Reset bit 11 */
#define GPIO_BRR_BR12                       ((uint16_t)0x1000)            /*!< Port x Reset bit 12 */
#define GPIO_BRR_BR13                       ((uint16_t)0x2000)            /*!< Port x Reset bit 13 */
#define GPIO_BRR_BR14                       ((uint16_t)0x4000)            /*!< Port x Reset bit 14 */
#define GPIO_BRR_BR15                       ((uint16_t)0x8000)            /*!< Port x Reset bit 15 */

/******************  Bit definition for GPIO_LCKR register  *******************/
#define GPIO_LCKR_LCK0                      ((uint32_t)0x00000001)        /*!< Port x Lock bit 0 */
#define GPIO_LCKR_LCK1                      ((uint32_t)0x00000002)        /*!< Port x Lock bit 1 */
#define GPIO_LCKR_LCK2                      ((uint32_t)0x00000004)        /*!< Port x Lock bit 2 */
#define GPIO_LCKR_LCK3                      ((uint32_t)0x00000008)        /*!< Port x Lock bit 3 */
#define GPIO_LCKR_LCK4                      ((uint32_t)0x00000010)        /*!< Port x Lock bit 4 */
#define GPIO_LCKR_LCK5                      ((uint32_t)0x00000020)        /*!< Port x Lock bit 5 */
#define GPIO_LCKR_LCK6                      ((uint32_t)0x00000040)        /*!< Port x Lock bit 6 */
#define GPIO_LCKR_LCK7                      ((uint32_t)0x00000080)        /*!< Port x Lock bit 7 */
#define GPIO_LCKR_LCK8                      ((uint32_t)0x00000100)        /*!< Port x Lock bit 8 */
#define GPIO_LCKR_LCK9                      ((uint32_t)0x00000200)        /*!< Port x Lock bit 9 */
#define GPIO_LCKR_LCK10                     ((uint32_t)0x00000400)        /*!< Port x Lock bit 10 */
#define GPIO_LCKR_LCK11                     ((uint32_t)0x00000800)        /*!< Port x Lock bit 11 */
#define GPIO_LCKR_LCK12                     ((uint32_t)0x00001000)        /*!< Port x Lock bit 12 */
#define GPIO_LCKR_LCK13                     ((uint32_t)0x00002000)        /*!< Port x Lock bit 13 */
#define GPIO_LCKR_LCK14                     ((uint32_t)0x00004000)        /*!< Port x Lock bit 14 */
#define GPIO_LCKR_LCK15                     ((uint32_t)0x00008000)        /*!< Port x Lock bit 15 */
#define GPIO_LCKR_LCKK                      ((uint32_t)0x00010000)        /*!< Lock key */

/*----------------------------------------------------------------------------*/

/******************  Bit definition for AFIO_EVCR register  *******************/
#define AFIO_EVCR_PIN                       ((uint8_t)0x0F)               /*!< PIN[3:0] bits (Pin selection) */
#define AFIO_EVCR_PIN_0                     ((uint8_t)0x01)               /*!< Bit 0 */
#define AFIO_EVCR_PIN_1                     ((uint8_t)0x02)               /*!< Bit 1 */
#define AFIO_EVCR_PIN_2                     ((uint8_t)0x04)               /*!< Bit 2 */
#define AFIO_EVCR_PIN_3                     ((uint8_t)0x08)               /*!< Bit 3 */

/*!< PIN configuration */
#define AFIO_EVCR_PIN_PX0                   ((uint8_t)0x00)               /*!< Pin 0 selected */
#define AFIO_EVCR_PIN_PX1                   ((uint8_t)0x01)               /*!< Pin 1 selected */
#define AFIO_EVCR_PIN_PX2                   ((uint8_t)0x02)               /*!< Pin 2 selected */
#define AFIO_EVCR_PIN_PX3                   ((uint8_t)0x03)               /*!< Pin 3 selected */
#define AFIO_EVCR_PIN_PX4                   ((uint8_t)0x04)               /*!< Pin 4 selected */
#define AFIO_EVCR_PIN_PX5                   ((uint8_t)0x05)               /*!< Pin 5 selected */
#define AFIO_EVCR_PIN_PX6                   ((uint8_t)0x06)               /*!< Pin 6 selected */
#define AFIO_EVCR_PIN_PX7                   ((uint8_t)0x07)               /*!< Pin 7 selected */
#define AFIO_EVCR_PIN_PX8                   ((uint8_t)0x08)               /*!< Pin 8 selected */
#define AFIO_EVCR_PIN_PX9                   ((uint8_t)0x09)               /*!< Pin 9 selected */
#define AFIO_EVCR_PIN_PX10                  ((uint8_t)0x0A)               /*!< Pin 10 selected */
#define AFIO_EVCR_PIN_PX11                  ((uint8_t)0x0B)               /*!< Pin 11 selected */
#define AFIO_EVCR_PIN_PX12                  ((uint8_t)0x0C)               /*!< Pin 12 selected */
#define AFIO_EVCR_PIN_PX13                  ((uint8_t)0x0D)               /*!< Pin 13 selected */
#define AFIO_EVCR_PIN_PX14                  ((uint8_t)0x0E)               /*!< Pin 14 selected */
#define AFIO_EVCR_PIN_PX15                  ((uint8_t)0x0F)               /*!< Pin 15 selected */

#define AFIO_EVCR_PORT                      ((uint8_t)0x70)               /*!< PORT[2:0] bits (Port selection) */
#define AFIO_EVCR_PORT_0                    ((uint8_t)0x10)               /*!< Bit 0 */
#define AFIO_EVCR_PORT_1                    ((uint8_t)0x20)               /*!< Bit 1 */
#define AFIO_EVCR_PORT_2                    ((uint8_t)0x40)               /*!< Bit 2 */

/*!< PORT configuration */
#define AFIO_EVCR_PORT_PA                   ((uint8_t)0x00)               /*!< Port A selected */
#define AFIO_EVCR_PORT_PB                   ((uint8_t)0x10)               /*!< Port B selected */
#define AFIO_EVCR_PORT_PC                   ((uint8_t)0x20)               /*!< Port C selected */
#define AFIO_EVCR_PORT_PD                   ((uint8_t)0x30)               /*!< Port D selected */
#define AFIO_EVCR_PORT_PE                   ((uint8_t)0x40)               /*!< Port E selected */
```

```
#define AFIO_EVCR_EVOE                       ((uint8_t)0x80)              /*!< Event Output Enable */

/******************  Bit definition for AFIO_MAPR register  *******************/
#define AFIO_MAPR_SPI1_REMAP                 ((uint32_t)0x00000001)       /*!< SPI1 remapping */
#define AFIO_MAPR_I2C1_REMAP                 ((uint32_t)0x00000002)       /*!< I2C1 remapping */
#define AFIO_MAPR_USART1_REMAP               ((uint32_t)0x00000004)       /*!< USART1 remapping */
#define AFIO_MAPR_USART2_REMAP               ((uint32_t)0x00000008)       /*!< USART2 remapping */

#define AFIO_MAPR_USART3_REMAP               ((uint32_t)0x00000030)       /*!< USART3_REMAP[1:0] bits (USART3 remapping) */
#define AFIO_MAPR_USART3_REMAP_0             ((uint32_t)0x00000010)       /*!< Bit 0 */
#define AFIO_MAPR_USART3_REMAP_1             ((uint32_t)0x00000020)       /*!< Bit 1 */

/* USART3_REMAP configuration */
#define AFIO_MAPR_USART3_REMAP_NOREMAP       ((uint32_t)0x00000000)       /*!< No remap (TX/PB10, RX/PB11, CK/PB12, CTS/PB13, RTS/PB14) */
#define AFIO_MAPR_USART3_REMAP_PARTIALREMAP  ((uint32_t)0x00000010)       /*!< Partial remap (TX/PC10, RX/PC11, CK/PC12, CTS/PB13, RTS/PB14) */
#define AFIO_MAPR_USART3_REMAP_FULLREMAP     ((uint32_t)0x00000030)       /*!< Full remap (TX/PD8, RX/PD9, CK/PD10, CTS/PD11, RTS/PD12) */

#define AFIO_MAPR_TIM1_REMAP                 ((uint32_t)0x000000C0)       /*!< TIM1_REMAP[1:0] bits (TIM1 remapping) */
#define AFIO_MAPR_TIM1_REMAP_0               ((uint32_t)0x00000040)       /*!< Bit 0 */
#define AFIO_MAPR_TIM1_REMAP_1               ((uint32_t)0x00000080)       /*!< Bit 1 */

/*!< TIM1_REMAP configuration */
#define AFIO_MAPR_TIM1_REMAP_NOREMAP         ((uint32_t)0x00000000)       /*!< No remap (ETR/PA12, CH1/PA8, CH2/PA9, CH3/PA10, CH4/PA11, BKIN/PB12, CH1N/PB13, CH2N/PB14, CH3N/PB15) */
#define AFIO_MAPR_TIM1_REMAP_PARTIALREMAP    ((uint32_t)0x00000040)       /*!< Partial remap (ETR/PA12, CH1/PA8, CH2/PA9, CH3/PA10, CH4/PA11, BKIN/PA6, CH1N/PA7, CH2N/PB0, CH3N/PB1) */
#define AFIO_MAPR_TIM1_REMAP_FULLREMAP       ((uint32_t)0x000000C0)       /*!< Full remap (ETR/PE7, CH1/PE9, CH2/PE11, CH3/PE13, CH4/PE14, BKIN/PE15, CH1N/PE8, CH2N/PE10, CH3N/PE12) */

#define AFIO_MAPR_TIM2_REMAP                 ((uint32_t)0x00000300)       /*!< TIM2_REMAP[1:0] bits (TIM2 remapping) */
#define AFIO_MAPR_TIM2_REMAP_0               ((uint32_t)0x00000100)       /*!< Bit 0 */
#define AFIO_MAPR_TIM2_REMAP_1               ((uint32_t)0x00000200)       /*!< Bit 1 */

/*!< TIM2_REMAP configuration */
#define AFIO_MAPR_TIM2_REMAP_NOREMAP         ((uint32_t)0x00000000)       /*!< No remap (CH1/ETR/PA0, CH2/PA1, CH3/PA2, CH4/PA3) */
#define AFIO_MAPR_TIM2_REMAP_PARTIALREMAP1   ((uint32_t)0x00000100)       /*!< Partial remap (CH1/ETR/PA15, CH2/PB3, CH3/PA2, CH4/PA3) */
#define AFIO_MAPR_TIM2_REMAP_PARTIALREMAP2   ((uint32_t)0x00000200)       /*!< Partial remap (CH1/ETR/PA0, CH2/PA1, CH3/PB10, CH4/PB11) */
#define AFIO_MAPR_TIM2_REMAP_FULLREMAP       ((uint32_t)0x00000300)       /*!< Full remap (CH1/ETR/PA15, CH2/PB3, CH3/PB10, CH4/PB11) */

#define AFIO_MAPR_TIM3_REMAP                 ((uint32_t)0x00000C00)       /*!< TIM3_REMAP[1:0] bits (TIM3 remapping) */
#define AFIO_MAPR_TIM3_REMAP_0               ((uint32_t)0x00000400)       /*!< Bit 0 */
#define AFIO_MAPR_TIM3_REMAP_1               ((uint32_t)0x00000800)       /*!< Bit 1 */

/*!< TIM3_REMAP configuration */
#define AFIO_MAPR_TIM3_REMAP_NOREMAP         ((uint32_t)0x00000000)       /*!< No remap (CH1/PA6, CH2/PA7, CH3/PB0, CH4/PB1) */
#define AFIO_MAPR_TIM3_REMAP_PARTIALREMAP    ((uint32_t)0x00000800)       /*!< Partial remap (CH1/PB4, CH2/PB5, CH3/PB0, CH4/PB1) */
#define AFIO_MAPR_TIM3_REMAP_FULLREMAP       ((uint32_t)0x00000C00)       /*!< Full remap (CH1/PC6, CH2/PC7, CH3/PC8, CH4/PC9) */

#define AFIO_MAPR_TIM4_REMAP                 ((uint32_t)0x00001000)       /*!< TIM4_REMAP bit (TIM4 remapping) */

#define AFIO_MAPR_CAN_REMAP                  ((uint32_t)0x00006000)       /*!< CAN_REMAP[1:0] bits (CAN Alternate function remapping) */
#define AFIO_MAPR_CAN_REMAP_0                ((uint32_t)0x00002000)       /*!< Bit 0 */
#define AFIO_MAPR_CAN_REMAP_1                ((uint32_t)0x00004000)       /*!< Bit 1 */

/*!< CAN_REMAP configuration */
#define AFIO_MAPR_CAN_REMAP_REMAP1           ((uint32_t)0x00000000)       /*!< CANRX mapped to PA11,
```

```
CANTX mapped to PA12 */
#define AFIO_MAPR_CAN_REMAP_REMAP2          ((uint32_t)0x00004000)        /*!< CANRX mapped to PB8, CANTX mapped to PB9 */
#define AFIO_MAPR_CAN_REMAP_REMAP3          ((uint32_t)0x00006000)        /*!< CANRX mapped to PD0, CANTX mapped to PD1 */

#define AFIO_MAPR_PD01_REMAP                ((uint32_t)0x00008000)        /*!< Port D0/Port D1 mapping on OSC_IN/OSC_OUT */
#define AFIO_MAPR_TIM5CH4_IREMAP            ((uint32_t)0x00010000)        /*!< TIM5 Channel4 Internal Remap */
#define AFIO_MAPR_ADC1_ETRGINJ_REMAP        ((uint32_t)0x00020000)        /*!< ADC 1 External Trigger Injected Conversion remapping */
#define AFIO_MAPR_ADC1_ETRGREG_REMAP        ((uint32_t)0x00040000)        /*!< ADC 1 External Trigger Regular Conversion remapping */
#define AFIO_MAPR_ADC2_ETRGINJ_REMAP        ((uint32_t)0x00080000)        /*!< ADC 2 External Trigger Injected Conversion remapping */
#define AFIO_MAPR_ADC2_ETRGREG_REMAP        ((uint32_t)0x00100000)        /*!< ADC 2 External Trigger Regular Conversion remapping */

/*!< SWJ_CFG configuration */
#define AFIO_MAPR_SWJ_CFG                   ((uint32_t)0x07000000)        /*!< SWJ_CFG[2:0] bits (Serial Wire JTAG configuration) */
#define AFIO_MAPR_SWJ_CFG_0                 ((uint32_t)0x01000000)        /*!< Bit 0 */
#define AFIO_MAPR_SWJ_CFG_1                 ((uint32_t)0x02000000)        /*!< Bit 1 */
#define AFIO_MAPR_SWJ_CFG_2                 ((uint32_t)0x04000000)        /*!< Bit 2 */

#define AFIO_MAPR_SWJ_CFG_RESET             ((uint32_t)0x00000000)        /*!< Full SWJ (JTAG-DP + SW-DP) : Reset State */
#define AFIO_MAPR_SWJ_CFG_NOJNTRST          ((uint32_t)0x01000000)        /*!< Full SWJ (JTAG-DP + SW-DP) but without JNTRST */
#define AFIO_MAPR_SWJ_CFG_JTAGDISABLE       ((uint32_t)0x02000000)        /*!< JTAG-DP Disabled and SW-DP Enabled */
#define AFIO_MAPR_SWJ_CFG_DISABLE           ((uint32_t)0x04000000)        /*!< JTAG-DP Disabled and SW-DP Disabled */

#ifdef STM32F10X_CL
/*!< ETH_REMAP configuration */
 #define AFIO_MAPR_ETH_REMAP                 ((uint32_t)0x00200000)        /*!< SPI3_REMAP bit (Ethernet MAC I/O remapping) */

/*!< CAN2_REMAP configuration */
 #define AFIO_MAPR_CAN2_REMAP                ((uint32_t)0x00400000)        /*!< CAN2_REMAP bit (CAN2 I/O remapping) */

/*!< MII_RMII_SEL configuration */
 #define AFIO_MAPR_MII_RMII_SEL              ((uint32_t)0x00800000)        /*!< MII_RMII_SEL bit (Ethernet MII or RMII selection) */

/*!< SPI3_REMAP configuration */
 #define AFIO_MAPR_SPI3_REMAP                ((uint32_t)0x10000000)        /*!< SPI3_REMAP bit (SPI3 remapping) */

/*!< TIM2ITR1_IREMAP configuration */
 #define AFIO_MAPR_TIM2ITR1_IREMAP           ((uint32_t)0x20000000)        /*!< TIM2ITR1_IREMAP bit (TIM2 internal trigger 1 remapping) */

/*!< PTP_PPS_REMAP configuration */
 #define AFIO_MAPR_PTP_PPS_REMAP             ((uint32_t)0x40000000)        /*!< PTP_PPS_REMAP bit (Ethernet PTP PPS remapping) */
#endif

/*****************  Bit definition for AFIO_EXTICR1 register  *****************/
#define AFIO_EXTICR1_EXTI0                  ((uint16_t)0x000F)            /*!< EXTI 0 configuration */
#define AFIO_EXTICR1_EXTI1                  ((uint16_t)0x00F0)            /*!< EXTI 1 configuration */
#define AFIO_EXTICR1_EXTI2                  ((uint16_t)0x0F00)            /*!< EXTI 2 configuration */
#define AFIO_EXTICR1_EXTI3                  ((uint16_t)0xF000)            /*!< EXTI 3 configuration */

/*!< EXTI0 configuration */
#define AFIO_EXTICR1_EXTI0_PA               ((uint16_t)0x0000)            /*!< PA[0] pin */
#define AFIO_EXTICR1_EXTI0_PB               ((uint16_t)0x0001)            /*!< PB[0] pin */
#define AFIO_EXTICR1_EXTI0_PC               ((uint16_t)0x0002)            /*!< PC[0] pin */
#define AFIO_EXTICR1_EXTI0_PD               ((uint16_t)0x0003)            /*!< PD[0] pin */
#define AFIO_EXTICR1_EXTI0_PE               ((uint16_t)0x0004)            /*!< PE[0] pin */
```

```
#define AFIO_EXTICR1_EXTI0_PF                ((uint16_t)0x0005)            /*!< PF[0] pin */
#define AFIO_EXTICR1_EXTI0_PG                ((uint16_t)0x0006)            /*!< PG[0] pin */

/*!< EXTI1 configuration */
#define AFIO_EXTICR1_EXTI1_PA                ((uint16_t)0x0000)            /*!< PA[1] pin */
#define AFIO_EXTICR1_EXTI1_PB                ((uint16_t)0x0010)            /*!< PB[1] pin */
#define AFIO_EXTICR1_EXTI1_PC                ((uint16_t)0x0020)            /*!< PC[1] pin */
#define AFIO_EXTICR1_EXTI1_PD                ((uint16_t)0x0030)            /*!< PD[1] pin */
#define AFIO_EXTICR1_EXTI1_PE                ((uint16_t)0x0040)            /*!< PE[1] pin */
#define AFIO_EXTICR1_EXTI1_PF                ((uint16_t)0x0050)            /*!< PF[1] pin */
#define AFIO_EXTICR1_EXTI1_PG                ((uint16_t)0x0060)            /*!< PG[1] pin */

/*!< EXTI2 configuration */
#define AFIO_EXTICR1_EXTI2_PA                ((uint16_t)0x0000)            /*!< PA[2] pin */
#define AFIO_EXTICR1_EXTI2_PB                ((uint16_t)0x0100)            /*!< PB[2] pin */
#define AFIO_EXTICR1_EXTI2_PC                ((uint16_t)0x0200)            /*!< PC[2] pin */
#define AFIO_EXTICR1_EXTI2_PD                ((uint16_t)0x0300)            /*!< PD[2] pin */
#define AFIO_EXTICR1_EXTI2_PE                ((uint16_t)0x0400)            /*!< PE[2] pin */
#define AFIO_EXTICR1_EXTI2_PF                ((uint16_t)0x0500)            /*!< PF[2] pin */
#define AFIO_EXTICR1_EXTI2_PG                ((uint16_t)0x0600)            /*!< PG[2] pin */

/*!< EXTI3 configuration */
#define AFIO_EXTICR1_EXTI3_PA                ((uint16_t)0x0000)            /*!< PA[3] pin */
#define AFIO_EXTICR1_EXTI3_PB                ((uint16_t)0x1000)            /*!< PB[3] pin */
#define AFIO_EXTICR1_EXTI3_PC                ((uint16_t)0x2000)            /*!< PC[3] pin */
#define AFIO_EXTICR1_EXTI3_PD                ((uint16_t)0x3000)            /*!< PD[3] pin */
#define AFIO_EXTICR1_EXTI3_PE                ((uint16_t)0x4000)            /*!< PE[3] pin */
#define AFIO_EXTICR1_EXTI3_PF                ((uint16_t)0x5000)            /*!< PF[3] pin */
#define AFIO_EXTICR1_EXTI3_PG                ((uint16_t)0x6000)            /*!< PG[3] pin */

/*****************  Bit definition for AFIO_EXTICR2 register  *****************/
#define AFIO_EXTICR2_EXTI4                   ((uint16_t)0x000F)            /*!< EXTI 4 configuration */
#define AFIO_EXTICR2_EXTI5                   ((uint16_t)0x00F0)            /*!< EXTI 5 configuration */
#define AFIO_EXTICR2_EXTI6                   ((uint16_t)0x0F00)            /*!< EXTI 6 configuration */
#define AFIO_EXTICR2_EXTI7                   ((uint16_t)0xF000)            /*!< EXTI 7 configuration */

/*!< EXTI4 configuration */
#define AFIO_EXTICR2_EXTI4_PA                ((uint16_t)0x0000)            /*!< PA[4] pin */
#define AFIO_EXTICR2_EXTI4_PB                ((uint16_t)0x0001)            /*!< PB[4] pin */
#define AFIO_EXTICR2_EXTI4_PC                ((uint16_t)0x0002)            /*!< PC[4] pin */
#define AFIO_EXTICR2_EXTI4_PD                ((uint16_t)0x0003)            /*!< PD[4] pin */
#define AFIO_EXTICR2_EXTI4_PE                ((uint16_t)0x0004)            /*!< PE[4] pin */
#define AFIO_EXTICR2_EXTI4_PF                ((uint16_t)0x0005)            /*!< PF[4] pin */
#define AFIO_EXTICR2_EXTI4_PG                ((uint16_t)0x0006)            /*!< PG[4] pin */

/* EXTI5 configuration */
#define AFIO_EXTICR2_EXTI5_PA                ((uint16_t)0x0000)            /*!< PA[5] pin */
#define AFIO_EXTICR2_EXTI5_PB                ((uint16_t)0x0010)            /*!< PB[5] pin */
#define AFIO_EXTICR2_EXTI5_PC                ((uint16_t)0x0020)            /*!< PC[5] pin */
#define AFIO_EXTICR2_EXTI5_PD                ((uint16_t)0x0030)            /*!< PD[5] pin */
#define AFIO_EXTICR2_EXTI5_PE                ((uint16_t)0x0040)            /*!< PE[5] pin */
#define AFIO_EXTICR2_EXTI5_PF                ((uint16_t)0x0050)            /*!< PF[5] pin */
#define AFIO_EXTICR2_EXTI5_PG                ((uint16_t)0x0060)            /*!< PG[5] pin */

/*!< EXTI6 configuration */
#define AFIO_EXTICR2_EXTI6_PA                ((uint16_t)0x0000)            /*!< PA[6] pin */
#define AFIO_EXTICR2_EXTI6_PB                ((uint16_t)0x0100)            /*!< PB[6] pin */
#define AFIO_EXTICR2_EXTI6_PC                ((uint16_t)0x0200)            /*!< PC[6] pin */
#define AFIO_EXTICR2_EXTI6_PD                ((uint16_t)0x0300)            /*!< PD[6] pin */
#define AFIO_EXTICR2_EXTI6_PE                ((uint16_t)0x0400)            /*!< PE[6] pin */
#define AFIO_EXTICR2_EXTI6_PF                ((uint16_t)0x0500)            /*!< PF[6] pin */
#define AFIO_EXTICR2_EXTI6_PG                ((uint16_t)0x0600)            /*!< PG[6] pin */

/*!< EXTI7 configuration */
#define AFIO_EXTICR2_EXTI7_PA                ((uint16_t)0x0000)            /*!< PA[7] pin */
#define AFIO_EXTICR2_EXTI7_PB                ((uint16_t)0x1000)            /*!< PB[7] pin */
#define AFIO_EXTICR2_EXTI7_PC                ((uint16_t)0x2000)            /*!< PC[7] pin */
#define AFIO_EXTICR2_EXTI7_PD                ((uint16_t)0x3000)            /*!< PD[7] pin */
#define AFIO_EXTICR2_EXTI7_PE                ((uint16_t)0x4000)            /*!< PE[7] pin */
#define AFIO_EXTICR2_EXTI7_PF                ((uint16_t)0x5000)            /*!< PF[7] pin */
#define AFIO_EXTICR2_EXTI7_PG                ((uint16_t)0x6000)            /*!< PG[7] pin */

/*****************  Bit definition for AFIO_EXTICR3 register  *****************/
```

```
#define AFIO_EXTICR3_EXTI8                  ((uint16_t)0x000F)          /*!< EXTI 8 configuration */
#define AFIO_EXTICR3_EXTI9                  ((uint16_t)0x00F0)          /*!< EXTI 9 configuration */
#define AFIO_EXTICR3_EXTI10                 ((uint16_t)0x0F00)          /*!< EXTI 10 configuration */
#define AFIO_EXTICR3_EXTI11                 ((uint16_t)0xF000)          /*!< EXTI 11 configuration */

/*!< EXTI8 configuration */
#define AFIO_EXTICR3_EXTI8_PA               ((uint16_t)0x0000)          /*!< PA[8] pin */
#define AFIO_EXTICR3_EXTI8_PB               ((uint16_t)0x0001)          /*!< PB[8] pin */
#define AFIO_EXTICR3_EXTI8_PC               ((uint16_t)0x0002)          /*!< PC[8] pin */
#define AFIO_EXTICR3_EXTI8_PD               ((uint16_t)0x0003)          /*!< PD[8] pin */
#define AFIO_EXTICR3_EXTI8_PE               ((uint16_t)0x0004)          /*!< PE[8] pin */
#define AFIO_EXTICR3_EXTI8_PF               ((uint16_t)0x0005)          /*!< PF[8] pin */
#define AFIO_EXTICR3_EXTI8_PG               ((uint16_t)0x0006)          /*!< PG[8] pin */

/*!< EXTI9 configuration */
#define AFIO_EXTICR3_EXTI9_PA               ((uint16_t)0x0000)          /*!< PA[9] pin */
#define AFIO_EXTICR3_EXTI9_PB               ((uint16_t)0x0010)          /*!< PB[9] pin */
#define AFIO_EXTICR3_EXTI9_PC               ((uint16_t)0x0020)          /*!< PC[9] pin */
#define AFIO_EXTICR3_EXTI9_PD               ((uint16_t)0x0030)          /*!< PD[9] pin */
#define AFIO_EXTICR3_EXTI9_PE               ((uint16_t)0x0040)          /*!< PE[9] pin */
#define AFIO_EXTICR3_EXTI9_PF               ((uint16_t)0x0050)          /*!< PF[9] pin */
#define AFIO_EXTICR3_EXTI9_PG               ((uint16_t)0x0060)          /*!< PG[9] pin */

/*!< EXTI10 configuration */
#define AFIO_EXTICR3_EXTI10_PA              ((uint16_t)0x0000)          /*!< PA[10] pin */
#define AFIO_EXTICR3_EXTI10_PB              ((uint16_t)0x0100)          /*!< PB[10] pin */
#define AFIO_EXTICR3_EXTI10_PC              ((uint16_t)0x0200)          /*!< PC[10] pin */
#define AFIO_EXTICR3_EXTI10_PD              ((uint16_t)0x0300)          /*!< PD[10] pin */
#define AFIO_EXTICR3_EXTI10_PE              ((uint16_t)0x0400)          /*!< PE[10] pin */
#define AFIO_EXTICR3_EXTI10_PF              ((uint16_t)0x0500)          /*!< PF[10] pin */
#define AFIO_EXTICR3_EXTI10_PG              ((uint16_t)0x0600)          /*!< PG[10] pin */

/*!< EXTI11 configuration */
#define AFIO_EXTICR3_EXTI11_PA              ((uint16_t)0x0000)          /*!< PA[11] pin */
#define AFIO_EXTICR3_EXTI11_PB              ((uint16_t)0x1000)          /*!< PB[11] pin */
#define AFIO_EXTICR3_EXTI11_PC              ((uint16_t)0x2000)          /*!< PC[11] pin */
#define AFIO_EXTICR3_EXTI11_PD              ((uint16_t)0x3000)          /*!< PD[11] pin */
#define AFIO_EXTICR3_EXTI11_PE              ((uint16_t)0x4000)          /*!< PE[11] pin */
#define AFIO_EXTICR3_EXTI11_PF              ((uint16_t)0x5000)          /*!< PF[11] pin */
#define AFIO_EXTICR3_EXTI11_PG              ((uint16_t)0x6000)          /*!< PG[11] pin */

/*****************  Bit definition for AFIO_EXTICR4 register  *****************/
#define AFIO_EXTICR4_EXTI12                 ((uint16_t)0x000F)          /*!< EXTI 12 configuration */
#define AFIO_EXTICR4_EXTI13                 ((uint16_t)0x00F0)          /*!< EXTI 13 configuration */
#define AFIO_EXTICR4_EXTI14                 ((uint16_t)0x0F00)          /*!< EXTI 14 configuration */
#define AFIO_EXTICR4_EXTI15                 ((uint16_t)0xF000)          /*!< EXTI 15 configuration */

/* EXTI12 configuration */
#define AFIO_EXTICR4_EXTI12_PA              ((uint16_t)0x0000)          /*!< PA[12] pin */
#define AFIO_EXTICR4_EXTI12_PB              ((uint16_t)0x0001)          /*!< PB[12] pin */
#define AFIO_EXTICR4_EXTI12_PC              ((uint16_t)0x0002)          /*!< PC[12] pin */
#define AFIO_EXTICR4_EXTI12_PD              ((uint16_t)0x0003)          /*!< PD[12] pin */
#define AFIO_EXTICR4_EXTI12_PE              ((uint16_t)0x0004)          /*!< PE[12] pin */
#define AFIO_EXTICR4_EXTI12_PF              ((uint16_t)0x0005)          /*!< PF[12] pin */
#define AFIO_EXTICR4_EXTI12_PG              ((uint16_t)0x0006)          /*!< PG[12] pin */

/* EXTI13 configuration */
#define AFIO_EXTICR4_EXTI13_PA              ((uint16_t)0x0000)          /*!< PA[13] pin */
#define AFIO_EXTICR4_EXTI13_PB              ((uint16_t)0x0010)          /*!< PB[13] pin */
#define AFIO_EXTICR4_EXTI13_PC              ((uint16_t)0x0020)          /*!< PC[13] pin */
#define AFIO_EXTICR4_EXTI13_PD              ((uint16_t)0x0030)          /*!< PD[13] pin */
#define AFIO_EXTICR4_EXTI13_PE              ((uint16_t)0x0040)          /*!< PE[13] pin */
#define AFIO_EXTICR4_EXTI13_PF              ((uint16_t)0x0050)          /*!< PF[13] pin */
#define AFIO_EXTICR4_EXTI13_PG              ((uint16_t)0x0060)          /*!< PG[13] pin */

/*!< EXTI14 configuration */
#define AFIO_EXTICR4_EXTI14_PA              ((uint16_t)0x0000)          /*!< PA[14] pin */
#define AFIO_EXTICR4_EXTI14_PB              ((uint16_t)0x0100)          /*!< PB[14] pin */
#define AFIO_EXTICR4_EXTI14_PC              ((uint16_t)0x0200)          /*!< PC[14] pin */
#define AFIO_EXTICR4_EXTI14_PD              ((uint16_t)0x0300)          /*!< PD[14] pin */
#define AFIO_EXTICR4_EXTI14_PE              ((uint16_t)0x0400)          /*!< PE[14] pin */
#define AFIO_EXTICR4_EXTI14_PF              ((uint16_t)0x0500)          /*!< PF[14] pin */
#define AFIO_EXTICR4_EXTI14_PG              ((uint16_t)0x0600)          /*!< PG[14] pin */
```

```
/*!< EXTI15 configuration */
#define AFIO_EXTICR4_EXTI15_PA              ((uint16_t)0x0000)            /*!< PA[15] pin */
#define AFIO_EXTICR4_EXTI15_PB              ((uint16_t)0x1000)            /*!< PB[15] pin */
#define AFIO_EXTICR4_EXTI15_PC              ((uint16_t)0x2000)            /*!< PC[15] pin */
#define AFIO_EXTICR4_EXTI15_PD              ((uint16_t)0x3000)            /*!< PD[15] pin */
#define AFIO_EXTICR4_EXTI15_PE              ((uint16_t)0x4000)            /*!< PE[15] pin */
#define AFIO_EXTICR4_EXTI15_PF              ((uint16_t)0x5000)            /*!< PF[15] pin */
#define AFIO_EXTICR4_EXTI15_PG              ((uint16_t)0x6000)            /*!< PG[15] pin */

#if defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL) || defined (STM32F10X_HD_VL)
/******************  Bit definition for AFIO_MAPR2 register  ******************/
#define AFIO_MAPR2_TIM15_REMAP              ((uint32_t)0x00000001)        /*!< TIM15 remapping */
#define AFIO_MAPR2_TIM16_REMAP              ((uint32_t)0x00000002)        /*!< TIM16 remapping */
#define AFIO_MAPR2_TIM17_REMAP              ((uint32_t)0x00000004)        /*!< TIM17 remapping */
#define AFIO_MAPR2_CEC_REMAP                ((uint32_t)0x00000008)        /*!< CEC remapping */
#define AFIO_MAPR2_TIM1_DMA_REMAP           ((uint32_t)0x00000010)        /*!< TIM1_DMA remapping */
#endif

#ifdef STM32F10X_HD_VL
#define AFIO_MAPR2_TIM13_REMAP              ((uint32_t)0x00000100)        /*!< TIM13 remapping */
#define AFIO_MAPR2_TIM14_REMAP              ((uint32_t)0x00000200)        /*!< TIM14 remapping */
#define AFIO_MAPR2_FSMC_NADV_REMAP          ((uint32_t)0x00000400)        /*!< FSMC NADV remapping */
#define AFIO_MAPR2_TIM67_DAC_DMA_REMAP      ((uint32_t)0x00000800)        /*!< TIM6/TIM7 and DAC DMA remapping */
#define AFIO_MAPR2_TIM12_REMAP              ((uint32_t)0x00001000)        /*!< TIM12 remapping */
#define AFIO_MAPR2_MISC_REMAP               ((uint32_t)0x00002000)        /*!< Miscellaneous remapping */
#endif

#ifdef STM32F10X_XL
/******************  Bit definition for AFIO_MAPR2 register  ******************/
#define AFIO_MAPR2_TIM9_REMAP               ((uint32_t)0x00000020)        /*!< TIM9 remapping */
#define AFIO_MAPR2_TIM10_REMAP              ((uint32_t)0x00000040)        /*!< TIM10 remapping */
#define AFIO_MAPR2_TIM11_REMAP              ((uint32_t)0x00000080)        /*!< TIM11 remapping */
#define AFIO_MAPR2_TIM13_REMAP              ((uint32_t)0x00000100)        /*!< TIM13 remapping */
#define AFIO_MAPR2_TIM14_REMAP              ((uint32_t)0x00000200)        /*!< TIM14 remapping */
#define AFIO_MAPR2_FSMC_NADV_REMAP          ((uint32_t)0x00000400)        /*!< FSMC NADV remapping */
#endif

/******************************************************************************/
/*                                                                            */
/*                               SystemTick                                   */
/*                                                                            */
/******************************************************************************/

/*****************  Bit definition for SysTick_CTRL register  *****************/
#define  SysTick_CTRL_ENABLE                ((uint32_t)0x00000001)        /*!< Counter enable */
#define  SysTick_CTRL_TICKINT               ((uint32_t)0x00000002)        /*!< Counting down to 0 pends the SysTick handler */
#define  SysTick_CTRL_CLKSOURCE             ((uint32_t)0x00000004)        /*!< Clock source */
#define  SysTick_CTRL_COUNTFLAG             ((uint32_t)0x00010000)        /*!< Count Flag */

/*****************  Bit definition for SysTick_LOAD register  *****************/
#define  SysTick_LOAD_RELOAD                ((uint32_t)0x00FFFFFF)        /*!< Value to load into the SysTick Current Value Register when the counter reaches 0 */

/*****************  Bit definition for SysTick_VAL register  ******************/
#define  SysTick_VAL_CURRENT                ((uint32_t)0x00FFFFFF)        /*!< Current value at the time the register is accessed */

/*****************  Bit definition for SysTick_CALIB register  ****************/
#define  SysTick_CALIB_TENMS                ((uint32_t)0x00FFFFFF)        /*!< Reload value to use for 10ms timing */
#define  SysTick_CALIB_SKEW                 ((uint32_t)0x40000000)        /*!< Calibration value is not exactly 10 ms */
#define  SysTick_CALIB_NOREF                ((uint32_t)0x80000000)        /*!< The reference clock is not provided */

/******************************************************************************/
/*                                                                            */
/*                  Nested Vectored Interrupt Controller                      */
/*                                                                            */
```

```
/******************************************************************************/

/*******************  Bit definition for NVIC_ISER register  *******************/
#define  NVIC_ISER_SETENA                    ((uint32_t)0xFFFFFFFF)        /*!< Interrupt set enable bits */
#define  NVIC_ISER_SETENA_0                  ((uint32_t)0x00000001)        /*!< bit 0 */
#define  NVIC_ISER_SETENA_1                  ((uint32_t)0x00000002)        /*!< bit 1 */
#define  NVIC_ISER_SETENA_2                  ((uint32_t)0x00000004)        /*!< bit 2 */
#define  NVIC_ISER_SETENA_3                  ((uint32_t)0x00000008)        /*!< bit 3 */
#define  NVIC_ISER_SETENA_4                  ((uint32_t)0x00000010)        /*!< bit 4 */
#define  NVIC_ISER_SETENA_5                  ((uint32_t)0x00000020)        /*!< bit 5 */
#define  NVIC_ISER_SETENA_6                  ((uint32_t)0x00000040)        /*!< bit 6 */
#define  NVIC_ISER_SETENA_7                  ((uint32_t)0x00000080)        /*!< bit 7 */
#define  NVIC_ISER_SETENA_8                  ((uint32_t)0x00000100)        /*!< bit 8 */
#define  NVIC_ISER_SETENA_9                  ((uint32_t)0x00000200)        /*!< bit 9 */
#define  NVIC_ISER_SETENA_10                 ((uint32_t)0x00000400)        /*!< bit 10 */
#define  NVIC_ISER_SETENA_11                 ((uint32_t)0x00000800)        /*!< bit 11 */
#define  NVIC_ISER_SETENA_12                 ((uint32_t)0x00001000)        /*!< bit 12 */
#define  NVIC_ISER_SETENA_13                 ((uint32_t)0x00002000)        /*!< bit 13 */
#define  NVIC_ISER_SETENA_14                 ((uint32_t)0x00004000)        /*!< bit 14 */
#define  NVIC_ISER_SETENA_15                 ((uint32_t)0x00008000)        /*!< bit 15 */
#define  NVIC_ISER_SETENA_16                 ((uint32_t)0x00010000)        /*!< bit 16 */
#define  NVIC_ISER_SETENA_17                 ((uint32_t)0x00020000)        /*!< bit 17 */
#define  NVIC_ISER_SETENA_18                 ((uint32_t)0x00040000)        /*!< bit 18 */
#define  NVIC_ISER_SETENA_19                 ((uint32_t)0x00080000)        /*!< bit 19 */
#define  NVIC_ISER_SETENA_20                 ((uint32_t)0x00100000)        /*!< bit 20 */
#define  NVIC_ISER_SETENA_21                 ((uint32_t)0x00200000)        /*!< bit 21 */
#define  NVIC_ISER_SETENA_22                 ((uint32_t)0x00400000)        /*!< bit 22 */
#define  NVIC_ISER_SETENA_23                 ((uint32_t)0x00800000)        /*!< bit 23 */
#define  NVIC_ISER_SETENA_24                 ((uint32_t)0x01000000)        /*!< bit 24 */
#define  NVIC_ISER_SETENA_25                 ((uint32_t)0x02000000)        /*!< bit 25 */
#define  NVIC_ISER_SETENA_26                 ((uint32_t)0x04000000)        /*!< bit 26 */
#define  NVIC_ISER_SETENA_27                 ((uint32_t)0x08000000)        /*!< bit 27 */
#define  NVIC_ISER_SETENA_28                 ((uint32_t)0x10000000)        /*!< bit 28 */
#define  NVIC_ISER_SETENA_29                 ((uint32_t)0x20000000)        /*!< bit 29 */
#define  NVIC_ISER_SETENA_30                 ((uint32_t)0x40000000)        /*!< bit 30 */
#define  NVIC_ISER_SETENA_31                 ((uint32_t)0x80000000)        /*!< bit 31 */

/*******************  Bit definition for NVIC_ICER register  *******************/
#define  NVIC_ICER_CLRENA                    ((uint32_t)0xFFFFFFFF)        /*!< Interrupt clear-enable bits */
#define  NVIC_ICER_CLRENA_0                  ((uint32_t)0x00000001)        /*!< bit 0 */
#define  NVIC_ICER_CLRENA_1                  ((uint32_t)0x00000002)        /*!< bit 1 */
#define  NVIC_ICER_CLRENA_2                  ((uint32_t)0x00000004)        /*!< bit 2 */
#define  NVIC_ICER_CLRENA_3                  ((uint32_t)0x00000008)        /*!< bit 3 */
#define  NVIC_ICER_CLRENA_4                  ((uint32_t)0x00000010)        /*!< bit 4 */
#define  NVIC_ICER_CLRENA_5                  ((uint32_t)0x00000020)        /*!< bit 5 */
#define  NVIC_ICER_CLRENA_6                  ((uint32_t)0x00000040)        /*!< bit 6 */
#define  NVIC_ICER_CLRENA_7                  ((uint32_t)0x00000080)        /*!< bit 7 */
#define  NVIC_ICER_CLRENA_8                  ((uint32_t)0x00000100)        /*!< bit 8 */
#define  NVIC_ICER_CLRENA_9                  ((uint32_t)0x00000200)        /*!< bit 9 */
#define  NVIC_ICER_CLRENA_10                 ((uint32_t)0x00000400)        /*!< bit 10 */
#define  NVIC_ICER_CLRENA_11                 ((uint32_t)0x00000800)        /*!< bit 11 */
#define  NVIC_ICER_CLRENA_12                 ((uint32_t)0x00001000)        /*!< bit 12 */
#define  NVIC_ICER_CLRENA_13                 ((uint32_t)0x00002000)        /*!< bit 13 */
#define  NVIC_ICER_CLRENA_14                 ((uint32_t)0x00004000)        /*!< bit 14 */
#define  NVIC_ICER_CLRENA_15                 ((uint32_t)0x00008000)        /*!< bit 15 */
#define  NVIC_ICER_CLRENA_16                 ((uint32_t)0x00010000)        /*!< bit 16 */
#define  NVIC_ICER_CLRENA_17                 ((uint32_t)0x00020000)        /*!< bit 17 */
#define  NVIC_ICER_CLRENA_18                 ((uint32_t)0x00040000)        /*!< bit 18 */
#define  NVIC_ICER_CLRENA_19                 ((uint32_t)0x00080000)        /*!< bit 19 */
#define  NVIC_ICER_CLRENA_20                 ((uint32_t)0x00100000)        /*!< bit 20 */
#define  NVIC_ICER_CLRENA_21                 ((uint32_t)0x00200000)        /*!< bit 21 */
#define  NVIC_ICER_CLRENA_22                 ((uint32_t)0x00400000)        /*!< bit 22 */
#define  NVIC_ICER_CLRENA_23                 ((uint32_t)0x00800000)        /*!< bit 23 */
#define  NVIC_ICER_CLRENA_24                 ((uint32_t)0x01000000)        /*!< bit 24 */
#define  NVIC_ICER_CLRENA_25                 ((uint32_t)0x02000000)        /*!< bit 25 */
#define  NVIC_ICER_CLRENA_26                 ((uint32_t)0x04000000)        /*!< bit 26 */
#define  NVIC_ICER_CLRENA_27                 ((uint32_t)0x08000000)        /*!< bit 27 */
#define  NVIC_ICER_CLRENA_28                 ((uint32_t)0x10000000)        /*!< bit 28 */
#define  NVIC_ICER_CLRENA_29                 ((uint32_t)0x20000000)        /*!< bit 29 */
#define  NVIC_ICER_CLRENA_30                 ((uint32_t)0x40000000)        /*!< bit 30 */
#define  NVIC_ICER_CLRENA_31                 ((uint32_t)0x80000000)        /*!< bit 31 */
```

```
/******************  Bit definition for NVIC_ISPR register  *******************/
#define  NVIC_ISPR_SETPEND                    ((uint32_t)0xFFFFFFFF)        /*!< Interrupt set-pending bits */
#define  NVIC_ISPR_SETPEND_0                  ((uint32_t)0x00000001)        /*!< bit 0 */
#define  NVIC_ISPR_SETPEND_1                  ((uint32_t)0x00000002)        /*!< bit 1 */
#define  NVIC_ISPR_SETPEND_2                  ((uint32_t)0x00000004)        /*!< bit 2 */
#define  NVIC_ISPR_SETPEND_3                  ((uint32_t)0x00000008)        /*!< bit 3 */
#define  NVIC_ISPR_SETPEND_4                  ((uint32_t)0x00000010)        /*!< bit 4 */
#define  NVIC_ISPR_SETPEND_5                  ((uint32_t)0x00000020)        /*!< bit 5 */
#define  NVIC_ISPR_SETPEND_6                  ((uint32_t)0x00000040)        /*!< bit 6 */
#define  NVIC_ISPR_SETPEND_7                  ((uint32_t)0x00000080)        /*!< bit 7 */
#define  NVIC_ISPR_SETPEND_8                  ((uint32_t)0x00000100)        /*!< bit 8 */
#define  NVIC_ISPR_SETPEND_9                  ((uint32_t)0x00000200)        /*!< bit 9 */
#define  NVIC_ISPR_SETPEND_10                 ((uint32_t)0x00000400)        /*!< bit 10 */
#define  NVIC_ISPR_SETPEND_11                 ((uint32_t)0x00000800)        /*!< bit 11 */
#define  NVIC_ISPR_SETPEND_12                 ((uint32_t)0x00001000)        /*!< bit 12 */
#define  NVIC_ISPR_SETPEND_13                 ((uint32_t)0x00002000)        /*!< bit 13 */
#define  NVIC_ISPR_SETPEND_14                 ((uint32_t)0x00004000)        /*!< bit 14 */
#define  NVIC_ISPR_SETPEND_15                 ((uint32_t)0x00008000)        /*!< bit 15 */
#define  NVIC_ISPR_SETPEND_16                 ((uint32_t)0x00010000)        /*!< bit 16 */
#define  NVIC_ISPR_SETPEND_17                 ((uint32_t)0x00020000)        /*!< bit 17 */
#define  NVIC_ISPR_SETPEND_18                 ((uint32_t)0x00040000)        /*!< bit 18 */
#define  NVIC_ISPR_SETPEND_19                 ((uint32_t)0x00080000)        /*!< bit 19 */
#define  NVIC_ISPR_SETPEND_20                 ((uint32_t)0x00100000)        /*!< bit 20 */
#define  NVIC_ISPR_SETPEND_21                 ((uint32_t)0x00200000)        /*!< bit 21 */
#define  NVIC_ISPR_SETPEND_22                 ((uint32_t)0x00400000)        /*!< bit 22 */
#define  NVIC_ISPR_SETPEND_23                 ((uint32_t)0x00800000)        /*!< bit 23 */
#define  NVIC_ISPR_SETPEND_24                 ((uint32_t)0x01000000)        /*!< bit 24 */
#define  NVIC_ISPR_SETPEND_25                 ((uint32_t)0x02000000)        /*!< bit 25 */
#define  NVIC_ISPR_SETPEND_26                 ((uint32_t)0x04000000)        /*!< bit 26 */
#define  NVIC_ISPR_SETPEND_27                 ((uint32_t)0x08000000)        /*!< bit 27 */
#define  NVIC_ISPR_SETPEND_28                 ((uint32_t)0x10000000)        /*!< bit 28 */
#define  NVIC_ISPR_SETPEND_29                 ((uint32_t)0x20000000)        /*!< bit 29 */
#define  NVIC_ISPR_SETPEND_30                 ((uint32_t)0x40000000)        /*!< bit 30 */
#define  NVIC_ISPR_SETPEND_31                 ((uint32_t)0x80000000)        /*!< bit 31 */

/******************  Bit definition for NVIC_ICPR register  *******************/
#define  NVIC_ICPR_CLRPEND                    ((uint32_t)0xFFFFFFFF)        /*!< Interrupt clear-pending bits */
#define  NVIC_ICPR_CLRPEND_0                  ((uint32_t)0x00000001)        /*!< bit 0 */
#define  NVIC_ICPR_CLRPEND_1                  ((uint32_t)0x00000002)        /*!< bit 1 */
#define  NVIC_ICPR_CLRPEND_2                  ((uint32_t)0x00000004)        /*!< bit 2 */
#define  NVIC_ICPR_CLRPEND_3                  ((uint32_t)0x00000008)        /*!< bit 3 */
#define  NVIC_ICPR_CLRPEND_4                  ((uint32_t)0x00000010)        /*!< bit 4 */
#define  NVIC_ICPR_CLRPEND_5                  ((uint32_t)0x00000020)        /*!< bit 5 */
#define  NVIC_ICPR_CLRPEND_6                  ((uint32_t)0x00000040)        /*!< bit 6 */
#define  NVIC_ICPR_CLRPEND_7                  ((uint32_t)0x00000080)        /*!< bit 7 */
#define  NVIC_ICPR_CLRPEND_8                  ((uint32_t)0x00000100)        /*!< bit 8 */
#define  NVIC_ICPR_CLRPEND_9                  ((uint32_t)0x00000200)        /*!< bit 9 */
#define  NVIC_ICPR_CLRPEND_10                 ((uint32_t)0x00000400)        /*!< bit 10 */
#define  NVIC_ICPR_CLRPEND_11                 ((uint32_t)0x00000800)        /*!< bit 11 */
#define  NVIC_ICPR_CLRPEND_12                 ((uint32_t)0x00001000)        /*!< bit 12 */
#define  NVIC_ICPR_CLRPEND_13                 ((uint32_t)0x00002000)        /*!< bit 13 */
#define  NVIC_ICPR_CLRPEND_14                 ((uint32_t)0x00004000)        /*!< bit 14 */
#define  NVIC_ICPR_CLRPEND_15                 ((uint32_t)0x00008000)        /*!< bit 15 */
#define  NVIC_ICPR_CLRPEND_16                 ((uint32_t)0x00010000)        /*!< bit 16 */
#define  NVIC_ICPR_CLRPEND_17                 ((uint32_t)0x00020000)        /*!< bit 17 */
#define  NVIC_ICPR_CLRPEND_18                 ((uint32_t)0x00040000)        /*!< bit 18 */
#define  NVIC_ICPR_CLRPEND_19                 ((uint32_t)0x00080000)        /*!< bit 19 */
#define  NVIC_ICPR_CLRPEND_20                 ((uint32_t)0x00100000)        /*!< bit 20 */
#define  NVIC_ICPR_CLRPEND_21                 ((uint32_t)0x00200000)        /*!< bit 21 */
#define  NVIC_ICPR_CLRPEND_22                 ((uint32_t)0x00400000)        /*!< bit 22 */
#define  NVIC_ICPR_CLRPEND_23                 ((uint32_t)0x00800000)        /*!< bit 23 */
#define  NVIC_ICPR_CLRPEND_24                 ((uint32_t)0x01000000)        /*!< bit 24 */
#define  NVIC_ICPR_CLRPEND_25                 ((uint32_t)0x02000000)        /*!< bit 25 */
#define  NVIC_ICPR_CLRPEND_26                 ((uint32_t)0x04000000)        /*!< bit 26 */
#define  NVIC_ICPR_CLRPEND_27                 ((uint32_t)0x08000000)        /*!< bit 27 */
#define  NVIC_ICPR_CLRPEND_28                 ((uint32_t)0x10000000)        /*!< bit 28 */
#define  NVIC_ICPR_CLRPEND_29                 ((uint32_t)0x20000000)        /*!< bit 29 */
#define  NVIC_ICPR_CLRPEND_30                 ((uint32_t)0x40000000)        /*!< bit 30 */
#define  NVIC_ICPR_CLRPEND_31                 ((uint32_t)0x80000000)        /*!< bit 31 */
```

```
/******************  Bit definition for NVIC_IABR register  *******************/
#define  NVIC_IABR_ACTIVE                    ((uint32_t)0xFFFFFFFF)        /*!< Interrupt active flags */
#define  NVIC_IABR_ACTIVE_0                  ((uint32_t)0x00000001)        /*!< bit 0 */
#define  NVIC_IABR_ACTIVE_1                  ((uint32_t)0x00000002)        /*!< bit 1 */
#define  NVIC_IABR_ACTIVE_2                  ((uint32_t)0x00000004)        /*!< bit 2 */
#define  NVIC_IABR_ACTIVE_3                  ((uint32_t)0x00000008)        /*!< bit 3 */
#define  NVIC_IABR_ACTIVE_4                  ((uint32_t)0x00000010)        /*!< bit 4 */
#define  NVIC_IABR_ACTIVE_5                  ((uint32_t)0x00000020)        /*!< bit 5 */
#define  NVIC_IABR_ACTIVE_6                  ((uint32_t)0x00000040)        /*!< bit 6 */
#define  NVIC_IABR_ACTIVE_7                  ((uint32_t)0x00000080)        /*!< bit 7 */
#define  NVIC_IABR_ACTIVE_8                  ((uint32_t)0x00000100)        /*!< bit 8 */
#define  NVIC_IABR_ACTIVE_9                  ((uint32_t)0x00000200)        /*!< bit 9 */
#define  NVIC_IABR_ACTIVE_10                 ((uint32_t)0x00000400)        /*!< bit 10 */
#define  NVIC_IABR_ACTIVE_11                 ((uint32_t)0x00000800)        /*!< bit 11 */
#define  NVIC_IABR_ACTIVE_12                 ((uint32_t)0x00001000)        /*!< bit 12 */
#define  NVIC_IABR_ACTIVE_13                 ((uint32_t)0x00002000)        /*!< bit 13 */
#define  NVIC_IABR_ACTIVE_14                 ((uint32_t)0x00004000)        /*!< bit 14 */
#define  NVIC_IABR_ACTIVE_15                 ((uint32_t)0x00008000)        /*!< bit 15 */
#define  NVIC_IABR_ACTIVE_16                 ((uint32_t)0x00010000)        /*!< bit 16 */
#define  NVIC_IABR_ACTIVE_17                 ((uint32_t)0x00020000)        /*!< bit 17 */
#define  NVIC_IABR_ACTIVE_18                 ((uint32_t)0x00040000)        /*!< bit 18 */
#define  NVIC_IABR_ACTIVE_19                 ((uint32_t)0x00080000)        /*!< bit 19 */
#define  NVIC_IABR_ACTIVE_20                 ((uint32_t)0x00100000)        /*!< bit 20 */
#define  NVIC_IABR_ACTIVE_21                 ((uint32_t)0x00200000)        /*!< bit 21 */
#define  NVIC_IABR_ACTIVE_22                 ((uint32_t)0x00400000)        /*!< bit 22 */
#define  NVIC_IABR_ACTIVE_23                 ((uint32_t)0x00800000)        /*!< bit 23 */
#define  NVIC_IABR_ACTIVE_24                 ((uint32_t)0x01000000)        /*!< bit 24 */
#define  NVIC_IABR_ACTIVE_25                 ((uint32_t)0x02000000)        /*!< bit 25 */
#define  NVIC_IABR_ACTIVE_26                 ((uint32_t)0x04000000)        /*!< bit 26 */
#define  NVIC_IABR_ACTIVE_27                 ((uint32_t)0x08000000)        /*!< bit 27 */
#define  NVIC_IABR_ACTIVE_28                 ((uint32_t)0x10000000)        /*!< bit 28 */
#define  NVIC_IABR_ACTIVE_29                 ((uint32_t)0x20000000)        /*!< bit 29 */
#define  NVIC_IABR_ACTIVE_30                 ((uint32_t)0x40000000)        /*!< bit 30 */
#define  NVIC_IABR_ACTIVE_31                 ((uint32_t)0x80000000)        /*!< bit 31 */

/******************  Bit definition for NVIC_PRI0 register  *******************/
#define  NVIC_IPR0_PRI_0                     ((uint32_t)0x000000FF)        /*!< Priority of interrupt 0 */
#define  NVIC_IPR0_PRI_1                     ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 1 */
#define  NVIC_IPR0_PRI_2                     ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 2 */
#define  NVIC_IPR0_PRI_3                     ((uint32_t)0xFF000000)        /*!< Priority of interrupt 3 */

/******************  Bit definition for NVIC_PRI1 register  *******************/
#define  NVIC_IPR1_PRI_4                     ((uint32_t)0x000000FF)        /*!< Priority of interrupt 4 */
#define  NVIC_IPR1_PRI_5                     ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 5 */
#define  NVIC_IPR1_PRI_6                     ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 6 */
#define  NVIC_IPR1_PRI_7                     ((uint32_t)0xFF000000)        /*!< Priority of interrupt 7 */

/******************  Bit definition for NVIC_PRI2 register  *******************/
#define  NVIC_IPR2_PRI_8                     ((uint32_t)0x000000FF)        /*!< Priority of interrupt 8 */
#define  NVIC_IPR2_PRI_9                     ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 9 */
#define  NVIC_IPR2_PRI_10                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 10 */
#define  NVIC_IPR2_PRI_11                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt 11 */

/******************  Bit definition for NVIC_PRI3 register  *******************/
#define  NVIC_IPR3_PRI_12                    ((uint32_t)0x000000FF)        /*!< Priority of interrupt 12 */
#define  NVIC_IPR3_PRI_13                    ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 13 */
#define  NVIC_IPR3_PRI_14                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 14 */
#define  NVIC_IPR3_PRI_15                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt
```

```
15 */

/******************  Bit definition for NVIC_PRI4 register  *******************/
#define  NVIC_IPR4_PRI_16                    ((uint32_t)0x000000FF)        /*!< Priority of interrupt 16 */
#define  NVIC_IPR4_PRI_17                    ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 17 */
#define  NVIC_IPR4_PRI_18                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 18 */
#define  NVIC_IPR4_PRI_19                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt 19 */

/******************  Bit definition for NVIC_PRI5 register  *******************/
#define  NVIC_IPR5_PRI_20                    ((uint32_t)0x000000FF)        /*!< Priority of interrupt 20 */
#define  NVIC_IPR5_PRI_21                    ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 21 */
#define  NVIC_IPR5_PRI_22                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 22 */
#define  NVIC_IPR5_PRI_23                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt 23 */

/******************  Bit definition for NVIC_PRI6 register  *******************/
#define  NVIC_IPR6_PRI_24                    ((uint32_t)0x000000FF)        /*!< Priority of interrupt 24 */
#define  NVIC_IPR6_PRI_25                    ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 25 */
#define  NVIC_IPR6_PRI_26                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 26 */
#define  NVIC_IPR6_PRI_27                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt 27 */

/******************  Bit definition for NVIC_PRI7 register  *******************/
#define  NVIC_IPR7_PRI_28                    ((uint32_t)0x000000FF)        /*!< Priority of interrupt 28 */
#define  NVIC_IPR7_PRI_29                    ((uint32_t)0x0000FF00)        /*!< Priority of interrupt 29 */
#define  NVIC_IPR7_PRI_30                    ((uint32_t)0x00FF0000)        /*!< Priority of interrupt 30 */
#define  NVIC_IPR7_PRI_31                    ((uint32_t)0xFF000000)        /*!< Priority of interrupt 31 */

/******************  Bit definition for SCB_CPUID register  *******************/
#define  SCB_CPUID_REVISION                  ((uint32_t)0x0000000F)        /*!< Implementation defined revision number */
#define  SCB_CPUID_PARTNO                    ((uint32_t)0x0000FFF0)        /*!< Number of processor within family */
#define  SCB_CPUID_Constant                  ((uint32_t)0x000F0000)        /*!< Reads as 0x0F */
#define  SCB_CPUID_VARIANT                   ((uint32_t)0x00F00000)        /*!< Implementation defined variant number */
#define  SCB_CPUID_IMPLEMENTER               ((uint32_t)0xFF000000)        /*!< Implementer code. ARM is 0x41 */

/*******************  Bit definition for SCB_ICSR register  *******************/
#define  SCB_ICSR_VECTACTIVE                 ((uint32_t)0x000001FF)        /*!< Active ISR number field */
#define  SCB_ICSR_RETTOBASE                  ((uint32_t)0x00000800)        /*!< All active exceptions minus the IPSR_current_exception yields the empty set */
#define  SCB_ICSR_VECTPENDING                ((uint32_t)0x003FF000)        /*!< Pending ISR number field */
#define  SCB_ICSR_ISRPENDING                 ((uint32_t)0x00400000)        /*!< Interrupt pending flag */
#define  SCB_ICSR_ISRPREEMPT                 ((uint32_t)0x00800000)        /*!< It indicates that a pending interrupt becomes active in the next running cycle */
#define  SCB_ICSR_PENDSTCLR                  ((uint32_t)0x02000000)        /*!< Clear pending SysTick bit */
#define  SCB_ICSR_PENDSTSET                  ((uint32_t)0x04000000)        /*!< Set pending SysTick bit */
#define  SCB_ICSR_PENDSVCLR                  ((uint32_t)0x08000000)        /*!< Clear pending pendSV bit */
#define  SCB_ICSR_PENDSVSET                  ((uint32_t)0x10000000)        /*!< Set pending pendSV bit */
#define  SCB_ICSR_NMIPENDSET                 ((uint32_t)0x80000000)        /*!< Set pending NMI bit */

/*******************  Bit definition for SCB_VTOR register  *******************/
```

```
#define  SCB_VTOR_TBLOFF                    ((uint32_t)0x1FFFFF80)        /*!< Vector table base offset field */
#define  SCB_VTOR_TBLBASE                   ((uint32_t)0x20000000)        /*!< Table base in code(0) or RAM(1) */

/*!<*****************  Bit definition for SCB_AIRCR register  *******************/
#define  SCB_AIRCR_VECTRESET                ((uint32_t)0x00000001)        /*!< System Reset bit */
#define  SCB_AIRCR_VECTCLRACTIVE            ((uint32_t)0x00000002)        /*!< Clear active vector bit */
#define  SCB_AIRCR_SYSRESETREQ              ((uint32_t)0x00000004)        /*!< Requests chip control logic to generate a reset */

#define  SCB_AIRCR_PRIGROUP                 ((uint32_t)0x00000700)        /*!< PRIGROUP[2:0] bits (Priority group) */
#define  SCB_AIRCR_PRIGROUP_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  SCB_AIRCR_PRIGROUP_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  SCB_AIRCR_PRIGROUP_2               ((uint32_t)0x00000400)        /*!< Bit 2  */

/* prority group configuration */
#define  SCB_AIRCR_PRIGROUP0                ((uint32_t)0x00000000)        /*!< Priority group=0 (7 bits of pre-emption priority, 1 bit of subpriority) */
#define  SCB_AIRCR_PRIGROUP1                ((uint32_t)0x00000100)        /*!< Priority group=1 (6 bits of pre-emption priority, 2 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP2                ((uint32_t)0x00000200)        /*!< Priority group=2 (5 bits of pre-emption priority, 3 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP3                ((uint32_t)0x00000300)        /*!< Priority group=3 (4 bits of pre-emption priority, 4 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP4                ((uint32_t)0x00000400)        /*!< Priority group=4 (3 bits of pre-emption priority, 5 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP5                ((uint32_t)0x00000500)        /*!< Priority group=5 (2 bits of pre-emption priority, 6 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP6                ((uint32_t)0x00000600)        /*!< Priority group=6 (1 bit of pre-emption priority, 7 bits of subpriority) */
#define  SCB_AIRCR_PRIGROUP7                ((uint32_t)0x00000700)        /*!< Priority group=7 (no pre-emption priority, 8 bits of subpriority) */

#define  SCB_AIRCR_ENDIANESS                ((uint32_t)0x00008000)        /*!< Data endianness bit */
#define  SCB_AIRCR_VECTKEY                  ((uint32_t)0xFFFF0000)        /*!< Register key (VECTKEY) - Reads as 0xFA05 (VECTKEYSTAT) */

/*******************  Bit definition for SCB_SCR register  ********************/
#define  SCB_SCR_SLEEPONEXIT                ((uint8_t)0x02)               /*!< Sleep on exit bit */
#define  SCB_SCR_SLEEPDEEP                  ((uint8_t)0x04)               /*!< Sleep deep bit */
#define  SCB_SCR_SEVONPEND                  ((uint8_t)0x10)               /*!< Wake up from WFE */

/********************  Bit definition for SCB_CCR register  *******************/
#define  SCB_CCR_NONBASETHRDENA             ((uint16_t)0x0001)            /*!< Thread mode can be entered from any level in Handler mode by controlled return value */
#define  SCB_CCR_USERSETMPEND               ((uint16_t)0x0002)            /*!< Enables user code to write the Software Trigger Interrupt register to trigger (pend) a Main exception */
#define  SCB_CCR_UNALIGN_TRP                ((uint16_t)0x0008)            /*!< Trap for unaligned access */
#define  SCB_CCR_DIV_0_TRP                  ((uint16_t)0x0010)            /*!< Trap on Divide by 0 */
#define  SCB_CCR_BFHFNMIGN                  ((uint16_t)0x0100)            /*!< Handlers running at priority -1 and -2 */
#define  SCB_CCR_STKALIGN                   ((uint16_t)0x0200)            /*!< On exception entry, the SP used prior to the exception is adjusted to be 8-byte aligned */

/*******************  Bit definition for SCB_SHPR register ********************/
#define  SCB_SHPR_PRI_N                     ((uint32_t)0x000000FF)        /*!< Priority of system handler 4,8, and 12. Mem Manage, reserved and Debug Monitor */
#define  SCB_SHPR_PRI_N1                    ((uint32_t)0x0000FF00)        /*!< Priority of system handler 5,9, and 13. Bus Fault, reserved and reserved */
#define  SCB_SHPR_PRI_N2                    ((uint32_t)0x00FF0000)        /*!< Priority of system handler 6,10, and 14. Usage Fault, reserved and PendSV */
#define  SCB_SHPR_PRI_N3                    ((uint32_t)0xFF000000)        /*!< Priority of system handler 7,11, and 15. Reserved, SVCall and SysTick */

/******************  Bit definition for SCB_SHCSR register  ******************/
#define  SCB_SHCSR_MEMFAULTACT              ((uint32_t)0x00000001)        /*!< MemManage is active */
#define  SCB_SHCSR_BUSFAULTACT              ((uint32_t)0x00000002)        /*!< BusFault is active */
#define  SCB_SHCSR_USGFAULTACT              ((uint32_t)0x00000008)        /*!< UsageFault is active */
#define  SCB_SHCSR_SVCALLACT                ((uint32_t)0x00000080)        /*!< SVCall is active */
```

```
#define  SCB_SHCSR_MONITORACT                ((uint32_t)0x00000100)        /*!< Monitor is active */
#define  SCB_SHCSR_PENDSVACT                 ((uint32_t)0x00000400)        /*!< PendSV is active */
#define  SCB_SHCSR_SYSTICKACT                ((uint32_t)0x00000800)        /*!< SysTick is active */
#define  SCB_SHCSR_USGFAULTPENDED            ((uint32_t)0x00001000)        /*!< Usage Fault is pended */
#define  SCB_SHCSR_MEMFAULTPENDED            ((uint32_t)0x00002000)        /*!< MemManage is pended */
#define  SCB_SHCSR_BUSFAULTPENDED            ((uint32_t)0x00004000)        /*!< Bus Fault is pended */
#define  SCB_SHCSR_SVCALLPENDED              ((uint32_t)0x00008000)        /*!< SVCall is pended */
#define  SCB_SHCSR_MEMFAULTENA               ((uint32_t)0x00010000)        /*!< MemManage enable */
#define  SCB_SHCSR_BUSFAULTENA               ((uint32_t)0x00020000)        /*!< Bus Fault enable */
#define  SCB_SHCSR_USGFAULTENA               ((uint32_t)0x00040000)        /*!< UsageFault enable */

/*******************  Bit definition for SCB_CFSR register  *******************/
/*!< MFSR */
#define  SCB_CFSR_IACCVIOL                   ((uint32_t)0x00000001)        /*!< Instruction access violation */
#define  SCB_CFSR_DACCVIOL                   ((uint32_t)0x00000002)        /*!< Data access violation */
#define  SCB_CFSR_MUNSTKERR                  ((uint32_t)0x00000008)        /*!< Unstacking error */
#define  SCB_CFSR_MSTKERR                    ((uint32_t)0x00000010)        /*!< Stacking error */
#define  SCB_CFSR_MMARVALID                  ((uint32_t)0x00000080)        /*!< Memory Manage Address Register address valid flag */
/*!< BFSR */
#define  SCB_CFSR_IBUSERR                    ((uint32_t)0x00000100)        /*!< Instruction bus error flag */
#define  SCB_CFSR_PRECISERR                  ((uint32_t)0x00000200)        /*!< Precise data bus error */
#define  SCB_CFSR_IMPRECISERR                ((uint32_t)0x00000400)        /*!< Imprecise data bus error */
#define  SCB_CFSR_UNSTKERR                   ((uint32_t)0x00000800)        /*!< Unstacking error */
#define  SCB_CFSR_STKERR                     ((uint32_t)0x00001000)        /*!< Stacking error */
#define  SCB_CFSR_BFARVALID                  ((uint32_t)0x00008000)        /*!< Bus Fault Address Register address valid flag */
/*!< UFSR */
#define  SCB_CFSR_UNDEFINSTR                 ((uint32_t)0x00010000)        /*!< The processor attempt to execute an undefined instruction */
#define  SCB_CFSR_INVSTATE                   ((uint32_t)0x00020000)        /*!< Invalid combination of EPSR and instruction */
#define  SCB_CFSR_INVPC                      ((uint32_t)0x00040000)        /*!< Attempt to load EXC_RETURN into pc illegally */
#define  SCB_CFSR_NOCP                       ((uint32_t)0x00080000)        /*!< Attempt to use a coprocessor instruction */
#define  SCB_CFSR_UNALIGNED                  ((uint32_t)0x01000000)        /*!< Fault occurs when there is an attempt to make an unaligned memory access */
#define  SCB_CFSR_DIVBYZERO                  ((uint32_t)0x02000000)        /*!< Fault occurs when SDIV or DIV instruction is used with a divisor of 0 */

/*******************  Bit definition for SCB_HFSR register  *******************/
#define  SCB_HFSR_VECTTBL                    ((uint32_t)0x00000002)        /*!< Fault occurs because of vector table read on exception processing */
#define  SCB_HFSR_FORCED                     ((uint32_t)0x40000000)        /*!< Hard Fault activated when a configurable Fault was received and cannot activate */
#define  SCB_HFSR_DEBUGEVT                   ((uint32_t)0x80000000)        /*!< Fault related to debug */

/*******************  Bit definition for SCB_DFSR register  *******************/
#define  SCB_DFSR_HALTED                     ((uint8_t)0x01)               /*!< Halt request flag */
#define  SCB_DFSR_BKPT                       ((uint8_t)0x02)               /*!< BKPT flag */
#define  SCB_DFSR_DWTTRAP                    ((uint8_t)0x04)               /*!< Data Watchpoint and Trace (DWT) flag */
#define  SCB_DFSR_VCATCH                     ((uint8_t)0x08)               /*!< Vector catch flag */
#define  SCB_DFSR_EXTERNAL                   ((uint8_t)0x10)               /*!< External debug request flag */

/*******************  Bit definition for SCB_MMFAR register  ******************/
#define  SCB_MMFAR_ADDRESS                   ((uint32_t)0xFFFFFFFF)        /*!< Mem Manage fault address field */

/*******************  Bit definition for SCB_BFAR register  *******************/
#define  SCB_BFAR_ADDRESS                    ((uint32_t)0xFFFFFFFF)        /*!< Bus fault address field */

/*******************  Bit definition for SCB_afsr register  *******************/
#define  SCB_AFSR_IMPDEF                     ((uint32_t)0xFFFFFFFF)        /*!< Implementation defined */

/******************************************************************************/
/*                                                                            */
```

```
/*                    External Interrupt/Event Controller                    */
/*                                                                           */
/*****************************************************************************/

/*******************  Bit definition for EXTI_IMR register  *******************/
#define  EXTI_IMR_MR0                        ((uint32_t)0x00000001)        /*!< Interrupt Mask on line 0 */
#define  EXTI_IMR_MR1                        ((uint32_t)0x00000002)        /*!< Interrupt Mask on line 1 */
#define  EXTI_IMR_MR2                        ((uint32_t)0x00000004)        /*!< Interrupt Mask on line 2 */
#define  EXTI_IMR_MR3                        ((uint32_t)0x00000008)        /*!< Interrupt Mask on line 3 */
#define  EXTI_IMR_MR4                        ((uint32_t)0x00000010)        /*!< Interrupt Mask on line 4 */
#define  EXTI_IMR_MR5                        ((uint32_t)0x00000020)        /*!< Interrupt Mask on line 5 */
#define  EXTI_IMR_MR6                        ((uint32_t)0x00000040)        /*!< Interrupt Mask on line 6 */
#define  EXTI_IMR_MR7                        ((uint32_t)0x00000080)        /*!< Interrupt Mask on line 7 */
#define  EXTI_IMR_MR8                        ((uint32_t)0x00000100)        /*!< Interrupt Mask on line 8 */
#define  EXTI_IMR_MR9                        ((uint32_t)0x00000200)        /*!< Interrupt Mask on line 9 */
#define  EXTI_IMR_MR10                       ((uint32_t)0x00000400)        /*!< Interrupt Mask on line 10 */
#define  EXTI_IMR_MR11                       ((uint32_t)0x00000800)        /*!< Interrupt Mask on line 11 */
#define  EXTI_IMR_MR12                       ((uint32_t)0x00001000)        /*!< Interrupt Mask on line 12 */
#define  EXTI_IMR_MR13                       ((uint32_t)0x00002000)        /*!< Interrupt Mask on line 13 */
#define  EXTI_IMR_MR14                       ((uint32_t)0x00004000)        /*!< Interrupt Mask on line 14 */
#define  EXTI_IMR_MR15                       ((uint32_t)0x00008000)        /*!< Interrupt Mask on line 15 */
#define  EXTI_IMR_MR16                       ((uint32_t)0x00010000)        /*!< Interrupt Mask on line 16 */
#define  EXTI_IMR_MR17                       ((uint32_t)0x00020000)        /*!< Interrupt Mask on line 17 */
#define  EXTI_IMR_MR18                       ((uint32_t)0x00040000)        /*!< Interrupt Mask on line 18 */
#define  EXTI_IMR_MR19                       ((uint32_t)0x00080000)        /*!< Interrupt Mask on line 19 */

/*******************  Bit definition for EXTI_EMR register  *******************/
#define  EXTI_EMR_MR0                        ((uint32_t)0x00000001)        /*!< Event Mask on line 0 */
#define  EXTI_EMR_MR1                        ((uint32_t)0x00000002)        /*!< Event Mask on line 1 */
#define  EXTI_EMR_MR2                        ((uint32_t)0x00000004)        /*!< Event Mask on line 2 */
#define  EXTI_EMR_MR3                        ((uint32_t)0x00000008)        /*!< Event Mask on line 3 */
#define  EXTI_EMR_MR4                        ((uint32_t)0x00000010)        /*!< Event Mask on line 4 */
#define  EXTI_EMR_MR5                        ((uint32_t)0x00000020)        /*!< Event Mask on line 5 */
#define  EXTI_EMR_MR6                        ((uint32_t)0x00000040)        /*!< Event Mask on line 6 */
#define  EXTI_EMR_MR7                        ((uint32_t)0x00000080)        /*!< Event Mask on line 7 */
#define  EXTI_EMR_MR8                        ((uint32_t)0x00000100)        /*!< Event Mask on line 8 */
#define  EXTI_EMR_MR9                        ((uint32_t)0x00000200)        /*!< Event Mask on line 9 */
#define  EXTI_EMR_MR10                       ((uint32_t)0x00000400)        /*!< Event Mask on line 10 */
#define  EXTI_EMR_MR11                       ((uint32_t)0x00000800)        /*!< Event Mask on line 11 */
#define  EXTI_EMR_MR12                       ((uint32_t)0x00001000)        /*!< Event Mask on line 12 */
#define  EXTI_EMR_MR13                       ((uint32_t)0x00002000)        /*!< Event Mask on line 13 */
#define  EXTI_EMR_MR14                       ((uint32_t)0x00004000)        /*!< Event Mask on line 14 */
#define  EXTI_EMR_MR15                       ((uint32_t)0x00008000)        /*!< Event Mask on line 15 */
#define  EXTI_EMR_MR16                       ((uint32_t)0x00010000)        /*!< Event Mask on line 16 */
#define  EXTI_EMR_MR17                       ((uint32_t)0x00020000)        /*!< Event Mask on line 17 */
#define  EXTI_EMR_MR18                       ((uint32_t)0x00040000)        /*!< Event Mask on line 18 */
#define  EXTI_EMR_MR19                       ((uint32_t)0x00080000)        /*!< Event Mask on line 19 */

/******************  Bit definition for EXTI_RTSR register  ******************/
#define  EXTI_RTSR_TR0                       ((uint32_t)0x00000001)        /*!< Rising trigger event configuration bit of line 0 */
#define  EXTI_RTSR_TR1                       ((uint32_t)0x00000002)        /*!< Rising trigger event configuration bit of line 1 */
```

```
#define  EXTI_RTSR_TR2                       ((uint32_t)0x00000004)        /*!< Rising trigger event configuration bit of line 2 */
#define  EXTI_RTSR_TR3                       ((uint32_t)0x00000008)        /*!< Rising trigger event configuration bit of line 3 */
#define  EXTI_RTSR_TR4                       ((uint32_t)0x00000010)        /*!< Rising trigger event configuration bit of line 4 */
#define  EXTI_RTSR_TR5                       ((uint32_t)0x00000020)        /*!< Rising trigger event configuration bit of line 5 */
#define  EXTI_RTSR_TR6                       ((uint32_t)0x00000040)        /*!< Rising trigger event configuration bit of line 6 */
#define  EXTI_RTSR_TR7                       ((uint32_t)0x00000080)        /*!< Rising trigger event configuration bit of line 7 */
#define  EXTI_RTSR_TR8                       ((uint32_t)0x00000100)        /*!< Rising trigger event configuration bit of line 8 */
#define  EXTI_RTSR_TR9                       ((uint32_t)0x00000200)        /*!< Rising trigger event configuration bit of line 9 */
#define  EXTI_RTSR_TR10                      ((uint32_t)0x00000400)        /*!< Rising trigger event configuration bit of line 10 */
#define  EXTI_RTSR_TR11                      ((uint32_t)0x00000800)        /*!< Rising trigger event configuration bit of line 11 */
#define  EXTI_RTSR_TR12                      ((uint32_t)0x00001000)        /*!< Rising trigger event configuration bit of line 12 */
#define  EXTI_RTSR_TR13                      ((uint32_t)0x00002000)        /*!< Rising trigger event configuration bit of line 13 */
#define  EXTI_RTSR_TR14                      ((uint32_t)0x00004000)        /*!< Rising trigger event configuration bit of line 14 */
#define  EXTI_RTSR_TR15                      ((uint32_t)0x00008000)        /*!< Rising trigger event configuration bit of line 15 */
#define  EXTI_RTSR_TR16                      ((uint32_t)0x00010000)        /*!< Rising trigger event configuration bit of line 16 */
#define  EXTI_RTSR_TR17                      ((uint32_t)0x00020000)        /*!< Rising trigger event configuration bit of line 17 */
#define  EXTI_RTSR_TR18                      ((uint32_t)0x00040000)        /*!< Rising trigger event configuration bit of line 18 */
#define  EXTI_RTSR_TR19                      ((uint32_t)0x00080000)        /*!< Rising trigger event configuration bit of line 19 */

/******************  Bit definition for EXTI_FTSR register  ******************/
#define  EXTI_FTSR_TR0                       ((uint32_t)0x00000001)        /*!< Falling trigger event configuration bit of line 0 */
#define  EXTI_FTSR_TR1                       ((uint32_t)0x00000002)        /*!< Falling trigger event configuration bit of line 1 */
#define  EXTI_FTSR_TR2                       ((uint32_t)0x00000004)        /*!< Falling trigger event configuration bit of line 2 */
#define  EXTI_FTSR_TR3                       ((uint32_t)0x00000008)        /*!< Falling trigger event configuration bit of line 3 */
#define  EXTI_FTSR_TR4                       ((uint32_t)0x00000010)        /*!< Falling trigger event configuration bit of line 4 */
#define  EXTI_FTSR_TR5                       ((uint32_t)0x00000020)        /*!< Falling trigger event configuration bit of line 5 */
#define  EXTI_FTSR_TR6                       ((uint32_t)0x00000040)        /*!< Falling trigger event configuration bit of line 6 */
#define  EXTI_FTSR_TR7                       ((uint32_t)0x00000080)        /*!< Falling trigger event configuration bit of line 7 */
#define  EXTI_FTSR_TR8                       ((uint32_t)0x00000100)        /*!< Falling trigger event configuration bit of line 8 */
#define  EXTI_FTSR_TR9                       ((uint32_t)0x00000200)        /*!< Falling trigger event configuration bit of line 9 */
#define  EXTI_FTSR_TR10                      ((uint32_t)0x00000400)        /*!< Falling trigger event configuration bit of line 10 */
#define  EXTI_FTSR_TR11                      ((uint32_t)0x00000800)        /*!< Falling trigger event configuration bit of line 11 */
#define  EXTI_FTSR_TR12                      ((uint32_t)0x00001000)        /*!< Falling trigger event configuration bit of line 12 */
#define  EXTI_FTSR_TR13                      ((uint32_t)0x00002000)        /*!< Falling trigger event configuration bit of line 13 */
#define  EXTI_FTSR_TR14                      ((uint32_t)0x00004000)        /*!< Falling trigger event configuration bit of line 14 */
#define  EXTI_FTSR_TR15                      ((uint32_t)0x00008000)        /*!< Falling trigger event configuration bit of line 15 */
#define  EXTI_FTSR_TR16                      ((uint32_t)0x00010000)        /*!< Falling trigger event configuration bit of line 16 */
#define  EXTI_FTSR_TR17                      ((uint32_t)0x00020000)        /*!< Falling trigger event
```

```
configuration bit of line 17 */
#define  EXTI_FTSR_TR18                      ((uint32_t)0x00040000)        /*!< Falling trigger event configuration bit of line 18 */
#define  EXTI_FTSR_TR19                      ((uint32_t)0x00080000)        /*!< Falling trigger event configuration bit of line 19 */

/******************  Bit definition for EXTI_SWIER register  ******************/
#define  EXTI_SWIER_SWIER0                   ((uint32_t)0x00000001)        /*!< Software Interrupt on line 0 */
#define  EXTI_SWIER_SWIER1                   ((uint32_t)0x00000002)        /*!< Software Interrupt on line 1 */
#define  EXTI_SWIER_SWIER2                   ((uint32_t)0x00000004)        /*!< Software Interrupt on line 2 */
#define  EXTI_SWIER_SWIER3                   ((uint32_t)0x00000008)        /*!< Software Interrupt on line 3 */
#define  EXTI_SWIER_SWIER4                   ((uint32_t)0x00000010)        /*!< Software Interrupt on line 4 */
#define  EXTI_SWIER_SWIER5                   ((uint32_t)0x00000020)        /*!< Software Interrupt on line 5 */
#define  EXTI_SWIER_SWIER6                   ((uint32_t)0x00000040)        /*!< Software Interrupt on line 6 */
#define  EXTI_SWIER_SWIER7                   ((uint32_t)0x00000080)        /*!< Software Interrupt on line 7 */
#define  EXTI_SWIER_SWIER8                   ((uint32_t)0x00000100)        /*!< Software Interrupt on line 8 */
#define  EXTI_SWIER_SWIER9                   ((uint32_t)0x00000200)        /*!< Software Interrupt on line 9 */
#define  EXTI_SWIER_SWIER10                  ((uint32_t)0x00000400)        /*!< Software Interrupt on line 10 */
#define  EXTI_SWIER_SWIER11                  ((uint32_t)0x00000800)        /*!< Software Interrupt on line 11 */
#define  EXTI_SWIER_SWIER12                  ((uint32_t)0x00001000)        /*!< Software Interrupt on line 12 */
#define  EXTI_SWIER_SWIER13                  ((uint32_t)0x00002000)        /*!< Software Interrupt on line 13 */
#define  EXTI_SWIER_SWIER14                  ((uint32_t)0x00004000)        /*!< Software Interrupt on line 14 */
#define  EXTI_SWIER_SWIER15                  ((uint32_t)0x00008000)        /*!< Software Interrupt on line 15 */
#define  EXTI_SWIER_SWIER16                  ((uint32_t)0x00010000)        /*!< Software Interrupt on line 16 */
#define  EXTI_SWIER_SWIER17                  ((uint32_t)0x00020000)        /*!< Software Interrupt on line 17 */
#define  EXTI_SWIER_SWIER18                  ((uint32_t)0x00040000)        /*!< Software Interrupt on line 18 */
#define  EXTI_SWIER_SWIER19                  ((uint32_t)0x00080000)        /*!< Software Interrupt on line 19 */

/*******************  Bit definition for EXTI_PR register  ********************/
#define  EXTI_PR_PR0                         ((uint32_t)0x00000001)        /*!< Pending bit for line 0 */
#define  EXTI_PR_PR1                         ((uint32_t)0x00000002)        /*!< Pending bit for line 1 */
#define  EXTI_PR_PR2                         ((uint32_t)0x00000004)        /*!< Pending bit for line 2 */
#define  EXTI_PR_PR3                         ((uint32_t)0x00000008)        /*!< Pending bit for line 3 */
#define  EXTI_PR_PR4                         ((uint32_t)0x00000010)        /*!< Pending bit for line 4 */
#define  EXTI_PR_PR5                         ((uint32_t)0x00000020)        /*!< Pending bit for line 5 */
#define  EXTI_PR_PR6                         ((uint32_t)0x00000040)        /*!< Pending bit for line 6 */
#define  EXTI_PR_PR7                         ((uint32_t)0x00000080)        /*!< Pending bit for line 7 */
#define  EXTI_PR_PR8                         ((uint32_t)0x00000100)        /*!< Pending bit for line 8 */
#define  EXTI_PR_PR9                         ((uint32_t)0x00000200)        /*!< Pending bit for line 9 */
#define  EXTI_PR_PR10                        ((uint32_t)0x00000400)        /*!< Pending bit for line 10 */
#define  EXTI_PR_PR11                        ((uint32_t)0x00000800)        /*!< Pending bit for line 11 */
#define  EXTI_PR_PR12                        ((uint32_t)0x00001000)        /*!< Pending bit for line 12 */
#define  EXTI_PR_PR13                        ((uint32_t)0x00002000)        /*!< Pending bit for line 13 */
#define  EXTI_PR_PR14                        ((uint32_t)0x00004000)        /*!< Pending bit for line 14 */
#define  EXTI_PR_PR15                        ((uint32_t)0x00008000)        /*!< Pending bit for line 15 */
#define  EXTI_PR_PR16                        ((uint32_t)0x00010000)        /*!< Pending bit for line 16 */
```

```
#define  EXTI_PR_PR17                        ((uint32_t)0x00020000)        /*!< Pending bit for line 17 */
#define  EXTI_PR_PR18                        ((uint32_t)0x00040000)        /*!< Pending bit for line 18 */
#define  EXTI_PR_PR19                        ((uint32_t)0x00080000)        /*!< Pending bit for line 19 */

/******************************************************************************/
/*                                                                            */
/*                              DMA Controller                                */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for DMA_ISR register  *******************/
#define  DMA_ISR_GIF1                        ((uint32_t)0x00000001)        /*!< Channel 1 Global interrupt flag */
#define  DMA_ISR_TCIF1                       ((uint32_t)0x00000002)        /*!< Channel 1 Transfer Complete flag */
#define  DMA_ISR_HTIF1                       ((uint32_t)0x00000004)        /*!< Channel 1 Half Transfer flag */
#define  DMA_ISR_TEIF1                       ((uint32_t)0x00000008)        /*!< Channel 1 Transfer Error flag */
#define  DMA_ISR_GIF2                        ((uint32_t)0x00000010)        /*!< Channel 2 Global interrupt flag */
#define  DMA_ISR_TCIF2                       ((uint32_t)0x00000020)        /*!< Channel 2 Transfer Complete flag */
#define  DMA_ISR_HTIF2                       ((uint32_t)0x00000040)        /*!< Channel 2 Half Transfer flag */
#define  DMA_ISR_TEIF2                       ((uint32_t)0x00000080)        /*!< Channel 2 Transfer Error flag */
#define  DMA_ISR_GIF3                        ((uint32_t)0x00000100)        /*!< Channel 3 Global interrupt flag */
#define  DMA_ISR_TCIF3                       ((uint32_t)0x00000200)        /*!< Channel 3 Transfer Complete flag */
#define  DMA_ISR_HTIF3                       ((uint32_t)0x00000400)        /*!< Channel 3 Half Transfer flag */
#define  DMA_ISR_TEIF3                       ((uint32_t)0x00000800)        /*!< Channel 3 Transfer Error flag */
#define  DMA_ISR_GIF4                        ((uint32_t)0x00001000)        /*!< Channel 4 Global interrupt flag */
#define  DMA_ISR_TCIF4                       ((uint32_t)0x00002000)        /*!< Channel 4 Transfer Complete flag */
#define  DMA_ISR_HTIF4                       ((uint32_t)0x00004000)        /*!< Channel 4 Half Transfer flag */
#define  DMA_ISR_TEIF4                       ((uint32_t)0x00008000)        /*!< Channel 4 Transfer Error flag */
#define  DMA_ISR_GIF5                        ((uint32_t)0x00010000)        /*!< Channel 5 Global interrupt flag */
#define  DMA_ISR_TCIF5                       ((uint32_t)0x00020000)        /*!< Channel 5 Transfer Complete flag */
#define  DMA_ISR_HTIF5                       ((uint32_t)0x00040000)        /*!< Channel 5 Half Transfer flag */
#define  DMA_ISR_TEIF5                       ((uint32_t)0x00080000)        /*!< Channel 5 Transfer Error flag */
#define  DMA_ISR_GIF6                        ((uint32_t)0x00100000)        /*!< Channel 6 Global interrupt flag */
#define  DMA_ISR_TCIF6                       ((uint32_t)0x00200000)        /*!< Channel 6 Transfer Complete flag */
#define  DMA_ISR_HTIF6                       ((uint32_t)0x00400000)        /*!< Channel 6 Half Transfer flag */
#define  DMA_ISR_TEIF6                       ((uint32_t)0x00800000)        /*!< Channel 6 Transfer Error flag */
#define  DMA_ISR_GIF7                        ((uint32_t)0x01000000)        /*!< Channel 7 Global interrupt flag */
#define  DMA_ISR_TCIF7                       ((uint32_t)0x02000000)        /*!< Channel 7 Transfer Complete flag */
#define  DMA_ISR_HTIF7                       ((uint32_t)0x04000000)        /*!< Channel 7 Half Transfer flag */
#define  DMA_ISR_TEIF7                       ((uint32_t)0x08000000)        /*!< Channel 7 Transfer Error flag */

/*******************  Bit definition for DMA_IFCR register  *******************/
#define  DMA_IFCR_CGIF1                      ((uint32_t)0x00000001)        /*!< Channel 1 Global
```

```
interrupt clear */
#define  DMA_IFCR_CTCIF1                    ((uint32_t)0x00000002)        /*!< Channel 1 Transfer Complete clear */
#define  DMA_IFCR_CHTIF1                    ((uint32_t)0x00000004)        /*!< Channel 1 Half Transfer clear */
#define  DMA_IFCR_CTEIF1                    ((uint32_t)0x00000008)        /*!< Channel 1 Transfer Error clear */
#define  DMA_IFCR_CGIF2                     ((uint32_t)0x00000010)        /*!< Channel 2 Global interrupt clear */
#define  DMA_IFCR_CTCIF2                    ((uint32_t)0x00000020)        /*!< Channel 2 Transfer Complete clear */
#define  DMA_IFCR_CHTIF2                    ((uint32_t)0x00000040)        /*!< Channel 2 Half Transfer clear */
#define  DMA_IFCR_CTEIF2                    ((uint32_t)0x00000080)        /*!< Channel 2 Transfer Error clear */
#define  DMA_IFCR_CGIF3                     ((uint32_t)0x00000100)        /*!< Channel 3 Global interrupt clear */
#define  DMA_IFCR_CTCIF3                    ((uint32_t)0x00000200)        /*!< Channel 3 Transfer Complete clear */
#define  DMA_IFCR_CHTIF3                    ((uint32_t)0x00000400)        /*!< Channel 3 Half Transfer clear */
#define  DMA_IFCR_CTEIF3                    ((uint32_t)0x00000800)        /*!< Channel 3 Transfer Error clear */
#define  DMA_IFCR_CGIF4                     ((uint32_t)0x00001000)        /*!< Channel 4 Global interrupt clear */
#define  DMA_IFCR_CTCIF4                    ((uint32_t)0x00002000)        /*!< Channel 4 Transfer Complete clear */
#define  DMA_IFCR_CHTIF4                    ((uint32_t)0x00004000)        /*!< Channel 4 Half Transfer clear */
#define  DMA_IFCR_CTEIF4                    ((uint32_t)0x00008000)        /*!< Channel 4 Transfer Error clear */
#define  DMA_IFCR_CGIF5                     ((uint32_t)0x00010000)        /*!< Channel 5 Global interrupt clear */
#define  DMA_IFCR_CTCIF5                    ((uint32_t)0x00020000)        /*!< Channel 5 Transfer Complete clear */
#define  DMA_IFCR_CHTIF5                    ((uint32_t)0x00040000)        /*!< Channel 5 Half Transfer clear */
#define  DMA_IFCR_CTEIF5                    ((uint32_t)0x00080000)        /*!< Channel 5 Transfer Error clear */
#define  DMA_IFCR_CGIF6                     ((uint32_t)0x00100000)        /*!< Channel 6 Global interrupt clear */
#define  DMA_IFCR_CTCIF6                    ((uint32_t)0x00200000)        /*!< Channel 6 Transfer Complete clear */
#define  DMA_IFCR_CHTIF6                    ((uint32_t)0x00400000)        /*!< Channel 6 Half Transfer clear */
#define  DMA_IFCR_CTEIF6                    ((uint32_t)0x00800000)        /*!< Channel 6 Transfer Error clear */
#define  DMA_IFCR_CGIF7                     ((uint32_t)0x01000000)        /*!< Channel 7 Global interrupt clear */
#define  DMA_IFCR_CTCIF7                    ((uint32_t)0x02000000)        /*!< Channel 7 Transfer Complete clear */
#define  DMA_IFCR_CHTIF7                    ((uint32_t)0x04000000)        /*!< Channel 7 Half Transfer clear */
#define  DMA_IFCR_CTEIF7                    ((uint32_t)0x08000000)        /*!< Channel 7 Transfer Error clear */

/*******************  Bit definition for DMA_CCR1 register  *******************/
#define  DMA_CCR1_EN                        ((uint16_t)0x0001)            /*!< Channel enable*/
#define  DMA_CCR1_TCIE                      ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR1_HTIE                      ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR1_TEIE                      ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR1_DIR                       ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR1_CIRC                      ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR1_PINC                      ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR1_MINC                      ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR1_PSIZE                     ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
```

```
#define  DMA_CCR1_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR1_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR1_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR1_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR1_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR1_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits(Channel Priority level) */
#define  DMA_CCR1_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR1_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR1_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode */

/*******************  Bit definition for DMA_CCR2 register  *******************/
#define  DMA_CCR2_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
#define  DMA_CCR2_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR2_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR2_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR2_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR2_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR2_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR2_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR2_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR2_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR2_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR2_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR2_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR2_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR2_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel Priority level) */
#define  DMA_CCR2_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR2_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR2_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode */

/*******************  Bit definition for DMA_CCR3 register  *******************/
#define  DMA_CCR3_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
#define  DMA_CCR3_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR3_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR3_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR3_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR3_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR3_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR3_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR3_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR3_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR3_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR3_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR3_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR3_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR3_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel
```

```
Priority level) */
#define  DMA_CCR3_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR3_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR3_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode */

/*!<******************  Bit definition for DMA_CCR4 register  *******************/
#define  DMA_CCR4_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
#define  DMA_CCR4_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR4_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR4_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR4_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR4_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR4_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR4_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR4_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR4_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR4_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR4_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR4_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR4_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR4_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel Priority level) */
#define  DMA_CCR4_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR4_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR4_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode */

/******************  Bit definition for DMA_CCR5 register  *******************/
#define  DMA_CCR5_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
#define  DMA_CCR5_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR5_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR5_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR5_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR5_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR5_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR5_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR5_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR5_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR5_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR5_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR5_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR5_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR5_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel Priority level) */
#define  DMA_CCR5_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR5_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR5_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode enable */

/******************  Bit definition for DMA_CCR6 register  *******************/
#define  DMA_CCR6_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
```

```
#define  DMA_CCR6_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR6_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR6_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR6_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR6_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR6_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR6_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR6_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR6_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR6_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR6_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR6_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR6_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR6_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel Priority level) */
#define  DMA_CCR6_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR6_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR6_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode */

/*******************  Bit definition for DMA_CCR7 register  *******************/
#define  DMA_CCR7_EN                         ((uint16_t)0x0001)            /*!< Channel enable */
#define  DMA_CCR7_TCIE                       ((uint16_t)0x0002)            /*!< Transfer complete interrupt enable */
#define  DMA_CCR7_HTIE                       ((uint16_t)0x0004)            /*!< Half Transfer interrupt enable */
#define  DMA_CCR7_TEIE                       ((uint16_t)0x0008)            /*!< Transfer error interrupt enable */
#define  DMA_CCR7_DIR                        ((uint16_t)0x0010)            /*!< Data transfer direction */
#define  DMA_CCR7_CIRC                       ((uint16_t)0x0020)            /*!< Circular mode */
#define  DMA_CCR7_PINC                       ((uint16_t)0x0040)            /*!< Peripheral increment mode */
#define  DMA_CCR7_MINC                       ((uint16_t)0x0080)            /*!< Memory increment mode */

#define  DMA_CCR7_PSIZE                      ((uint16_t)0x0300)            /*!< PSIZE[1:0] bits (Peripheral size) */
#define  DMA_CCR7_PSIZE_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  DMA_CCR7_PSIZE_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  DMA_CCR7_MSIZE                      ((uint16_t)0x0C00)            /*!< MSIZE[1:0] bits (Memory size) */
#define  DMA_CCR7_MSIZE_0                    ((uint16_t)0x0400)            /*!< Bit 0 */
#define  DMA_CCR7_MSIZE_1                    ((uint16_t)0x0800)            /*!< Bit 1 */

#define  DMA_CCR7_PL                         ((uint16_t)0x3000)            /*!< PL[1:0] bits (Channel Priority level) */
#define  DMA_CCR7_PL_0                       ((uint16_t)0x1000)            /*!< Bit 0 */
#define  DMA_CCR7_PL_1                       ((uint16_t)0x2000)            /*!< Bit 1 */

#define  DMA_CCR7_MEM2MEM                    ((uint16_t)0x4000)            /*!< Memory to memory mode enable */

/******************  Bit definition for DMA_CNDTR1 register  ******************/
#define  DMA_CNDTR1_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CNDTR2 register  ******************/
#define  DMA_CNDTR2_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CNDTR3 register  ******************/
#define  DMA_CNDTR3_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to
```

```
Transfer */

/******************  Bit definition for DMA_CNDTR4 register  ******************/
#define  DMA_CNDTR4_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CNDTR5 register  ******************/
#define  DMA_CNDTR5_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CNDTR6 register  ******************/
#define  DMA_CNDTR6_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CNDTR7 register  ******************/
#define  DMA_CNDTR7_NDT                      ((uint16_t)0xFFFF)            /*!< Number of data to Transfer */

/******************  Bit definition for DMA_CPAR1 register  *******************/
#define  DMA_CPAR1_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */

/******************  Bit definition for DMA_CPAR2 register  *******************/
#define  DMA_CPAR2_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */

/******************  Bit definition for DMA_CPAR3 register  *******************/
#define  DMA_CPAR3_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */


/******************  Bit definition for DMA_CPAR4 register  *******************/
#define  DMA_CPAR4_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */

/******************  Bit definition for DMA_CPAR5 register  *******************/
#define  DMA_CPAR5_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */

/******************  Bit definition for DMA_CPAR6 register  *******************/
#define  DMA_CPAR6_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */


/******************  Bit definition for DMA_CPAR7 register  *******************/
#define  DMA_CPAR7_PA                        ((uint32_t)0xFFFFFFFF)        /*!< Peripheral Address */

/******************  Bit definition for DMA_CMAR1 register  *******************/
#define  DMA_CMAR1_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************  Bit definition for DMA_CMAR2 register  *******************/
#define  DMA_CMAR2_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************  Bit definition for DMA_CMAR3 register  *******************/
#define  DMA_CMAR3_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */


/******************  Bit definition for DMA_CMAR4 register  *******************/
#define  DMA_CMAR4_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************  Bit definition for DMA_CMAR5 register  *******************/
#define  DMA_CMAR5_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************  Bit definition for DMA_CMAR6 register  *******************/
#define  DMA_CMAR6_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************  Bit definition for DMA_CMAR7 register  *******************/
#define  DMA_CMAR7_MA                        ((uint32_t)0xFFFFFFFF)        /*!< Memory Address */

/******************************************************************************/
/*                                                                            */
/*                        Analog to Digital Converter                         */
/*                                                                            */
/******************************************************************************/

/********************  Bit definition for ADC_SR register  ********************/
#define  ADC_SR_AWD                          ((uint8_t)0x01)               /*!< Analog watchdog flag */
#define  ADC_SR_EOC                          ((uint8_t)0x02)               /*!< End of conversion */
#define  ADC_SR_JEOC                         ((uint8_t)0x04)               /*!< Injected channel end of
```

```
conversion */
#define  ADC_SR_JSTRT                        ((uint8_t)0x08)              /*!< Injected channel Start flag */
#define  ADC_SR_STRT                         ((uint8_t)0x10)              /*!< Regular channel Start flag */

/*******************  Bit definition for ADC_CR1 register  ********************/
#define  ADC_CR1_AWDCH                       ((uint32_t)0x0000001F)       /*!< AWDCH[4:0] bits (Analog watchdog channel select bits) */
#define  ADC_CR1_AWDCH_0                     ((uint32_t)0x00000001)       /*!< Bit 0 */
#define  ADC_CR1_AWDCH_1                     ((uint32_t)0x00000002)       /*!< Bit 1 */
#define  ADC_CR1_AWDCH_2                     ((uint32_t)0x00000004)       /*!< Bit 2 */
#define  ADC_CR1_AWDCH_3                     ((uint32_t)0x00000008)       /*!< Bit 3 */
#define  ADC_CR1_AWDCH_4                     ((uint32_t)0x00000010)       /*!< Bit 4 */

#define  ADC_CR1_EOCIE                       ((uint32_t)0x00000020)       /*!< Interrupt enable for EOC */
#define  ADC_CR1_AWDIE                       ((uint32_t)0x00000040)       /*!< Analog Watchdog interrupt enable */
#define  ADC_CR1_JEOCIE                      ((uint32_t)0x00000080)       /*!< Interrupt enable for injected channels */
#define  ADC_CR1_SCAN                        ((uint32_t)0x00000100)       /*!< Scan mode */
#define  ADC_CR1_AWDSGL                      ((uint32_t)0x00000200)       /*!< Enable the watchdog on a single channel in scan mode */
#define  ADC_CR1_JAUTO                       ((uint32_t)0x00000400)       /*!< Automatic injected group conversion */
#define  ADC_CR1_DISCEN                      ((uint32_t)0x00000800)       /*!< Discontinuous mode on regular channels */
#define  ADC_CR1_JDISCEN                     ((uint32_t)0x00001000)       /*!< Discontinuous mode on injected channels */

#define  ADC_CR1_DISCNUM                     ((uint32_t)0x0000E000)       /*!< DISCNUM[2:0] bits (Discontinuous mode channel count) */
#define  ADC_CR1_DISCNUM_0                   ((uint32_t)0x00002000)       /*!< Bit 0 */
#define  ADC_CR1_DISCNUM_1                   ((uint32_t)0x00004000)       /*!< Bit 1 */
#define  ADC_CR1_DISCNUM_2                   ((uint32_t)0x00008000)       /*!< Bit 2 */

#define  ADC_CR1_DUALMOD                     ((uint32_t)0x000F0000)       /*!< DUALMOD[3:0] bits (Dual mode selection) */
#define  ADC_CR1_DUALMOD_0                   ((uint32_t)0x00010000)       /*!< Bit 0 */
#define  ADC_CR1_DUALMOD_1                   ((uint32_t)0x00020000)       /*!< Bit 1 */
#define  ADC_CR1_DUALMOD_2                   ((uint32_t)0x00040000)       /*!< Bit 2 */
#define  ADC_CR1_DUALMOD_3                   ((uint32_t)0x00080000)       /*!< Bit 3 */

#define  ADC_CR1_JAWDEN                      ((uint32_t)0x00400000)       /*!< Analog watchdog enable on injected channels */
#define  ADC_CR1_AWDEN                       ((uint32_t)0x00800000)       /*!< Analog watchdog enable on regular channels */


/*******************  Bit definition for ADC_CR2 register  ********************/
#define  ADC_CR2_ADON                        ((uint32_t)0x00000001)       /*!< A/D Converter ON / OFF */
#define  ADC_CR2_CONT                        ((uint32_t)0x00000002)       /*!< Continuous Conversion */
#define  ADC_CR2_CAL                         ((uint32_t)0x00000004)       /*!< A/D Calibration */
#define  ADC_CR2_RSTCAL                      ((uint32_t)0x00000008)       /*!< Reset Calibration */
#define  ADC_CR2_DMA                         ((uint32_t)0x00000100)       /*!< Direct Memory access mode */
#define  ADC_CR2_ALIGN                       ((uint32_t)0x00000800)       /*!< Data Alignment */

#define  ADC_CR2_JEXTSEL                     ((uint32_t)0x00007000)       /*!< JEXTSEL[2:0] bits (External event select for injected group) */
#define  ADC_CR2_JEXTSEL_0                   ((uint32_t)0x00001000)       /*!< Bit 0 */
#define  ADC_CR2_JEXTSEL_1                   ((uint32_t)0x00002000)       /*!< Bit 1 */
#define  ADC_CR2_JEXTSEL_2                   ((uint32_t)0x00004000)       /*!< Bit 2 */

#define  ADC_CR2_JEXTTRIG                    ((uint32_t)0x00008000)       /*!< External Trigger Conversion mode for injected channels */

#define  ADC_CR2_EXTSEL                      ((uint32_t)0x000E0000)       /*!< EXTSEL[2:0] bits (External Event Select for regular group) */
#define  ADC_CR2_EXTSEL_0                    ((uint32_t)0x00020000)       /*!< Bit 0 */
#define  ADC_CR2_EXTSEL_1                    ((uint32_t)0x00040000)       /*!< Bit 1 */
#define  ADC_CR2_EXTSEL_2                    ((uint32_t)0x00080000)       /*!< Bit 2 */
```

```
#define  ADC_CR2_EXTTRIG                      ((uint32_t)0x00100000)        /*!< External Trigger Conversion mode for regular channels */
#define  ADC_CR2_JSWSTART                     ((uint32_t)0x00200000)        /*!< Start Conversion of injected channels */
#define  ADC_CR2_SWSTART                      ((uint32_t)0x00400000)        /*!< Start Conversion of regular channels */
#define  ADC_CR2_TSVREFE                      ((uint32_t)0x00800000)        /*!< Temperature Sensor and VREFINT Enable */

/******************  Bit definition for ADC_SMPR1 register  *******************/
#define  ADC_SMPR1_SMP10                      ((uint32_t)0x00000007)        /*!< SMP10[2:0] bits (Channel 10 Sample time selection) */
#define  ADC_SMPR1_SMP10_0                    ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP10_1                    ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP10_2                    ((uint32_t)0x00000004)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP11                      ((uint32_t)0x00000038)        /*!< SMP11[2:0] bits (Channel 11 Sample time selection) */
#define  ADC_SMPR1_SMP11_0                    ((uint32_t)0x00000008)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP11_1                    ((uint32_t)0x00000010)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP11_2                    ((uint32_t)0x00000020)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP12                      ((uint32_t)0x000001C0)        /*!< SMP12[2:0] bits (Channel 12 Sample time selection) */
#define  ADC_SMPR1_SMP12_0                    ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP12_1                    ((uint32_t)0x00000080)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP12_2                    ((uint32_t)0x00000100)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP13                      ((uint32_t)0x00000E00)        /*!< SMP13[2:0] bits (Channel 13 Sample time selection) */
#define  ADC_SMPR1_SMP13_0                    ((uint32_t)0x00000200)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP13_1                    ((uint32_t)0x00000400)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP13_2                    ((uint32_t)0x00000800)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP14                      ((uint32_t)0x00007000)        /*!< SMP14[2:0] bits (Channel 14 Sample time selection) */
#define  ADC_SMPR1_SMP14_0                    ((uint32_t)0x00001000)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP14_1                    ((uint32_t)0x00002000)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP14_2                    ((uint32_t)0x00004000)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP15                      ((uint32_t)0x00038000)        /*!< SMP15[2:0] bits (Channel 15 Sample time selection) */
#define  ADC_SMPR1_SMP15_0                    ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP15_1                    ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP15_2                    ((uint32_t)0x00020000)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP16                      ((uint32_t)0x001C0000)        /*!< SMP16[2:0] bits (Channel 16 Sample time selection) */
#define  ADC_SMPR1_SMP16_0                    ((uint32_t)0x00040000)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP16_1                    ((uint32_t)0x00080000)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP16_2                    ((uint32_t)0x00100000)        /*!< Bit 2 */

#define  ADC_SMPR1_SMP17                      ((uint32_t)0x00E00000)        /*!< SMP17[2:0] bits (Channel 17 Sample time selection) */
#define  ADC_SMPR1_SMP17_0                    ((uint32_t)0x00200000)        /*!< Bit 0 */
#define  ADC_SMPR1_SMP17_1                    ((uint32_t)0x00400000)        /*!< Bit 1 */
#define  ADC_SMPR1_SMP17_2                    ((uint32_t)0x00800000)        /*!< Bit 2 */

/******************  Bit definition for ADC_SMPR2 register  *******************/
#define  ADC_SMPR2_SMP0                       ((uint32_t)0x00000007)        /*!< SMP0[2:0] bits (Channel 0 Sample time selection) */
#define  ADC_SMPR2_SMP0_0                     ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP0_1                     ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP0_2                     ((uint32_t)0x00000004)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP1                       ((uint32_t)0x00000038)        /*!< SMP1[2:0] bits (Channel 1 Sample time selection) */
#define  ADC_SMPR2_SMP1_0                     ((uint32_t)0x00000008)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP1_1                     ((uint32_t)0x00000010)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP1_2                     ((uint32_t)0x00000020)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP2                       ((uint32_t)0x000001C0)        /*!< SMP2[2:0] bits (Channel
```

```
2 Sample time selection) */
#define  ADC_SMPR2_SMP2_0                    ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP2_1                    ((uint32_t)0x00000080)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP2_2                    ((uint32_t)0x00000100)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP3                      ((uint32_t)0x00000E00)        /*!< SMP3[2:0] bits (Channel
3 Sample time selection) */
#define  ADC_SMPR2_SMP3_0                    ((uint32_t)0x00000200)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP3_1                    ((uint32_t)0x00000400)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP3_2                    ((uint32_t)0x00000800)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP4                      ((uint32_t)0x00007000)        /*!< SMP4[2:0] bits (Channel
4 Sample time selection) */
#define  ADC_SMPR2_SMP4_0                    ((uint32_t)0x00001000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP4_1                    ((uint32_t)0x00002000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP4_2                    ((uint32_t)0x00004000)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP5                      ((uint32_t)0x00038000)        /*!< SMP5[2:0] bits (Channel
5 Sample time selection) */
#define  ADC_SMPR2_SMP5_0                    ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP5_1                    ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP5_2                    ((uint32_t)0x00020000)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP6                      ((uint32_t)0x001C0000)        /*!< SMP6[2:0] bits (Channel
6 Sample time selection) */
#define  ADC_SMPR2_SMP6_0                    ((uint32_t)0x00040000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP6_1                    ((uint32_t)0x00080000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP6_2                    ((uint32_t)0x00100000)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP7                      ((uint32_t)0x00E00000)        /*!< SMP7[2:0] bits (Channel
7 Sample time selection) */
#define  ADC_SMPR2_SMP7_0                    ((uint32_t)0x00200000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP7_1                    ((uint32_t)0x00400000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP7_2                    ((uint32_t)0x00800000)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP8                      ((uint32_t)0x07000000)        /*!< SMP8[2:0] bits (Channel
8 Sample time selection) */
#define  ADC_SMPR2_SMP8_0                    ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP8_1                    ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP8_2                    ((uint32_t)0x04000000)        /*!< Bit 2 */

#define  ADC_SMPR2_SMP9                      ((uint32_t)0x38000000)        /*!< SMP9[2:0] bits (Channel
9 Sample time selection) */
#define  ADC_SMPR2_SMP9_0                    ((uint32_t)0x08000000)        /*!< Bit 0 */
#define  ADC_SMPR2_SMP9_1                    ((uint32_t)0x10000000)        /*!< Bit 1 */
#define  ADC_SMPR2_SMP9_2                    ((uint32_t)0x20000000)        /*!< Bit 2 */

/******************  Bit definition for ADC_JOFR1 register  *******************/
#define  ADC_JOFR1_JOFFSET1                  ((uint16_t)0x0FFF)            /*!< Data offset for
injected channel 1 */

/******************  Bit definition for ADC_JOFR2 register  *******************/
#define  ADC_JOFR2_JOFFSET2                  ((uint16_t)0x0FFF)            /*!< Data offset for
injected channel 2 */

/******************  Bit definition for ADC_JOFR3 register  *******************/
#define  ADC_JOFR3_JOFFSET3                  ((uint16_t)0x0FFF)            /*!< Data offset for
injected channel 3 */

/******************  Bit definition for ADC_JOFR4 register  *******************/
#define  ADC_JOFR4_JOFFSET4                  ((uint16_t)0x0FFF)            /*!< Data offset for
injected channel 4 */

/*******************  Bit definition for ADC_HTR register  ********************/
#define  ADC_HTR_HT                          ((uint16_t)0x0FFF)            /*!< Analog watchdog high
threshold */

/*******************  Bit definition for ADC_LTR register  ********************/
#define  ADC_LTR_LT                          ((uint16_t)0x0FFF)            /*!< Analog watchdog low
threshold */

/*******************  Bit definition for ADC_SQR1 register  *******************/
#define  ADC_SQR1_SQ13                       ((uint32_t)0x0000001F)        /*!< SQ13[4:0] bits (13th
```

```
conversion in regular sequence) */
#define  ADC_SQR1_SQ13_0                     ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_SQR1_SQ13_1                     ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_SQR1_SQ13_2                     ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  ADC_SQR1_SQ13_3                     ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  ADC_SQR1_SQ13_4                     ((uint32_t)0x00000010)        /*!< Bit 4 */

#define  ADC_SQR1_SQ14                       ((uint32_t)0x000003E0)        /*!< SQ14[4:0] bits (14th
conversion in regular sequence) */
#define  ADC_SQR1_SQ14_0                     ((uint32_t)0x00000020)        /*!< Bit 0 */
#define  ADC_SQR1_SQ14_1                     ((uint32_t)0x00000040)        /*!< Bit 1 */
#define  ADC_SQR1_SQ14_2                     ((uint32_t)0x00000080)        /*!< Bit 2 */
#define  ADC_SQR1_SQ14_3                     ((uint32_t)0x00000100)        /*!< Bit 3 */
#define  ADC_SQR1_SQ14_4                     ((uint32_t)0x00000200)        /*!< Bit 4 */

#define  ADC_SQR1_SQ15                       ((uint32_t)0x00007C00)        /*!< SQ15[4:0] bits (15th
conversion in regular sequence) */
#define  ADC_SQR1_SQ15_0                     ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  ADC_SQR1_SQ15_1                     ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  ADC_SQR1_SQ15_2                     ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  ADC_SQR1_SQ15_3                     ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  ADC_SQR1_SQ15_4                     ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  ADC_SQR1_SQ16                       ((uint32_t)0x000F8000)        /*!< SQ16[4:0] bits (16th
conversion in regular sequence) */
#define  ADC_SQR1_SQ16_0                     ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_SQR1_SQ16_1                     ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_SQR1_SQ16_2                     ((uint32_t)0x00020000)        /*!< Bit 2 */
#define  ADC_SQR1_SQ16_3                     ((uint32_t)0x00040000)        /*!< Bit 3 */
#define  ADC_SQR1_SQ16_4                     ((uint32_t)0x00080000)        /*!< Bit 4 */

#define  ADC_SQR1_L                          ((uint32_t)0x00F00000)        /*!< L[3:0] bits (Regular
channel sequence length) */
#define  ADC_SQR1_L_0                        ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  ADC_SQR1_L_1                        ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  ADC_SQR1_L_2                        ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  ADC_SQR1_L_3                        ((uint32_t)0x00800000)        /*!< Bit 3 */

/*******************  Bit definition for ADC_SQR2 register  *******************/
#define  ADC_SQR2_SQ7                        ((uint32_t)0x0000001F)        /*!< SQ7[4:0] bits (7th
conversion in regular sequence) */
#define  ADC_SQR2_SQ7_0                      ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_SQR2_SQ7_1                      ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_SQR2_SQ7_2                      ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  ADC_SQR2_SQ7_3                      ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  ADC_SQR2_SQ7_4                      ((uint32_t)0x00000010)        /*!< Bit 4 */

#define  ADC_SQR2_SQ8                        ((uint32_t)0x000003E0)        /*!< SQ8[4:0] bits (8th
conversion in regular sequence) */
#define  ADC_SQR2_SQ8_0                      ((uint32_t)0x00000020)        /*!< Bit 0 */
#define  ADC_SQR2_SQ8_1                      ((uint32_t)0x00000040)        /*!< Bit 1 */
#define  ADC_SQR2_SQ8_2                      ((uint32_t)0x00000080)        /*!< Bit 2 */
#define  ADC_SQR2_SQ8_3                      ((uint32_t)0x00000100)        /*!< Bit 3 */
#define  ADC_SQR2_SQ8_4                      ((uint32_t)0x00000200)        /*!< Bit 4 */

#define  ADC_SQR2_SQ9                        ((uint32_t)0x00007C00)        /*!< SQ9[4:0] bits (9th
conversion in regular sequence) */
#define  ADC_SQR2_SQ9_0                      ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  ADC_SQR2_SQ9_1                      ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  ADC_SQR2_SQ9_2                      ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  ADC_SQR2_SQ9_3                      ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  ADC_SQR2_SQ9_4                      ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  ADC_SQR2_SQ10                       ((uint32_t)0x000F8000)        /*!< SQ10[4:0] bits (10th
conversion in regular sequence) */
#define  ADC_SQR2_SQ10_0                     ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_SQR2_SQ10_1                     ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_SQR2_SQ10_2                     ((uint32_t)0x00020000)        /*!< Bit 2 */
#define  ADC_SQR2_SQ10_3                     ((uint32_t)0x00040000)        /*!< Bit 3 */
#define  ADC_SQR2_SQ10_4                     ((uint32_t)0x00080000)        /*!< Bit 4 */

#define  ADC_SQR2_SQ11                       ((uint32_t)0x01F00000)        /*!< SQ11[4:0] bits (11th
conversion in regular sequence) */
```

```
#define  ADC_SQR2_SQ11_0                     ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  ADC_SQR2_SQ11_1                     ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  ADC_SQR2_SQ11_2                     ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  ADC_SQR2_SQ11_3                     ((uint32_t)0x00800000)        /*!< Bit 3 */
#define  ADC_SQR2_SQ11_4                     ((uint32_t)0x01000000)        /*!< Bit 4 */

#define  ADC_SQR2_SQ12                       ((uint32_t)0x3E000000)        /*!< SQ12[4:0] bits (12th conversion in regular sequence) */
#define  ADC_SQR2_SQ12_0                     ((uint32_t)0x02000000)        /*!< Bit 0 */
#define  ADC_SQR2_SQ12_1                     ((uint32_t)0x04000000)        /*!< Bit 1 */
#define  ADC_SQR2_SQ12_2                     ((uint32_t)0x08000000)        /*!< Bit 2 */
#define  ADC_SQR2_SQ12_3                     ((uint32_t)0x10000000)        /*!< Bit 3 */
#define  ADC_SQR2_SQ12_4                     ((uint32_t)0x20000000)        /*!< Bit 4 */

/*******************  Bit definition for ADC_SQR3 register  *******************/
#define  ADC_SQR3_SQ1                        ((uint32_t)0x0000001F)        /*!< SQ1[4:0] bits (1st conversion in regular sequence) */
#define  ADC_SQR3_SQ1_0                      ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_SQR3_SQ1_1                      ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_SQR3_SQ1_2                      ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  ADC_SQR3_SQ1_3                      ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  ADC_SQR3_SQ1_4                      ((uint32_t)0x00000010)        /*!< Bit 4 */

#define  ADC_SQR3_SQ2                        ((uint32_t)0x000003E0)        /*!< SQ2[4:0] bits (2nd conversion in regular sequence) */
#define  ADC_SQR3_SQ2_0                      ((uint32_t)0x00000020)        /*!< Bit 0 */
#define  ADC_SQR3_SQ2_1                      ((uint32_t)0x00000040)        /*!< Bit 1 */
#define  ADC_SQR3_SQ2_2                      ((uint32_t)0x00000080)        /*!< Bit 2 */
#define  ADC_SQR3_SQ2_3                      ((uint32_t)0x00000100)        /*!< Bit 3 */
#define  ADC_SQR3_SQ2_4                      ((uint32_t)0x00000200)        /*!< Bit 4 */

#define  ADC_SQR3_SQ3                        ((uint32_t)0x00007C00)        /*!< SQ3[4:0] bits (3rd conversion in regular sequence) */
#define  ADC_SQR3_SQ3_0                      ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  ADC_SQR3_SQ3_1                      ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  ADC_SQR3_SQ3_2                      ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  ADC_SQR3_SQ3_3                      ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  ADC_SQR3_SQ3_4                      ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  ADC_SQR3_SQ4                        ((uint32_t)0x000F8000)        /*!< SQ4[4:0] bits (4th conversion in regular sequence) */
#define  ADC_SQR3_SQ4_0                      ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_SQR3_SQ4_1                      ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_SQR3_SQ4_2                      ((uint32_t)0x00020000)        /*!< Bit 2 */
#define  ADC_SQR3_SQ4_3                      ((uint32_t)0x00040000)        /*!< Bit 3 */
#define  ADC_SQR3_SQ4_4                      ((uint32_t)0x00080000)        /*!< Bit 4 */

#define  ADC_SQR3_SQ5                        ((uint32_t)0x01F00000)        /*!< SQ5[4:0] bits (5th conversion in regular sequence) */
#define  ADC_SQR3_SQ5_0                      ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  ADC_SQR3_SQ5_1                      ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  ADC_SQR3_SQ5_2                      ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  ADC_SQR3_SQ5_3                      ((uint32_t)0x00800000)        /*!< Bit 3 */
#define  ADC_SQR3_SQ5_4                      ((uint32_t)0x01000000)        /*!< Bit 4 */

#define  ADC_SQR3_SQ6                        ((uint32_t)0x3E000000)        /*!< SQ6[4:0] bits (6th conversion in regular sequence) */
#define  ADC_SQR3_SQ6_0                      ((uint32_t)0x02000000)        /*!< Bit 0 */
#define  ADC_SQR3_SQ6_1                      ((uint32_t)0x04000000)        /*!< Bit 1 */
#define  ADC_SQR3_SQ6_2                      ((uint32_t)0x08000000)        /*!< Bit 2 */
#define  ADC_SQR3_SQ6_3                      ((uint32_t)0x10000000)        /*!< Bit 3 */
#define  ADC_SQR3_SQ6_4                      ((uint32_t)0x20000000)        /*!< Bit 4 */

/*******************  Bit definition for ADC_JSQR register  *******************/
#define  ADC_JSQR_JSQ1                       ((uint32_t)0x0000001F)        /*!< JSQ1[4:0] bits (1st conversion in injected sequence) */
#define  ADC_JSQR_JSQ1_0                     ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  ADC_JSQR_JSQ1_1                     ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  ADC_JSQR_JSQ1_2                     ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  ADC_JSQR_JSQ1_3                     ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  ADC_JSQR_JSQ1_4                     ((uint32_t)0x00000010)        /*!< Bit 4 */

#define  ADC_JSQR_JSQ2                       ((uint32_t)0x000003E0)        /*!< JSQ2[4:0] bits (2nd
```

```
conversion in injected sequence) */
#define  ADC_JSQR_JSQ2_0                      ((uint32_t)0x00000020)        /*!< Bit 0 */
#define  ADC_JSQR_JSQ2_1                      ((uint32_t)0x00000040)        /*!< Bit 1 */
#define  ADC_JSQR_JSQ2_2                      ((uint32_t)0x00000080)        /*!< Bit 2 */
#define  ADC_JSQR_JSQ2_3                      ((uint32_t)0x00000100)        /*!< Bit 3 */
#define  ADC_JSQR_JSQ2_4                      ((uint32_t)0x00000200)        /*!< Bit 4 */

#define  ADC_JSQR_JSQ3                        ((uint32_t)0x00007C00)        /*!< JSQ3[4:0] bits (3rd conversion in injected sequence) */
#define  ADC_JSQR_JSQ3_0                      ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  ADC_JSQR_JSQ3_1                      ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  ADC_JSQR_JSQ3_2                      ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  ADC_JSQR_JSQ3_3                      ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  ADC_JSQR_JSQ3_4                      ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  ADC_JSQR_JSQ4                        ((uint32_t)0x000F8000)        /*!< JSQ4[4:0] bits (4th conversion in injected sequence) */
#define  ADC_JSQR_JSQ4_0                      ((uint32_t)0x00008000)        /*!< Bit 0 */
#define  ADC_JSQR_JSQ4_1                      ((uint32_t)0x00010000)        /*!< Bit 1 */
#define  ADC_JSQR_JSQ4_2                      ((uint32_t)0x00020000)        /*!< Bit 2 */
#define  ADC_JSQR_JSQ4_3                      ((uint32_t)0x00040000)        /*!< Bit 3 */
#define  ADC_JSQR_JSQ4_4                      ((uint32_t)0x00080000)        /*!< Bit 4 */

#define  ADC_JSQR_JL                          ((uint32_t)0x00300000)        /*!< JL[1:0] bits (Injected Sequence length) */
#define  ADC_JSQR_JL_0                        ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  ADC_JSQR_JL_1                        ((uint32_t)0x00200000)        /*!< Bit 1 */

/********************  Bit definition for ADC_JDR1 register  ********************/
#define  ADC_JDR1_JDATA                       ((uint16_t)0xFFFF)            /*!< Injected data */

/********************  Bit definition for ADC_JDR2 register  ********************/
#define  ADC_JDR2_JDATA                       ((uint16_t)0xFFFF)            /*!< Injected data */

/********************  Bit definition for ADC_JDR3 register  ********************/
#define  ADC_JDR3_JDATA                       ((uint16_t)0xFFFF)            /*!< Injected data */

/********************  Bit definition for ADC_JDR4 register  ********************/
#define  ADC_JDR4_JDATA                       ((uint16_t)0xFFFF)            /*!< Injected data */

/*********************  Bit definition for ADC_DR register  *********************/
#define  ADC_DR_DATA                          ((uint32_t)0x0000FFFF)        /*!< Regular data */
#define  ADC_DR_ADC2DATA                      ((uint32_t)0xFFFF0000)        /*!< ADC2 data */

/******************************************************************************/
/*                                                                            */
/*                        Digital to Analog Converter                         */
/*                                                                            */
/******************************************************************************/

/*********************  Bit definition for DAC_CR register  *********************/
#define  DAC_CR_EN1                           ((uint32_t)0x00000001)        /*!< DAC channel1 enable */
#define  DAC_CR_BOFF1                         ((uint32_t)0x00000002)        /*!< DAC channel1 output buffer disable */
#define  DAC_CR_TEN1                          ((uint32_t)0x00000004)        /*!< DAC channel1 Trigger enable */

#define  DAC_CR_TSEL1                         ((uint32_t)0x00000038)        /*!< TSEL1[2:0] (DAC channel1 Trigger selection) */
#define  DAC_CR_TSEL1_0                       ((uint32_t)0x00000008)        /*!< Bit 0 */
#define  DAC_CR_TSEL1_1                       ((uint32_t)0x00000010)        /*!< Bit 1 */
#define  DAC_CR_TSEL1_2                       ((uint32_t)0x00000020)        /*!< Bit 2 */

#define  DAC_CR_WAVE1                         ((uint32_t)0x000000C0)        /*!< WAVE1[1:0] (DAC channel1 noise/triangle wave generation enable) */
#define  DAC_CR_WAVE1_0                       ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  DAC_CR_WAVE1_1                       ((uint32_t)0x00000080)        /*!< Bit 1 */

#define  DAC_CR_MAMP1                         ((uint32_t)0x00000F00)        /*!< MAMP1[3:0] (DAC channel1 Mask/Amplitude selector) */
#define  DAC_CR_MAMP1_0                       ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  DAC_CR_MAMP1_1                       ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  DAC_CR_MAMP1_2                       ((uint32_t)0x00000400)        /*!< Bit 2 */
```

```
#define  DAC_CR_MAMP1_3                      ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  DAC_CR_DMAEN1                       ((uint32_t)0x00001000)        /*!< DAC channel1 DMA enable */
#define  DAC_CR_EN2                          ((uint32_t)0x00010000)        /*!< DAC channel2 enable */
#define  DAC_CR_BOFF2                        ((uint32_t)0x00020000)        /*!< DAC channel2 output buffer disable */
#define  DAC_CR_TEN2                         ((uint32_t)0x00040000)        /*!< DAC channel2 Trigger enable */

#define  DAC_CR_TSEL2                        ((uint32_t)0x00380000)        /*!< TSEL2[2:0] (DAC channel2 Trigger selection) */
#define  DAC_CR_TSEL2_0                      ((uint32_t)0x00080000)        /*!< Bit 0 */
#define  DAC_CR_TSEL2_1                      ((uint32_t)0x00100000)        /*!< Bit 1 */
#define  DAC_CR_TSEL2_2                      ((uint32_t)0x00200000)        /*!< Bit 2 */

#define  DAC_CR_WAVE2                        ((uint32_t)0x00C00000)        /*!< WAVE2[1:0] (DAC channel2 noise/triangle wave generation enable) */
#define  DAC_CR_WAVE2_0                      ((uint32_t)0x00400000)        /*!< Bit 0 */
#define  DAC_CR_WAVE2_1                      ((uint32_t)0x00800000)        /*!< Bit 1 */

#define  DAC_CR_MAMP2                        ((uint32_t)0x0F000000)        /*!< MAMP2[3:0] (DAC channel2 Mask/Amplitude selector) */
#define  DAC_CR_MAMP2_0                      ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  DAC_CR_MAMP2_1                      ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  DAC_CR_MAMP2_2                      ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  DAC_CR_MAMP2_3                      ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  DAC_CR_DMAEN2                       ((uint32_t)0x10000000)        /*!< DAC channel2 DMA enabled */

/*****************  Bit definition for DAC_SWTRIGR register  ******************/
#define  DAC_SWTRIGR_SWTRIG1                 ((uint8_t)0x01)               /*!< DAC channel1 software trigger */
#define  DAC_SWTRIGR_SWTRIG2                 ((uint8_t)0x02)               /*!< DAC channel2 software trigger */

/*****************  Bit definition for DAC_DHR12R1 register  ******************/
#define  DAC_DHR12R1_DACC1DHR                ((uint16_t)0x0FFF)            /*!< DAC channel1 12-bit Right aligned data */

/*****************  Bit definition for DAC_DHR12L1 register  ******************/
#define  DAC_DHR12L1_DACC1DHR                ((uint16_t)0xFFF0)            /*!< DAC channel1 12-bit Left aligned data */

/******************  Bit definition for DAC_DHR8R1 register  ******************/
#define  DAC_DHR8R1_DACC1DHR                 ((uint8_t)0xFF)               /*!< DAC channel1 8-bit Right aligned data */

/*****************  Bit definition for DAC_DHR12R2 register  ******************/
#define  DAC_DHR12R2_DACC2DHR                ((uint16_t)0x0FFF)            /*!< DAC channel2 12-bit Right aligned data */

/*****************  Bit definition for DAC_DHR12L2 register  ******************/
#define  DAC_DHR12L2_DACC2DHR                ((uint16_t)0xFFF0)            /*!< DAC channel2 12-bit Left aligned data */

/******************  Bit definition for DAC_DHR8R2 register  ******************/
#define  DAC_DHR8R2_DACC2DHR                 ((uint8_t)0xFF)               /*!< DAC channel2 8-bit Right aligned data */

/*****************  Bit definition for DAC_DHR12RD register  ******************/
#define  DAC_DHR12RD_DACC1DHR                ((uint32_t)0x00000FFF)        /*!< DAC channel1 12-bit Right aligned data */
#define  DAC_DHR12RD_DACC2DHR                ((uint32_t)0x0FFF0000)        /*!< DAC channel2 12-bit Right aligned data */

/*****************  Bit definition for DAC_DHR12LD register  ******************/
#define  DAC_DHR12LD_DACC1DHR                ((uint32_t)0x0000FFF0)        /*!< DAC channel1 12-bit Left aligned data */
#define  DAC_DHR12LD_DACC2DHR                ((uint32_t)0xFFF00000)        /*!< DAC channel2 12-bit Left aligned data */

```

```
/*******************  Bit definition for DAC_DHR8RD register  *******************/
#define  DAC_DHR8RD_DACC1DHR                 ((uint16_t)0x00FF)            /*!< DAC channel1 8-bit Right aligned data */
#define  DAC_DHR8RD_DACC2DHR                 ((uint16_t)0xFF00)            /*!< DAC channel2 8-bit Right aligned data */

/*******************  Bit definition for DAC_DOR1 register  *******************/
#define  DAC_DOR1_DACC1DOR                   ((uint16_t)0x0FFF)            /*!< DAC channel1 data output */

/*******************  Bit definition for DAC_DOR2 register  *******************/
#define  DAC_DOR2_DACC2DOR                   ((uint16_t)0x0FFF)            /*!< DAC channel2 data output */

/********************  Bit definition for DAC_SR register  ********************/
#define  DAC_SR_DMAUDR1                      ((uint32_t)0x00002000)        /*!< DAC channel1 DMA underrun flag */
#define  DAC_SR_DMAUDR2                      ((uint32_t)0x20000000)        /*!< DAC channel2 DMA underrun flag */

/******************************************************************************/
/*                                                                            */
/*                                      CEC                                   */
/*                                                                            */
/******************************************************************************/
/********************  Bit definition for CEC_CFGR register  *******************/
#define  CEC_CFGR_PE              ((uint16_t)0x0001)     /*!<  Peripheral Enable */
#define  CEC_CFGR_IE              ((uint16_t)0x0002)     /*!<  Interrupt Enable */
#define  CEC_CFGR_BTEM            ((uint16_t)0x0004)     /*!<  Bit Timing Error Mode */
#define  CEC_CFGR_BPEM            ((uint16_t)0x0008)     /*!<  Bit Period Error Mode */

/********************  Bit definition for CEC_OAR register  *******************/
#define  CEC_OAR_OA               ((uint16_t)0x000F)     /*!<  OA[3:0]: Own Address */
#define  CEC_OAR_OA_0             ((uint16_t)0x0001)     /*!<  Bit 0 */
#define  CEC_OAR_OA_1             ((uint16_t)0x0002)     /*!<  Bit 1 */
#define  CEC_OAR_OA_2             ((uint16_t)0x0004)     /*!<  Bit 2 */
#define  CEC_OAR_OA_3             ((uint16_t)0x0008)     /*!<  Bit 3 */

/********************  Bit definition for CEC_PRES register  *******************/
#define  CEC_PRES_PRES            ((uint16_t)0x3FFF)   /*!<  Prescaler Counter Value */

/********************  Bit definition for CEC_ESR register  *******************/
#define  CEC_ESR_BTE              ((uint16_t)0x0001)     /*!<  Bit Timing Error */
#define  CEC_ESR_BPE              ((uint16_t)0x0002)     /*!<  Bit Period Error */
#define  CEC_ESR_RBTFE            ((uint16_t)0x0004)     /*!<  Rx Block Transfer Finished Error */
#define  CEC_ESR_SBE              ((uint16_t)0x0008)     /*!<  Start Bit Error */
#define  CEC_ESR_ACKE             ((uint16_t)0x0010)     /*!<  Block Acknowledge Error */
#define  CEC_ESR_LINE             ((uint16_t)0x0020)     /*!<  Line Error */
#define  CEC_ESR_TBTFE            ((uint16_t)0x0040)     /*!<  Tx Block Transfer Finished Error */

/********************  Bit definition for CEC_CSR register  *******************/
#define  CEC_CSR_TSOM             ((uint16_t)0x0001)     /*!<  Tx Start Of Message */
#define  CEC_CSR_TEOM             ((uint16_t)0x0002)     /*!<  Tx End Of Message */
#define  CEC_CSR_TERR             ((uint16_t)0x0004)     /*!<  Tx Error */
#define  CEC_CSR_TBTRF            ((uint16_t)0x0008)     /*!<  Tx Byte Transfer Request or Block Transfer Finished */
#define  CEC_CSR_RSOM             ((uint16_t)0x0010)     /*!<  Rx Start Of Message */
#define  CEC_CSR_REOM             ((uint16_t)0x0020)     /*!<  Rx End Of Message */
#define  CEC_CSR_RERR             ((uint16_t)0x0040)     /*!<  Rx Error */
#define  CEC_CSR_RBTF             ((uint16_t)0x0080)     /*!<  Rx Block Transfer Finished */

/********************  Bit definition for CEC_TXD register  *******************/
#define  CEC_TXD_TXD              ((uint16_t)0x00FF)     /*!<  Tx Data register */

/********************  Bit definition for CEC_RXD register  *******************/
#define  CEC_RXD_RXD              ((uint16_t)0x00FF)     /*!<  Rx Data register */

/******************************************************************************/
/*                                                                            */
/*                                      TIM                                   */
/*                                                                            */
/******************************************************************************/

```

```
/*******************  Bit definition for TIM_CR1 register  *******************/
#define  TIM_CR1_CEN                         ((uint16_t)0x0001)            /*!< Counter enable */
#define  TIM_CR1_UDIS                        ((uint16_t)0x0002)            /*!< Update disable */
#define  TIM_CR1_URS                         ((uint16_t)0x0004)            /*!< Update request source */
#define  TIM_CR1_OPM                         ((uint16_t)0x0008)            /*!< One pulse mode */
#define  TIM_CR1_DIR                         ((uint16_t)0x0010)            /*!< Direction */

#define  TIM_CR1_CMS                         ((uint16_t)0x0060)            /*!< CMS[1:0] bits (Center-aligned mode selection) */
#define  TIM_CR1_CMS_0                       ((uint16_t)0x0020)            /*!< Bit 0 */
#define  TIM_CR1_CMS_1                       ((uint16_t)0x0040)            /*!< Bit 1 */

#define  TIM_CR1_ARPE                        ((uint16_t)0x0080)            /*!< Auto-reload preload enable */

#define  TIM_CR1_CKD                         ((uint16_t)0x0300)            /*!< CKD[1:0] bits (clock division) */
#define  TIM_CR1_CKD_0                       ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_CR1_CKD_1                       ((uint16_t)0x0200)            /*!< Bit 1 */

/*******************  Bit definition for TIM_CR2 register  *******************/
#define  TIM_CR2_CCPC                        ((uint16_t)0x0001)            /*!< Capture/Compare Preloaded Control */
#define  TIM_CR2_CCUS                        ((uint16_t)0x0004)            /*!< Capture/Compare Control Update Selection */
#define  TIM_CR2_CCDS                        ((uint16_t)0x0008)            /*!< Capture/Compare DMA Selection */

#define  TIM_CR2_MMS                         ((uint16_t)0x0070)            /*!< MMS[2:0] bits (Master Mode Selection) */
#define  TIM_CR2_MMS_0                       ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_CR2_MMS_1                       ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_CR2_MMS_2                       ((uint16_t)0x0040)            /*!< Bit 2 */

#define  TIM_CR2_TI1S                        ((uint16_t)0x0080)            /*!< TI1 Selection */
#define  TIM_CR2_OIS1                        ((uint16_t)0x0100)            /*!< Output Idle state 1 (OC1 output) */
#define  TIM_CR2_OIS1N                       ((uint16_t)0x0200)            /*!< Output Idle state 1 (OC1N output) */
#define  TIM_CR2_OIS2                        ((uint16_t)0x0400)            /*!< Output Idle state 2 (OC2 output) */
#define  TIM_CR2_OIS2N                       ((uint16_t)0x0800)            /*!< Output Idle state 2 (OC2N output) */
#define  TIM_CR2_OIS3                        ((uint16_t)0x1000)            /*!< Output Idle state 3 (OC3 output) */
#define  TIM_CR2_OIS3N                       ((uint16_t)0x2000)            /*!< Output Idle state 3 (OC3N output) */
#define  TIM_CR2_OIS4                        ((uint16_t)0x4000)            /*!< Output Idle state 4 (OC4 output) */

/*******************  Bit definition for TIM_SMCR register  *******************/
#define  TIM_SMCR_SMS                        ((uint16_t)0x0007)            /*!< SMS[2:0] bits (Slave mode selection) */
#define  TIM_SMCR_SMS_0                      ((uint16_t)0x0001)            /*!< Bit 0 */
#define  TIM_SMCR_SMS_1                      ((uint16_t)0x0002)            /*!< Bit 1 */
#define  TIM_SMCR_SMS_2                      ((uint16_t)0x0004)            /*!< Bit 2 */

#define  TIM_SMCR_TS                         ((uint16_t)0x0070)            /*!< TS[2:0] bits (Trigger selection) */
#define  TIM_SMCR_TS_0                       ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_SMCR_TS_1                       ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_SMCR_TS_2                       ((uint16_t)0x0040)            /*!< Bit 2 */

#define  TIM_SMCR_MSM                        ((uint16_t)0x0080)            /*!< Master/slave mode */

#define  TIM_SMCR_ETF                        ((uint16_t)0x0F00)            /*!< ETF[3:0] bits (External trigger filter) */
#define  TIM_SMCR_ETF_0                      ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_SMCR_ETF_1                      ((uint16_t)0x0200)            /*!< Bit 1 */
#define  TIM_SMCR_ETF_2                      ((uint16_t)0x0400)            /*!< Bit 2 */
#define  TIM_SMCR_ETF_3                      ((uint16_t)0x0800)            /*!< Bit 3 */

#define  TIM_SMCR_ETPS                       ((uint16_t)0x3000)            /*!< ETPS[1:0] bits
```

```
(External trigger prescaler) */
#define  TIM_SMCR_ETPS_0                      ((uint16_t)0x1000)            /*!< Bit 0 */
#define  TIM_SMCR_ETPS_1                      ((uint16_t)0x2000)            /*!< Bit 1 */

#define  TIM_SMCR_ECE                         ((uint16_t)0x4000)            /*!< External clock enable */
#define  TIM_SMCR_ETP                         ((uint16_t)0x8000)            /*!< External trigger polarity */

/*******************  Bit definition for TIM_DIER register  *******************/
#define  TIM_DIER_UIE                         ((uint16_t)0x0001)            /*!< Update interrupt enable */
#define  TIM_DIER_CC1IE                       ((uint16_t)0x0002)            /*!< Capture/Compare 1 interrupt enable */
#define  TIM_DIER_CC2IE                       ((uint16_t)0x0004)            /*!< Capture/Compare 2 interrupt enable */
#define  TIM_DIER_CC3IE                       ((uint16_t)0x0008)            /*!< Capture/Compare 3 interrupt enable */
#define  TIM_DIER_CC4IE                       ((uint16_t)0x0010)            /*!< Capture/Compare 4 interrupt enable */
#define  TIM_DIER_COMIE                       ((uint16_t)0x0020)            /*!< COM interrupt enable */
#define  TIM_DIER_TIE                         ((uint16_t)0x0040)            /*!< Trigger interrupt enable */
#define  TIM_DIER_BIE                         ((uint16_t)0x0080)            /*!< Break interrupt enable */
#define  TIM_DIER_UDE                         ((uint16_t)0x0100)            /*!< Update DMA request enable */
#define  TIM_DIER_CC1DE                       ((uint16_t)0x0200)            /*!< Capture/Compare 1 DMA request enable */
#define  TIM_DIER_CC2DE                       ((uint16_t)0x0400)            /*!< Capture/Compare 2 DMA request enable */
#define  TIM_DIER_CC3DE                       ((uint16_t)0x0800)            /*!< Capture/Compare 3 DMA request enable */
#define  TIM_DIER_CC4DE                       ((uint16_t)0x1000)            /*!< Capture/Compare 4 DMA request enable */
#define  TIM_DIER_COMDE                       ((uint16_t)0x2000)            /*!< COM DMA request enable */
#define  TIM_DIER_TDE                         ((uint16_t)0x4000)            /*!< Trigger DMA request enable */

/********************  Bit definition for TIM_SR register  ********************/
#define  TIM_SR_UIF                           ((uint16_t)0x0001)            /*!< Update interrupt Flag */
#define  TIM_SR_CC1IF                         ((uint16_t)0x0002)            /*!< Capture/Compare 1 interrupt Flag */
#define  TIM_SR_CC2IF                         ((uint16_t)0x0004)            /*!< Capture/Compare 2 interrupt Flag */
#define  TIM_SR_CC3IF                         ((uint16_t)0x0008)            /*!< Capture/Compare 3 interrupt Flag */
#define  TIM_SR_CC4IF                         ((uint16_t)0x0010)            /*!< Capture/Compare 4 interrupt Flag */
#define  TIM_SR_COMIF                         ((uint16_t)0x0020)            /*!< COM interrupt Flag */
#define  TIM_SR_TIF                           ((uint16_t)0x0040)            /*!< Trigger interrupt Flag */
#define  TIM_SR_BIF                           ((uint16_t)0x0080)            /*!< Break interrupt Flag */
#define  TIM_SR_CC1OF                         ((uint16_t)0x0200)            /*!< Capture/Compare 1 Overcapture Flag */
#define  TIM_SR_CC2OF                         ((uint16_t)0x0400)            /*!< Capture/Compare 2 Overcapture Flag */
#define  TIM_SR_CC3OF                         ((uint16_t)0x0800)            /*!< Capture/Compare 3 Overcapture Flag */
#define  TIM_SR_CC4OF                         ((uint16_t)0x1000)            /*!< Capture/Compare 4 Overcapture Flag */

/*******************  Bit definition for TIM_EGR register  ********************/
#define  TIM_EGR_UG                           ((uint8_t)0x01)               /*!< Update Generation */
#define  TIM_EGR_CC1G                         ((uint8_t)0x02)               /*!< Capture/Compare 1 Generation */
#define  TIM_EGR_CC2G                         ((uint8_t)0x04)               /*!< Capture/Compare 2 Generation */
#define  TIM_EGR_CC3G                         ((uint8_t)0x08)               /*!< Capture/Compare 3 Generation */
#define  TIM_EGR_CC4G                         ((uint8_t)0x10)               /*!< Capture/Compare 4 Generation */
#define  TIM_EGR_COMG                         ((uint8_t)0x20)               /*!< Capture/Compare Control Update Generation */
#define  TIM_EGR_TG                           ((uint8_t)0x40)               /*!< Trigger Generation */
#define  TIM_EGR_BG                           ((uint8_t)0x80)               /*!< Break Generation */
```

```
/******************  Bit definition for TIM_CCMR1 register  *******************/
#define  TIM_CCMR1_CC1S                      ((uint16_t)0x0003)            /*!< CC1S[1:0] bits (Capture/Compare 1 Selection) */
#define  TIM_CCMR1_CC1S_0                    ((uint16_t)0x0001)            /*!< Bit 0 */
#define  TIM_CCMR1_CC1S_1                    ((uint16_t)0x0002)            /*!< Bit 1 */

#define  TIM_CCMR1_OC1FE                     ((uint16_t)0x0004)            /*!< Output Compare 1 Fast enable */
#define  TIM_CCMR1_OC1PE                     ((uint16_t)0x0008)            /*!< Output Compare 1 Preload enable */

#define  TIM_CCMR1_OC1M                      ((uint16_t)0x0070)            /*!< OC1M[2:0] bits (Output Compare 1 Mode) */
#define  TIM_CCMR1_OC1M_0                    ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_CCMR1_OC1M_1                    ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_CCMR1_OC1M_2                    ((uint16_t)0x0040)            /*!< Bit 2 */

#define  TIM_CCMR1_OC1CE                     ((uint16_t)0x0080)            /*!< Output Compare 1Clear Enable */

#define  TIM_CCMR1_CC2S                      ((uint16_t)0x0300)            /*!< CC2S[1:0] bits (Capture/Compare 2 Selection) */
#define  TIM_CCMR1_CC2S_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_CCMR1_CC2S_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  TIM_CCMR1_OC2FE                     ((uint16_t)0x0400)            /*!< Output Compare 2 Fast enable */
#define  TIM_CCMR1_OC2PE                     ((uint16_t)0x0800)            /*!< Output Compare 2 Preload enable */

#define  TIM_CCMR1_OC2M                      ((uint16_t)0x7000)            /*!< OC2M[2:0] bits (Output Compare 2 Mode) */
#define  TIM_CCMR1_OC2M_0                    ((uint16_t)0x1000)            /*!< Bit 0 */
#define  TIM_CCMR1_OC2M_1                    ((uint16_t)0x2000)            /*!< Bit 1 */
#define  TIM_CCMR1_OC2M_2                    ((uint16_t)0x4000)            /*!< Bit 2 */

#define  TIM_CCMR1_OC2CE                     ((uint16_t)0x8000)            /*!< Output Compare 2 Clear Enable */

/*----------------------------------------------------------------------------*/

#define  TIM_CCMR1_IC1PSC                    ((uint16_t)0x000C)            /*!< IC1PSC[1:0] bits (Input Capture 1 Prescaler) */
#define  TIM_CCMR1_IC1PSC_0                  ((uint16_t)0x0004)            /*!< Bit 0 */
#define  TIM_CCMR1_IC1PSC_1                  ((uint16_t)0x0008)            /*!< Bit 1 */

#define  TIM_CCMR1_IC1F                      ((uint16_t)0x00F0)            /*!< IC1F[3:0] bits (Input Capture 1 Filter) */
#define  TIM_CCMR1_IC1F_0                    ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_CCMR1_IC1F_1                    ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_CCMR1_IC1F_2                    ((uint16_t)0x0040)            /*!< Bit 2 */
#define  TIM_CCMR1_IC1F_3                    ((uint16_t)0x0080)            /*!< Bit 3 */

#define  TIM_CCMR1_IC2PSC                    ((uint16_t)0x0C00)            /*!< IC2PSC[1:0] bits (Input Capture 2 Prescaler) */
#define  TIM_CCMR1_IC2PSC_0                  ((uint16_t)0x0400)            /*!< Bit 0 */
#define  TIM_CCMR1_IC2PSC_1                  ((uint16_t)0x0800)            /*!< Bit 1 */

#define  TIM_CCMR1_IC2F                      ((uint16_t)0xF000)            /*!< IC2F[3:0] bits (Input Capture 2 Filter) */
#define  TIM_CCMR1_IC2F_0                    ((uint16_t)0x1000)            /*!< Bit 0 */
#define  TIM_CCMR1_IC2F_1                    ((uint16_t)0x2000)            /*!< Bit 1 */
#define  TIM_CCMR1_IC2F_2                    ((uint16_t)0x4000)            /*!< Bit 2 */
#define  TIM_CCMR1_IC2F_3                    ((uint16_t)0x8000)            /*!< Bit 3 */

/******************  Bit definition for TIM_CCMR2 register  *******************/
#define  TIM_CCMR2_CC3S                      ((uint16_t)0x0003)            /*!< CC3S[1:0] bits (Capture/Compare 3 Selection) */
#define  TIM_CCMR2_CC3S_0                    ((uint16_t)0x0001)            /*!< Bit 0 */
#define  TIM_CCMR2_CC3S_1                    ((uint16_t)0x0002)            /*!< Bit 1 */

#define  TIM_CCMR2_OC3FE                     ((uint16_t)0x0004)            /*!< Output Compare 3 Fast
```

```
enable */
#define  TIM_CCMR2_OC3PE                     ((uint16_t)0x0008)            /*!< Output Compare 3 Preload enable */

#define  TIM_CCMR2_OC3M                      ((uint16_t)0x0070)            /*!< OC3M[2:0] bits (Output Compare 3 Mode) */
#define  TIM_CCMR2_OC3M_0                    ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_CCMR2_OC3M_1                    ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_CCMR2_OC3M_2                    ((uint16_t)0x0040)            /*!< Bit 2 */

#define  TIM_CCMR2_OC3CE                     ((uint16_t)0x0080)            /*!< Output Compare 3 Clear Enable */

#define  TIM_CCMR2_CC4S                      ((uint16_t)0x0300)            /*!< CC4S[1:0] bits (Capture/Compare 4 Selection) */
#define  TIM_CCMR2_CC4S_0                    ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_CCMR2_CC4S_1                    ((uint16_t)0x0200)            /*!< Bit 1 */

#define  TIM_CCMR2_OC4FE                     ((uint16_t)0x0400)            /*!< Output Compare 4 Fast enable */
#define  TIM_CCMR2_OC4PE                     ((uint16_t)0x0800)            /*!< Output Compare 4 Preload enable */

#define  TIM_CCMR2_OC4M                      ((uint16_t)0x7000)            /*!< OC4M[2:0] bits (Output Compare 4 Mode) */
#define  TIM_CCMR2_OC4M_0                    ((uint16_t)0x1000)            /*!< Bit 0 */
#define  TIM_CCMR2_OC4M_1                    ((uint16_t)0x2000)            /*!< Bit 1 */
#define  TIM_CCMR2_OC4M_2                    ((uint16_t)0x4000)            /*!< Bit 2 */

#define  TIM_CCMR2_OC4CE                     ((uint16_t)0x8000)            /*!< Output Compare 4 Clear Enable */

/*----------------------------------------------------------------------------*/

#define  TIM_CCMR2_IC3PSC                    ((uint16_t)0x000C)            /*!< IC3PSC[1:0] bits (Input Capture 3 Prescaler) */
#define  TIM_CCMR2_IC3PSC_0                  ((uint16_t)0x0004)            /*!< Bit 0 */
#define  TIM_CCMR2_IC3PSC_1                  ((uint16_t)0x0008)            /*!< Bit 1 */

#define  TIM_CCMR2_IC3F                      ((uint16_t)0x00F0)            /*!< IC3F[3:0] bits (Input Capture 3 Filter) */
#define  TIM_CCMR2_IC3F_0                    ((uint16_t)0x0010)            /*!< Bit 0 */
#define  TIM_CCMR2_IC3F_1                    ((uint16_t)0x0020)            /*!< Bit 1 */
#define  TIM_CCMR2_IC3F_2                    ((uint16_t)0x0040)            /*!< Bit 2 */
#define  TIM_CCMR2_IC3F_3                    ((uint16_t)0x0080)            /*!< Bit 3 */

#define  TIM_CCMR2_IC4PSC                    ((uint16_t)0x0C00)            /*!< IC4PSC[1:0] bits (Input Capture 4 Prescaler) */
#define  TIM_CCMR2_IC4PSC_0                  ((uint16_t)0x0400)            /*!< Bit 0 */
#define  TIM_CCMR2_IC4PSC_1                  ((uint16_t)0x0800)            /*!< Bit 1 */

#define  TIM_CCMR2_IC4F                      ((uint16_t)0xF000)            /*!< IC4F[3:0] bits (Input Capture 4 Filter) */
#define  TIM_CCMR2_IC4F_0                    ((uint16_t)0x1000)            /*!< Bit 0 */
#define  TIM_CCMR2_IC4F_1                    ((uint16_t)0x2000)            /*!< Bit 1 */
#define  TIM_CCMR2_IC4F_2                    ((uint16_t)0x4000)            /*!< Bit 2 */
#define  TIM_CCMR2_IC4F_3                    ((uint16_t)0x8000)            /*!< Bit 3 */

/*******************  Bit definition for TIM_CCER register  *******************/
#define  TIM_CCER_CC1E                       ((uint16_t)0x0001)            /*!< Capture/Compare 1 output enable */
#define  TIM_CCER_CC1P                       ((uint16_t)0x0002)            /*!< Capture/Compare 1 output Polarity */
#define  TIM_CCER_CC1NE                      ((uint16_t)0x0004)            /*!< Capture/Compare 1 Complementary output enable */
#define  TIM_CCER_CC1NP                      ((uint16_t)0x0008)            /*!< Capture/Compare 1 Complementary output Polarity */
#define  TIM_CCER_CC2E                       ((uint16_t)0x0010)            /*!< Capture/Compare 2 output enable */
#define  TIM_CCER_CC2P                       ((uint16_t)0x0020)            /*!< Capture/Compare 2 output Polarity */
#define  TIM_CCER_CC2NE                      ((uint16_t)0x0040)            /*!< Capture/Compare 2 Complementary output enable */
```

```
#define  TIM_CCER_CC2NP                      ((uint16_t)0x0080)            /*!< Capture/Compare 2 Complementary output Polarity */
#define  TIM_CCER_CC3E                       ((uint16_t)0x0100)            /*!< Capture/Compare 3 output enable */
#define  TIM_CCER_CC3P                       ((uint16_t)0x0200)            /*!< Capture/Compare 3 output Polarity */
#define  TIM_CCER_CC3NE                      ((uint16_t)0x0400)            /*!< Capture/Compare 3 Complementary output enable */
#define  TIM_CCER_CC3NP                      ((uint16_t)0x0800)            /*!< Capture/Compare 3 Complementary output Polarity */
#define  TIM_CCER_CC4E                       ((uint16_t)0x1000)            /*!< Capture/Compare 4 output enable */
#define  TIM_CCER_CC4P                       ((uint16_t)0x2000)            /*!< Capture/Compare 4 output Polarity */
#define  TIM_CCER_CC4NP                      ((uint16_t)0x8000)            /*!< Capture/Compare 4 Complementary output Polarity */

/*******************  Bit definition for TIM_CNT register  *******************/
#define  TIM_CNT_CNT                         ((uint16_t)0xFFFF)            /*!< Counter Value */

/*******************  Bit definition for TIM_PSC register  *******************/
#define  TIM_PSC_PSC                         ((uint16_t)0xFFFF)            /*!< Prescaler Value */

/*******************  Bit definition for TIM_ARR register  *******************/
#define  TIM_ARR_ARR                         ((uint16_t)0xFFFF)            /*!< actual auto-reload Value */

/*******************  Bit definition for TIM_RCR register  *******************/
#define  TIM_RCR_REP                         ((uint8_t)0xFF)               /*!< Repetition Counter Value */

/*******************  Bit definition for TIM_CCR1 register  ******************/
#define  TIM_CCR1_CCR1                       ((uint16_t)0xFFFF)            /*!< Capture/Compare 1 Value */

/*******************  Bit definition for TIM_CCR2 register  ******************/
#define  TIM_CCR2_CCR2                       ((uint16_t)0xFFFF)            /*!< Capture/Compare 2 Value */

/*******************  Bit definition for TIM_CCR3 register  ******************/
#define  TIM_CCR3_CCR3                       ((uint16_t)0xFFFF)            /*!< Capture/Compare 3 Value */

/*******************  Bit definition for TIM_CCR4 register  ******************/
#define  TIM_CCR4_CCR4                       ((uint16_t)0xFFFF)            /*!< Capture/Compare 4 Value */

/*******************  Bit definition for TIM_BDTR register  ******************/
#define  TIM_BDTR_DTG                        ((uint16_t)0x00FF)            /*!< DTG[0:7] bits (Dead-Time Generator set-up) */
#define  TIM_BDTR_DTG_0                      ((uint16_t)0x0001)            /*!< Bit 0 */
#define  TIM_BDTR_DTG_1                      ((uint16_t)0x0002)            /*!< Bit 1 */
#define  TIM_BDTR_DTG_2                      ((uint16_t)0x0004)            /*!< Bit 2 */
#define  TIM_BDTR_DTG_3                      ((uint16_t)0x0008)            /*!< Bit 3 */
#define  TIM_BDTR_DTG_4                      ((uint16_t)0x0010)            /*!< Bit 4 */
#define  TIM_BDTR_DTG_5                      ((uint16_t)0x0020)            /*!< Bit 5 */
#define  TIM_BDTR_DTG_6                      ((uint16_t)0x0040)            /*!< Bit 6 */
#define  TIM_BDTR_DTG_7                      ((uint16_t)0x0080)            /*!< Bit 7 */

#define  TIM_BDTR_LOCK                       ((uint16_t)0x0300)            /*!< LOCK[1:0] bits (Lock Configuration) */
#define  TIM_BDTR_LOCK_0                     ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_BDTR_LOCK_1                     ((uint16_t)0x0200)            /*!< Bit 1 */

#define  TIM_BDTR_OSSI                       ((uint16_t)0x0400)            /*!< Off-State Selection for Idle mode */
#define  TIM_BDTR_OSSR                       ((uint16_t)0x0800)            /*!< Off-State Selection for Run mode */
#define  TIM_BDTR_BKE                        ((uint16_t)0x1000)            /*!< Break enable */
#define  TIM_BDTR_BKP                        ((uint16_t)0x2000)            /*!< Break Polarity */
#define  TIM_BDTR_AOE                        ((uint16_t)0x4000)            /*!< Automatic Output enable */
#define  TIM_BDTR_MOE                        ((uint16_t)0x8000)            /*!< Main Output enable */
```

```
/*******************  Bit definition for TIM_DCR register  *******************/
#define  TIM_DCR_DBA                         ((uint16_t)0x001F)            /*!< DBA[4:0] bits (DMA Base Address) */
#define  TIM_DCR_DBA_0                       ((uint16_t)0x0001)            /*!< Bit 0 */
#define  TIM_DCR_DBA_1                       ((uint16_t)0x0002)            /*!< Bit 1 */
#define  TIM_DCR_DBA_2                       ((uint16_t)0x0004)            /*!< Bit 2 */
#define  TIM_DCR_DBA_3                       ((uint16_t)0x0008)            /*!< Bit 3 */
#define  TIM_DCR_DBA_4                       ((uint16_t)0x0010)            /*!< Bit 4 */

#define  TIM_DCR_DBL                         ((uint16_t)0x1F00)            /*!< DBL[4:0] bits (DMA Burst Length) */
#define  TIM_DCR_DBL_0                       ((uint16_t)0x0100)            /*!< Bit 0 */
#define  TIM_DCR_DBL_1                       ((uint16_t)0x0200)            /*!< Bit 1 */
#define  TIM_DCR_DBL_2                       ((uint16_t)0x0400)            /*!< Bit 2 */
#define  TIM_DCR_DBL_3                       ((uint16_t)0x0800)            /*!< Bit 3 */
#define  TIM_DCR_DBL_4                       ((uint16_t)0x1000)            /*!< Bit 4 */

/*******************  Bit definition for TIM_DMAR register  *******************/
#define  TIM_DMAR_DMAB                       ((uint16_t)0xFFFF)            /*!< DMA register for burst accesses */

/******************************************************************************/
/*                                                                            */
/*                             Real-Time Clock                                */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for RTC_CRH register  *******************/
#define  RTC_CRH_SECIE                       ((uint8_t)0x01)               /*!< Second Interrupt Enable */
#define  RTC_CRH_ALRIE                       ((uint8_t)0x02)               /*!< Alarm Interrupt Enable */
#define  RTC_CRH_OWIE                        ((uint8_t)0x04)               /*!< OverfloW Interrupt Enable */

/*******************  Bit definition for RTC_CRL register  *******************/
#define  RTC_CRL_SECF                        ((uint8_t)0x01)               /*!< Second Flag */
#define  RTC_CRL_ALRF                        ((uint8_t)0x02)               /*!< Alarm Flag */
#define  RTC_CRL_OWF                         ((uint8_t)0x04)               /*!< OverfloW Flag */
#define  RTC_CRL_RSF                         ((uint8_t)0x08)               /*!< Registers Synchronized Flag */
#define  RTC_CRL_CNF                         ((uint8_t)0x10)               /*!< Configuration Flag */
#define  RTC_CRL_RTOFF                       ((uint8_t)0x20)               /*!< RTC operation OFF */

/*******************  Bit definition for RTC_PRLH register  *******************/
#define  RTC_PRLH_PRL                        ((uint16_t)0x000F)            /*!< RTC Prescaler Reload Value High */

/*******************  Bit definition for RTC_PRLL register  *******************/
#define  RTC_PRLL_PRL                        ((uint16_t)0xFFFF)            /*!< RTC Prescaler Reload Value Low */

/*******************  Bit definition for RTC_DIVH register  *******************/
#define  RTC_DIVH_RTC_DIV                    ((uint16_t)0x000F)            /*!< RTC Clock Divider High */

/*******************  Bit definition for RTC_DIVL register  *******************/
#define  RTC_DIVL_RTC_DIV                    ((uint16_t)0xFFFF)            /*!< RTC Clock Divider Low */

/*******************  Bit definition for RTC_CNTH register  *******************/
#define  RTC_CNTH_RTC_CNT                    ((uint16_t)0xFFFF)            /*!< RTC Counter High */

/*******************  Bit definition for RTC_CNTL register  *******************/
#define  RTC_CNTL_RTC_CNT                    ((uint16_t)0xFFFF)            /*!< RTC Counter Low */

/*******************  Bit definition for RTC_ALRH register  *******************/
#define  RTC_ALRH_RTC_ALR                    ((uint16_t)0xFFFF)            /*!< RTC Alarm High */

/*******************  Bit definition for RTC_ALRL register  *******************/
#define  RTC_ALRL_RTC_ALR                    ((uint16_t)0xFFFF)            /*!< RTC Alarm Low */

/******************************************************************************/
/*                                                                            */
/*                          Independent WATCHDOG                              */
```

```
  /*                                                                          */
  /****************************************************************************/

  /*******************  Bit definition for IWDG_KR register  *******************/
  #define  IWDG_KR_KEY                         ((uint16_t)0xFFFF)            /*!< Key value (write only, read 0000h) */

  /*******************  Bit definition for IWDG_PR register  *******************/
  #define  IWDG_PR_PR                          ((uint8_t)0x07)               /*!< PR[2:0] (Prescaler divider) */
  #define  IWDG_PR_PR_0                        ((uint8_t)0x01)               /*!< Bit 0 */
  #define  IWDG_PR_PR_1                        ((uint8_t)0x02)               /*!< Bit 1 */
  #define  IWDG_PR_PR_2                        ((uint8_t)0x04)               /*!< Bit 2 */

  /*******************  Bit definition for IWDG_RLR register  *******************/
  #define  IWDG_RLR_RL                         ((uint16_t)0x0FFF)            /*!< Watchdog counter reload value */

  /*******************  Bit definition for IWDG_SR register  ********************/
  #define  IWDG_SR_PVU                         ((uint8_t)0x01)               /*!< Watchdog prescaler value update */
  #define  IWDG_SR_RVU                         ((uint8_t)0x02)               /*!< Watchdog counter reload value update */

  /****************************************************************************/
  /*                                                                          */
  /*                             Window WATCHDOG                              */
  /*                                                                          */
  /****************************************************************************/

  /*******************  Bit definition for WWDG_CR register  ********************/
  #define  WWDG_CR_T                           ((uint8_t)0x7F)               /*!< T[6:0] bits (7-Bit counter (MSB to LSB)) */
  #define  WWDG_CR_T0                          ((uint8_t)0x01)               /*!< Bit 0 */
  #define  WWDG_CR_T1                          ((uint8_t)0x02)               /*!< Bit 1 */
  #define  WWDG_CR_T2                          ((uint8_t)0x04)               /*!< Bit 2 */
  #define  WWDG_CR_T3                          ((uint8_t)0x08)               /*!< Bit 3 */
  #define  WWDG_CR_T4                          ((uint8_t)0x10)               /*!< Bit 4 */
  #define  WWDG_CR_T5                          ((uint8_t)0x20)               /*!< Bit 5 */
  #define  WWDG_CR_T6                          ((uint8_t)0x40)               /*!< Bit 6 */

  #define  WWDG_CR_WDGA                        ((uint8_t)0x80)               /*!< Activation bit */

  /*******************  Bit definition for WWDG_CFR register  *******************/
  #define  WWDG_CFR_W                          ((uint16_t)0x007F)            /*!< W[6:0] bits (7-bit window value) */
  #define  WWDG_CFR_W0                         ((uint16_t)0x0001)            /*!< Bit 0 */
  #define  WWDG_CFR_W1                         ((uint16_t)0x0002)            /*!< Bit 1 */
  #define  WWDG_CFR_W2                         ((uint16_t)0x0004)            /*!< Bit 2 */
  #define  WWDG_CFR_W3                         ((uint16_t)0x0008)            /*!< Bit 3 */
  #define  WWDG_CFR_W4                         ((uint16_t)0x0010)            /*!< Bit 4 */
  #define  WWDG_CFR_W5                         ((uint16_t)0x0020)            /*!< Bit 5 */
  #define  WWDG_CFR_W6                         ((uint16_t)0x0040)            /*!< Bit 6 */

  #define  WWDG_CFR_WDGTB                      ((uint16_t)0x0180)            /*!< WDGTB[1:0] bits (Timer Base) */
  #define  WWDG_CFR_WDGTB0                     ((uint16_t)0x0080)            /*!< Bit 0 */
  #define  WWDG_CFR_WDGTB1                     ((uint16_t)0x0100)            /*!< Bit 1 */

  #define  WWDG_CFR_EWI                        ((uint16_t)0x0200)            /*!< Early Wakeup Interrupt */

  /*******************  Bit definition for WWDG_SR register  ********************/
  #define  WWDG_SR_EWIF                        ((uint8_t)0x01)               /*!< Early Wakeup Interrupt Flag */

  /****************************************************************************/
  /*                                                                          */
  /*                       Flexible Static Memory Controller                  */
  /*                                                                          */
  /****************************************************************************/

  /*******************  Bit definition for FSMC_BCR1 register  *******************/
  #define  FSMC_BCR1_MBKEN                     ((uint32_t)0x00000001)        /*!< Memory bank enable bit */
```

```
#define  FSMC_BCR1_MUXEN                     ((uint32_t)0x00000002)        /*!< Address/data multiplexing enable bit */

#define  FSMC_BCR1_MTYP                      ((uint32_t)0x0000000C)        /*!< MTYP[1:0] bits (Memory type) */
#define  FSMC_BCR1_MTYP_0                    ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  FSMC_BCR1_MTYP_1                    ((uint32_t)0x00000008)        /*!< Bit 1 */

#define  FSMC_BCR1_MWID                      ((uint32_t)0x00000030)        /*!< MWID[1:0] bits (Memory data bus width) */
#define  FSMC_BCR1_MWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BCR1_MWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_BCR1_FACCEN                    ((uint32_t)0x00000040)        /*!< Flash access enable */
#define  FSMC_BCR1_BURSTEN                   ((uint32_t)0x00000100)        /*!< Burst enable bit */
#define  FSMC_BCR1_WAITPOL                   ((uint32_t)0x00000200)        /*!< Wait signal polarity bit */
#define  FSMC_BCR1_WRAPMOD                   ((uint32_t)0x00000400)        /*!< Wrapped burst mode support */
#define  FSMC_BCR1_WAITCFG                   ((uint32_t)0x00000800)        /*!< Wait timing configuration */
#define  FSMC_BCR1_WREN                      ((uint32_t)0x00001000)        /*!< Write enable bit */
#define  FSMC_BCR1_WAITEN                    ((uint32_t)0x00002000)        /*!< Wait enable bit */
#define  FSMC_BCR1_EXTMOD                    ((uint32_t)0x00004000)        /*!< Extended mode enable */
#define  FSMC_BCR1_ASYNCWAIT                 ((uint32_t)0x00008000)       /*!< Asynchronous wait */
#define  FSMC_BCR1_CBURSTRW                  ((uint32_t)0x00080000)        /*!< Write burst enable */

/******************  Bit definition for FSMC_BCR2 register  *******************/
#define  FSMC_BCR2_MBKEN                     ((uint32_t)0x00000001)        /*!< Memory bank enable bit */
#define  FSMC_BCR2_MUXEN                     ((uint32_t)0x00000002)        /*!< Address/data multiplexing enable bit */

#define  FSMC_BCR2_MTYP                      ((uint32_t)0x0000000C)        /*!< MTYP[1:0] bits (Memory type) */
#define  FSMC_BCR2_MTYP_0                    ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  FSMC_BCR2_MTYP_1                    ((uint32_t)0x00000008)        /*!< Bit 1 */

#define  FSMC_BCR2_MWID                      ((uint32_t)0x00000030)        /*!< MWID[1:0] bits (Memory data bus width) */
#define  FSMC_BCR2_MWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BCR2_MWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_BCR2_FACCEN                    ((uint32_t)0x00000040)        /*!< Flash access enable */
#define  FSMC_BCR2_BURSTEN                   ((uint32_t)0x00000100)        /*!< Burst enable bit */
#define  FSMC_BCR2_WAITPOL                   ((uint32_t)0x00000200)        /*!< Wait signal polarity bit */
#define  FSMC_BCR2_WRAPMOD                   ((uint32_t)0x00000400)        /*!< Wrapped burst mode support */
#define  FSMC_BCR2_WAITCFG                   ((uint32_t)0x00000800)        /*!< Wait timing configuration */
#define  FSMC_BCR2_WREN                      ((uint32_t)0x00001000)        /*!< Write enable bit */
#define  FSMC_BCR2_WAITEN                    ((uint32_t)0x00002000)        /*!< Wait enable bit */
#define  FSMC_BCR2_EXTMOD                    ((uint32_t)0x00004000)        /*!< Extended mode enable */
#define  FSMC_BCR2_ASYNCWAIT                 ((uint32_t)0x00008000)       /*!< Asynchronous wait */
#define  FSMC_BCR2_CBURSTRW                  ((uint32_t)0x00080000)        /*!< Write burst enable */

/******************  Bit definition for FSMC_BCR3 register  *******************/
#define  FSMC_BCR3_MBKEN                     ((uint32_t)0x00000001)        /*!< Memory bank enable bit */
#define  FSMC_BCR3_MUXEN                     ((uint32_t)0x00000002)        /*!< Address/data multiplexing enable bit */

#define  FSMC_BCR3_MTYP                      ((uint32_t)0x0000000C)        /*!< MTYP[1:0] bits (Memory type) */
#define  FSMC_BCR3_MTYP_0                    ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  FSMC_BCR3_MTYP_1                    ((uint32_t)0x00000008)        /*!< Bit 1 */

#define  FSMC_BCR3_MWID                      ((uint32_t)0x00000030)        /*!< MWID[1:0] bits (Memory data bus width) */
#define  FSMC_BCR3_MWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BCR3_MWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_BCR3_FACCEN                    ((uint32_t)0x00000040)        /*!< Flash access enable */
#define  FSMC_BCR3_BURSTEN                   ((uint32_t)0x00000100)        /*!< Burst enable bit */
```

```
#define  FSMC_BCR3_WAITPOL                    ((uint32_t)0x00000200)        /*!< Wait signal polarity bit. */
#define  FSMC_BCR3_WRAPMOD                    ((uint32_t)0x00000400)        /*!< Wrapped burst mode support */
#define  FSMC_BCR3_WAITCFG                    ((uint32_t)0x00000800)        /*!< Wait timing configuration */
#define  FSMC_BCR3_WREN                       ((uint32_t)0x00001000)        /*!< Write enable bit */
#define  FSMC_BCR3_WAITEN                     ((uint32_t)0x00002000)        /*!< Wait enable bit */
#define  FSMC_BCR3_EXTMOD                     ((uint32_t)0x00004000)        /*!< Extended mode enable */
#define  FSMC_BCR3_ASYNCWAIT                  ((uint32_t)0x00008000)       /*!< Asynchronous wait */
#define  FSMC_BCR3_CBURSTRW                   ((uint32_t)0x00080000)        /*!< Write burst enable */

/******************  Bit definition for FSMC_BCR4 register  ******************/
#define  FSMC_BCR4_MBKEN                      ((uint32_t)0x00000001)        /*!< Memory bank enable bit */
#define  FSMC_BCR4_MUXEN                      ((uint32_t)0x00000002)        /*!< Address/data multiplexing enable bit */

#define  FSMC_BCR4_MTYP                       ((uint32_t)0x0000000C)        /*!< MTYP[1:0] bits (Memory type) */
#define  FSMC_BCR4_MTYP_0                     ((uint32_t)0x00000004)        /*!< Bit 0 */
#define  FSMC_BCR4_MTYP_1                     ((uint32_t)0x00000008)        /*!< Bit 1 */

#define  FSMC_BCR4_MWID                       ((uint32_t)0x00000030)        /*!< MWID[1:0] bits (Memory data bus width) */
#define  FSMC_BCR4_MWID_0                     ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BCR4_MWID_1                     ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_BCR4_FACCEN                     ((uint32_t)0x00000040)        /*!< Flash access enable */
#define  FSMC_BCR4_BURSTEN                    ((uint32_t)0x00000100)        /*!< Burst enable bit */
#define  FSMC_BCR4_WAITPOL                    ((uint32_t)0x00000200)        /*!< Wait signal polarity bit */
#define  FSMC_BCR4_WRAPMOD                    ((uint32_t)0x00000400)        /*!< Wrapped burst mode support */
#define  FSMC_BCR4_WAITCFG                    ((uint32_t)0x00000800)        /*!< Wait timing configuration */
#define  FSMC_BCR4_WREN                       ((uint32_t)0x00001000)        /*!< Write enable bit */
#define  FSMC_BCR4_WAITEN                     ((uint32_t)0x00002000)        /*!< Wait enable bit */
#define  FSMC_BCR4_EXTMOD                     ((uint32_t)0x00004000)        /*!< Extended mode enable */
#define  FSMC_BCR4_ASYNCWAIT                  ((uint32_t)0x00008000)       /*!< Asynchronous wait */
#define  FSMC_BCR4_CBURSTRW                   ((uint32_t)0x00080000)        /*!< Write burst enable */

/******************  Bit definition for FSMC_BTR1 register  ******************/
#define  FSMC_BTR1_ADDSET                     ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BTR1_ADDSET_0                   ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BTR1_ADDSET_1                   ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BTR1_ADDSET_2                   ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BTR1_ADDSET_3                   ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BTR1_ADDHLD                     ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits (Address-hold phase duration) */
#define  FSMC_BTR1_ADDHLD_0                   ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BTR1_ADDHLD_1                   ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BTR1_ADDHLD_2                   ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BTR1_ADDHLD_3                   ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BTR1_DATAST                     ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BTR1_DATAST_0                   ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BTR1_DATAST_1                   ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BTR1_DATAST_2                   ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BTR1_DATAST_3                   ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BTR1_BUSTURN                    ((uint32_t)0x000F0000)        /*!< BUSTURN[3:0] bits (Bus turnaround phase duration) */
#define  FSMC_BTR1_BUSTURN_0                  ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_BTR1_BUSTURN_1                  ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_BTR1_BUSTURN_2                  ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_BTR1_BUSTURN_3                  ((uint32_t)0x00080000)        /*!< Bit 3 */

#define  FSMC_BTR1_CLKDIV                     ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BTR1_CLKDIV_0                   ((uint32_t)0x00100000)        /*!< Bit 0 */
```

```
#define  FSMC_BTR1_CLKDIV_1                  ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BTR1_CLKDIV_2                  ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BTR1_CLKDIV_3                  ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BTR1_DATLAT                    ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BTR1_DATLAT_0                  ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BTR1_DATLAT_1                  ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BTR1_DATLAT_2                  ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BTR1_DATLAT_3                  ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BTR1_ACCMOD                    ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BTR1_ACCMOD_0                  ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BTR1_ACCMOD_1                  ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BTR2 register  *******************/
#define  FSMC_BTR2_ADDSET                    ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BTR2_ADDSET_0                  ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BTR2_ADDSET_1                  ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BTR2_ADDSET_2                  ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BTR2_ADDSET_3                  ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BTR2_ADDHLD                    ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits (Address-hold phase duration) */
#define  FSMC_BTR2_ADDHLD_0                  ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BTR2_ADDHLD_1                  ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BTR2_ADDHLD_2                  ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BTR2_ADDHLD_3                  ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BTR2_DATAST                    ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BTR2_DATAST_0                  ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BTR2_DATAST_1                  ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BTR2_DATAST_2                  ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BTR2_DATAST_3                  ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BTR2_BUSTURN                   ((uint32_t)0x000F0000)        /*!< BUSTURN[3:0] bits (Bus turnaround phase duration) */
#define  FSMC_BTR2_BUSTURN_0                 ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_BTR2_BUSTURN_1                 ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_BTR2_BUSTURN_2                 ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_BTR2_BUSTURN_3                 ((uint32_t)0x00080000)        /*!< Bit 3 */

#define  FSMC_BTR2_CLKDIV                    ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BTR2_CLKDIV_0                  ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BTR2_CLKDIV_1                  ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BTR2_CLKDIV_2                  ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BTR2_CLKDIV_3                  ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BTR2_DATLAT                    ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BTR2_DATLAT_0                  ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BTR2_DATLAT_1                  ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BTR2_DATLAT_2                  ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BTR2_DATLAT_3                  ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BTR2_ACCMOD                    ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BTR2_ACCMOD_0                  ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BTR2_ACCMOD_1                  ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BTR3 register  *******************/
#define  FSMC_BTR3_ADDSET                    ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BTR3_ADDSET_0                  ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BTR3_ADDSET_1                  ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BTR3_ADDSET_2                  ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BTR3_ADDSET_3                  ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BTR3_ADDHLD                    ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits
```

```
(Address-hold phase duration) */
#define  FSMC_BTR3_ADDHLD_0                  ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BTR3_ADDHLD_1                  ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BTR3_ADDHLD_2                  ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BTR3_ADDHLD_3                  ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BTR3_DATAST                    ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits
(Data-phase duration) */
#define  FSMC_BTR3_DATAST_0                  ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BTR3_DATAST_1                  ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BTR3_DATAST_2                  ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BTR3_DATAST_3                  ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BTR3_BUSTURN                   ((uint32_t)0x000F0000)        /*!< BUSTURN[3:0] bits (Bus
turnaround phase duration) */
#define  FSMC_BTR3_BUSTURN_0                 ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_BTR3_BUSTURN_1                 ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_BTR3_BUSTURN_2                 ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_BTR3_BUSTURN_3                 ((uint32_t)0x00080000)        /*!< Bit 3 */

#define  FSMC_BTR3_CLKDIV                    ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock
divide ratio) */
#define  FSMC_BTR3_CLKDIV_0                  ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BTR3_CLKDIV_1                  ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BTR3_CLKDIV_2                  ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BTR3_CLKDIV_3                  ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BTR3_DATLAT                    ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data
latency) */
#define  FSMC_BTR3_DATLAT_0                  ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BTR3_DATLAT_1                  ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BTR3_DATLAT_2                  ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BTR3_DATLAT_3                  ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BTR3_ACCMOD                    ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits
(Access mode) */
#define  FSMC_BTR3_ACCMOD_0                  ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BTR3_ACCMOD_1                  ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BTR4 register  *******************/
#define  FSMC_BTR4_ADDSET                    ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits
(Address setup phase duration) */
#define  FSMC_BTR4_ADDSET_0                  ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BTR4_ADDSET_1                  ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BTR4_ADDSET_2                  ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BTR4_ADDSET_3                  ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BTR4_ADDHLD                    ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits
(Address-hold phase duration) */
#define  FSMC_BTR4_ADDHLD_0                  ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BTR4_ADDHLD_1                  ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BTR4_ADDHLD_2                  ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BTR4_ADDHLD_3                  ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BTR4_DATAST                    ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits
(Data-phase duration) */
#define  FSMC_BTR4_DATAST_0                  ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BTR4_DATAST_1                  ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BTR4_DATAST_2                  ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BTR4_DATAST_3                  ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BTR4_BUSTURN                   ((uint32_t)0x000F0000)        /*!< BUSTURN[3:0] bits (Bus
turnaround phase duration) */
#define  FSMC_BTR4_BUSTURN_0                 ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_BTR4_BUSTURN_1                 ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_BTR4_BUSTURN_2                 ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_BTR4_BUSTURN_3                 ((uint32_t)0x00080000)        /*!< Bit 3 */

#define  FSMC_BTR4_CLKDIV                    ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock
divide ratio) */
#define  FSMC_BTR4_CLKDIV_0                  ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BTR4_CLKDIV_1                  ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BTR4_CLKDIV_2                  ((uint32_t)0x00400000)        /*!< Bit 2 */
```

```
#define  FSMC_BTR4_CLKDIV_3                  ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BTR4_DATLAT                    ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BTR4_DATLAT_0                  ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BTR4_DATLAT_1                  ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BTR4_DATLAT_2                  ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BTR4_DATLAT_3                  ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BTR4_ACCMOD                    ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BTR4_ACCMOD_0                  ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BTR4_ACCMOD_1                  ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BWTR1 register  ******************/
#define  FSMC_BWTR1_ADDSET                   ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BWTR1_ADDSET_0                 ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BWTR1_ADDSET_1                 ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BWTR1_ADDSET_2                 ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BWTR1_ADDSET_3                 ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BWTR1_ADDHLD                   ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits (Address-hold phase duration) */
#define  FSMC_BWTR1_ADDHLD_0                 ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BWTR1_ADDHLD_1                 ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BWTR1_ADDHLD_2                 ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BWTR1_ADDHLD_3                 ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BWTR1_DATAST                   ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BWTR1_DATAST_0                 ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BWTR1_DATAST_1                 ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BWTR1_DATAST_2                 ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BWTR1_DATAST_3                 ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BWTR1_CLKDIV                   ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BWTR1_CLKDIV_0                 ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BWTR1_CLKDIV_1                 ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BWTR1_CLKDIV_2                 ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BWTR1_CLKDIV_3                 ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BWTR1_DATLAT                   ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BWTR1_DATLAT_0                 ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BWTR1_DATLAT_1                 ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BWTR1_DATLAT_2                 ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BWTR1_DATLAT_3                 ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BWTR1_ACCMOD                   ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BWTR1_ACCMOD_0                 ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BWTR1_ACCMOD_1                 ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BWTR2 register  ******************/
#define  FSMC_BWTR2_ADDSET                   ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BWTR2_ADDSET_0                 ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BWTR2_ADDSET_1                 ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BWTR2_ADDSET_2                 ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BWTR2_ADDSET_3                 ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BWTR2_ADDHLD                   ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits (Address-hold phase duration) */
#define  FSMC_BWTR2_ADDHLD_0                 ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BWTR2_ADDHLD_1                 ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BWTR2_ADDHLD_2                 ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BWTR2_ADDHLD_3                 ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BWTR2_DATAST                   ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BWTR2_DATAST_0                 ((uint32_t)0x00000100)        /*!< Bit 0 */
```

```
#define  FSMC_BWTR2_DATAST_1                 ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BWTR2_DATAST_2                 ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BWTR2_DATAST_3                 ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BWTR2_CLKDIV                   ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BWTR2_CLKDIV_0                 ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BWTR2_CLKDIV_1                 ((uint32_t)0x00200000)        /*!< Bit 1*/
#define  FSMC_BWTR2_CLKDIV_2                 ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BWTR2_CLKDIV_3                 ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BWTR2_DATLAT                   ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BWTR2_DATLAT_0                 ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BWTR2_DATLAT_1                 ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BWTR2_DATLAT_2                 ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BWTR2_DATLAT_3                 ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BWTR2_ACCMOD                   ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BWTR2_ACCMOD_0                 ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BWTR2_ACCMOD_1                 ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BWTR3 register  ******************/
#define  FSMC_BWTR3_ADDSET                   ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BWTR3_ADDSET_0                 ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BWTR3_ADDSET_1                 ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BWTR3_ADDSET_2                 ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BWTR3_ADDSET_3                 ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BWTR3_ADDHLD                   ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits (Address-hold phase duration) */
#define  FSMC_BWTR3_ADDHLD_0                 ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BWTR3_ADDHLD_1                 ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BWTR3_ADDHLD_2                 ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BWTR3_ADDHLD_3                 ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BWTR3_DATAST                   ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BWTR3_DATAST_0                 ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BWTR3_DATAST_1                 ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BWTR3_DATAST_2                 ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BWTR3_DATAST_3                 ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BWTR3_CLKDIV                   ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BWTR3_CLKDIV_0                 ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BWTR3_CLKDIV_1                 ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BWTR3_CLKDIV_2                 ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BWTR3_CLKDIV_3                 ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BWTR3_DATLAT                   ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BWTR3_DATLAT_0                 ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BWTR3_DATLAT_1                 ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BWTR3_DATLAT_2                 ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BWTR3_DATLAT_3                 ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BWTR3_ACCMOD                   ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BWTR3_ACCMOD_0                 ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BWTR3_ACCMOD_1                 ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_BWTR4 register  ******************/
#define  FSMC_BWTR4_ADDSET                   ((uint32_t)0x0000000F)        /*!< ADDSET[3:0] bits (Address setup phase duration) */
#define  FSMC_BWTR4_ADDSET_0                 ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_BWTR4_ADDSET_1                 ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_BWTR4_ADDSET_2                 ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_BWTR4_ADDSET_3                 ((uint32_t)0x00000008)        /*!< Bit 3 */

#define  FSMC_BWTR4_ADDHLD                   ((uint32_t)0x000000F0)        /*!< ADDHLD[3:0] bits
```

```
(Address-hold phase duration) */
#define  FSMC_BWTR4_ADDHLD_0                 ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_BWTR4_ADDHLD_1                 ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  FSMC_BWTR4_ADDHLD_2                 ((uint32_t)0x00000040)        /*!< Bit 2 */
#define  FSMC_BWTR4_ADDHLD_3                 ((uint32_t)0x00000080)        /*!< Bit 3 */

#define  FSMC_BWTR4_DATAST                   ((uint32_t)0x0000FF00)        /*!< DATAST [3:0] bits (Data-phase duration) */
#define  FSMC_BWTR4_DATAST_0                 ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_BWTR4_DATAST_1                 ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_BWTR4_DATAST_2                 ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_BWTR4_DATAST_3                 ((uint32_t)0x00000800)        /*!< Bit 3 */

#define  FSMC_BWTR4_CLKDIV                   ((uint32_t)0x00F00000)        /*!< CLKDIV[3:0] bits (Clock divide ratio) */
#define  FSMC_BWTR4_CLKDIV_0                 ((uint32_t)0x00100000)        /*!< Bit 0 */
#define  FSMC_BWTR4_CLKDIV_1                 ((uint32_t)0x00200000)        /*!< Bit 1 */
#define  FSMC_BWTR4_CLKDIV_2                 ((uint32_t)0x00400000)        /*!< Bit 2 */
#define  FSMC_BWTR4_CLKDIV_3                 ((uint32_t)0x00800000)        /*!< Bit 3 */

#define  FSMC_BWTR4_DATLAT                   ((uint32_t)0x0F000000)        /*!< DATLA[3:0] bits (Data latency) */
#define  FSMC_BWTR4_DATLAT_0                 ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_BWTR4_DATLAT_1                 ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_BWTR4_DATLAT_2                 ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_BWTR4_DATLAT_3                 ((uint32_t)0x08000000)        /*!< Bit 3 */

#define  FSMC_BWTR4_ACCMOD                   ((uint32_t)0x30000000)        /*!< ACCMOD[1:0] bits (Access mode) */
#define  FSMC_BWTR4_ACCMOD_0                 ((uint32_t)0x10000000)        /*!< Bit 0 */
#define  FSMC_BWTR4_ACCMOD_1                 ((uint32_t)0x20000000)        /*!< Bit 1 */

/******************  Bit definition for FSMC_PCR2 register  *******************/
#define  FSMC_PCR2_PWAITEN                   ((uint32_t)0x00000002)        /*!< Wait feature enable bit */
#define  FSMC_PCR2_PBKEN                     ((uint32_t)0x00000004)        /*!< PC Card/NAND Flash memory bank enable bit */
#define  FSMC_PCR2_PTYP                      ((uint32_t)0x00000008)        /*!< Memory type */

#define  FSMC_PCR2_PWID                      ((uint32_t)0x00000030)        /*!< PWID[1:0] bits (NAND Flash databus width) */
#define  FSMC_PCR2_PWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_PCR2_PWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_PCR2_ECCEN                     ((uint32_t)0x00000040)        /*!< ECC computation logic enable bit */

#define  FSMC_PCR2_TCLR                      ((uint32_t)0x00001E00)        /*!< TCLR[3:0] bits (CLE to RE delay) */
#define  FSMC_PCR2_TCLR_0                    ((uint32_t)0x00000200)        /*!< Bit 0 */
#define  FSMC_PCR2_TCLR_1                    ((uint32_t)0x00000400)        /*!< Bit 1 */
#define  FSMC_PCR2_TCLR_2                    ((uint32_t)0x00000800)        /*!< Bit 2 */
#define  FSMC_PCR2_TCLR_3                    ((uint32_t)0x00001000)        /*!< Bit 3 */

#define  FSMC_PCR2_TAR                       ((uint32_t)0x0001E000)        /*!< TAR[3:0] bits (ALE to RE delay) */
#define  FSMC_PCR2_TAR_0                     ((uint32_t)0x00002000)        /*!< Bit 0 */
#define  FSMC_PCR2_TAR_1                     ((uint32_t)0x00004000)        /*!< Bit 1 */
#define  FSMC_PCR2_TAR_2                     ((uint32_t)0x00008000)        /*!< Bit 2 */
#define  FSMC_PCR2_TAR_3                     ((uint32_t)0x00010000)        /*!< Bit 3 */

#define  FSMC_PCR2_ECCPS                     ((uint32_t)0x000E0000)        /*!< ECCPS[1:0] bits (ECC page size) */
#define  FSMC_PCR2_ECCPS_0                   ((uint32_t)0x00020000)        /*!< Bit 0 */
#define  FSMC_PCR2_ECCPS_1                   ((uint32_t)0x00040000)        /*!< Bit 1 */
#define  FSMC_PCR2_ECCPS_2                   ((uint32_t)0x00080000)        /*!< Bit 2 */

/******************  Bit definition for FSMC_PCR3 register  *******************/
#define  FSMC_PCR3_PWAITEN                   ((uint32_t)0x00000002)        /*!< Wait feature enable bit */
#define  FSMC_PCR3_PBKEN                     ((uint32_t)0x00000004)        /*!< PC Card/NAND Flash memory bank enable bit */
#define  FSMC_PCR3_PTYP                      ((uint32_t)0x00000008)        /*!< Memory type */
```

```
#define  FSMC_PCR3_PWID                      ((uint32_t)0x00000030)        /*!< PWID[1:0] bits (NAND Flash databus width) */
#define  FSMC_PCR3_PWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_PCR3_PWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_PCR3_ECCEN                     ((uint32_t)0x00000040)        /*!< ECC computation logic enable bit */

#define  FSMC_PCR3_TCLR                      ((uint32_t)0x00001E00)        /*!< TCLR[3:0] bits (CLE to RE delay) */
#define  FSMC_PCR3_TCLR_0                    ((uint32_t)0x00000200)        /*!< Bit 0 */
#define  FSMC_PCR3_TCLR_1                    ((uint32_t)0x00000400)        /*!< Bit 1 */
#define  FSMC_PCR3_TCLR_2                    ((uint32_t)0x00000800)        /*!< Bit 2 */
#define  FSMC_PCR3_TCLR_3                    ((uint32_t)0x00001000)        /*!< Bit 3 */

#define  FSMC_PCR3_TAR                       ((uint32_t)0x0001E000)        /*!< TAR[3:0] bits (ALE to RE delay) */
#define  FSMC_PCR3_TAR_0                     ((uint32_t)0x00002000)        /*!< Bit 0 */
#define  FSMC_PCR3_TAR_1                     ((uint32_t)0x00004000)        /*!< Bit 1 */
#define  FSMC_PCR3_TAR_2                     ((uint32_t)0x00008000)        /*!< Bit 2 */
#define  FSMC_PCR3_TAR_3                     ((uint32_t)0x00010000)        /*!< Bit 3 */

#define  FSMC_PCR3_ECCPS                     ((uint32_t)0x000E0000)        /*!< ECCPS[2:0] bits (ECC page size) */
#define  FSMC_PCR3_ECCPS_0                   ((uint32_t)0x00020000)        /*!< Bit 0 */
#define  FSMC_PCR3_ECCPS_1                   ((uint32_t)0x00040000)        /*!< Bit 1 */
#define  FSMC_PCR3_ECCPS_2                   ((uint32_t)0x00080000)        /*!< Bit 2 */

/******************  Bit definition for FSMC_PCR4 register  *******************/
#define  FSMC_PCR4_PWAITEN                   ((uint32_t)0x00000002)        /*!< Wait feature enable bit */
#define  FSMC_PCR4_PBKEN                     ((uint32_t)0x00000004)        /*!< PC Card/NAND Flash memory bank enable bit */
#define  FSMC_PCR4_PTYP                      ((uint32_t)0x00000008)        /*!< Memory type */

#define  FSMC_PCR4_PWID                      ((uint32_t)0x00000030)        /*!< PWID[1:0] bits (NAND Flash databus width) */
#define  FSMC_PCR4_PWID_0                    ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  FSMC_PCR4_PWID_1                    ((uint32_t)0x00000020)        /*!< Bit 1 */

#define  FSMC_PCR4_ECCEN                     ((uint32_t)0x00000040)        /*!< ECC computation logic enable bit */

#define  FSMC_PCR4_TCLR                      ((uint32_t)0x00001E00)        /*!< TCLR[3:0] bits (CLE to RE delay) */
#define  FSMC_PCR4_TCLR_0                    ((uint32_t)0x00000200)        /*!< Bit 0 */
#define  FSMC_PCR4_TCLR_1                    ((uint32_t)0x00000400)        /*!< Bit 1 */
#define  FSMC_PCR4_TCLR_2                    ((uint32_t)0x00000800)        /*!< Bit 2 */
#define  FSMC_PCR4_TCLR_3                    ((uint32_t)0x00001000)        /*!< Bit 3 */

#define  FSMC_PCR4_TAR                       ((uint32_t)0x0001E000)        /*!< TAR[3:0] bits (ALE to RE delay) */
#define  FSMC_PCR4_TAR_0                     ((uint32_t)0x00002000)        /*!< Bit 0 */
#define  FSMC_PCR4_TAR_1                     ((uint32_t)0x00004000)        /*!< Bit 1 */
#define  FSMC_PCR4_TAR_2                     ((uint32_t)0x00008000)        /*!< Bit 2 */
#define  FSMC_PCR4_TAR_3                     ((uint32_t)0x00010000)        /*!< Bit 3 */

#define  FSMC_PCR4_ECCPS                     ((uint32_t)0x000E0000)        /*!< ECCPS[2:0] bits (ECC page size) */
#define  FSMC_PCR4_ECCPS_0                   ((uint32_t)0x00020000)        /*!< Bit 0 */
#define  FSMC_PCR4_ECCPS_1                   ((uint32_t)0x00040000)        /*!< Bit 1 */
#define  FSMC_PCR4_ECCPS_2                   ((uint32_t)0x00080000)        /*!< Bit 2 */

/******************  Bit definition for FSMC_SR2 register  *******************/
#define  FSMC_SR2_IRS                        ((uint8_t)0x01)               /*!< Interrupt Rising Edge status */
#define  FSMC_SR2_ILS                        ((uint8_t)0x02)               /*!< Interrupt Level status */
#define  FSMC_SR2_IFS                        ((uint8_t)0x04)               /*!< Interrupt Falling Edge status */
#define  FSMC_SR2_IREN                       ((uint8_t)0x08)               /*!< Interrupt Rising Edge detection Enable bit */
#define  FSMC_SR2_ILEN                       ((uint8_t)0x10)               /*!< Interrupt Level
```

```
detection Enable bit */
#define  FSMC_SR2_IFEN                       ((uint8_t)0x20)              /*!< Interrupt Falling Edge detection Enable bit */
#define  FSMC_SR2_FEMPT                      ((uint8_t)0x40)              /*!< FIFO empty */

/*******************  Bit definition for FSMC_SR3 register  *******************/
#define  FSMC_SR3_IRS                        ((uint8_t)0x01)              /*!< Interrupt Rising Edge status */
#define  FSMC_SR3_ILS                        ((uint8_t)0x02)              /*!< Interrupt Level status */
#define  FSMC_SR3_IFS                        ((uint8_t)0x04)              /*!< Interrupt Falling Edge status */
#define  FSMC_SR3_IREN                       ((uint8_t)0x08)              /*!< Interrupt Rising Edge detection Enable bit */
#define  FSMC_SR3_ILEN                       ((uint8_t)0x10)              /*!< Interrupt Level detection Enable bit */
#define  FSMC_SR3_IFEN                       ((uint8_t)0x20)              /*!< Interrupt Falling Edge detection Enable bit */
#define  FSMC_SR3_FEMPT                      ((uint8_t)0x40)              /*!< FIFO empty */

/*******************  Bit definition for FSMC_SR4 register  *******************/
#define  FSMC_SR4_IRS                        ((uint8_t)0x01)              /*!< Interrupt Rising Edge status */
#define  FSMC_SR4_ILS                        ((uint8_t)0x02)              /*!< Interrupt Level status */
#define  FSMC_SR4_IFS                        ((uint8_t)0x04)              /*!< Interrupt Falling Edge status */
#define  FSMC_SR4_IREN                       ((uint8_t)0x08)              /*!< Interrupt Rising Edge detection Enable bit */
#define  FSMC_SR4_ILEN                       ((uint8_t)0x10)              /*!< Interrupt Level detection Enable bit */
#define  FSMC_SR4_IFEN                       ((uint8_t)0x20)              /*!< Interrupt Falling Edge detection Enable bit */
#define  FSMC_SR4_FEMPT                      ((uint8_t)0x40)              /*!< FIFO empty */

/******************  Bit definition for FSMC_PMEM2 register  ******************/
#define  FSMC_PMEM2_MEMSET2                  ((uint32_t)0x000000FF)       /*!< MEMSET2[7:0] bits (Common memory 2 setup time) */
#define  FSMC_PMEM2_MEMSET2_0                ((uint32_t)0x00000001)       /*!< Bit 0 */
#define  FSMC_PMEM2_MEMSET2_1                ((uint32_t)0x00000002)       /*!< Bit 1 */
#define  FSMC_PMEM2_MEMSET2_2                ((uint32_t)0x00000004)       /*!< Bit 2 */
#define  FSMC_PMEM2_MEMSET2_3                ((uint32_t)0x00000008)       /*!< Bit 3 */
#define  FSMC_PMEM2_MEMSET2_4                ((uint32_t)0x00000010)       /*!< Bit 4 */
#define  FSMC_PMEM2_MEMSET2_5                ((uint32_t)0x00000020)       /*!< Bit 5 */
#define  FSMC_PMEM2_MEMSET2_6                ((uint32_t)0x00000040)       /*!< Bit 6 */
#define  FSMC_PMEM2_MEMSET2_7                ((uint32_t)0x00000080)       /*!< Bit 7 */

#define  FSMC_PMEM2_MEMWAIT2                 ((uint32_t)0x0000FF00)       /*!< MEMWAIT2[7:0] bits (Common memory 2 wait time) */
#define  FSMC_PMEM2_MEMWAIT2_0               ((uint32_t)0x00000100)       /*!< Bit 0 */
#define  FSMC_PMEM2_MEMWAIT2_1               ((uint32_t)0x00000200)       /*!< Bit 1 */
#define  FSMC_PMEM2_MEMWAIT2_2               ((uint32_t)0x00000400)       /*!< Bit 2 */
#define  FSMC_PMEM2_MEMWAIT2_3               ((uint32_t)0x00000800)       /*!< Bit 3 */
#define  FSMC_PMEM2_MEMWAIT2_4               ((uint32_t)0x00001000)       /*!< Bit 4 */
#define  FSMC_PMEM2_MEMWAIT2_5               ((uint32_t)0x00002000)       /*!< Bit 5 */
#define  FSMC_PMEM2_MEMWAIT2_6               ((uint32_t)0x00004000)       /*!< Bit 6 */
#define  FSMC_PMEM2_MEMWAIT2_7               ((uint32_t)0x00008000)       /*!< Bit 7 */

#define  FSMC_PMEM2_MEMHOLD2                 ((uint32_t)0x00FF0000)       /*!< MEMHOLD2[7:0] bits (Common memory 2 hold time) */
#define  FSMC_PMEM2_MEMHOLD2_0               ((uint32_t)0x00010000)       /*!< Bit 0 */
#define  FSMC_PMEM2_MEMHOLD2_1               ((uint32_t)0x00020000)       /*!< Bit 1 */
#define  FSMC_PMEM2_MEMHOLD2_2               ((uint32_t)0x00040000)       /*!< Bit 2 */
#define  FSMC_PMEM2_MEMHOLD2_3               ((uint32_t)0x00080000)       /*!< Bit 3 */
#define  FSMC_PMEM2_MEMHOLD2_4               ((uint32_t)0x00100000)       /*!< Bit 4 */
#define  FSMC_PMEM2_MEMHOLD2_5               ((uint32_t)0x00200000)       /*!< Bit 5 */
#define  FSMC_PMEM2_MEMHOLD2_6               ((uint32_t)0x00400000)       /*!< Bit 6 */
#define  FSMC_PMEM2_MEMHOLD2_7               ((uint32_t)0x00800000)       /*!< Bit 7 */

#define  FSMC_PMEM2_MEMHIZ2                  ((uint32_t)0xFF000000)       /*!< MEMHIZ2[7:0] bits (Common memory 2 databus HiZ time) */
#define  FSMC_PMEM2_MEMHIZ2_0                ((uint32_t)0x01000000)       /*!< Bit 0 */
#define  FSMC_PMEM2_MEMHIZ2_1                ((uint32_t)0x02000000)       /*!< Bit 1 */
#define  FSMC_PMEM2_MEMHIZ2_2                ((uint32_t)0x04000000)       /*!< Bit 2 */
#define  FSMC_PMEM2_MEMHIZ2_3                ((uint32_t)0x08000000)       /*!< Bit 3 */
```

```
#define  FSMC_PMEM2_MEMHIZ2_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PMEM2_MEMHIZ2_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PMEM2_MEMHIZ2_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PMEM2_MEMHIZ2_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PMEM3 register  ******************/
#define  FSMC_PMEM3_MEMSET3                  ((uint32_t)0x000000FF)        /*!< MEMSET3[7:0] bits (Common memory 3 setup time) */
#define  FSMC_PMEM3_MEMSET3_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PMEM3_MEMSET3_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PMEM3_MEMSET3_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_PMEM3_MEMSET3_3                ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PMEM3_MEMSET3_4                ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PMEM3_MEMSET3_5                ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PMEM3_MEMSET3_6                ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PMEM3_MEMSET3_7                ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PMEM3_MEMWAIT3                 ((uint32_t)0x0000FF00)        /*!< MEMWAIT3[7:0] bits (Common memory 3 wait time) */
#define  FSMC_PMEM3_MEMWAIT3_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PMEM3_MEMWAIT3_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PMEM3_MEMWAIT3_2               ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PMEM3_MEMWAIT3_3               ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PMEM3_MEMWAIT3_4               ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PMEM3_MEMWAIT3_5               ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PMEM3_MEMWAIT3_6               ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PMEM3_MEMWAIT3_7               ((uint32_t)0x00008000)        /*!< Bit 7 */

#define  FSMC_PMEM3_MEMHOLD3                 ((uint32_t)0x00FF0000)        /*!< MEMHOLD3[7:0] bits (Common memory 3 hold time) */
#define  FSMC_PMEM3_MEMHOLD3_0               ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PMEM3_MEMHOLD3_1               ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PMEM3_MEMHOLD3_2               ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PMEM3_MEMHOLD3_3               ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PMEM3_MEMHOLD3_4               ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PMEM3_MEMHOLD3_5               ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PMEM3_MEMHOLD3_6               ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PMEM3_MEMHOLD3_7               ((uint32_t)0x00800000)        /*!< Bit 7 */

#define  FSMC_PMEM3_MEMHIZ3                  ((uint32_t)0xFF000000)        /*!< MEMHIZ3[7:0] bits (Common memory 3 databus HiZ time) */
#define  FSMC_PMEM3_MEMHIZ3_0                ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PMEM3_MEMHIZ3_1                ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PMEM3_MEMHIZ3_2                ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PMEM3_MEMHIZ3_3                ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PMEM3_MEMHIZ3_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PMEM3_MEMHIZ3_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PMEM3_MEMHIZ3_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PMEM3_MEMHIZ3_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PMEM4 register  ******************/
#define  FSMC_PMEM4_MEMSET4                  ((uint32_t)0x000000FF)        /*!< MEMSET4[7:0] bits (Common memory 4 setup time) */
#define  FSMC_PMEM4_MEMSET4_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PMEM4_MEMSET4_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PMEM4_MEMSET4_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_PMEM4_MEMSET4_3                ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PMEM4_MEMSET4_4                ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PMEM4_MEMSET4_5                ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PMEM4_MEMSET4_6                ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PMEM4_MEMSET4_7                ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PMEM4_MEMWAIT4                 ((uint32_t)0x0000FF00)        /*!< MEMWAIT4[7:0] bits (Common memory 4 wait time) */
#define  FSMC_PMEM4_MEMWAIT4_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PMEM4_MEMWAIT4_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PMEM4_MEMWAIT4_2               ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PMEM4_MEMWAIT4_3               ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PMEM4_MEMWAIT4_4               ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PMEM4_MEMWAIT4_5               ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PMEM4_MEMWAIT4_6               ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PMEM4_MEMWAIT4_7               ((uint32_t)0x00008000)        /*!< Bit 7 */

```

```
#define  FSMC_PMEM4_MEMHOLD4                 ((uint32_t)0x00FF0000)        /*!< MEMHOLD4[7:0] bits (Common memory 4 hold time) */
#define  FSMC_PMEM4_MEMHOLD4_0               ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PMEM4_MEMHOLD4_1               ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PMEM4_MEMHOLD4_2               ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PMEM4_MEMHOLD4_3               ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PMEM4_MEMHOLD4_4               ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PMEM4_MEMHOLD4_5               ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PMEM4_MEMHOLD4_6               ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PMEM4_MEMHOLD4_7               ((uint32_t)0x00800000)        /*!< Bit 7 */

#define  FSMC_PMEM4_MEMHIZ4                  ((uint32_t)0xFF000000)        /*!< MEMHIZ4[7:0] bits (Common memory 4 databus HiZ time) */
#define  FSMC_PMEM4_MEMHIZ4_0                ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PMEM4_MEMHIZ4_1                ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PMEM4_MEMHIZ4_2                ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PMEM4_MEMHIZ4_3                ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PMEM4_MEMHIZ4_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PMEM4_MEMHIZ4_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PMEM4_MEMHIZ4_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PMEM4_MEMHIZ4_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PATT2 register  ******************/
#define  FSMC_PATT2_ATTSET2                  ((uint32_t)0x000000FF)        /*!< ATTSET2[7:0] bits (Attribute memory 2 setup time) */
#define  FSMC_PATT2_ATTSET2_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PATT2_ATTSET2_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PATT2_ATTSET2_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_PATT2_ATTSET2_3                ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PATT2_ATTSET2_4                ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PATT2_ATTSET2_5                ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PATT2_ATTSET2_6                ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PATT2_ATTSET2_7                ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PATT2_ATTWAIT2                 ((uint32_t)0x0000FF00)        /*!< ATTWAIT2[7:0] bits (Attribute memory 2 wait time) */
#define  FSMC_PATT2_ATTWAIT2_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PATT2_ATTWAIT2_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PATT2_ATTWAIT2_2               ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PATT2_ATTWAIT2_3               ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PATT2_ATTWAIT2_4               ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PATT2_ATTWAIT2_5               ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PATT2_ATTWAIT2_6               ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PATT2_ATTWAIT2_7               ((uint32_t)0x00008000)        /*!< Bit 7 */

#define  FSMC_PATT2_ATTHOLD2                 ((uint32_t)0x00FF0000)        /*!< ATTHOLD2[7:0] bits (Attribute memory 2 hold time) */
#define  FSMC_PATT2_ATTHOLD2_0               ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PATT2_ATTHOLD2_1               ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PATT2_ATTHOLD2_2               ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PATT2_ATTHOLD2_3               ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PATT2_ATTHOLD2_4               ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PATT2_ATTHOLD2_5               ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PATT2_ATTHOLD2_6               ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PATT2_ATTHOLD2_7               ((uint32_t)0x00800000)        /*!< Bit 7 */

#define  FSMC_PATT2_ATTHIZ2                  ((uint32_t)0xFF000000)        /*!< ATTHIZ2[7:0] bits (Attribute memory 2 databus HiZ time) */
#define  FSMC_PATT2_ATTHIZ2_0                ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PATT2_ATTHIZ2_1                ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PATT2_ATTHIZ2_2                ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PATT2_ATTHIZ2_3                ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PATT2_ATTHIZ2_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PATT2_ATTHIZ2_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PATT2_ATTHIZ2_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PATT2_ATTHIZ2_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PATT3 register  ******************/
#define  FSMC_PATT3_ATTSET3                  ((uint32_t)0x000000FF)        /*!< ATTSET3[7:0] bits (Attribute memory 3 setup time) */
#define  FSMC_PATT3_ATTSET3_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PATT3_ATTSET3_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PATT3_ATTSET3_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
```

```
#define  FSMC_PATT3_ATTSET3_3                ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PATT3_ATTSET3_4                ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PATT3_ATTSET3_5                ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PATT3_ATTSET3_6                ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PATT3_ATTSET3_7                ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PATT3_ATTWAIT3                 ((uint32_t)0x0000FF00)        /*!< ATTWAIT3[7:0] bits (Attribute memory 3 wait time) */
#define  FSMC_PATT3_ATTWAIT3_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PATT3_ATTWAIT3_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PATT3_ATTWAIT3_2               ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PATT3_ATTWAIT3_3               ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PATT3_ATTWAIT3_4               ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PATT3_ATTWAIT3_5               ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PATT3_ATTWAIT3_6               ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PATT3_ATTWAIT3_7               ((uint32_t)0x00008000)        /*!< Bit 7 */

#define  FSMC_PATT3_ATTHOLD3                 ((uint32_t)0x00FF0000)        /*!< ATTHOLD3[7:0] bits (Attribute memory 3 hold time) */
#define  FSMC_PATT3_ATTHOLD3_0               ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PATT3_ATTHOLD3_1               ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PATT3_ATTHOLD3_2               ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PATT3_ATTHOLD3_3               ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PATT3_ATTHOLD3_4               ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PATT3_ATTHOLD3_5               ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PATT3_ATTHOLD3_6               ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PATT3_ATTHOLD3_7               ((uint32_t)0x00800000)        /*!< Bit 7 */

#define  FSMC_PATT3_ATTHIZ3                  ((uint32_t)0xFF000000)        /*!< ATTHIZ3[7:0] bits (Attribute memory 3 databus HiZ time) */
#define  FSMC_PATT3_ATTHIZ3_0                ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PATT3_ATTHIZ3_1                ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PATT3_ATTHIZ3_2                ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PATT3_ATTHIZ3_3                ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PATT3_ATTHIZ3_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PATT3_ATTHIZ3_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PATT3_ATTHIZ3_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PATT3_ATTHIZ3_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PATT4 register  ******************/
#define  FSMC_PATT4_ATTSET4                  ((uint32_t)0x000000FF)        /*!< ATTSET4[7:0] bits (Attribute memory 4 setup time) */
#define  FSMC_PATT4_ATTSET4_0                ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PATT4_ATTSET4_1                ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PATT4_ATTSET4_2                ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_PATT4_ATTSET4_3                ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PATT4_ATTSET4_4                ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PATT4_ATTSET4_5                ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PATT4_ATTSET4_6                ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PATT4_ATTSET4_7                ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PATT4_ATTWAIT4                 ((uint32_t)0x0000FF00)        /*!< ATTWAIT4[7:0] bits (Attribute memory 4 wait time) */
#define  FSMC_PATT4_ATTWAIT4_0               ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PATT4_ATTWAIT4_1               ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PATT4_ATTWAIT4_2               ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PATT4_ATTWAIT4_3               ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PATT4_ATTWAIT4_4               ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PATT4_ATTWAIT4_5               ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PATT4_ATTWAIT4_6               ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PATT4_ATTWAIT4_7               ((uint32_t)0x00008000)        /*!< Bit 7 */

#define  FSMC_PATT4_ATTHOLD4                 ((uint32_t)0x00FF0000)        /*!< ATTHOLD4[7:0] bits (Attribute memory 4 hold time) */
#define  FSMC_PATT4_ATTHOLD4_0               ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PATT4_ATTHOLD4_1               ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PATT4_ATTHOLD4_2               ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PATT4_ATTHOLD4_3               ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PATT4_ATTHOLD4_4               ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PATT4_ATTHOLD4_5               ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PATT4_ATTHOLD4_6               ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PATT4_ATTHOLD4_7               ((uint32_t)0x00800000)        /*!< Bit 7 */
```

```
#define  FSMC_PATT4_ATTHIZ4                  ((uint32_t)0xFF000000)        /*!< ATTHIZ4[7:0] bits (Attribute memory 4 databus HiZ time) */
#define  FSMC_PATT4_ATTHIZ4_0                ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PATT4_ATTHIZ4_1                ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PATT4_ATTHIZ4_2                ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PATT4_ATTHIZ4_3                ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PATT4_ATTHIZ4_4                ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PATT4_ATTHIZ4_5                ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PATT4_ATTHIZ4_6                ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PATT4_ATTHIZ4_7                ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_PIO4 register  *******************/
#define  FSMC_PIO4_IOSET4                    ((uint32_t)0x000000FF)        /*!< IOSET4[7:0] bits (I/O 4 setup time) */
#define  FSMC_PIO4_IOSET4_0                  ((uint32_t)0x00000001)        /*!< Bit 0 */
#define  FSMC_PIO4_IOSET4_1                  ((uint32_t)0x00000002)        /*!< Bit 1 */
#define  FSMC_PIO4_IOSET4_2                  ((uint32_t)0x00000004)        /*!< Bit 2 */
#define  FSMC_PIO4_IOSET4_3                  ((uint32_t)0x00000008)        /*!< Bit 3 */
#define  FSMC_PIO4_IOSET4_4                  ((uint32_t)0x00000010)        /*!< Bit 4 */
#define  FSMC_PIO4_IOSET4_5                  ((uint32_t)0x00000020)        /*!< Bit 5 */
#define  FSMC_PIO4_IOSET4_6                  ((uint32_t)0x00000040)        /*!< Bit 6 */
#define  FSMC_PIO4_IOSET4_7                  ((uint32_t)0x00000080)        /*!< Bit 7 */

#define  FSMC_PIO4_IOWAIT4                   ((uint32_t)0x0000FF00)        /*!< IOWAIT4[7:0] bits (I/O 4 wait time) */
#define  FSMC_PIO4_IOWAIT4_0                 ((uint32_t)0x00000100)        /*!< Bit 0 */
#define  FSMC_PIO4_IOWAIT4_1                 ((uint32_t)0x00000200)        /*!< Bit 1 */
#define  FSMC_PIO4_IOWAIT4_2                 ((uint32_t)0x00000400)        /*!< Bit 2 */
#define  FSMC_PIO4_IOWAIT4_3                 ((uint32_t)0x00000800)        /*!< Bit 3 */
#define  FSMC_PIO4_IOWAIT4_4                 ((uint32_t)0x00001000)        /*!< Bit 4 */
#define  FSMC_PIO4_IOWAIT4_5                 ((uint32_t)0x00002000)        /*!< Bit 5 */
#define  FSMC_PIO4_IOWAIT4_6                 ((uint32_t)0x00004000)        /*!< Bit 6 */
#define  FSMC_PIO4_IOWAIT4_7                 ((uint32_t)0x00008000)        /*!< Bit 7 */

#define  FSMC_PIO4_IOHOLD4                   ((uint32_t)0x00FF0000)        /*!< IOHOLD4[7:0] bits (I/O 4 hold time) */
#define  FSMC_PIO4_IOHOLD4_0                 ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  FSMC_PIO4_IOHOLD4_1                 ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  FSMC_PIO4_IOHOLD4_2                 ((uint32_t)0x00040000)        /*!< Bit 2 */
#define  FSMC_PIO4_IOHOLD4_3                 ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  FSMC_PIO4_IOHOLD4_4                 ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  FSMC_PIO4_IOHOLD4_5                 ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  FSMC_PIO4_IOHOLD4_6                 ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  FSMC_PIO4_IOHOLD4_7                 ((uint32_t)0x00800000)        /*!< Bit 7 */

#define  FSMC_PIO4_IOHIZ4                    ((uint32_t)0xFF000000)        /*!< IOHIZ4[7:0] bits (I/O 4 databus HiZ time) */
#define  FSMC_PIO4_IOHIZ4_0                  ((uint32_t)0x01000000)        /*!< Bit 0 */
#define  FSMC_PIO4_IOHIZ4_1                  ((uint32_t)0x02000000)        /*!< Bit 1 */
#define  FSMC_PIO4_IOHIZ4_2                  ((uint32_t)0x04000000)        /*!< Bit 2 */
#define  FSMC_PIO4_IOHIZ4_3                  ((uint32_t)0x08000000)        /*!< Bit 3 */
#define  FSMC_PIO4_IOHIZ4_4                  ((uint32_t)0x10000000)        /*!< Bit 4 */
#define  FSMC_PIO4_IOHIZ4_5                  ((uint32_t)0x20000000)        /*!< Bit 5 */
#define  FSMC_PIO4_IOHIZ4_6                  ((uint32_t)0x40000000)        /*!< Bit 6 */
#define  FSMC_PIO4_IOHIZ4_7                  ((uint32_t)0x80000000)        /*!< Bit 7 */

/******************  Bit definition for FSMC_ECCR2 register  ******************/
#define  FSMC_ECCR2_ECC2                     ((uint32_t)0xFFFFFFFF)        /*!< ECC result */

/******************  Bit definition for FSMC_ECCR3 register  ******************/
#define  FSMC_ECCR3_ECC3                     ((uint32_t)0xFFFFFFFF)        /*!< ECC result */

/******************************************************************************/
/*                                                                            */
/*                          SD host Interface                                 */
/*                                                                            */
/******************************************************************************/

/******************  Bit definition for SDIO_POWER register  ******************/
#define  SDIO_POWER_PWRCTRL                  ((uint8_t)0x03)               /*!< PWRCTRL[1:0] bits (Power supply control bits) */
#define  SDIO_POWER_PWRCTRL_0                ((uint8_t)0x01)               /*!< Bit 0 */
#define  SDIO_POWER_PWRCTRL_1                ((uint8_t)0x02)               /*!< Bit 1 */
```

```
/******************  Bit definition for SDIO_CLKCR register  ******************/
#define  SDIO_CLKCR_CLKDIV                   ((uint16_t)0x00FF)            /*!< Clock divide factor */
#define  SDIO_CLKCR_CLKEN                    ((uint16_t)0x0100)            /*!< Clock enable bit */
#define  SDIO_CLKCR_PWRSAV                   ((uint16_t)0x0200)            /*!< Power saving configuration bit */
#define  SDIO_CLKCR_BYPASS                   ((uint16_t)0x0400)            /*!< Clock divider bypass enable bit */

#define  SDIO_CLKCR_WIDBUS                   ((uint16_t)0x1800)            /*!< WIDBUS[1:0] bits (Wide bus mode enable bit) */
#define  SDIO_CLKCR_WIDBUS_0                 ((uint16_t)0x0800)            /*!< Bit 0 */
#define  SDIO_CLKCR_WIDBUS_1                 ((uint16_t)0x1000)            /*!< Bit 1 */

#define  SDIO_CLKCR_NEGEDGE                  ((uint16_t)0x2000)            /*!< SDIO_CK dephasing selection bit */
#define  SDIO_CLKCR_HWFC_EN                  ((uint16_t)0x4000)            /*!< HW Flow Control enable */

/*******************  Bit definition for SDIO_ARG register  *******************/
#define  SDIO_ARG_CMDARG                     ((uint32_t)0xFFFFFFFF)            /*!< Command argument */

/*******************  Bit definition for SDIO_CMD register  *******************/
#define  SDIO_CMD_CMDINDEX                   ((uint16_t)0x003F)            /*!< Command Index */

#define  SDIO_CMD_WAITRESP                   ((uint16_t)0x00C0)            /*!< WAITRESP[1:0] bits (Wait for response bits) */
#define  SDIO_CMD_WAITRESP_0                 ((uint16_t)0x0040)            /*!<  Bit 0 */
#define  SDIO_CMD_WAITRESP_1                 ((uint16_t)0x0080)            /*!<  Bit 1 */

#define  SDIO_CMD_WAITINT                    ((uint16_t)0x0100)            /*!< CPSM Waits for Interrupt Request */
#define  SDIO_CMD_WAITPEND                   ((uint16_t)0x0200)            /*!< CPSM Waits for ends of data transfer (CmdPend internal signal) */
#define  SDIO_CMD_CPSMEN                     ((uint16_t)0x0400)            /*!< Command path state machine (CPSM) Enable bit */
#define  SDIO_CMD_SDIOSUSPEND                ((uint16_t)0x0800)            /*!< SD I/O suspend command */
#define  SDIO_CMD_ENCMDCOMPL                 ((uint16_t)0x1000)            /*!< Enable CMD completion */
#define  SDIO_CMD_NIEN                       ((uint16_t)0x2000)            /*!< Not Interrupt Enable */
#define  SDIO_CMD_CEATACMD                   ((uint16_t)0x4000)            /*!< CE-ATA command */

/*****************  Bit definition for SDIO_RESPCMD register  *****************/
#define  SDIO_RESPCMD_RESPCMD                ((uint8_t)0x3F)               /*!< Response command index */

/******************  Bit definition for SDIO_RESP0 register  ******************/
#define  SDIO_RESP0_CARDSTATUS0              ((uint32_t)0xFFFFFFFF)        /*!< Card Status */

/******************  Bit definition for SDIO_RESP1 register  ******************/
#define  SDIO_RESP1_CARDSTATUS1              ((uint32_t)0xFFFFFFFF)        /*!< Card Status */

/******************  Bit definition for SDIO_RESP2 register  ******************/
#define  SDIO_RESP2_CARDSTATUS2              ((uint32_t)0xFFFFFFFF)        /*!< Card Status */

/******************  Bit definition for SDIO_RESP3 register  ******************/
#define  SDIO_RESP3_CARDSTATUS3              ((uint32_t)0xFFFFFFFF)        /*!< Card Status */

/******************  Bit definition for SDIO_RESP4 register  ******************/
#define  SDIO_RESP4_CARDSTATUS4              ((uint32_t)0xFFFFFFFF)        /*!< Card Status */

/*****************  Bit definition for SDIO_DTIMER register  *****************/
#define  SDIO_DTIMER_DATATIME                ((uint32_t)0xFFFFFFFF)        /*!< Data timeout period. */

/******************  Bit definition for SDIO_DLEN register  *******************/
#define  SDIO_DLEN_DATALENGTH                ((uint32_t)0x01FFFFFF)        /*!< Data length value */

/******************  Bit definition for SDIO_DCTRL register  ******************/
#define  SDIO_DCTRL_DTEN                     ((uint16_t)0x0001)            /*!< Data transfer enabled bit */
#define  SDIO_DCTRL_DTDIR                    ((uint16_t)0x0002)            /*!< Data transfer direction selection */
#define  SDIO_DCTRL_DTMODE                   ((uint16_t)0x0004)            /*!< Data transfer mode selection */
#define  SDIO_DCTRL_DMAEN                    ((uint16_t)0x0008)            /*!< DMA enabled bit */

```

```
#define  SDIO_DCTRL_DBLOCKSIZE               ((uint16_t)0x00F0)            /*!< DBLOCKSIZE[3:0] bits (Data block size) */
#define  SDIO_DCTRL_DBLOCKSIZE_0             ((uint16_t)0x0010)            /*!< Bit 0 */
#define  SDIO_DCTRL_DBLOCKSIZE_1             ((uint16_t)0x0020)            /*!< Bit 1 */
#define  SDIO_DCTRL_DBLOCKSIZE_2             ((uint16_t)0x0040)            /*!< Bit 2 */
#define  SDIO_DCTRL_DBLOCKSIZE_3             ((uint16_t)0x0080)            /*!< Bit 3 */

#define  SDIO_DCTRL_RWSTART                  ((uint16_t)0x0100)            /*!< Read wait start */
#define  SDIO_DCTRL_RWSTOP                   ((uint16_t)0x0200)            /*!< Read wait stop */
#define  SDIO_DCTRL_RWMOD                    ((uint16_t)0x0400)            /*!< Read wait mode */
#define  SDIO_DCTRL_SDIOEN                   ((uint16_t)0x0800)            /*!< SD I/O enable functions */

/******************  Bit definition for SDIO_DCOUNT register  *****************/
#define  SDIO_DCOUNT_DATACOUNT               ((uint32_t)0x01FFFFFF)        /*!< Data count value */

/******************  Bit definition for SDIO_STA register  ********************/
#define  SDIO_STA_CCRCFAIL                   ((uint32_t)0x00000001)        /*!< Command response received (CRC check failed) */
#define  SDIO_STA_DCRCFAIL                   ((uint32_t)0x00000002)        /*!< Data block sent/received (CRC check failed) */
#define  SDIO_STA_CTIMEOUT                   ((uint32_t)0x00000004)        /*!< Command response timeout */
#define  SDIO_STA_DTIMEOUT                   ((uint32_t)0x00000008)        /*!< Data timeout */
#define  SDIO_STA_TXUNDERR                   ((uint32_t)0x00000010)        /*!< Transmit FIFO underrun error */
#define  SDIO_STA_RXOVERR                    ((uint32_t)0x00000020)        /*!< Received FIFO overrun error */
#define  SDIO_STA_CMDREND                    ((uint32_t)0x00000040)        /*!< Command response received (CRC check passed) */
#define  SDIO_STA_CMDSENT                    ((uint32_t)0x00000080)        /*!< Command sent (no response required) */
#define  SDIO_STA_DATAEND                    ((uint32_t)0x00000100)        /*!< Data end (data counter, SDIDCOUNT, is zero) */
#define  SDIO_STA_STBITERR                   ((uint32_t)0x00000200)        /*!< Start bit not detected on all data signals in wide bus mode */
#define  SDIO_STA_DBCKEND                    ((uint32_t)0x00000400)        /*!< Data block sent/received (CRC check passed) */
#define  SDIO_STA_CMDACT                     ((uint32_t)0x00000800)        /*!< Command transfer in progress */
#define  SDIO_STA_TXACT                      ((uint32_t)0x00001000)        /*!< Data transmit in progress */
#define  SDIO_STA_RXACT                      ((uint32_t)0x00002000)        /*!< Data receive in progress */
#define  SDIO_STA_TXFIFOHE                   ((uint32_t)0x00004000)        /*!< Transmit FIFO Half Empty: at least 8 words can be written into the FIFO */
#define  SDIO_STA_RXFIFOHF                   ((uint32_t)0x00008000)        /*!< Receive FIFO Half Full: there are at least 8 words in the FIFO */
#define  SDIO_STA_TXFIFOF                    ((uint32_t)0x00010000)        /*!< Transmit FIFO full */
#define  SDIO_STA_RXFIFOF                    ((uint32_t)0x00020000)        /*!< Receive FIFO full */
#define  SDIO_STA_TXFIFOE                    ((uint32_t)0x00040000)        /*!< Transmit FIFO empty */
#define  SDIO_STA_RXFIFOE                    ((uint32_t)0x00080000)        /*!< Receive FIFO empty */
#define  SDIO_STA_TXDAVL                     ((uint32_t)0x00100000)        /*!< Data available in transmit FIFO */
#define  SDIO_STA_RXDAVL                     ((uint32_t)0x00200000)        /*!< Data available in receive FIFO */
#define  SDIO_STA_SDIOIT                     ((uint32_t)0x00400000)        /*!< SDIO interrupt received */
#define  SDIO_STA_CEATAEND                   ((uint32_t)0x00800000)        /*!< CE-ATA command completion signal received for CMD61 */

/******************  Bit definition for SDIO_ICR register  ********************/
#define  SDIO_ICR_CCRCFAILC                  ((uint32_t)0x00000001)        /*!< CCRCFAIL flag clear bit */
#define  SDIO_ICR_DCRCFAILC                  ((uint32_t)0x00000002)        /*!< DCRCFAIL flag clear bit */
#define  SDIO_ICR_CTIMEOUTC                  ((uint32_t)0x00000004)        /*!< CTIMEOUT flag clear bit */
#define  SDIO_ICR_DTIMEOUTC                  ((uint32_t)0x00000008)        /*!< DTIMEOUT flag clear bit */
#define  SDIO_ICR_TXUNDERRC                  ((uint32_t)0x00000010)        /*!< TXUNDERR flag clear bit */
#define  SDIO_ICR_RXOVERRC                   ((uint32_t)0x00000020)        /*!< RXOVERR flag clear bit */
```

```
#define  SDIO_ICR_CMDRENDC                   ((uint32_t)0x00000040)        /*!< CMDREND flag clear bit */
#define  SDIO_ICR_CMDSENTC                   ((uint32_t)0x00000080)        /*!< CMDSENT flag clear bit */
#define  SDIO_ICR_DATAENDC                   ((uint32_t)0x00000100)        /*!< DATAEND flag clear bit */
#define  SDIO_ICR_STBITERRC                  ((uint32_t)0x00000200)        /*!< STBITERR flag clear bit */
#define  SDIO_ICR_DBCKENDC                   ((uint32_t)0x00000400)        /*!< DBCKEND flag clear bit */
#define  SDIO_ICR_SDIOITC                    ((uint32_t)0x00400000)        /*!< SDIOIT flag clear bit */
#define  SDIO_ICR_CEATAENDC                  ((uint32_t)0x00800000)        /*!< CEATAEND flag clear bit */

/******************  Bit definition for SDIO_MASK register  *******************/
#define  SDIO_MASK_CCRCFAILIE                ((uint32_t)0x00000001)        /*!< Command CRC Fail Interrupt Enable */
#define  SDIO_MASK_DCRCFAILIE                ((uint32_t)0x00000002)        /*!< Data CRC Fail Interrupt Enable */
#define  SDIO_MASK_CTIMEOUTIE                ((uint32_t)0x00000004)        /*!< Command TimeOut Interrupt Enable */
#define  SDIO_MASK_DTIMEOUTIE                ((uint32_t)0x00000008)        /*!< Data TimeOut Interrupt Enable */
#define  SDIO_MASK_TXUNDERRIE                ((uint32_t)0x00000010)        /*!< Tx FIFO UnderRun Error Interrupt Enable */
#define  SDIO_MASK_RXOVERRIE                 ((uint32_t)0x00000020)        /*!< Rx FIFO OverRun Error Interrupt Enable */
#define  SDIO_MASK_CMDRENDIE                 ((uint32_t)0x00000040)        /*!< Command Response Received Interrupt Enable */
#define  SDIO_MASK_CMDSENTIE                 ((uint32_t)0x00000080)        /*!< Command Sent Interrupt Enable */
#define  SDIO_MASK_DATAENDIE                 ((uint32_t)0x00000100)        /*!< Data End Interrupt Enable */
#define  SDIO_MASK_STBITERRIE                ((uint32_t)0x00000200)        /*!< Start Bit Error Interrupt Enable */
#define  SDIO_MASK_DBCKENDIE                 ((uint32_t)0x00000400)        /*!< Data Block End Interrupt Enable */
#define  SDIO_MASK_CMDACTIE                  ((uint32_t)0x00000800)        /*!< Command Acting Interrupt Enable */
#define  SDIO_MASK_TXACTIE                   ((uint32_t)0x00001000)        /*!< Data Transmit Acting Interrupt Enable */
#define  SDIO_MASK_RXACTIE                   ((uint32_t)0x00002000)        /*!< Data receive acting interrupt enabled */
#define  SDIO_MASK_TXFIFOHEIE                ((uint32_t)0x00004000)        /*!< Tx FIFO Half Empty interrupt Enable */
#define  SDIO_MASK_RXFIFOHFIE                ((uint32_t)0x00008000)        /*!< Rx FIFO Half Full interrupt Enable */
#define  SDIO_MASK_TXFIFOFIE                 ((uint32_t)0x00010000)        /*!< Tx FIFO Full interrupt Enable */
#define  SDIO_MASK_RXFIFOFIE                 ((uint32_t)0x00020000)        /*!< Rx FIFO Full interrupt Enable */
#define  SDIO_MASK_TXFIFOEIE                 ((uint32_t)0x00040000)        /*!< Tx FIFO Empty interrupt Enable */
#define  SDIO_MASK_RXFIFOEIE                 ((uint32_t)0x00080000)        /*!< Rx FIFO Empty interrupt Enable */
#define  SDIO_MASK_TXDAVLIE                  ((uint32_t)0x00100000)        /*!< Data available in Tx FIFO interrupt Enable */
#define  SDIO_MASK_RXDAVLIE                  ((uint32_t)0x00200000)        /*!< Data available in Rx FIFO interrupt Enable */
#define  SDIO_MASK_SDIOITIE                  ((uint32_t)0x00400000)        /*!< SDIO Mode Interrupt Received interrupt Enable */
#define  SDIO_MASK_CEATAENDIE                ((uint32_t)0x00800000)        /*!< CE-ATA command completion signal received Interrupt Enable */

/*****************  Bit definition for SDIO_FIFOCNT register  *****************/
#define  SDIO_FIFOCNT_FIFOCOUNT              ((uint32_t)0x00FFFFFF)        /*!< Remaining number of words to be written to or read from the FIFO */

/******************  Bit definition for SDIO_FIFO register  *******************/
#define  SDIO_FIFO_FIFODATA                  ((uint32_t)0xFFFFFFFF)        /*!< Receive and transmit FIFO data */

/******************************************************************************/
/*                                                                            */
/*                                 USB Device FS                              */
/*                                                                            */
/******************************************************************************/
```

```
/*!< Endpoint-specific registers */
/*******************  Bit definition for USB_EP0R register  *******************/
#define  USB_EP0R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP0R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP0R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP0R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP0R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP0R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP0R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP0R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP0R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP0R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP0R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP0R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP0R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP0R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP0R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP0R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP1R register  *******************/
#define  USB_EP1R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP1R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP1R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP1R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP1R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP1R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP1R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP1R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP1R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP1R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP1R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP1R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP1R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP1R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP1R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP1R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP2R register  *******************/
#define  USB_EP2R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP2R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP2R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP2R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */
```

```
#define  USB_EP2R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP2R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP2R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP2R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP2R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP2R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP2R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP2R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP2R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP2R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP2R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP2R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP3R register  *******************/
#define  USB_EP3R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP3R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP3R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP3R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP3R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP3R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP3R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP3R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP3R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP3R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP3R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP3R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP3R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP3R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP3R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP3R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP4R register  *******************/
#define  USB_EP4R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP4R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP4R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP4R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP4R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP4R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP4R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP4R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
```

```
#define  USB_EP4R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP4R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP4R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP4R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP4R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP4R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP4R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP4R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP5R register  *******************/
#define  USB_EP5R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP5R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP5R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP5R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP5R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP5R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP5R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP5R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP5R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP5R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP5R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP5R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP5R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP5R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP5R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP5R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP6R register  *******************/
#define  USB_EP6R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP6R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP6R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP6R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP6R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP6R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP6R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP6R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP6R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP6R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP6R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP6R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP6R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
```

```
#define  USB_EP6R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP6R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP6R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*******************  Bit definition for USB_EP7R register  *******************/
#define  USB_EP7R_EA                         ((uint16_t)0x000F)            /*!< Endpoint Address */

#define  USB_EP7R_STAT_TX                    ((uint16_t)0x0030)            /*!< STAT_TX[1:0] bits (Status bits, for transmission transfers) */
#define  USB_EP7R_STAT_TX_0                  ((uint16_t)0x0010)            /*!< Bit 0 */
#define  USB_EP7R_STAT_TX_1                  ((uint16_t)0x0020)            /*!< Bit 1 */

#define  USB_EP7R_DTOG_TX                    ((uint16_t)0x0040)            /*!< Data Toggle, for transmission transfers */
#define  USB_EP7R_CTR_TX                     ((uint16_t)0x0080)            /*!< Correct Transfer for transmission */
#define  USB_EP7R_EP_KIND                    ((uint16_t)0x0100)            /*!< Endpoint Kind */

#define  USB_EP7R_EP_TYPE                    ((uint16_t)0x0600)            /*!< EP_TYPE[1:0] bits (Endpoint type) */
#define  USB_EP7R_EP_TYPE_0                  ((uint16_t)0x0200)            /*!< Bit 0 */
#define  USB_EP7R_EP_TYPE_1                  ((uint16_t)0x0400)            /*!< Bit 1 */

#define  USB_EP7R_SETUP                      ((uint16_t)0x0800)            /*!< Setup transaction completed */

#define  USB_EP7R_STAT_RX                    ((uint16_t)0x3000)            /*!< STAT_RX[1:0] bits (Status bits, for reception transfers) */
#define  USB_EP7R_STAT_RX_0                  ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USB_EP7R_STAT_RX_1                  ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USB_EP7R_DTOG_RX                    ((uint16_t)0x4000)            /*!< Data Toggle, for reception transfers */
#define  USB_EP7R_CTR_RX                     ((uint16_t)0x8000)            /*!< Correct Transfer for reception */

/*!< Common registers */
/*******************  Bit definition for USB_CNTR register  *******************/
#define  USB_CNTR_FRES                       ((uint16_t)0x0001)            /*!< Force USB Reset */
#define  USB_CNTR_PDWN                       ((uint16_t)0x0002)            /*!< Power down */
#define  USB_CNTR_LP_MODE                    ((uint16_t)0x0004)            /*!< Low-power mode */
#define  USB_CNTR_FSUSP                      ((uint16_t)0x0008)            /*!< Force suspend */
#define  USB_CNTR_RESUME                     ((uint16_t)0x0010)            /*!< Resume request */
#define  USB_CNTR_ESOFM                      ((uint16_t)0x0100)            /*!< Expected Start Of Frame Interrupt Mask */
#define  USB_CNTR_SOFM                       ((uint16_t)0x0200)            /*!< Start Of Frame Interrupt Mask */
#define  USB_CNTR_RESETM                     ((uint16_t)0x0400)            /*!< RESET Interrupt Mask */
#define  USB_CNTR_SUSPM                      ((uint16_t)0x0800)            /*!< Suspend mode Interrupt Mask */
#define  USB_CNTR_WKUPM                      ((uint16_t)0x1000)            /*!< Wakeup Interrupt Mask */
#define  USB_CNTR_ERRM                       ((uint16_t)0x2000)            /*!< Error Interrupt Mask */
#define  USB_CNTR_PMAOVRM                    ((uint16_t)0x4000)            /*!< Packet Memory Area Over / Underrun Interrupt Mask */
#define  USB_CNTR_CTRM                       ((uint16_t)0x8000)            /*!< Correct Transfer Interrupt Mask */

/*******************  Bit definition for USB_ISTR register  *******************/
#define  USB_ISTR_EP_ID                      ((uint16_t)0x000F)            /*!< Endpoint Identifier */
#define  USB_ISTR_DIR                        ((uint16_t)0x0010)            /*!< Direction of transaction */
#define  USB_ISTR_ESOF                       ((uint16_t)0x0100)            /*!< Expected Start Of Frame */
#define  USB_ISTR_SOF                        ((uint16_t)0x0200)            /*!< Start Of Frame */
#define  USB_ISTR_RESET                      ((uint16_t)0x0400)            /*!< USB RESET request */
#define  USB_ISTR_SUSP                       ((uint16_t)0x0800)            /*!< Suspend mode request */
#define  USB_ISTR_WKUP                       ((uint16_t)0x1000)            /*!< Wake up */
#define  USB_ISTR_ERR                        ((uint16_t)0x2000)            /*!< Error */
#define  USB_ISTR_PMAOVR                     ((uint16_t)0x4000)            /*!< Packet Memory Area Over / Underrun */
```

```
#define  USB_ISTR_CTR                        ((uint16_t)0x8000)            /*!< Correct Transfer */

/*******************  Bit definition for USB_FNR register  *******************/
#define  USB_FNR_FN                          ((uint16_t)0x07FF)            /*!< Frame Number */
#define  USB_FNR_LSOF                        ((uint16_t)0x1800)            /*!< Lost SOF */
#define  USB_FNR_LCK                         ((uint16_t)0x2000)            /*!< Locked */
#define  USB_FNR_RXDM                        ((uint16_t)0x4000)            /*!< Receive Data - Line Status */
#define  USB_FNR_RXDP                        ((uint16_t)0x8000)            /*!< Receive Data + Line Status */

/******************  Bit definition for USB_DADDR register  ******************/
#define  USB_DADDR_ADD                       ((uint8_t)0x7F)               /*!< ADD[6:0] bits (Device Address) */
#define  USB_DADDR_ADD0                      ((uint8_t)0x01)               /*!< Bit 0 */
#define  USB_DADDR_ADD1                      ((uint8_t)0x02)               /*!< Bit 1 */
#define  USB_DADDR_ADD2                      ((uint8_t)0x04)               /*!< Bit 2 */
#define  USB_DADDR_ADD3                      ((uint8_t)0x08)               /*!< Bit 3 */
#define  USB_DADDR_ADD4                      ((uint8_t)0x10)               /*!< Bit 4 */
#define  USB_DADDR_ADD5                      ((uint8_t)0x20)               /*!< Bit 5 */
#define  USB_DADDR_ADD6                      ((uint8_t)0x40)               /*!< Bit 6 */

#define  USB_DADDR_EF                        ((uint8_t)0x80)               /*!< Enable Function */

/******************  Bit definition for USB_BTABLE register  *****************/
#define  USB_BTABLE_BTABLE                   ((uint16_t)0xFFF8)            /*!< Buffer Table */

/*!< Buffer descriptor table */
/*****************  Bit definition for USB_ADDR0_TX register  ****************/
#define  USB_ADDR0_TX_ADDR0_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 0 */

/*****************  Bit definition for USB_ADDR1_TX register  ****************/
#define  USB_ADDR1_TX_ADDR1_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 1 */

/*****************  Bit definition for USB_ADDR2_TX register  ****************/
#define  USB_ADDR2_TX_ADDR2_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 2 */

/*****************  Bit definition for USB_ADDR3_TX register  ****************/
#define  USB_ADDR3_TX_ADDR3_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 3 */

/*****************  Bit definition for USB_ADDR4_TX register  ****************/
#define  USB_ADDR4_TX_ADDR4_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 4 */

/*****************  Bit definition for USB_ADDR5_TX register  ****************/
#define  USB_ADDR5_TX_ADDR5_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 5 */

/*****************  Bit definition for USB_ADDR6_TX register  ****************/
#define  USB_ADDR6_TX_ADDR6_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 6 */

/*****************  Bit definition for USB_ADDR7_TX register  ****************/
#define  USB_ADDR7_TX_ADDR7_TX               ((uint16_t)0xFFFE)            /*!< Transmission Buffer Address 7 */

/*-----------------------------------------------------------------------------*/

/*****************  Bit definition for USB_COUNT0_TX register  ***************/
#define  USB_COUNT0_TX_COUNT0_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 0 */

/*****************  Bit definition for USB_COUNT1_TX register  ***************/
#define  USB_COUNT1_TX_COUNT1_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 1 */

/*****************  Bit definition for USB_COUNT2_TX register  ***************/
#define  USB_COUNT2_TX_COUNT2_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 2 */
```

```
/*****************  Bit definition for USB_COUNT3_TX register  ****************/
#define  USB_COUNT3_TX_COUNT3_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 3 */

/*****************  Bit definition for USB_COUNT4_TX register  ****************/
#define  USB_COUNT4_TX_COUNT4_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 4 */

/*****************  Bit definition for USB_COUNT5_TX register  ****************/
#define  USB_COUNT5_TX_COUNT5_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 5 */

/*****************  Bit definition for USB_COUNT6_TX register  ****************/
#define  USB_COUNT6_TX_COUNT6_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 6 */

/*****************  Bit definition for USB_COUNT7_TX register  ****************/
#define  USB_COUNT7_TX_COUNT7_TX             ((uint16_t)0x03FF)            /*!< Transmission Byte Count 7 */

/*-------------------------------------------------------------------------------*/

/****************  Bit definition for USB_COUNT0_TX_0 register  ***************/
#define  USB_COUNT0_TX_0_COUNT0_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 0 (low) */

/****************  Bit definition for USB_COUNT0_TX_1 register  ***************/
#define  USB_COUNT0_TX_1_COUNT0_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 0 (high) */

/****************  Bit definition for USB_COUNT1_TX_0 register  ***************/
#define  USB_COUNT1_TX_0_COUNT1_TX_0          ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 1 (low) */

/****************  Bit definition for USB_COUNT1_TX_1 register  ***************/
#define  USB_COUNT1_TX_1_COUNT1_TX_1          ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 1 (high) */

/****************  Bit definition for USB_COUNT2_TX_0 register  ***************/
#define  USB_COUNT2_TX_0_COUNT2_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 2 (low) */

/****************  Bit definition for USB_COUNT2_TX_1 register  ***************/
#define  USB_COUNT2_TX_1_COUNT2_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 2 (high) */

/****************  Bit definition for USB_COUNT3_TX_0 register  ***************/
#define  USB_COUNT3_TX_0_COUNT3_TX_0         ((uint16_t)0x000003FF)        /*!< Transmission Byte Count 3 (low) */

/****************  Bit definition for USB_COUNT3_TX_1 register  ***************/
#define  USB_COUNT3_TX_1_COUNT3_TX_1         ((uint16_t)0x03FF0000)        /*!< Transmission Byte Count 3 (high) */

/****************  Bit definition for USB_COUNT4_TX_0 register  ***************/
#define  USB_COUNT4_TX_0_COUNT4_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 4 (low) */

/****************  Bit definition for USB_COUNT4_TX_1 register  ***************/
#define  USB_COUNT4_TX_1_COUNT4_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 4 (high) */

/****************  Bit definition for USB_COUNT5_TX_0 register  ***************/
#define  USB_COUNT5_TX_0_COUNT5_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 5 (low) */

/****************  Bit definition for USB_COUNT5_TX_1 register  ***************/
#define  USB_COUNT5_TX_1_COUNT5_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 5 (high) */

/****************  Bit definition for USB_COUNT6_TX_0 register  ***************/
#define  USB_COUNT6_TX_0_COUNT6_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count
```

```
6 (low) */

/****************  Bit definition for USB_COUNT6_TX_1 register  ***************/
#define  USB_COUNT6_TX_1_COUNT6_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 6 (high) */

/****************  Bit definition for USB_COUNT7_TX_0 register  ***************/
#define  USB_COUNT7_TX_0_COUNT7_TX_0         ((uint32_t)0x000003FF)        /*!< Transmission Byte Count 7 (low) */

/****************  Bit definition for USB_COUNT7_TX_1 register  ***************/
#define  USB_COUNT7_TX_1_COUNT7_TX_1         ((uint32_t)0x03FF0000)        /*!< Transmission Byte Count 7 (high) */

/*-----------------------------------------------------------------------------*/

/******************  Bit definition for USB_ADDR0_RX register  *****************/
#define  USB_ADDR0_RX_ADDR0_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 0 */

/******************  Bit definition for USB_ADDR1_RX register  *****************/
#define  USB_ADDR1_RX_ADDR1_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 1 */

/******************  Bit definition for USB_ADDR2_RX register  *****************/
#define  USB_ADDR2_RX_ADDR2_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 2 */

/******************  Bit definition for USB_ADDR3_RX register  *****************/
#define  USB_ADDR3_RX_ADDR3_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 3 */

/******************  Bit definition for USB_ADDR4_RX register  *****************/
#define  USB_ADDR4_RX_ADDR4_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 4 */

/******************  Bit definition for USB_ADDR5_RX register  *****************/
#define  USB_ADDR5_RX_ADDR5_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 5 */

/******************  Bit definition for USB_ADDR6_RX register  *****************/
#define  USB_ADDR6_RX_ADDR6_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 6 */

/******************  Bit definition for USB_ADDR7_RX register  *****************/
#define  USB_ADDR7_RX_ADDR7_RX               ((uint16_t)0xFFFE)            /*!< Reception Buffer Address 7 */

/*-----------------------------------------------------------------------------*/

/******************  Bit definition for USB_COUNT0_RX register  ****************/
#define  USB_COUNT0_RX_COUNT0_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT0_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT0_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT0_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT0_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT0_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT0_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT0_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/******************  Bit definition for USB_COUNT1_RX register  ****************/
#define  USB_COUNT1_RX_COUNT1_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT1_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT1_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT1_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT1_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT1_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT1_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */
```

```
#define  USB_COUNT1_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT2_RX register  ****************/
#define  USB_COUNT2_RX_COUNT2_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT2_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT2_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT2_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT2_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT2_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT2_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT2_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT3_RX register  ****************/
#define  USB_COUNT3_RX_COUNT3_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT3_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT3_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT3_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT3_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT3_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT3_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT3_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT4_RX register  ****************/
#define  USB_COUNT4_RX_COUNT4_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT4_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT4_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT4_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT4_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT4_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT4_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT4_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT5_RX register  ****************/
#define  USB_COUNT5_RX_COUNT5_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT5_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT5_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT5_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT5_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT5_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT5_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT5_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT6_RX register  ****************/
#define  USB_COUNT6_RX_COUNT6_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT6_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
#define  USB_COUNT6_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT6_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT6_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT6_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT6_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT6_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*****************  Bit definition for USB_COUNT7_RX register  ****************/
#define  USB_COUNT7_RX_COUNT7_RX             ((uint16_t)0x03FF)            /*!< Reception Byte Count */

#define  USB_COUNT7_RX_NUM_BLOCK             ((uint16_t)0x7C00)            /*!< NUM_BLOCK[4:0] bits (Number of blocks) */
```

```
#define  USB_COUNT7_RX_NUM_BLOCK_0           ((uint16_t)0x0400)            /*!< Bit 0 */
#define  USB_COUNT7_RX_NUM_BLOCK_1           ((uint16_t)0x0800)            /*!< Bit 1 */
#define  USB_COUNT7_RX_NUM_BLOCK_2           ((uint16_t)0x1000)            /*!< Bit 2 */
#define  USB_COUNT7_RX_NUM_BLOCK_3           ((uint16_t)0x2000)            /*!< Bit 3 */
#define  USB_COUNT7_RX_NUM_BLOCK_4           ((uint16_t)0x4000)            /*!< Bit 4 */

#define  USB_COUNT7_RX_BLSIZE                ((uint16_t)0x8000)            /*!< BLock SIZE */

/*-----------------------------------------------------------------------------*/

/****************  Bit definition for USB_COUNT0_RX_0 register  ***************/
#define  USB_COUNT0_RX_0_COUNT0_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT0_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT0_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT0_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT0_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT0_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT0_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT0_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT0_RX_1 register  ***************/
#define  USB_COUNT0_RX_1_COUNT0_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT0_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT0_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 1 */
#define  USB_COUNT0_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT0_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT0_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT0_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT0_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/****************  Bit definition for USB_COUNT1_RX_0 register  ***************/
#define  USB_COUNT1_RX_0_COUNT1_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT1_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT1_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT1_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT1_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT1_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT1_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT1_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT1_RX_1 register  ***************/
#define  USB_COUNT1_RX_1_COUNT1_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT1_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT1_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT1_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT1_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT1_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT1_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT1_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/****************  Bit definition for USB_COUNT2_RX_0 register  ***************/
#define  USB_COUNT2_RX_0_COUNT2_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT2_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT2_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
```

```
#define  USB_COUNT2_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT2_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT2_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT2_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT2_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT2_RX_1 register  ***************/
#define  USB_COUNT2_RX_1_COUNT2_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT2_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT2_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT2_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT2_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT2_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT2_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT2_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/****************  Bit definition for USB_COUNT3_RX_0 register  ***************/
#define  USB_COUNT3_RX_0_COUNT3_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT3_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT3_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT3_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT3_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT3_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT3_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT3_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT3_RX_1 register  ***************/
#define  USB_COUNT3_RX_1_COUNT3_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT3_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT3_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT3_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT3_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT3_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT3_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT3_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/****************  Bit definition for USB_COUNT4_RX_0 register  ***************/
#define  USB_COUNT4_RX_0_COUNT4_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT4_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT4_RX_0_NUM_BLOCK_0_0      ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT4_RX_0_NUM_BLOCK_0_1      ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT4_RX_0_NUM_BLOCK_0_2      ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT4_RX_0_NUM_BLOCK_0_3      ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT4_RX_0_NUM_BLOCK_0_4      ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT4_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT4_RX_1 register  ***************/
#define  USB_COUNT4_RX_1_COUNT4_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT4_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT4_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT4_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT4_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT4_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
```

```
#define  USB_COUNT4_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT4_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/****************  Bit definition for USB_COUNT5_RX_0 register  ***************/
#define  USB_COUNT5_RX_0_COUNT5_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT5_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT5_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT5_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT5_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT5_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT5_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT5_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT5_RX_1 register  ***************/
#define  USB_COUNT5_RX_1_COUNT5_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT5_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT5_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT5_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT5_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT5_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT5_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT5_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/***************  Bit definition for USB_COUNT6_RX_0  register  ***************/
#define  USB_COUNT6_RX_0_COUNT6_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT6_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT6_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT6_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT6_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT6_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT6_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT6_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */

/****************  Bit definition for USB_COUNT6_RX_1 register  ***************/
#define  USB_COUNT6_RX_1_COUNT6_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT6_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT6_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT6_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT6_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT6_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT6_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT6_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/***************  Bit definition for USB_COUNT7_RX_0 register  ***************/
#define  USB_COUNT7_RX_0_COUNT7_RX_0         ((uint32_t)0x000003FF)        /*!< Reception Byte Count (low) */

#define  USB_COUNT7_RX_0_NUM_BLOCK_0         ((uint32_t)0x00007C00)        /*!< NUM_BLOCK_0[4:0] bits (Number of blocks) (low) */
#define  USB_COUNT7_RX_0_NUM_BLOCK_0_0       ((uint32_t)0x00000400)        /*!< Bit 0 */
#define  USB_COUNT7_RX_0_NUM_BLOCK_0_1       ((uint32_t)0x00000800)        /*!< Bit 1 */
#define  USB_COUNT7_RX_0_NUM_BLOCK_0_2       ((uint32_t)0x00001000)        /*!< Bit 2 */
#define  USB_COUNT7_RX_0_NUM_BLOCK_0_3       ((uint32_t)0x00002000)        /*!< Bit 3 */
#define  USB_COUNT7_RX_0_NUM_BLOCK_0_4       ((uint32_t)0x00004000)        /*!< Bit 4 */

#define  USB_COUNT7_RX_0_BLSIZE_0            ((uint32_t)0x00008000)        /*!< BLock SIZE (low) */
```

```
/****************  Bit definition for USB_COUNT7_RX_1 register  ****************/
#define  USB_COUNT7_RX_1_COUNT7_RX_1         ((uint32_t)0x03FF0000)        /*!< Reception Byte Count (high) */

#define  USB_COUNT7_RX_1_NUM_BLOCK_1         ((uint32_t)0x7C000000)        /*!< NUM_BLOCK_1[4:0] bits (Number of blocks) (high) */
#define  USB_COUNT7_RX_1_NUM_BLOCK_1_0       ((uint32_t)0x04000000)        /*!< Bit 0 */
#define  USB_COUNT7_RX_1_NUM_BLOCK_1_1       ((uint32_t)0x08000000)        /*!< Bit 1 */
#define  USB_COUNT7_RX_1_NUM_BLOCK_1_2       ((uint32_t)0x10000000)        /*!< Bit 2 */
#define  USB_COUNT7_RX_1_NUM_BLOCK_1_3       ((uint32_t)0x20000000)        /*!< Bit 3 */
#define  USB_COUNT7_RX_1_NUM_BLOCK_1_4       ((uint32_t)0x40000000)        /*!< Bit 4 */

#define  USB_COUNT7_RX_1_BLSIZE_1            ((uint32_t)0x80000000)        /*!< BLock SIZE (high) */

/******************************************************************************/
/*                                                                            */
/*                         Controller Area Network                            */
/*                                                                            */
/******************************************************************************/

/*!< CAN control and status registers */
/*******************  Bit definition for CAN_MCR register  *******************/
#define  CAN_MCR_INRQ                        ((uint16_t)0x0001)            /*!< Initialization Request */
#define  CAN_MCR_SLEEP                       ((uint16_t)0x0002)            /*!< Sleep Mode Request */
#define  CAN_MCR_TXFP                        ((uint16_t)0x0004)            /*!< Transmit FIFO Priority */
#define  CAN_MCR_RFLM                        ((uint16_t)0x0008)            /*!< Receive FIFO Locked Mode */
#define  CAN_MCR_NART                        ((uint16_t)0x0010)            /*!< No Automatic Retransmission */
#define  CAN_MCR_AWUM                        ((uint16_t)0x0020)            /*!< Automatic Wakeup Mode */
#define  CAN_MCR_ABOM                        ((uint16_t)0x0040)            /*!< Automatic Bus-Off Management */
#define  CAN_MCR_TTCM                        ((uint16_t)0x0080)            /*!< Time Triggered Communication Mode */
#define  CAN_MCR_RESET                       ((uint16_t)0x8000)            /*!< CAN software master reset */

/*******************  Bit definition for CAN_MSR register  *******************/
#define  CAN_MSR_INAK                        ((uint16_t)0x0001)            /*!< Initialization Acknowledge */
#define  CAN_MSR_SLAK                        ((uint16_t)0x0002)            /*!< Sleep Acknowledge */
#define  CAN_MSR_ERRI                        ((uint16_t)0x0004)            /*!< Error Interrupt */
#define  CAN_MSR_WKUI                        ((uint16_t)0x0008)            /*!< Wakeup Interrupt */
#define  CAN_MSR_SLAKI                       ((uint16_t)0x0010)            /*!< Sleep Acknowledge Interrupt */
#define  CAN_MSR_TXM                         ((uint16_t)0x0100)            /*!< Transmit Mode */
#define  CAN_MSR_RXM                         ((uint16_t)0x0200)            /*!< Receive Mode */
#define  CAN_MSR_SAMP                        ((uint16_t)0x0400)            /*!< Last Sample Point */
#define  CAN_MSR_RX                          ((uint16_t)0x0800)            /*!< CAN Rx Signal */

/*******************  Bit definition for CAN_TSR register  *******************/
#define  CAN_TSR_RQCP0                       ((uint32_t)0x00000001)        /*!< Request Completed Mailbox0 */
#define  CAN_TSR_TXOK0                       ((uint32_t)0x00000002)        /*!< Transmission OK of Mailbox0 */
#define  CAN_TSR_ALST0                       ((uint32_t)0x00000004)        /*!< Arbitration Lost for Mailbox0 */
#define  CAN_TSR_TERR0                       ((uint32_t)0x00000008)        /*!< Transmission Error of Mailbox0 */
#define  CAN_TSR_ABRQ0                       ((uint32_t)0x00000080)        /*!< Abort Request for Mailbox0 */
#define  CAN_TSR_RQCP1                       ((uint32_t)0x00000100)        /*!< Request Completed Mailbox1 */
#define  CAN_TSR_TXOK1                       ((uint32_t)0x00000200)        /*!< Transmission OK of Mailbox1 */
#define  CAN_TSR_ALST1                       ((uint32_t)0x00000400)        /*!< Arbitration Lost for Mailbox1 */
#define  CAN_TSR_TERR1                       ((uint32_t)0x00000800)        /*!< Transmission Error of Mailbox1 */
#define  CAN_TSR_ABRQ1                       ((uint32_t)0x00008000)        /*!< Abort Request for Mailbox 1 */
#define  CAN_TSR_RQCP2                       ((uint32_t)0x00010000)        /*!< Request Completed
```

```
Mailbox2 */
#define  CAN_TSR_TXOK2                       ((uint32_t)0x00020000)        /*!< Transmission OK of Mailbox 2 */
#define  CAN_TSR_ALST2                       ((uint32_t)0x00040000)        /*!< Arbitration Lost for mailbox 2 */
#define  CAN_TSR_TERR2                       ((uint32_t)0x00080000)        /*!< Transmission Error of Mailbox 2 */
#define  CAN_TSR_ABRQ2                       ((uint32_t)0x00800000)        /*!< Abort Request for Mailbox 2 */
#define  CAN_TSR_CODE                        ((uint32_t)0x03000000)        /*!< Mailbox Code */

#define  CAN_TSR_TME                         ((uint32_t)0x1C000000)        /*!< TME[2:0] bits */
#define  CAN_TSR_TME0                        ((uint32_t)0x04000000)        /*!< Transmit Mailbox 0 Empty */
#define  CAN_TSR_TME1                        ((uint32_t)0x08000000)        /*!< Transmit Mailbox 1 Empty */
#define  CAN_TSR_TME2                        ((uint32_t)0x10000000)        /*!< Transmit Mailbox 2 Empty */

#define  CAN_TSR_LOW                         ((uint32_t)0xE0000000)        /*!< LOW[2:0] bits */
#define  CAN_TSR_LOW0                        ((uint32_t)0x20000000)        /*!< Lowest Priority Flag for Mailbox 0 */
#define  CAN_TSR_LOW1                        ((uint32_t)0x40000000)        /*!< Lowest Priority Flag for Mailbox 1 */
#define  CAN_TSR_LOW2                        ((uint32_t)0x80000000)        /*!< Lowest Priority Flag for Mailbox 2 */

/*******************  Bit definition for CAN_RF0R register  *******************/
#define  CAN_RF0R_FMP0                       ((uint8_t)0x03)               /*!< FIFO 0 Message Pending */
#define  CAN_RF0R_FULL0                      ((uint8_t)0x08)               /*!< FIFO 0 Full */
#define  CAN_RF0R_FOVR0                      ((uint8_t)0x10)               /*!< FIFO 0 Overrun */
#define  CAN_RF0R_RFOM0                      ((uint8_t)0x20)               /*!< Release FIFO 0 Output Mailbox */

/*******************  Bit definition for CAN_RF1R register  *******************/
#define  CAN_RF1R_FMP1                       ((uint8_t)0x03)               /*!< FIFO 1 Message Pending */
#define  CAN_RF1R_FULL1                      ((uint8_t)0x08)               /*!< FIFO 1 Full */
#define  CAN_RF1R_FOVR1                      ((uint8_t)0x10)               /*!< FIFO 1 Overrun */
#define  CAN_RF1R_RFOM1                      ((uint8_t)0x20)               /*!< Release FIFO 1 Output Mailbox */

/********************  Bit definition for CAN_IER register  *******************/
#define  CAN_IER_TMEIE                       ((uint32_t)0x00000001)        /*!< Transmit Mailbox Empty Interrupt Enable */
#define  CAN_IER_FMPIE0                      ((uint32_t)0x00000002)        /*!< FIFO Message Pending Interrupt Enable */
#define  CAN_IER_FFIE0                       ((uint32_t)0x00000004)        /*!< FIFO Full Interrupt Enable */
#define  CAN_IER_FOVIE0                      ((uint32_t)0x00000008)        /*!< FIFO Overrun Interrupt Enable */
#define  CAN_IER_FMPIE1                      ((uint32_t)0x00000010)        /*!< FIFO Message Pending Interrupt Enable */
#define  CAN_IER_FFIE1                       ((uint32_t)0x00000020)        /*!< FIFO Full Interrupt Enable */
#define  CAN_IER_FOVIE1                      ((uint32_t)0x00000040)        /*!< FIFO Overrun Interrupt Enable */
#define  CAN_IER_EWGIE                       ((uint32_t)0x00000100)        /*!< Error Warning Interrupt Enable */
#define  CAN_IER_EPVIE                       ((uint32_t)0x00000200)        /*!< Error Passive Interrupt Enable */
#define  CAN_IER_BOFIE                       ((uint32_t)0x00000400)        /*!< Bus-Off Interrupt Enable */
#define  CAN_IER_LECIE                       ((uint32_t)0x00000800)        /*!< Last Error Code Interrupt Enable */
#define  CAN_IER_ERRIE                       ((uint32_t)0x00008000)        /*!< Error Interrupt Enable */
#define  CAN_IER_WKUIE                       ((uint32_t)0x00010000)        /*!< Wakeup Interrupt Enable */
#define  CAN_IER_SLKIE                       ((uint32_t)0x00020000)        /*!< Sleep Interrupt Enable */

/********************  Bit definition for CAN_ESR register  *******************/
#define  CAN_ESR_EWGF                        ((uint32_t)0x00000001)        /*!< Error Warning Flag */
#define  CAN_ESR_EPVF                        ((uint32_t)0x00000002)        /*!< Error Passive Flag */
#define  CAN_ESR_BOFF                        ((uint32_t)0x00000004)        /*!< Bus-Off Flag */
```

```
#define  CAN_ESR_LEC                         ((uint32_t)0x00000070)        /*!< LEC[2:0] bits (Last Error Code) */
#define  CAN_ESR_LEC_0                       ((uint32_t)0x00000010)        /*!< Bit 0 */
#define  CAN_ESR_LEC_1                       ((uint32_t)0x00000020)        /*!< Bit 1 */
#define  CAN_ESR_LEC_2                       ((uint32_t)0x00000040)        /*!< Bit 2 */

#define  CAN_ESR_TEC                         ((uint32_t)0x00FF0000)        /*!< Least significant byte of the 9-bit Transmit Error Counter */
#define  CAN_ESR_REC                         ((uint32_t)0xFF000000)        /*!< Receive Error Counter */

/*******************  Bit definition for CAN_BTR register  ********************/
#define  CAN_BTR_BRP                         ((uint32_t)0x000003FF)        /*!< Baud Rate Prescaler */
#define  CAN_BTR_TS1                         ((uint32_t)0x000F0000)        /*!< Time Segment 1 */
#define  CAN_BTR_TS2                         ((uint32_t)0x00700000)        /*!< Time Segment 2 */
#define  CAN_BTR_SJW                         ((uint32_t)0x03000000)        /*!< Resynchronization Jump Width */
#define  CAN_BTR_LBKM                        ((uint32_t)0x40000000)        /*!< Loop Back Mode (Debug) */
#define  CAN_BTR_SILM                        ((uint32_t)0x80000000)        /*!< Silent Mode */

/*!< Mailbox registers */
/*******************  Bit definition for CAN_TI0R register  ********************/
#define  CAN_TI0R_TXRQ                       ((uint32_t)0x00000001)        /*!< Transmit Mailbox Request */
#define  CAN_TI0R_RTR                        ((uint32_t)0x00000002)        /*!< Remote Transmission Request */
#define  CAN_TI0R_IDE                        ((uint32_t)0x00000004)        /*!< Identifier Extension */
#define  CAN_TI0R_EXID                       ((uint32_t)0x001FFFF8)        /*!< Extended Identifier */
#define  CAN_TI0R_STID                       ((uint32_t)0xFFE00000)        /*!< Standard Identifier or Extended Identifier */

/*******************  Bit definition for CAN_TDT0R register  *******************/
#define  CAN_TDT0R_DLC                       ((uint32_t)0x0000000F)        /*!< Data Length Code */
#define  CAN_TDT0R_TGT                       ((uint32_t)0x00000100)        /*!< Transmit Global Time */
#define  CAN_TDT0R_TIME                      ((uint32_t)0xFFFF0000)        /*!< Message Time Stamp */

/*******************  Bit definition for CAN_TDL0R register  *******************/
#define  CAN_TDL0R_DATA0                     ((uint32_t)0x000000FF)        /*!< Data byte 0 */
#define  CAN_TDL0R_DATA1                     ((uint32_t)0x0000FF00)        /*!< Data byte 1 */
#define  CAN_TDL0R_DATA2                     ((uint32_t)0x00FF0000)        /*!< Data byte 2 */
#define  CAN_TDL0R_DATA3                     ((uint32_t)0xFF000000)        /*!< Data byte 3 */

/*******************  Bit definition for CAN_TDH0R register  *******************/
#define  CAN_TDH0R_DATA4                     ((uint32_t)0x000000FF)        /*!< Data byte 4 */
#define  CAN_TDH0R_DATA5                     ((uint32_t)0x0000FF00)        /*!< Data byte 5 */
#define  CAN_TDH0R_DATA6                     ((uint32_t)0x00FF0000)        /*!< Data byte 6 */
#define  CAN_TDH0R_DATA7                     ((uint32_t)0xFF000000)        /*!< Data byte 7 */

/*******************  Bit definition for CAN_TI1R register  ********************/
#define  CAN_TI1R_TXRQ                       ((uint32_t)0x00000001)        /*!< Transmit Mailbox Request */
#define  CAN_TI1R_RTR                        ((uint32_t)0x00000002)        /*!< Remote Transmission Request */
#define  CAN_TI1R_IDE                        ((uint32_t)0x00000004)        /*!< Identifier Extension */
#define  CAN_TI1R_EXID                       ((uint32_t)0x001FFFF8)        /*!< Extended Identifier */
#define  CAN_TI1R_STID                       ((uint32_t)0xFFE00000)        /*!< Standard Identifier or Extended Identifier */

/*******************  Bit definition for CAN_TDT1R register  *******************/
#define  CAN_TDT1R_DLC                       ((uint32_t)0x0000000F)        /*!< Data Length Code */
#define  CAN_TDT1R_TGT                       ((uint32_t)0x00000100)        /*!< Transmit Global Time */
#define  CAN_TDT1R_TIME                      ((uint32_t)0xFFFF0000)        /*!< Message Time Stamp */

/*******************  Bit definition for CAN_TDL1R register  *******************/
#define  CAN_TDL1R_DATA0                     ((uint32_t)0x000000FF)        /*!< Data byte 0 */
#define  CAN_TDL1R_DATA1                     ((uint32_t)0x0000FF00)        /*!< Data byte 1 */
#define  CAN_TDL1R_DATA2                     ((uint32_t)0x00FF0000)        /*!< Data byte 2 */
#define  CAN_TDL1R_DATA3                     ((uint32_t)0xFF000000)        /*!< Data byte 3 */

/*******************  Bit definition for CAN_TDH1R register  *******************/
#define  CAN_TDH1R_DATA4                     ((uint32_t)0x000000FF)        /*!< Data byte 4 */
#define  CAN_TDH1R_DATA5                     ((uint32_t)0x0000FF00)        /*!< Data byte 5 */
#define  CAN_TDH1R_DATA6                     ((uint32_t)0x00FF0000)        /*!< Data byte 6 */
```

```
#define  CAN_TDH1R_DATA7                     ((uint32_t)0xFF000000)        /*!< Data byte 7 */

/*******************  Bit definition for CAN_TI2R register  *******************/
#define  CAN_TI2R_TXRQ                       ((uint32_t)0x00000001)        /*!< Transmit Mailbox Request */
#define  CAN_TI2R_RTR                        ((uint32_t)0x00000002)        /*!< Remote Transmission Request */
#define  CAN_TI2R_IDE                        ((uint32_t)0x00000004)        /*!< Identifier Extension */
#define  CAN_TI2R_EXID                       ((uint32_t)0x001FFFF8)        /*!< Extended identifier */
#define  CAN_TI2R_STID                       ((uint32_t)0xFFE00000)        /*!< Standard Identifier or Extended Identifier */

/*******************  Bit definition for CAN_TDT2R register  *******************/
#define  CAN_TDT2R_DLC                       ((uint32_t)0x0000000F)        /*!< Data Length Code */
#define  CAN_TDT2R_TGT                       ((uint32_t)0x00000100)        /*!< Transmit Global Time */
#define  CAN_TDT2R_TIME                      ((uint32_t)0xFFFF0000)        /*!< Message Time Stamp */

/*******************  Bit definition for CAN_TDL2R register  *******************/
#define  CAN_TDL2R_DATA0                     ((uint32_t)0x000000FF)        /*!< Data byte 0 */
#define  CAN_TDL2R_DATA1                     ((uint32_t)0x0000FF00)        /*!< Data byte 1 */
#define  CAN_TDL2R_DATA2                     ((uint32_t)0x00FF0000)        /*!< Data byte 2 */
#define  CAN_TDL2R_DATA3                     ((uint32_t)0xFF000000)        /*!< Data byte 3 */

/*******************  Bit definition for CAN_TDH2R register  *******************/
#define  CAN_TDH2R_DATA4                     ((uint32_t)0x000000FF)        /*!< Data byte 4 */
#define  CAN_TDH2R_DATA5                     ((uint32_t)0x0000FF00)        /*!< Data byte 5 */
#define  CAN_TDH2R_DATA6                     ((uint32_t)0x00FF0000)        /*!< Data byte 6 */
#define  CAN_TDH2R_DATA7                     ((uint32_t)0xFF000000)        /*!< Data byte 7 */

/*******************  Bit definition for CAN_RI0R register  *******************/
#define  CAN_RI0R_RTR                        ((uint32_t)0x00000002)        /*!< Remote Transmission Request */
#define  CAN_RI0R_IDE                        ((uint32_t)0x00000004)        /*!< Identifier Extension */
#define  CAN_RI0R_EXID                       ((uint32_t)0x001FFFF8)        /*!< Extended Identifier */
#define  CAN_RI0R_STID                       ((uint32_t)0xFFE00000)        /*!< Standard Identifier or Extended Identifier */

/*******************  Bit definition for CAN_RDT0R register  *******************/
#define  CAN_RDT0R_DLC                       ((uint32_t)0x0000000F)        /*!< Data Length Code */
#define  CAN_RDT0R_FMI                       ((uint32_t)0x0000FF00)        /*!< Filter Match Index */
#define  CAN_RDT0R_TIME                      ((uint32_t)0xFFFF0000)        /*!< Message Time Stamp */

/*******************  Bit definition for CAN_RDL0R register  *******************/
#define  CAN_RDL0R_DATA0                     ((uint32_t)0x000000FF)        /*!< Data byte 0 */
#define  CAN_RDL0R_DATA1                     ((uint32_t)0x0000FF00)        /*!< Data byte 1 */
#define  CAN_RDL0R_DATA2                     ((uint32_t)0x00FF0000)        /*!< Data byte 2 */
#define  CAN_RDL0R_DATA3                     ((uint32_t)0xFF000000)        /*!< Data byte 3 */

/*******************  Bit definition for CAN_RDH0R register  *******************/
#define  CAN_RDH0R_DATA4                     ((uint32_t)0x000000FF)        /*!< Data byte 4 */
#define  CAN_RDH0R_DATA5                     ((uint32_t)0x0000FF00)        /*!< Data byte 5 */
#define  CAN_RDH0R_DATA6                     ((uint32_t)0x00FF0000)        /*!< Data byte 6 */
#define  CAN_RDH0R_DATA7                     ((uint32_t)0xFF000000)        /*!< Data byte 7 */

/*******************  Bit definition for CAN_RI1R register  *******************/
#define  CAN_RI1R_RTR                        ((uint32_t)0x00000002)        /*!< Remote Transmission Request */
#define  CAN_RI1R_IDE                        ((uint32_t)0x00000004)        /*!< Identifier Extension */
#define  CAN_RI1R_EXID                       ((uint32_t)0x001FFFF8)        /*!< Extended identifier */
#define  CAN_RI1R_STID                       ((uint32_t)0xFFE00000)        /*!< Standard Identifier or Extended Identifier */

/*******************  Bit definition for CAN_RDT1R register  *******************/
#define  CAN_RDT1R_DLC                       ((uint32_t)0x0000000F)        /*!< Data Length Code */
#define  CAN_RDT1R_FMI                       ((uint32_t)0x0000FF00)        /*!< Filter Match Index */
#define  CAN_RDT1R_TIME                      ((uint32_t)0xFFFF0000)        /*!< Message Time Stamp */

/*******************  Bit definition for CAN_RDL1R register  *******************/
#define  CAN_RDL1R_DATA0                     ((uint32_t)0x000000FF)        /*!< Data byte 0 */
#define  CAN_RDL1R_DATA1                     ((uint32_t)0x0000FF00)        /*!< Data byte 1 */
#define  CAN_RDL1R_DATA2                     ((uint32_t)0x00FF0000)        /*!< Data byte 2 */
#define  CAN_RDL1R_DATA3                     ((uint32_t)0xFF000000)        /*!< Data byte 3 */
```

```
/*******************  Bit definition for CAN_RDH1R register  ******************/
#define  CAN_RDH1R_DATA4                     ((uint32_t)0x000000FF)        /*!< Data byte 4 */
#define  CAN_RDH1R_DATA5                     ((uint32_t)0x0000FF00)        /*!< Data byte 5 */
#define  CAN_RDH1R_DATA6                     ((uint32_t)0x00FF0000)        /*!< Data byte 6 */
#define  CAN_RDH1R_DATA7                     ((uint32_t)0xFF000000)        /*!< Data byte 7 */

/*!< CAN filter registers */
/*******************  Bit definition for CAN_FMR register  ********************/
#define  CAN_FMR_FINIT                       ((uint8_t)0x01)               /*!< Filter Init Mode */

/*******************  Bit definition for CAN_FM1R register  *******************/
#define  CAN_FM1R_FBM                        ((uint16_t)0x3FFF)            /*!< Filter Mode */
#define  CAN_FM1R_FBM0                       ((uint16_t)0x0001)            /*!< Filter Init Mode bit 0 */
#define  CAN_FM1R_FBM1                       ((uint16_t)0x0002)            /*!< Filter Init Mode bit 1 */
#define  CAN_FM1R_FBM2                       ((uint16_t)0x0004)            /*!< Filter Init Mode bit 2 */
#define  CAN_FM1R_FBM3                       ((uint16_t)0x0008)            /*!< Filter Init Mode bit 3 */
#define  CAN_FM1R_FBM4                       ((uint16_t)0x0010)            /*!< Filter Init Mode bit 4 */
#define  CAN_FM1R_FBM5                       ((uint16_t)0x0020)            /*!< Filter Init Mode bit 5 */
#define  CAN_FM1R_FBM6                       ((uint16_t)0x0040)            /*!< Filter Init Mode bit 6 */
#define  CAN_FM1R_FBM7                       ((uint16_t)0x0080)            /*!< Filter Init Mode bit 7 */
#define  CAN_FM1R_FBM8                       ((uint16_t)0x0100)            /*!< Filter Init Mode bit 8 */
#define  CAN_FM1R_FBM9                       ((uint16_t)0x0200)            /*!< Filter Init Mode bit 9 */
#define  CAN_FM1R_FBM10                      ((uint16_t)0x0400)            /*!< Filter Init Mode bit 10 */
#define  CAN_FM1R_FBM11                      ((uint16_t)0x0800)            /*!< Filter Init Mode bit 11 */
#define  CAN_FM1R_FBM12                      ((uint16_t)0x1000)            /*!< Filter Init Mode bit 12 */
#define  CAN_FM1R_FBM13                      ((uint16_t)0x2000)            /*!< Filter Init Mode bit 13 */

/*******************  Bit definition for CAN_FS1R register  *******************/
#define  CAN_FS1R_FSC                        ((uint16_t)0x3FFF)            /*!< Filter Scale Configuration */
#define  CAN_FS1R_FSC0                       ((uint16_t)0x0001)            /*!< Filter Scale Configuration bit 0 */
#define  CAN_FS1R_FSC1                       ((uint16_t)0x0002)            /*!< Filter Scale Configuration bit 1 */
#define  CAN_FS1R_FSC2                       ((uint16_t)0x0004)            /*!< Filter Scale Configuration bit 2 */
#define  CAN_FS1R_FSC3                       ((uint16_t)0x0008)            /*!< Filter Scale Configuration bit 3 */
#define  CAN_FS1R_FSC4                       ((uint16_t)0x0010)            /*!< Filter Scale Configuration bit 4 */
#define  CAN_FS1R_FSC5                       ((uint16_t)0x0020)            /*!< Filter Scale Configuration bit 5 */
#define  CAN_FS1R_FSC6                       ((uint16_t)0x0040)            /*!< Filter Scale Configuration bit 6 */
#define  CAN_FS1R_FSC7                       ((uint16_t)0x0080)            /*!< Filter Scale Configuration bit 7 */
#define  CAN_FS1R_FSC8                       ((uint16_t)0x0100)            /*!< Filter Scale Configuration bit 8 */
#define  CAN_FS1R_FSC9                       ((uint16_t)0x0200)            /*!< Filter Scale Configuration bit 9 */
#define  CAN_FS1R_FSC10                      ((uint16_t)0x0400)            /*!< Filter Scale Configuration bit 10 */
#define  CAN_FS1R_FSC11                      ((uint16_t)0x0800)            /*!< Filter Scale Configuration bit 11 */
#define  CAN_FS1R_FSC12                      ((uint16_t)0x1000)            /*!< Filter Scale Configuration bit 12 */
#define  CAN_FS1R_FSC13                      ((uint16_t)0x2000)            /*!< Filter Scale Configuration bit 13 */

/******************  Bit definition for CAN_FFA1R register  *******************/
#define  CAN_FFA1R_FFA                       ((uint16_t)0x3FFF)            /*!< Filter FIFO Assignment */
#define  CAN_FFA1R_FFA0                      ((uint16_t)0x0001)            /*!< Filter FIFO Assignment for Filter 0 */
#define  CAN_FFA1R_FFA1                      ((uint16_t)0x0002)            /*!< Filter FIFO Assignment for Filter 1 */
#define  CAN_FFA1R_FFA2                      ((uint16_t)0x0004)            /*!< Filter FIFO Assignment for Filter 2 */
#define  CAN_FFA1R_FFA3                      ((uint16_t)0x0008)            /*!< Filter FIFO Assignment for Filter 3 */
```

```
#define  CAN_FFA1R_FFA4                      ((uint16_t)0x0010)            /*!< Filter FIFO Assignment for Filter 4 */
#define  CAN_FFA1R_FFA5                      ((uint16_t)0x0020)            /*!< Filter FIFO Assignment for Filter 5 */
#define  CAN_FFA1R_FFA6                      ((uint16_t)0x0040)            /*!< Filter FIFO Assignment for Filter 6 */
#define  CAN_FFA1R_FFA7                      ((uint16_t)0x0080)            /*!< Filter FIFO Assignment for Filter 7 */
#define  CAN_FFA1R_FFA8                      ((uint16_t)0x0100)            /*!< Filter FIFO Assignment for Filter 8 */
#define  CAN_FFA1R_FFA9                      ((uint16_t)0x0200)            /*!< Filter FIFO Assignment for Filter 9 */
#define  CAN_FFA1R_FFA10                     ((uint16_t)0x0400)            /*!< Filter FIFO Assignment for Filter 10 */
#define  CAN_FFA1R_FFA11                     ((uint16_t)0x0800)            /*!< Filter FIFO Assignment for Filter 11 */
#define  CAN_FFA1R_FFA12                     ((uint16_t)0x1000)            /*!< Filter FIFO Assignment for Filter 12 */
#define  CAN_FFA1R_FFA13                     ((uint16_t)0x2000)            /*!< Filter FIFO Assignment for Filter 13 */

/*******************  Bit definition for CAN_FA1R register  *******************/
#define  CAN_FA1R_FACT                       ((uint16_t)0x3FFF)            /*!< Filter Active */
#define  CAN_FA1R_FACT0                      ((uint16_t)0x0001)            /*!< Filter 0 Active */
#define  CAN_FA1R_FACT1                      ((uint16_t)0x0002)            /*!< Filter 1 Active */
#define  CAN_FA1R_FACT2                      ((uint16_t)0x0004)            /*!< Filter 2 Active */
#define  CAN_FA1R_FACT3                      ((uint16_t)0x0008)            /*!< Filter 3 Active */
#define  CAN_FA1R_FACT4                      ((uint16_t)0x0010)            /*!< Filter 4 Active */
#define  CAN_FA1R_FACT5                      ((uint16_t)0x0020)            /*!< Filter 5 Active */
#define  CAN_FA1R_FACT6                      ((uint16_t)0x0040)            /*!< Filter 6 Active */
#define  CAN_FA1R_FACT7                      ((uint16_t)0x0080)            /*!< Filter 7 Active */
#define  CAN_FA1R_FACT8                      ((uint16_t)0x0100)            /*!< Filter 8 Active */
#define  CAN_FA1R_FACT9                      ((uint16_t)0x0200)            /*!< Filter 9 Active */
#define  CAN_FA1R_FACT10                     ((uint16_t)0x0400)            /*!< Filter 10 Active */
#define  CAN_FA1R_FACT11                     ((uint16_t)0x0800)            /*!< Filter 11 Active */
#define  CAN_FA1R_FACT12                     ((uint16_t)0x1000)            /*!< Filter 12 Active */
#define  CAN_FA1R_FACT13                     ((uint16_t)0x2000)            /*!< Filter 13 Active */

/*******************  Bit definition for CAN_F0R1 register  *******************/
#define  CAN_F0R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F0R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F0R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F0R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F0R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F0R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F0R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F0R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F0R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F0R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F0R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F0R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F0R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F0R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F0R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F0R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F0R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F0R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F0R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F0R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F0R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F0R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F0R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F0R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F0R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F0R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F0R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F0R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F0R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F0R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F0R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F0R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F1R1 register  *******************/
```

```
#define  CAN_F1R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F1R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F1R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F1R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F1R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F1R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F1R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F1R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F1R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F1R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F1R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F1R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F1R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F1R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F1R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F1R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F1R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F1R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F1R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F1R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F1R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F1R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F1R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F1R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F1R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F1R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F1R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F1R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F1R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F1R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F1R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F1R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F2R1 register  *******************/
#define  CAN_F2R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F2R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F2R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F2R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F2R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F2R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F2R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F2R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F2R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F2R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F2R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F2R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F2R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F2R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F2R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F2R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F2R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F2R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F2R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F2R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F2R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F2R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F2R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F2R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F2R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F2R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F2R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F2R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F2R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F2R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F2R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F2R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F3R1 register  *******************/
#define  CAN_F3R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F3R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F3R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F3R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F3R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
```

```
#define  CAN_F3R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F3R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F3R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F3R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F3R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F3R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F3R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F3R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F3R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F3R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F3R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F3R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F3R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F3R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F3R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F3R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F3R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F3R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F3R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F3R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F3R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F3R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F3R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F3R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F3R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F3R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F3R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F4R1 register  *******************/
#define  CAN_F4R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F4R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F4R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F4R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F4R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F4R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F4R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F4R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F4R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F4R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F4R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F4R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F4R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F4R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F4R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F4R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F4R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F4R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F4R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F4R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F4R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F4R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F4R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F4R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F4R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F4R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F4R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F4R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F4R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F4R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F4R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F4R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F5R1 register  *******************/
#define  CAN_F5R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F5R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F5R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F5R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F5R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F5R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F5R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F5R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F5R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F5R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
```

```
#define  CAN_F5R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F5R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F5R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F5R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F5R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F5R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F5R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F5R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F5R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F5R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F5R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F5R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F5R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F5R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F5R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F5R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F5R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F5R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F5R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F5R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F5R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F5R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F6R1 register  *******************/
#define  CAN_F6R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F6R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F6R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F6R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F6R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F6R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F6R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F6R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F6R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F6R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F6R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F6R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F6R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F6R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F6R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F6R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F6R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F6R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F6R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F6R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F6R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F6R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F6R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F6R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F6R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F6R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F6R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F6R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F6R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F6R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F6R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F6R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F7R1 register  *******************/
#define  CAN_F7R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F7R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F7R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F7R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F7R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F7R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F7R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F7R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F7R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F7R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F7R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F7R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F7R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F7R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F7R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
```

```
#define  CAN_F7R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F7R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F7R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F7R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F7R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F7R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F7R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F7R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F7R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F7R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F7R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F7R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F7R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F7R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F7R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F7R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F7R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F8R1 register  *******************/
#define  CAN_F8R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F8R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F8R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F8R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F8R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F8R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F8R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F8R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F8R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F8R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F8R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F8R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F8R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F8R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F8R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F8R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F8R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F8R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F8R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F8R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F8R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F8R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F8R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F8R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F8R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F8R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F8R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F8R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F8R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F8R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F8R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F8R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F9R1 register  *******************/
#define  CAN_F9R1_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F9R1_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F9R1_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F9R1_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F9R1_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F9R1_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F9R1_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F9R1_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F9R1_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F9R1_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F9R1_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F9R1_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F9R1_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F9R1_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F9R1_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F9R1_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F9R1_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F9R1_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F9R1_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F9R1_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
```

```
#define  CAN_F9R1_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F9R1_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F9R1_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F9R1_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F9R1_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F9R1_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F9R1_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F9R1_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F9R1_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F9R1_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F9R1_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F9R1_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F10R1 register  ******************/
#define  CAN_F10R1_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F10R1_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F10R1_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F10R1_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F10R1_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F10R1_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F10R1_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F10R1_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F10R1_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F10R1_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F10R1_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F10R1_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F10R1_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F10R1_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F10R1_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F10R1_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F10R1_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F10R1_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F10R1_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F10R1_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F10R1_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F10R1_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F10R1_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F10R1_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F10R1_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F10R1_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F10R1_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F10R1_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F10R1_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F10R1_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F10R1_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F10R1_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F11R1 register  ******************/
#define  CAN_F11R1_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F11R1_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F11R1_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F11R1_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F11R1_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F11R1_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F11R1_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F11R1_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F11R1_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F11R1_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F11R1_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F11R1_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F11R1_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F11R1_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F11R1_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F11R1_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F11R1_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F11R1_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F11R1_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F11R1_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F11R1_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F11R1_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F11R1_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F11R1_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F11R1_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
```

```
  #define  CAN_F11R1_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
  #define  CAN_F11R1_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
  #define  CAN_F11R1_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
  #define  CAN_F11R1_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
  #define  CAN_F11R1_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
  #define  CAN_F11R1_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
  #define  CAN_F11R1_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

  /*******************  Bit definition for CAN_F12R1 register  ******************/
  #define  CAN_F12R1_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
  #define  CAN_F12R1_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
  #define  CAN_F12R1_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
  #define  CAN_F12R1_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
  #define  CAN_F12R1_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
  #define  CAN_F12R1_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
  #define  CAN_F12R1_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
  #define  CAN_F12R1_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
  #define  CAN_F12R1_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
  #define  CAN_F12R1_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
  #define  CAN_F12R1_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
  #define  CAN_F12R1_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
  #define  CAN_F12R1_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
  #define  CAN_F12R1_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
  #define  CAN_F12R1_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
  #define  CAN_F12R1_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
  #define  CAN_F12R1_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
  #define  CAN_F12R1_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
  #define  CAN_F12R1_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
  #define  CAN_F12R1_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
  #define  CAN_F12R1_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
  #define  CAN_F12R1_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
  #define  CAN_F12R1_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
  #define  CAN_F12R1_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
  #define  CAN_F12R1_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
  #define  CAN_F12R1_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
  #define  CAN_F12R1_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
  #define  CAN_F12R1_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
  #define  CAN_F12R1_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
  #define  CAN_F12R1_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
  #define  CAN_F12R1_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
  #define  CAN_F12R1_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

  /*******************  Bit definition for CAN_F13R1 register  ******************/
  #define  CAN_F13R1_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
  #define  CAN_F13R1_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
  #define  CAN_F13R1_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
  #define  CAN_F13R1_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
  #define  CAN_F13R1_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
  #define  CAN_F13R1_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
  #define  CAN_F13R1_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
  #define  CAN_F13R1_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
  #define  CAN_F13R1_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
  #define  CAN_F13R1_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
  #define  CAN_F13R1_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
  #define  CAN_F13R1_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
  #define  CAN_F13R1_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
  #define  CAN_F13R1_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
  #define  CAN_F13R1_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
  #define  CAN_F13R1_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
  #define  CAN_F13R1_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
  #define  CAN_F13R1_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
  #define  CAN_F13R1_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
  #define  CAN_F13R1_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
  #define  CAN_F13R1_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
  #define  CAN_F13R1_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
  #define  CAN_F13R1_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
  #define  CAN_F13R1_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
  #define  CAN_F13R1_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
  #define  CAN_F13R1_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
  #define  CAN_F13R1_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
  #define  CAN_F13R1_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
  #define  CAN_F13R1_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
  #define  CAN_F13R1_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
```

```
#define  CAN_F13R1_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F13R1_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F0R2 register  *******************/
#define  CAN_F0R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F0R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F0R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F0R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F0R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F0R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F0R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F0R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F0R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F0R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F0R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F0R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F0R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F0R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F0R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F0R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F0R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F0R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F0R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F0R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F0R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F0R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F0R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F0R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F0R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F0R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F0R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F0R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F0R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F0R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F0R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F0R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F1R2 register  *******************/
#define  CAN_F1R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F1R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F1R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F1R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F1R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F1R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F1R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F1R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F1R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F1R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F1R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F1R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F1R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F1R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F1R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F1R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F1R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F1R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F1R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F1R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F1R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F1R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F1R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F1R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F1R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F1R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F1R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F1R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F1R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F1R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F1R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F1R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F2R2 register  *******************/
#define  CAN_F2R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
```

```
#define  CAN_F2R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F2R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F2R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F2R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F2R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F2R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F2R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F2R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F2R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F2R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F2R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F2R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F2R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F2R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F2R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F2R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F2R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F2R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F2R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F2R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F2R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F2R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F2R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F2R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F2R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F2R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F2R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F2R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F2R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F2R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F2R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F3R2 register  *******************/
#define  CAN_F3R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F3R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F3R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F3R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F3R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F3R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F3R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F3R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F3R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F3R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F3R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F3R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F3R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F3R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F3R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F3R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F3R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F3R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F3R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F3R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F3R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F3R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F3R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F3R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F3R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F3R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F3R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F3R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F3R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F3R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F3R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F3R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F4R2 register  *******************/
#define  CAN_F4R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F4R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F4R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F4R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F4R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F4R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
```

```
#define  CAN_F4R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F4R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F4R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F4R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F4R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F4R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F4R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F4R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F4R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F4R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F4R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F4R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F4R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F4R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F4R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F4R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F4R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F4R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F4R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F4R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F4R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F4R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F4R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F4R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F4R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F4R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F5R2 register  *******************/
#define  CAN_F5R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F5R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F5R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F5R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F5R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F5R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F5R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F5R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F5R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F5R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F5R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F5R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F5R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F5R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F5R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F5R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F5R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F5R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F5R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F5R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F5R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F5R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F5R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F5R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F5R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F5R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F5R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F5R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F5R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F5R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F5R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F5R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F6R2 register  *******************/
#define  CAN_F6R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F6R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F6R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F6R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F6R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F6R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F6R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F6R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F6R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F6R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F6R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
```

```
#define  CAN_F6R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F6R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F6R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F6R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F6R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F6R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F6R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F6R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F6R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F6R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F6R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F6R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F6R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F6R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F6R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F6R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F6R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F6R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F6R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F6R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F6R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F7R2 register  *******************/
#define  CAN_F7R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F7R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F7R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F7R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F7R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F7R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F7R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F7R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F7R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F7R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F7R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F7R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F7R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F7R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F7R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F7R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F7R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F7R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F7R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F7R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F7R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F7R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F7R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F7R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F7R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F7R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F7R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F7R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F7R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F7R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F7R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F7R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F8R2 register  *******************/
#define  CAN_F8R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F8R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F8R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F8R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F8R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F8R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F8R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F8R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F8R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F8R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F8R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F8R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F8R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F8R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F8R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F8R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
```

```
#define  CAN_F8R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F8R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F8R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F8R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F8R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F8R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F8R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F8R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F8R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F8R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F8R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F8R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F8R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F8R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F8R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F8R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F9R2 register  *******************/
#define  CAN_F9R2_FB0                        ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F9R2_FB1                        ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F9R2_FB2                        ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F9R2_FB3                        ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F9R2_FB4                        ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F9R2_FB5                        ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F9R2_FB6                        ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F9R2_FB7                        ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F9R2_FB8                        ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F9R2_FB9                        ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F9R2_FB10                       ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F9R2_FB11                       ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F9R2_FB12                       ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F9R2_FB13                       ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F9R2_FB14                       ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F9R2_FB15                       ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F9R2_FB16                       ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F9R2_FB17                       ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F9R2_FB18                       ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F9R2_FB19                       ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F9R2_FB20                       ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F9R2_FB21                       ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F9R2_FB22                       ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F9R2_FB23                       ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F9R2_FB24                       ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F9R2_FB25                       ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F9R2_FB26                       ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F9R2_FB27                       ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F9R2_FB28                       ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F9R2_FB29                       ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F9R2_FB30                       ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F9R2_FB31                       ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F10R2 register  ******************/
#define  CAN_F10R2_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F10R2_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F10R2_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F10R2_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F10R2_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F10R2_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F10R2_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F10R2_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F10R2_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F10R2_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F10R2_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F10R2_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F10R2_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F10R2_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F10R2_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F10R2_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F10R2_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F10R2_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F10R2_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F10R2_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F10R2_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
```

```
#define  CAN_F10R2_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F10R2_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F10R2_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F10R2_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F10R2_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F10R2_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F10R2_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F10R2_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F10R2_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F10R2_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F10R2_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F11R2 register  *******************/
#define  CAN_F11R2_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F11R2_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F11R2_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F11R2_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F11R2_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F11R2_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F11R2_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F11R2_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F11R2_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F11R2_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F11R2_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F11R2_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F11R2_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F11R2_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F11R2_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F11R2_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F11R2_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F11R2_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F11R2_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F11R2_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F11R2_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F11R2_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F11R2_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F11R2_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F11R2_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F11R2_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F11R2_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F11R2_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F11R2_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F11R2_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F11R2_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F11R2_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F12R2 register  *******************/
#define  CAN_F12R2_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F12R2_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F12R2_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F12R2_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F12R2_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F12R2_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F12R2_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F12R2_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F12R2_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F12R2_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F12R2_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F12R2_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F12R2_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F12R2_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F12R2_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F12R2_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F12R2_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F12R2_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F12R2_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F12R2_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F12R2_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F12R2_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F12R2_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F12R2_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F12R2_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F12R2_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
```

```
#define  CAN_F12R2_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F12R2_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F12R2_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F12R2_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F12R2_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F12R2_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/*******************  Bit definition for CAN_F13R2 register  ******************/
#define  CAN_F13R2_FB0                       ((uint32_t)0x00000001)        /*!< Filter bit 0 */
#define  CAN_F13R2_FB1                       ((uint32_t)0x00000002)        /*!< Filter bit 1 */
#define  CAN_F13R2_FB2                       ((uint32_t)0x00000004)        /*!< Filter bit 2 */
#define  CAN_F13R2_FB3                       ((uint32_t)0x00000008)        /*!< Filter bit 3 */
#define  CAN_F13R2_FB4                       ((uint32_t)0x00000010)        /*!< Filter bit 4 */
#define  CAN_F13R2_FB5                       ((uint32_t)0x00000020)        /*!< Filter bit 5 */
#define  CAN_F13R2_FB6                       ((uint32_t)0x00000040)        /*!< Filter bit 6 */
#define  CAN_F13R2_FB7                       ((uint32_t)0x00000080)        /*!< Filter bit 7 */
#define  CAN_F13R2_FB8                       ((uint32_t)0x00000100)        /*!< Filter bit 8 */
#define  CAN_F13R2_FB9                       ((uint32_t)0x00000200)        /*!< Filter bit 9 */
#define  CAN_F13R2_FB10                      ((uint32_t)0x00000400)        /*!< Filter bit 10 */
#define  CAN_F13R2_FB11                      ((uint32_t)0x00000800)        /*!< Filter bit 11 */
#define  CAN_F13R2_FB12                      ((uint32_t)0x00001000)        /*!< Filter bit 12 */
#define  CAN_F13R2_FB13                      ((uint32_t)0x00002000)        /*!< Filter bit 13 */
#define  CAN_F13R2_FB14                      ((uint32_t)0x00004000)        /*!< Filter bit 14 */
#define  CAN_F13R2_FB15                      ((uint32_t)0x00008000)        /*!< Filter bit 15 */
#define  CAN_F13R2_FB16                      ((uint32_t)0x00010000)        /*!< Filter bit 16 */
#define  CAN_F13R2_FB17                      ((uint32_t)0x00020000)        /*!< Filter bit 17 */
#define  CAN_F13R2_FB18                      ((uint32_t)0x00040000)        /*!< Filter bit 18 */
#define  CAN_F13R2_FB19                      ((uint32_t)0x00080000)        /*!< Filter bit 19 */
#define  CAN_F13R2_FB20                      ((uint32_t)0x00100000)        /*!< Filter bit 20 */
#define  CAN_F13R2_FB21                      ((uint32_t)0x00200000)        /*!< Filter bit 21 */
#define  CAN_F13R2_FB22                      ((uint32_t)0x00400000)        /*!< Filter bit 22 */
#define  CAN_F13R2_FB23                      ((uint32_t)0x00800000)        /*!< Filter bit 23 */
#define  CAN_F13R2_FB24                      ((uint32_t)0x01000000)        /*!< Filter bit 24 */
#define  CAN_F13R2_FB25                      ((uint32_t)0x02000000)        /*!< Filter bit 25 */
#define  CAN_F13R2_FB26                      ((uint32_t)0x04000000)        /*!< Filter bit 26 */
#define  CAN_F13R2_FB27                      ((uint32_t)0x08000000)        /*!< Filter bit 27 */
#define  CAN_F13R2_FB28                      ((uint32_t)0x10000000)        /*!< Filter bit 28 */
#define  CAN_F13R2_FB29                      ((uint32_t)0x20000000)        /*!< Filter bit 29 */
#define  CAN_F13R2_FB30                      ((uint32_t)0x40000000)        /*!< Filter bit 30 */
#define  CAN_F13R2_FB31                      ((uint32_t)0x80000000)        /*!< Filter bit 31 */

/******************************************************************************/
/*                                                                            */
/*                        Serial Peripheral Interface                         */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for SPI_CR1 register  ********************/
#define  SPI_CR1_CPHA                        ((uint16_t)0x0001)            /*!< Clock Phase */
#define  SPI_CR1_CPOL                        ((uint16_t)0x0002)            /*!< Clock Polarity */
#define  SPI_CR1_MSTR                        ((uint16_t)0x0004)            /*!< Master Selection */

#define  SPI_CR1_BR                          ((uint16_t)0x0038)            /*!< BR[2:0] bits (Baud Rate Control) */
#define  SPI_CR1_BR_0                        ((uint16_t)0x0008)            /*!< Bit 0 */
#define  SPI_CR1_BR_1                        ((uint16_t)0x0010)            /*!< Bit 1 */
#define  SPI_CR1_BR_2                        ((uint16_t)0x0020)            /*!< Bit 2 */

#define  SPI_CR1_SPE                         ((uint16_t)0x0040)            /*!< SPI Enable */
#define  SPI_CR1_LSBFIRST                    ((uint16_t)0x0080)            /*!< Frame Format */
#define  SPI_CR1_SSI                         ((uint16_t)0x0100)            /*!< Internal slave select */
#define  SPI_CR1_SSM                         ((uint16_t)0x0200)            /*!< Software slave management */
#define  SPI_CR1_RXONLY                      ((uint16_t)0x0400)            /*!< Receive only */
#define  SPI_CR1_DFF                         ((uint16_t)0x0800)            /*!< Data Frame Format */
#define  SPI_CR1_CRCNEXT                     ((uint16_t)0x1000)            /*!< Transmit CRC next */
#define  SPI_CR1_CRCEN                       ((uint16_t)0x2000)            /*!< Hardware CRC calculation enable */
#define  SPI_CR1_BIDIOE                      ((uint16_t)0x4000)            /*!< Output enable in bidirectional mode */
#define  SPI_CR1_BIDIMODE                    ((uint16_t)0x8000)            /*!< Bidirectional data mode enable */

```

```
/*******************  Bit definition for SPI_CR2 register  *******************/
#define  SPI_CR2_RXDMAEN                     ((uint8_t)0x01)            /*!< Rx Buffer DMA Enable */
#define  SPI_CR2_TXDMAEN                     ((uint8_t)0x02)            /*!< Tx Buffer DMA Enable */
#define  SPI_CR2_SSOE                        ((uint8_t)0x04)            /*!< SS Output Enable */
#define  SPI_CR2_ERRIE                       ((uint8_t)0x20)            /*!< Error Interrupt Enable */
#define  SPI_CR2_RXNEIE                      ((uint8_t)0x40)            /*!< RX buffer Not Empty Interrupt Enable */
#define  SPI_CR2_TXEIE                       ((uint8_t)0x80)            /*!< Tx buffer Empty Interrupt Enable */

/********************  Bit definition for SPI_SR register  ********************/
#define  SPI_SR_RXNE                         ((uint8_t)0x01)            /*!< Receive buffer Not Empty */
#define  SPI_SR_TXE                          ((uint8_t)0x02)            /*!< Transmit buffer Empty */
#define  SPI_SR_CHSIDE                       ((uint8_t)0x04)            /*!< Channel side */
#define  SPI_SR_UDR                          ((uint8_t)0x08)            /*!< Underrun flag */
#define  SPI_SR_CRCERR                       ((uint8_t)0x10)            /*!< CRC Error flag */
#define  SPI_SR_MODF                         ((uint8_t)0x20)            /*!< Mode fault */
#define  SPI_SR_OVR                          ((uint8_t)0x40)            /*!< Overrun flag */
#define  SPI_SR_BSY                          ((uint8_t)0x80)            /*!< Busy flag */

/********************  Bit definition for SPI_DR register  ********************/
#define  SPI_DR_DR                           ((uint16_t)0xFFFF)         /*!< Data Register */

/*******************  Bit definition for SPI_CRCPR register  *******************/
#define  SPI_CRCPR_CRCPOLY                   ((uint16_t)0xFFFF)         /*!< CRC polynomial register */

/******************  Bit definition for SPI_RXCRCR register  *******************/
#define  SPI_RXCRCR_RXCRC                    ((uint16_t)0xFFFF)         /*!< Rx CRC Register */

/******************  Bit definition for SPI_TXCRCR register  *******************/
#define  SPI_TXCRCR_TXCRC                    ((uint16_t)0xFFFF)         /*!< Tx CRC Register */

/******************  Bit definition for SPI_I2SCFGR register  *****************/
#define  SPI_I2SCFGR_CHLEN                   ((uint16_t)0x0001)         /*!< Channel length (number of bits per audio channel) */

#define  SPI_I2SCFGR_DATLEN                  ((uint16_t)0x0006)         /*!< DATLEN[1:0] bits (Data length to be transferred) */
#define  SPI_I2SCFGR_DATLEN_0                ((uint16_t)0x0002)         /*!< Bit 0 */
#define  SPI_I2SCFGR_DATLEN_1                ((uint16_t)0x0004)         /*!< Bit 1 */

#define  SPI_I2SCFGR_CKPOL                   ((uint16_t)0x0008)         /*!< steady state clock polarity */

#define  SPI_I2SCFGR_I2SSTD                  ((uint16_t)0x0030)         /*!< I2SSTD[1:0] bits (I2S standard selection) */
#define  SPI_I2SCFGR_I2SSTD_0                ((uint16_t)0x0010)         /*!< Bit 0 */
#define  SPI_I2SCFGR_I2SSTD_1                ((uint16_t)0x0020)         /*!< Bit 1 */

#define  SPI_I2SCFGR_PCMSYNC                 ((uint16_t)0x0080)         /*!< PCM frame synchronization */

#define  SPI_I2SCFGR_I2SCFG                  ((uint16_t)0x0300)         /*!< I2SCFG[1:0] bits (I2S configuration mode) */
#define  SPI_I2SCFGR_I2SCFG_0                ((uint16_t)0x0100)         /*!< Bit 0 */
#define  SPI_I2SCFGR_I2SCFG_1                ((uint16_t)0x0200)         /*!< Bit 1 */

#define  SPI_I2SCFGR_I2SE                    ((uint16_t)0x0400)         /*!< I2S Enable */
#define  SPI_I2SCFGR_I2SMOD                  ((uint16_t)0x0800)         /*!< I2S mode selection */

/******************  Bit definition for SPI_I2SPR register  *******************/
#define  SPI_I2SPR_I2SDIV                    ((uint16_t)0x00FF)         /*!< I2S Linear prescaler */
#define  SPI_I2SPR_ODD                       ((uint16_t)0x0100)         /*!< Odd factor for the prescaler */
#define  SPI_I2SPR_MCKOE                     ((uint16_t)0x0200)         /*!< Master Clock Output Enable */

/******************************************************************************/
/*                                                                            */
/*                        Inter-integrated Circuit Interface                  */
/*                                                                            */
```

```
/******************************************************************************/

/*******************  Bit definition for I2C_CR1 register  *******************/
#define  I2C_CR1_PE                          ((uint16_t)0x0001)            /*!< Peripheral Enable */
#define  I2C_CR1_SMBUS                       ((uint16_t)0x0002)            /*!< SMBus Mode */
#define  I2C_CR1_SMBTYPE                     ((uint16_t)0x0008)            /*!< SMBus Type */
#define  I2C_CR1_ENARP                       ((uint16_t)0x0010)            /*!< ARP Enable */
#define  I2C_CR1_ENPEC                       ((uint16_t)0x0020)            /*!< PEC Enable */
#define  I2C_CR1_ENGC                        ((uint16_t)0x0040)            /*!< General Call Enable */
#define  I2C_CR1_NOSTRETCH                   ((uint16_t)0x0080)            /*!< Clock Stretching Disable (Slave mode) */
#define  I2C_CR1_START                       ((uint16_t)0x0100)            /*!< Start Generation */
#define  I2C_CR1_STOP                        ((uint16_t)0x0200)            /*!< Stop Generation */
#define  I2C_CR1_ACK                         ((uint16_t)0x0400)            /*!< Acknowledge Enable */
#define  I2C_CR1_POS                         ((uint16_t)0x0800)            /*!< Acknowledge/PEC Position (for data reception) */
#define  I2C_CR1_PEC                         ((uint16_t)0x1000)            /*!< Packet Error Checking */
#define  I2C_CR1_ALERT                       ((uint16_t)0x2000)            /*!< SMBus Alert */
#define  I2C_CR1_SWRST                       ((uint16_t)0x8000)            /*!< Software Reset */

/*******************  Bit definition for I2C_CR2 register  *******************/
#define  I2C_CR2_FREQ                        ((uint16_t)0x003F)            /*!< FREQ[5:0] bits (Peripheral Clock Frequency) */
#define  I2C_CR2_FREQ_0                      ((uint16_t)0x0001)            /*!< Bit 0 */
#define  I2C_CR2_FREQ_1                      ((uint16_t)0x0002)            /*!< Bit 1 */
#define  I2C_CR2_FREQ_2                      ((uint16_t)0x0004)            /*!< Bit 2 */
#define  I2C_CR2_FREQ_3                      ((uint16_t)0x0008)            /*!< Bit 3 */
#define  I2C_CR2_FREQ_4                      ((uint16_t)0x0010)            /*!< Bit 4 */
#define  I2C_CR2_FREQ_5                      ((uint16_t)0x0020)            /*!< Bit 5 */

#define  I2C_CR2_ITERREN                     ((uint16_t)0x0100)            /*!< Error Interrupt Enable */
#define  I2C_CR2_ITEVTEN                     ((uint16_t)0x0200)            /*!< Event Interrupt Enable */
#define  I2C_CR2_ITBUFEN                     ((uint16_t)0x0400)            /*!< Buffer Interrupt Enable */
#define  I2C_CR2_DMAEN                       ((uint16_t)0x0800)            /*!< DMA Requests Enable */
#define  I2C_CR2_LAST                        ((uint16_t)0x1000)            /*!< DMA Last Transfer */

/*******************  Bit definition for I2C_OAR1 register  *******************/
#define  I2C_OAR1_ADD1_7                     ((uint16_t)0x00FE)            /*!< Interface Address */
#define  I2C_OAR1_ADD8_9                     ((uint16_t)0x0300)            /*!< Interface Address */

#define  I2C_OAR1_ADD0                       ((uint16_t)0x0001)            /*!< Bit 0 */
#define  I2C_OAR1_ADD1                       ((uint16_t)0x0002)            /*!< Bit 1 */
#define  I2C_OAR1_ADD2                       ((uint16_t)0x0004)            /*!< Bit 2 */
#define  I2C_OAR1_ADD3                       ((uint16_t)0x0008)            /*!< Bit 3 */
#define  I2C_OAR1_ADD4                       ((uint16_t)0x0010)            /*!< Bit 4 */
#define  I2C_OAR1_ADD5                       ((uint16_t)0x0020)            /*!< Bit 5 */
#define  I2C_OAR1_ADD6                       ((uint16_t)0x0040)            /*!< Bit 6 */
#define  I2C_OAR1_ADD7                       ((uint16_t)0x0080)            /*!< Bit 7 */
#define  I2C_OAR1_ADD8                       ((uint16_t)0x0100)            /*!< Bit 8 */
#define  I2C_OAR1_ADD9                       ((uint16_t)0x0200)            /*!< Bit 9 */

#define  I2C_OAR1_ADDMODE                    ((uint16_t)0x8000)            /*!< Addressing Mode (Slave mode) */

/*******************  Bit definition for I2C_OAR2 register  *******************/
#define  I2C_OAR2_ENDUAL                     ((uint8_t)0x01)               /*!< Dual addressing mode enable */
#define  I2C_OAR2_ADD2                       ((uint8_t)0xFE)               /*!< Interface address */

/*******************  Bit definition for I2C_DR register  *******************/
#define  I2C_DR_DR                           ((uint8_t)0xFF)               /*!< 8-bit Data Register */

/*******************  Bit definition for I2C_SR1 register  *******************/
#define  I2C_SR1_SB                          ((uint16_t)0x0001)            /*!< Start Bit (Master mode) */
#define  I2C_SR1_ADDR                        ((uint16_t)0x0002)            /*!< Address sent (master mode)/matched (slave mode) */
#define  I2C_SR1_BTF                         ((uint16_t)0x0004)            /*!< Byte Transfer Finished */
#define  I2C_SR1_ADD10                       ((uint16_t)0x0008)            /*!< 10-bit header sent (Master mode) */
#define  I2C_SR1_STOPF                       ((uint16_t)0x0010)            /*!< Stop detection (Slave mode) */
```

```
#define  I2C_SR1_RXNE                        ((uint16_t)0x0040)            /*!< Data Register not Empty (receivers) */
#define  I2C_SR1_TXE                         ((uint16_t)0x0080)            /*!< Data Register Empty (transmitters) */
#define  I2C_SR1_BERR                        ((uint16_t)0x0100)            /*!< Bus Error */
#define  I2C_SR1_ARLO                        ((uint16_t)0x0200)            /*!< Arbitration Lost (master mode) */
#define  I2C_SR1_AF                          ((uint16_t)0x0400)            /*!< Acknowledge Failure */
#define  I2C_SR1_OVR                         ((uint16_t)0x0800)            /*!< Overrun/Underrun */
#define  I2C_SR1_PECERR                      ((uint16_t)0x1000)            /*!< PEC Error in reception */
#define  I2C_SR1_TIMEOUT                     ((uint16_t)0x4000)            /*!< Timeout or Tlow Error */
#define  I2C_SR1_SMBALERT                    ((uint16_t)0x8000)            /*!< SMBus Alert */

/********************  Bit definition for I2C_SR2 register  ********************/
#define  I2C_SR2_MSL                         ((uint16_t)0x0001)            /*!< Master/Slave */
#define  I2C_SR2_BUSY                        ((uint16_t)0x0002)            /*!< Bus Busy */
#define  I2C_SR2_TRA                         ((uint16_t)0x0004)            /*!< Transmitter/Receiver */
#define  I2C_SR2_GENCALL                     ((uint16_t)0x0010)            /*!< General Call Address (Slave mode) */
#define  I2C_SR2_SMBDEFAULT                  ((uint16_t)0x0020)            /*!< SMBus Device Default Address (Slave mode) */
#define  I2C_SR2_SMBHOST                     ((uint16_t)0x0040)            /*!< SMBus Host Header (Slave mode) */
#define  I2C_SR2_DUALF                       ((uint16_t)0x0080)            /*!< Dual Flag (Slave mode) */
#define  I2C_SR2_PEC                         ((uint16_t)0xFF00)            /*!< Packet Error Checking Register */

/********************  Bit definition for I2C_CCR register  ********************/
#define  I2C_CCR_CCR                         ((uint16_t)0x0FFF)            /*!< Clock Control Register in Fast/Standard mode (Master mode) */
#define  I2C_CCR_DUTY                        ((uint16_t)0x4000)            /*!< Fast Mode Duty Cycle */
#define  I2C_CCR_FS                          ((uint16_t)0x8000)            /*!< I2C Master Mode Selection */

/********************  Bit definition for I2C_TRISE register  ********************/
#define  I2C_TRISE_TRISE                     ((uint8_t)0x3F)               /*!< Maximum Rise Time in Fast/Standard mode (Master mode) */

/******************************************************************************/
/*                                                                            */
/*         Universal Synchronous Asynchronous Receiver Transmitter            */
/*                                                                            */
/******************************************************************************/

/********************  Bit definition for USART_SR register  ********************/
#define  USART_SR_PE                         ((uint16_t)0x0001)            /*!< Parity Error */
#define  USART_SR_FE                         ((uint16_t)0x0002)            /*!< Framing Error */
#define  USART_SR_NE                         ((uint16_t)0x0004)            /*!< Noise Error Flag */
#define  USART_SR_ORE                        ((uint16_t)0x0008)            /*!< OverRun Error */
#define  USART_SR_IDLE                       ((uint16_t)0x0010)            /*!< IDLE line detected */
#define  USART_SR_RXNE                       ((uint16_t)0x0020)            /*!< Read Data Register Not Empty */
#define  USART_SR_TC                         ((uint16_t)0x0040)            /*!< Transmission Complete */
#define  USART_SR_TXE                        ((uint16_t)0x0080)            /*!< Transmit Data Register Empty */
#define  USART_SR_LBD                        ((uint16_t)0x0100)            /*!< LIN Break Detection Flag */
#define  USART_SR_CTS                        ((uint16_t)0x0200)            /*!< CTS Flag */

/********************  Bit definition for USART_DR register  ********************/
#define  USART_DR_DR                         ((uint16_t)0x01FF)            /*!< Data value */

/******************  Bit definition for USART_BRR register  ********************/
#define  USART_BRR_DIV_Fraction              ((uint16_t)0x000F)            /*!< Fraction of USARTDIV */
#define  USART_BRR_DIV_Mantissa              ((uint16_t)0xFFF0)            /*!< Mantissa of USARTDIV */

/******************  Bit definition for USART_CR1 register  ********************/
#define  USART_CR1_SBK                       ((uint16_t)0x0001)            /*!< Send Break */
#define  USART_CR1_RWU                       ((uint16_t)0x0002)            /*!< Receiver wakeup */
#define  USART_CR1_RE                        ((uint16_t)0x0004)            /*!< Receiver Enable */
#define  USART_CR1_TE                        ((uint16_t)0x0008)            /*!< Transmitter Enable */
#define  USART_CR1_IDLEIE                    ((uint16_t)0x0010)            /*!< IDLE Interrupt Enable */
#define  USART_CR1_RXNEIE                    ((uint16_t)0x0020)            /*!< RXNE Interrupt Enable */
```

```
#define  USART_CR1_TCIE                      ((uint16_t)0x0040)            /*!< Transmission Complete Interrupt Enable */
#define  USART_CR1_TXEIE                     ((uint16_t)0x0080)            /*!< PE Interrupt Enable */
#define  USART_CR1_PEIE                      ((uint16_t)0x0100)            /*!< PE Interrupt Enable */
#define  USART_CR1_PS                        ((uint16_t)0x0200)            /*!< Parity Selection */
#define  USART_CR1_PCE                       ((uint16_t)0x0400)            /*!< Parity Control Enable */
#define  USART_CR1_WAKE                      ((uint16_t)0x0800)            /*!< Wakeup method */
#define  USART_CR1_M                         ((uint16_t)0x1000)            /*!< Word length */
#define  USART_CR1_UE                        ((uint16_t)0x2000)            /*!< USART Enable */
#define  USART_CR1_OVER8                     ((uint16_t)0x8000)            /*!< USART Oversmapling 8-bits */

/******************  Bit definition for USART_CR2 register  *******************/
#define  USART_CR2_ADD                       ((uint16_t)0x000F)            /*!< Address of the USART node */
#define  USART_CR2_LBDL                      ((uint16_t)0x0020)            /*!< LIN Break Detection Length */
#define  USART_CR2_LBDIE                     ((uint16_t)0x0040)            /*!< LIN Break Detection Interrupt Enable */
#define  USART_CR2_LBCL                      ((uint16_t)0x0100)            /*!< Last Bit Clock pulse */
#define  USART_CR2_CPHA                      ((uint16_t)0x0200)            /*!< Clock Phase */
#define  USART_CR2_CPOL                      ((uint16_t)0x0400)            /*!< Clock Polarity */
#define  USART_CR2_CLKEN                     ((uint16_t)0x0800)            /*!< Clock Enable */

#define  USART_CR2_STOP                      ((uint16_t)0x3000)            /*!< STOP[1:0] bits (STOP bits) */
#define  USART_CR2_STOP_0                    ((uint16_t)0x1000)            /*!< Bit 0 */
#define  USART_CR2_STOP_1                    ((uint16_t)0x2000)            /*!< Bit 1 */

#define  USART_CR2_LINEN                     ((uint16_t)0x4000)            /*!< LIN mode enable */

/******************  Bit definition for USART_CR3 register  *******************/
#define  USART_CR3_EIE                       ((uint16_t)0x0001)            /*!< Error Interrupt Enable */
#define  USART_CR3_IREN                      ((uint16_t)0x0002)            /*!< IrDA mode Enable */
#define  USART_CR3_IRLP                      ((uint16_t)0x0004)            /*!< IrDA Low-Power */
#define  USART_CR3_HDSEL                     ((uint16_t)0x0008)            /*!< Half-Duplex Selection */
#define  USART_CR3_NACK                      ((uint16_t)0x0010)            /*!< Smartcard NACK enable */
#define  USART_CR3_SCEN                      ((uint16_t)0x0020)            /*!< Smartcard mode enable */
#define  USART_CR3_DMAR                      ((uint16_t)0x0040)            /*!< DMA Enable Receiver */
#define  USART_CR3_DMAT                      ((uint16_t)0x0080)            /*!< DMA Enable Transmitter */
#define  USART_CR3_RTSE                      ((uint16_t)0x0100)            /*!< RTS Enable */
#define  USART_CR3_CTSE                      ((uint16_t)0x0200)            /*!< CTS Enable */
#define  USART_CR3_CTSIE                     ((uint16_t)0x0400)            /*!< CTS Interrupt Enable */
#define  USART_CR3_ONEBIT                    ((uint16_t)0x0800)            /*!< One Bit method */

/******************  Bit definition for USART_GTPR register  ******************/
#define  USART_GTPR_PSC                      ((uint16_t)0x00FF)            /*!< PSC[7:0] bits (Prescaler value) */
#define  USART_GTPR_PSC_0                    ((uint16_t)0x0001)            /*!< Bit 0 */
#define  USART_GTPR_PSC_1                    ((uint16_t)0x0002)            /*!< Bit 1 */
#define  USART_GTPR_PSC_2                    ((uint16_t)0x0004)            /*!< Bit 2 */
#define  USART_GTPR_PSC_3                    ((uint16_t)0x0008)            /*!< Bit 3 */
#define  USART_GTPR_PSC_4                    ((uint16_t)0x0010)            /*!< Bit 4 */
#define  USART_GTPR_PSC_5                    ((uint16_t)0x0020)            /*!< Bit 5 */
#define  USART_GTPR_PSC_6                    ((uint16_t)0x0040)            /*!< Bit 6 */
#define  USART_GTPR_PSC_7                    ((uint16_t)0x0080)            /*!< Bit 7 */

#define  USART_GTPR_GT                       ((uint16_t)0xFF00)            /*!< Guard time value */

/******************************************************************************/
/*                                                                            */
/*                                 Debug MCU                                  */
/*                                                                            */
/******************************************************************************/

/****************  Bit definition for DBGMCU_IDCODE register  *****************/
#define  DBGMCU_IDCODE_DEV_ID                ((uint32_t)0x00000FFF)        /*!< Device Identifier */

#define  DBGMCU_IDCODE_REV_ID                ((uint32_t)0xFFFF0000)        /*!< REV_ID[15:0] bits (Revision Identifier) */
#define  DBGMCU_IDCODE_REV_ID_0              ((uint32_t)0x00010000)        /*!< Bit 0 */
#define  DBGMCU_IDCODE_REV_ID_1              ((uint32_t)0x00020000)        /*!< Bit 1 */
#define  DBGMCU_IDCODE_REV_ID_2              ((uint32_t)0x00040000)        /*!< Bit 2 */
```

```
#define  DBGMCU_IDCODE_REV_ID_3              ((uint32_t)0x00080000)        /*!< Bit 3 */
#define  DBGMCU_IDCODE_REV_ID_4              ((uint32_t)0x00100000)        /*!< Bit 4 */
#define  DBGMCU_IDCODE_REV_ID_5              ((uint32_t)0x00200000)        /*!< Bit 5 */
#define  DBGMCU_IDCODE_REV_ID_6              ((uint32_t)0x00400000)        /*!< Bit 6 */
#define  DBGMCU_IDCODE_REV_ID_7              ((uint32_t)0x00800000)        /*!< Bit 7 */
#define  DBGMCU_IDCODE_REV_ID_8              ((uint32_t)0x01000000)        /*!< Bit 8 */
#define  DBGMCU_IDCODE_REV_ID_9              ((uint32_t)0x02000000)        /*!< Bit 9 */
#define  DBGMCU_IDCODE_REV_ID_10             ((uint32_t)0x04000000)        /*!< Bit 10 */
#define  DBGMCU_IDCODE_REV_ID_11             ((uint32_t)0x08000000)        /*!< Bit 11 */
#define  DBGMCU_IDCODE_REV_ID_12             ((uint32_t)0x10000000)        /*!< Bit 12 */
#define  DBGMCU_IDCODE_REV_ID_13             ((uint32_t)0x20000000)        /*!< Bit 13 */
#define  DBGMCU_IDCODE_REV_ID_14             ((uint32_t)0x40000000)        /*!< Bit 14 */
#define  DBGMCU_IDCODE_REV_ID_15             ((uint32_t)0x80000000)        /*!< Bit 15 */

/******************  Bit definition for DBGMCU_CR register  *******************/
#define  DBGMCU_CR_DBG_SLEEP                 ((uint32_t)0x00000001)        /*!< Debug Sleep Mode */
#define  DBGMCU_CR_DBG_STOP                  ((uint32_t)0x00000002)        /*!< Debug Stop Mode */
#define  DBGMCU_CR_DBG_STANDBY               ((uint32_t)0x00000004)        /*!< Debug Standby mode */
#define  DBGMCU_CR_TRACE_IOEN                ((uint32_t)0x00000020)        /*!< Trace Pin Assignment Control */

#define  DBGMCU_CR_TRACE_MODE                ((uint32_t)0x000000C0)        /*!< TRACE_MODE[1:0] bits (Trace Pin Assignment Control) */
#define  DBGMCU_CR_TRACE_MODE_0              ((uint32_t)0x00000040)        /*!< Bit 0 */
#define  DBGMCU_CR_TRACE_MODE_1              ((uint32_t)0x00000080)        /*!< Bit 1 */

#define  DBGMCU_CR_DBG_IWDG_STOP             ((uint32_t)0x00000100)        /*!< Debug Independent Watchdog stopped when Core is halted */
#define  DBGMCU_CR_DBG_WWDG_STOP             ((uint32_t)0x00000200)        /*!< Debug Window Watchdog stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM1_STOP             ((uint32_t)0x00000400)        /*!< TIM1 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM2_STOP             ((uint32_t)0x00000800)        /*!< TIM2 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM3_STOP             ((uint32_t)0x00001000)        /*!< TIM3 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM4_STOP             ((uint32_t)0x00002000)        /*!< TIM4 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_CAN1_STOP             ((uint32_t)0x00004000)        /*!< Debug CAN1 stopped when Core is halted */
#define  DBGMCU_CR_DBG_I2C1_SMBUS_TIMEOUT    ((uint32_t)0x00008000)        /*!< SMBUS timeout mode stopped when Core is halted */
#define  DBGMCU_CR_DBG_I2C2_SMBUS_TIMEOUT    ((uint32_t)0x00010000)        /*!< SMBUS timeout mode stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM8_STOP             ((uint32_t)0x00020000)        /*!< TIM8 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM5_STOP             ((uint32_t)0x00040000)        /*!< TIM5 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM6_STOP             ((uint32_t)0x00080000)        /*!< TIM6 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_TIM7_STOP             ((uint32_t)0x00100000)        /*!< TIM7 counter stopped when core is halted */
#define  DBGMCU_CR_DBG_CAN2_STOP             ((uint32_t)0x00200000)        /*!< Debug CAN2 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM15_STOP            ((uint32_t)0x00400000)        /*!< Debug TIM15 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM16_STOP            ((uint32_t)0x00800000)        /*!< Debug TIM16 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM17_STOP            ((uint32_t)0x01000000)        /*!< Debug TIM17 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM12_STOP            ((uint32_t)0x02000000)        /*!< Debug TIM12 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM13_STOP            ((uint32_t)0x04000000)        /*!< Debug TIM13 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM14_STOP            ((uint32_t)0x08000000)        /*!< Debug TIM14 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM9_STOP             ((uint32_t)0x10000000)        /*!< Debug TIM9 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM10_STOP            ((uint32_t)0x20000000)        /*!< Debug TIM10 stopped when Core is halted */
#define  DBGMCU_CR_DBG_TIM11_STOP            ((uint32_t)0x40000000)        /*!< Debug TIM11 stopped when Core is halted */
```

```
/******************************************************************************/
/*                                                                            */
/*                        FLASH and Option Bytes Registers                    */
/*                                                                            */
/******************************************************************************/

/*******************  Bit definition for FLASH_ACR register  *******************/
#define  FLASH_ACR_LATENCY                   ((uint8_t)0x03)        /*!< LATENCY[2:0] bits (Latency) */
#define  FLASH_ACR_LATENCY_0                 ((uint8_t)0x00)        /*!< Bit 0 */
#define  FLASH_ACR_LATENCY_1                 ((uint8_t)0x01)        /*!< Bit 0 */
#define  FLASH_ACR_LATENCY_2                 ((uint8_t)0x02)        /*!< Bit 1 */

#define  FLASH_ACR_HLFCYA                    ((uint8_t)0x08)        /*!< Flash Half Cycle Access Enable */
#define  FLASH_ACR_PRFTBE                    ((uint8_t)0x10)        /*!< Prefetch Buffer Enable */
#define  FLASH_ACR_PRFTBS                    ((uint8_t)0x20)        /*!< Prefetch Buffer Status */

/*******************  Bit definition for FLASH_KEYR register  *******************/
#define  FLASH_KEYR_FKEYR                    ((uint32_t)0xFFFFFFFF)  /*!< FPEC Key */

/*****************  Bit definition for FLASH_OPTKEYR register  *****************/
#define  FLASH_OPTKEYR_OPTKEYR               ((uint32_t)0xFFFFFFFF)  /*!< Option Byte Key */

/*******************  Bit definition for FLASH_SR register  *******************/
#define  FLASH_SR_BSY                        ((uint8_t)0x01)        /*!< Busy */
#define  FLASH_SR_PGERR                      ((uint8_t)0x04)        /*!< Programming Error */
#define  FLASH_SR_WRPRTERR                   ((uint8_t)0x10)        /*!< Write Protection Error */
#define  FLASH_SR_EOP                        ((uint8_t)0x20)        /*!< End of operation */

/*******************  Bit definition for FLASH_CR register  *******************/
#define  FLASH_CR_PG                         ((uint16_t)0x0001)     /*!< Programming */
#define  FLASH_CR_PER                        ((uint16_t)0x0002)     /*!< Page Erase */
#define  FLASH_CR_MER                        ((uint16_t)0x0004)     /*!< Mass Erase */
#define  FLASH_CR_OPTPG                      ((uint16_t)0x0010)     /*!< Option Byte Programming */
#define  FLASH_CR_OPTER                      ((uint16_t)0x0020)     /*!< Option Byte Erase */
#define  FLASH_CR_STRT                       ((uint16_t)0x0040)     /*!< Start */
#define  FLASH_CR_LOCK                       ((uint16_t)0x0080)     /*!< Lock */
#define  FLASH_CR_OPTWRE                     ((uint16_t)0x0200)     /*!< Option Bytes Write Enable */
#define  FLASH_CR_ERRIE                      ((uint16_t)0x0400)     /*!< Error Interrupt Enable */
#define  FLASH_CR_EOPIE                      ((uint16_t)0x1000)     /*!< End of operation interrupt enable */

/*******************  Bit definition for FLASH_AR register  *******************/
#define  FLASH_AR_FAR                        ((uint32_t)0xFFFFFFFF)  /*!< Flash Address */

/*******************  Bit definition for FLASH_OBR register  *******************/
#define  FLASH_OBR_OPTERR                    ((uint16_t)0x0001)     /*!< Option Byte Error */
#define  FLASH_OBR_RDPRT                     ((uint16_t)0x0002)     /*!< Read protection */

#define  FLASH_OBR_USER                      ((uint16_t)0x03FC)     /*!< User Option Bytes */
#define  FLASH_OBR_WDG_SW                    ((uint16_t)0x0004)     /*!< WDG_SW */
#define  FLASH_OBR_nRST_STOP                 ((uint16_t)0x0008)     /*!< nRST_STOP */
#define  FLASH_OBR_nRST_STDBY                ((uint16_t)0x0010)     /*!< nRST_STDBY */
#define  FLASH_OBR_BFB2                      ((uint16_t)0x0020)     /*!< BFB2 */

/*******************  Bit definition for FLASH_WRPR register  *******************/
#define  FLASH_WRPR_WRP                        ((uint32_t)0xFFFFFFFF)  /*!< Write Protect */

/*----------------------------------------------------------------------------*/

/*******************  Bit definition for FLASH_RDP register  *******************/
#define  FLASH_RDP_RDP                       ((uint32_t)0x000000FF)  /*!< Read protection option byte */
#define  FLASH_RDP_nRDP                      ((uint32_t)0x0000FF00)  /*!< Read protection complemented option byte */

/*******************  Bit definition for FLASH_USER register  *******************/
#define  FLASH_USER_USER                     ((uint32_t)0x00FF0000)  /*!< User option byte */
#define  FLASH_USER_nUSER                    ((uint32_t)0xFF000000)  /*!< User complemented option byte */
```

```
/******************  Bit definition for FLASH_Data0 register  *****************/
#define  FLASH_Data0_Data0                    ((uint32_t)0x000000FF)        /*!< User data storage option byte */
#define  FLASH_Data0_nData0                   ((uint32_t)0x0000FF00)        /*!< User data storage complemented option byte */

/******************  Bit definition for FLASH_Data1 register  *****************/
#define  FLASH_Data1_Data1                    ((uint32_t)0x00FF0000)        /*!< User data storage option byte */
#define  FLASH_Data1_nData1                   ((uint32_t)0xFF000000)        /*!< User data storage complemented option byte */

/******************  Bit definition for FLASH_WRP0 register  ******************/
#define  FLASH_WRP0_WRP0                      ((uint32_t)0x000000FF)        /*!< Flash memory write protection option bytes */
#define  FLASH_WRP0_nWRP0                     ((uint32_t)0x0000FF00)        /*!< Flash memory write protection complemented option bytes */

/******************  Bit definition for FLASH_WRP1 register  ******************/
#define  FLASH_WRP1_WRP1                      ((uint32_t)0x00FF0000)        /*!< Flash memory write protection option bytes */
#define  FLASH_WRP1_nWRP1                     ((uint32_t)0xFF000000)        /*!< Flash memory write protection complemented option bytes */

/******************  Bit definition for FLASH_WRP2 register  ******************/
#define  FLASH_WRP2_WRP2                      ((uint32_t)0x000000FF)        /*!< Flash memory write protection option bytes */
#define  FLASH_WRP2_nWRP2                     ((uint32_t)0x0000FF00)        /*!< Flash memory write protection complemented option bytes */

/******************  Bit definition for FLASH_WRP3 register  ******************/
#define  FLASH_WRP3_WRP3                      ((uint32_t)0x00FF0000)        /*!< Flash memory write protection option bytes */
#define  FLASH_WRP3_nWRP3                     ((uint32_t)0xFF000000)        /*!< Flash memory write protection complemented option bytes */

#ifdef STM32F10X_CL
/*****************************************************************************/
/*                Ethernet MAC Registers bits definitions                    */
/*****************************************************************************/
/* Bit definition for Ethernet MAC Control Register register */
#define ETH_MACCR_WD      ((uint32_t)0x00800000)  /* Watchdog disable */
#define ETH_MACCR_JD      ((uint32_t)0x00400000)  /* Jabber disable */
#define ETH_MACCR_IFG     ((uint32_t)0x000E0000)  /* Inter-frame gap */
  #define ETH_MACCR_IFG_96Bit     ((uint32_t)0x00000000)  /* Minimum IFG between frames during transmission is 96Bit */
  #define ETH_MACCR_IFG_88Bit     ((uint32_t)0x00020000)  /* Minimum IFG between frames during transmission is 88Bit */
  #define ETH_MACCR_IFG_80Bit     ((uint32_t)0x00040000)  /* Minimum IFG between frames during transmission is 80Bit */
  #define ETH_MACCR_IFG_72Bit     ((uint32_t)0x00060000)  /* Minimum IFG between frames during transmission is 72Bit */
  #define ETH_MACCR_IFG_64Bit     ((uint32_t)0x00080000)  /* Minimum IFG between frames during transmission is 64Bit */
  #define ETH_MACCR_IFG_56Bit     ((uint32_t)0x000A0000)  /* Minimum IFG between frames during transmission is 56Bit */
  #define ETH_MACCR_IFG_48Bit     ((uint32_t)0x000C0000)  /* Minimum IFG between frames during transmission is 48Bit */
  #define ETH_MACCR_IFG_40Bit     ((uint32_t)0x000E0000)  /* Minimum IFG between frames during transmission is 40Bit */
#define ETH_MACCR_CSD     ((uint32_t)0x00010000)  /* Carrier sense disable (during transmission) */
#define ETH_MACCR_FES     ((uint32_t)0x00004000)  /* Fast ethernet speed */
#define ETH_MACCR_ROD     ((uint32_t)0x00002000)  /* Receive own disable */
#define ETH_MACCR_LM      ((uint32_t)0x00001000)  /* loopback mode */
#define ETH_MACCR_DM      ((uint32_t)0x00000800)  /* Duplex mode */
#define ETH_MACCR_IPCO    ((uint32_t)0x00000400)  /* IP Checksum offload */
#define ETH_MACCR_RD      ((uint32_t)0x00000200)  /* Retry disable */
#define ETH_MACCR_APCS    ((uint32_t)0x00000080)  /* Automatic Pad/CRC stripping */
#define ETH_MACCR_BL      ((uint32_t)0x00000060)  /* Back-off limit: random integer number (r) of slot time delays before rescheduling
                                                     a transmission attempt during retries after a collision: 0 =< r <2^k */
```

```
  #define ETH_MACCR_BL_10    ((uint32_t)0x00000000)  /* k = min (n, 10) */
  #define ETH_MACCR_BL_8     ((uint32_t)0x00000020)  /* k = min (n, 8) */
  #define ETH_MACCR_BL_4     ((uint32_t)0x00000040)  /* k = min (n, 4) */
  #define ETH_MACCR_BL_1     ((uint32_t)0x00000060)  /* k = min (n, 1) */
#define ETH_MACCR_DC      ((uint32_t)0x00000010)  /* Defferal check */
#define ETH_MACCR_TE      ((uint32_t)0x00000008)  /* Transmitter enable */
#define ETH_MACCR_RE      ((uint32_t)0x00000004)  /* Receiver enable */

/* Bit definition for Ethernet MAC Frame Filter Register */
#define ETH_MACFFR_RA     ((uint32_t)0x80000000)  /* Receive all */
#define ETH_MACFFR_HPF    ((uint32_t)0x00000400)  /* Hash or perfect filter */
#define ETH_MACFFR_SAF    ((uint32_t)0x00000200)  /* Source address filter enable */
#define ETH_MACFFR_SAIF   ((uint32_t)0x00000100)  /* SA inverse filtering */
#define ETH_MACFFR_PCF    ((uint32_t)0x000000C0)  /* Pass control frames: 3 cases */
  #define ETH_MACFFR_PCF_BlockAll                ((uint32_t)0x00000040)  /* MAC filters all control frames from reaching the application */
  #define ETH_MACFFR_PCF_ForwardAll              ((uint32_t)0x00000080)  /* MAC forwards all control frames to application even if they fail the Address Filter */
  #define ETH_MACFFR_PCF_ForwardPassedAddrFilter ((uint32_t)0x000000C0)  /* MAC forwards control frames that pass the Address Filter. */
#define ETH_MACFFR_BFD    ((uint32_t)0x00000020)  /* Broadcast frame disable */
#define ETH_MACFFR_PAM    ((uint32_t)0x00000010)  /* Pass all mutlicast */
#define ETH_MACFFR_DAIF   ((uint32_t)0x00000008)  /* DA Inverse filtering */
#define ETH_MACFFR_HM     ((uint32_t)0x00000004)  /* Hash multicast */
#define ETH_MACFFR_HU     ((uint32_t)0x00000002)  /* Hash unicast */
#define ETH_MACFFR_PM     ((uint32_t)0x00000001)  /* Promiscuous mode */

/* Bit definition for Ethernet MAC Hash Table High Register */
#define ETH_MACHTHR_HTH   ((uint32_t)0xFFFFFFFF)  /* Hash table high */

/* Bit definition for Ethernet MAC Hash Table Low Register */
#define ETH_MACHTLR_HTL   ((uint32_t)0xFFFFFFFF)  /* Hash table low */

/* Bit definition for Ethernet MAC MII Address Register */
#define ETH_MACMIIAR_PA   ((uint32_t)0x0000F800)  /* Physical layer address */
#define ETH_MACMIIAR_MR   ((uint32_t)0x000007C0)  /* MII register in the selected PHY */
#define ETH_MACMIIAR_CR   ((uint32_t)0x0000001C)  /* CR clock range: 6 cases */
  #define ETH_MACMIIAR_CR_Div42   ((uint32_t)0x00000000)  /* HCLK:60-72 MHz; MDC clock= HCLK/42 */
  #define ETH_MACMIIAR_CR_Div16   ((uint32_t)0x00000008)  /* HCLK:20-35 MHz; MDC clock= HCLK/16 */
  #define ETH_MACMIIAR_CR_Div26   ((uint32_t)0x0000000C)  /* HCLK:35-60 MHz; MDC clock= HCLK/26 */
#define ETH_MACMIIAR_MW   ((uint32_t)0x00000002)  /* MII write */
#define ETH_MACMIIAR_MB   ((uint32_t)0x00000001)  /* MII busy */

/* Bit definition for Ethernet MAC MII Data Register */
#define ETH_MACMIIDR_MD   ((uint32_t)0x0000FFFF)  /* MII data: read/write data from/to PHY */

/* Bit definition for Ethernet MAC Flow Control Register */
#define ETH_MACFCR_PT     ((uint32_t)0xFFFF0000)  /* Pause time */
#define ETH_MACFCR_ZQPD   ((uint32_t)0x00000080)  /* Zero-quanta pause disable */
#define ETH_MACFCR_PLT    ((uint32_t)0x00000030)  /* Pause low threshold: 4 cases */
  #define ETH_MACFCR_PLT_Minus4   ((uint32_t)0x00000000)  /* Pause time minus 4 slot times */
  #define ETH_MACFCR_PLT_Minus28  ((uint32_t)0x00000010)  /* Pause time minus 28 slot times */
  #define ETH_MACFCR_PLT_Minus144 ((uint32_t)0x00000020)  /* Pause time minus 144 slot times */
  #define ETH_MACFCR_PLT_Minus256 ((uint32_t)0x00000030)  /* Pause time minus 256 slot times */
#define ETH_MACFCR_UPFD   ((uint32_t)0x00000008)  /* Unicast pause frame detect */
#define ETH_MACFCR_RFCE   ((uint32_t)0x00000004)  /* Receive flow control enable */
#define ETH_MACFCR_TFCE   ((uint32_t)0x00000002)  /* Transmit flow control enable */
#define ETH_MACFCR_FCBBPA ((uint32_t)0x00000001)  /* Flow control busy/backpressure activate */

/* Bit definition for Ethernet MAC VLAN Tag Register */
#define ETH_MACVLANTR_VLANTC ((uint32_t)0x00010000)  /* 12-bit VLAN tag comparison */
#define ETH_MACVLANTR_VLANTI ((uint32_t)0x0000FFFF)  /* VLAN tag identifier (for receive frames) */

/* Bit definition for Ethernet MAC Remote Wake-UpFrame Filter Register */
#define ETH_MACRWUFFR_D   ((uint32_t)0xFFFFFFFF)  /* Wake-up frame filter register data */
/* Eight sequential Writes to this address (offset 0x28) will write all Wake-UpFrame Filter Registers.
   Eight sequential Reads from this address (offset 0x28) will read all Wake-UpFrame Filter Registers. */
/* Wake-UpFrame Filter Reg0 : Filter 0 Byte Mask
   Wake-UpFrame Filter Reg1 : Filter 1 Byte Mask
   Wake-UpFrame Filter Reg2 : Filter 2 Byte Mask
   Wake-UpFrame Filter Reg3 : Filter 3 Byte Mask
   Wake-UpFrame Filter Reg4 : RSVD - Filter3 Command - RSVD - Filter2 Command -
                              RSVD - Filter1 Command - RSVD - Filter0 Command
```

```
     Wake-UpFrame Filter Re5 : Filter3 Offset - Filter2 Offset - Filter1 Offset - Filter0 Offset
     Wake-UpFrame Filter Re6 : Filter1 CRC16 - Filter0 CRC16
     Wake-UpFrame Filter Re7 : Filter3 CRC16 - Filter2 CRC16 */

/* Bit definition for Ethernet MAC PMT Control and Status Register */
#define ETH_MACPMTCSR_WFFRPR ((uint32_t)0x80000000)  /* Wake-Up Frame Filter Register Pointer Reset */
#define ETH_MACPMTCSR_GU     ((uint32_t)0x00000200)  /* Global Unicast */
#define ETH_MACPMTCSR_WFR    ((uint32_t)0x00000040)  /* Wake-Up Frame Received */
#define ETH_MACPMTCSR_MPR    ((uint32_t)0x00000020)  /* Magic Packet Received */
#define ETH_MACPMTCSR_WFE    ((uint32_t)0x00000004)  /* Wake-Up Frame Enable */
#define ETH_MACPMTCSR_MPE    ((uint32_t)0x00000002)  /* Magic Packet Enable */
#define ETH_MACPMTCSR_PD     ((uint32_t)0x00000001)  /* Power Down */

/* Bit definition for Ethernet MAC Status Register */
#define ETH_MACSR_TSTS      ((uint32_t)0x00000200)  /* Time stamp trigger status */
#define ETH_MACSR_MMCTS     ((uint32_t)0x00000040)  /* MMC transmit status */
#define ETH_MACSR_MMMCRS    ((uint32_t)0x00000020)  /* MMC receive status */
#define ETH_MACSR_MMCS      ((uint32_t)0x00000010)  /* MMC status */
#define ETH_MACSR_PMTS      ((uint32_t)0x00000008)  /* PMT status */

/* Bit definition for Ethernet MAC Interrupt Mask Register */
#define ETH_MACIMR_TSTIM     ((uint32_t)0x00000200)  /* Time stamp trigger interrupt mask */
#define ETH_MACIMR_PMTIM     ((uint32_t)0x00000008)  /* PMT interrupt mask */

/* Bit definition for Ethernet MAC Address0 High Register */
#define ETH_MACA0HR_MACA0H   ((uint32_t)0x0000FFFF)  /* MAC address0 high */

/* Bit definition for Ethernet MAC Address0 Low Register */
#define ETH_MACA0LR_MACA0L   ((uint32_t)0xFFFFFFFF)  /* MAC address0 low */

/* Bit definition for Ethernet MAC Address1 High Register */
#define ETH_MACA1HR_AE       ((uint32_t)0x80000000)  /* Address enable */
#define ETH_MACA1HR_SA       ((uint32_t)0x40000000)  /* Source address */
#define ETH_MACA1HR_MBC      ((uint32_t)0x3F000000)  /* Mask byte control: bits to mask for comparison of the MAC Address bytes */
  #define ETH_MACA1HR_MBC_HBits15_8    ((uint32_t)0x20000000)  /* Mask MAC Address high reg bits [15:8] */
  #define ETH_MACA1HR_MBC_HBits7_0     ((uint32_t)0x10000000)  /* Mask MAC Address high reg bits [7:0] */
  #define ETH_MACA1HR_MBC_LBits31_24   ((uint32_t)0x08000000)  /* Mask MAC Address low reg bits [31:24] */
  #define ETH_MACA1HR_MBC_LBits23_16   ((uint32_t)0x04000000)  /* Mask MAC Address low reg bits [23:16] */
  #define ETH_MACA1HR_MBC_LBits15_8    ((uint32_t)0x02000000)  /* Mask MAC Address low reg bits [15:8] */
  #define ETH_MACA1HR_MBC_LBits7_0     ((uint32_t)0x01000000)  /* Mask MAC Address low reg bits [7:0] */
#define ETH_MACA1HR_MACA1H   ((uint32_t)0x0000FFFF)  /* MAC address1 high */

/* Bit definition for Ethernet MAC Address1 Low Register */
#define ETH_MACA1LR_MACA1L   ((uint32_t)0xFFFFFFFF)  /* MAC address1 low */

/* Bit definition for Ethernet MAC Address2 High Register */
#define ETH_MACA2HR_AE       ((uint32_t)0x80000000)  /* Address enable */
#define ETH_MACA2HR_SA       ((uint32_t)0x40000000)  /* Source address */
#define ETH_MACA2HR_MBC      ((uint32_t)0x3F000000)  /* Mask byte control */
  #define ETH_MACA2HR_MBC_HBits15_8    ((uint32_t)0x20000000)  /* Mask MAC Address high reg bits [15:8] */
  #define ETH_MACA2HR_MBC_HBits7_0     ((uint32_t)0x10000000)  /* Mask MAC Address high reg bits [7:0] */
  #define ETH_MACA2HR_MBC_LBits31_24   ((uint32_t)0x08000000)  /* Mask MAC Address low reg bits [31:24] */
  #define ETH_MACA2HR_MBC_LBits23_16   ((uint32_t)0x04000000)  /* Mask MAC Address low reg bits [23:16] */
  #define ETH_MACA2HR_MBC_LBits15_8    ((uint32_t)0x02000000)  /* Mask MAC Address low reg bits [15:8] */
  #define ETH_MACA2HR_MBC_LBits7_0     ((uint32_t)0x01000000)  /* Mask MAC Address low reg bits [70] */
#define ETH_MACA2HR_MACA2H   ((uint32_t)0x0000FFFF)  /* MAC address1 high */

/* Bit definition for Ethernet MAC Address2 Low Register */
#define ETH_MACA2LR_MACA2L   ((uint32_t)0xFFFFFFFF)  /* MAC address2 low */

/* Bit definition for Ethernet MAC Address3 High Register */
#define ETH_MACA3HR_AE       ((uint32_t)0x80000000)  /* Address enable */
#define ETH_MACA3HR_SA       ((uint32_t)0x40000000)  /* Source address */
#define ETH_MACA3HR_MBC      ((uint32_t)0x3F000000)  /* Mask byte control */
  #define ETH_MACA3HR_MBC_HBits15_8    ((uint32_t)0x20000000)  /* Mask MAC Address high reg bits [15:8] */
```

```
  #define ETH_MACA3HR_MBC_HBits7_0     ((uint32_t)0x10000000)  /* Mask MAC Address high reg bits [7:0] */
  #define ETH_MACA3HR_MBC_LBits31_24   ((uint32_t)0x08000000)  /* Mask MAC Address low reg bits [31:24] */
  #define ETH_MACA3HR_MBC_LBits23_16   ((uint32_t)0x04000000)  /* Mask MAC Address low reg bits [23:16] */
  #define ETH_MACA3HR_MBC_LBits15_8    ((uint32_t)0x02000000)  /* Mask MAC Address low reg bits [15:8] */
  #define ETH_MACA3HR_MBC_LBits7_0     ((uint32_t)0x01000000)  /* Mask MAC Address low reg bits [70] */
#define ETH_MACA3HR_MACA3H   ((uint32_t)0x0000FFFF)  /* MAC address3 high */

/* Bit definition for Ethernet MAC Address3 Low Register */
#define ETH_MACA3LR_MACA3L   ((uint32_t)0xFFFFFFFF)  /* MAC address3 low */

/******************************************************************************/
/*                Ethernet MMC Registers bits definition                      */
/******************************************************************************/

/* Bit definition for Ethernet MMC Contol Register */
#define ETH_MMCCR_MCF        ((uint32_t)0x00000008)  /* MMC Counter Freeze */
#define ETH_MMCCR_ROR        ((uint32_t)0x00000004)  /* Reset on Read */
#define ETH_MMCCR_CSR        ((uint32_t)0x00000002)  /* Counter Stop Rollover */
#define ETH_MMCCR_CR         ((uint32_t)0x00000001)  /* Counters Reset */

/* Bit definition for Ethernet MMC Receive Interrupt Register */
#define ETH_MMCRIR_RGUFS     ((uint32_t)0x00020000)  /* Set when Rx good unicast frames counter reaches half the maximum value */
#define ETH_MMCRIR_RFAES     ((uint32_t)0x00000040)  /* Set when Rx alignment error counter reaches half the maximum value */
#define ETH_MMCRIR_RFCES     ((uint32_t)0x00000020)  /* Set when Rx crc error counter reaches half the maximum value */

/* Bit definition for Ethernet MMC Transmit Interrupt Register */
#define ETH_MMCTIR_TGFS      ((uint32_t)0x00200000)  /* Set when Tx good frame count counter reaches half the maximum value */
#define ETH_MMCTIR_TGFMSCS   ((uint32_t)0x00008000)  /* Set when Tx good multi col counter reaches half the maximum value */
#define ETH_MMCTIR_TGFSCS    ((uint32_t)0x00004000)  /* Set when Tx good single col counter reaches half the maximum value */

/* Bit definition for Ethernet MMC Receive Interrupt Mask Register */
#define ETH_MMCRIMR_RGUFM    ((uint32_t)0x00020000)  /* Mask the interrupt when Rx good unicast frames counter reaches half the maximum value */
#define ETH_MMCRIMR_RFAEM    ((uint32_t)0x00000040)  /* Mask the interrupt when when Rx alignment error counter reaches half the maximum value */
#define ETH_MMCRIMR_RFCEM    ((uint32_t)0x00000020)  /* Mask the interrupt when Rx crc error counter reaches half the maximum value */

/* Bit definition for Ethernet MMC Transmit Interrupt Mask Register */
#define ETH_MMCTIMR_TGFM     ((uint32_t)0x00200000)  /* Mask the interrupt when Tx good frame count counter reaches half the maximum value */
#define ETH_MMCTIMR_TGFMSCM  ((uint32_t)0x00008000)  /* Mask the interrupt when Tx good multi col counter reaches half the maximum value */
#define ETH_MMCTIMR_TGFSCM   ((uint32_t)0x00004000)  /* Mask the interrupt when Tx good single col counter reaches half the maximum value */

/* Bit definition for Ethernet MMC Transmitted Good Frames after Single Collision Counter Register */
#define ETH_MMCTGFSCCR_TGFSCC     ((uint32_t)0xFFFFFFFF)  /* Number of successfully transmitted frames after a single collision in Half-duplex mode. */

/* Bit definition for Ethernet MMC Transmitted Good Frames after More than a Single Collision Counter Register */
#define ETH_MMCTGFMSCCR_TGFMSCC   ((uint32_t)0xFFFFFFFF)  /* Number of successfully transmitted frames after more than a single collision in Half-duplex mode. */

/* Bit definition for Ethernet MMC Transmitted Good Frames Counter Register */
#define ETH_MMCTGFCR_TGFC    ((uint32_t)0xFFFFFFFF)  /* Number of good frames transmitted. */

/* Bit definition for Ethernet MMC Received Frames with CRC Error Counter Register */
#define ETH_MMCRFCECR_RFCEC  ((uint32_t)0xFFFFFFFF)  /* Number of frames received with CRC error. */

/* Bit definition for Ethernet MMC Received Frames with Alignement Error Counter Register */
#define ETH_MMCRFAECR_RFAEC  ((uint32_t)0xFFFFFFFF)  /* Number of frames received with alignment (dribble) error */

```

```
/* Bit definition for Ethernet MMC Received Good Unicast Frames Counter Register */
#define ETH_MMCRGUFCR_RGUFC  ((uint32_t)0xFFFFFFFF)  /* Number of good unicast frames received. */

/******************************************************************************/
/*               Ethernet PTP Registers bits definition                       */
/******************************************************************************/

/* Bit definition for Ethernet PTP Time Stamp Contol Register */
#define ETH_PTPTSCR_TSARU    ((uint32_t)0x00000020)  /* Addend register update */
#define ETH_PTPTSCR_TSITE    ((uint32_t)0x00000010)  /* Time stamp interrupt trigger enable */
#define ETH_PTPTSCR_TSSTU    ((uint32_t)0x00000008)  /* Time stamp update */
#define ETH_PTPTSCR_TSSTI    ((uint32_t)0x00000004)  /* Time stamp initialize */
#define ETH_PTPTSCR_TSFCU    ((uint32_t)0x00000002)  /* Time stamp fine or coarse update */
#define ETH_PTPTSCR_TSE      ((uint32_t)0x00000001)  /* Time stamp enable */

/* Bit definition for Ethernet PTP Sub-Second Increment Register */
#define ETH_PTPSSIR_STSSI    ((uint32_t)0x000000FF)  /* System time Sub-second increment value */

/* Bit definition for Ethernet PTP Time Stamp High Register */
#define ETH_PTPTSHR_STS      ((uint32_t)0xFFFFFFFF)  /* System Time second */

/* Bit definition for Ethernet PTP Time Stamp Low Register */
#define ETH_PTPTSLR_STPNS    ((uint32_t)0x80000000)  /* System Time Positive or negative time */
#define ETH_PTPTSLR_STSS     ((uint32_t)0x7FFFFFFF)  /* System Time sub-seconds */

/* Bit definition for Ethernet PTP Time Stamp High Update Register */
#define ETH_PTPTSHUR_TSUS    ((uint32_t)0xFFFFFFFF)  /* Time stamp update seconds */

/* Bit definition for Ethernet PTP Time Stamp Low Update Register */
#define ETH_PTPTSLUR_TSUPNS  ((uint32_t)0x80000000)  /* Time stamp update Positive or negative time */
#define ETH_PTPTSLUR_TSUSS   ((uint32_t)0x7FFFFFFF)  /* Time stamp update sub-seconds */

/* Bit definition for Ethernet PTP Time Stamp Addend Register */
#define ETH_PTPTSAR_TSA      ((uint32_t)0xFFFFFFFF)  /* Time stamp addend */

/* Bit definition for Ethernet PTP Target Time High Register */
#define ETH_PTPTTHR_TTSH     ((uint32_t)0xFFFFFFFF)  /* Target time stamp high */

/* Bit definition for Ethernet PTP Target Time Low Register */
#define ETH_PTPTTLR_TTSL     ((uint32_t)0xFFFFFFFF)  /* Target time stamp low */

/******************************************************************************/
/*                 Ethernet DMA Registers bits definition                     */
/******************************************************************************/

/* Bit definition for Ethernet DMA Bus Mode Register */
#define ETH_DMABMR_AAB       ((uint32_t)0x02000000)  /* Address-Aligned beats */
#define ETH_DMABMR_FPM        ((uint32_t)0x01000000)  /* 4xPBL mode */
#define ETH_DMABMR_USP       ((uint32_t)0x00800000)  /* Use separate PBL */
#define ETH_DMABMR_RDP       ((uint32_t)0x007E0000)  /* RxDMA PBL */
  #define ETH_DMABMR_RDP_1Beat    ((uint32_t)0x00020000)  /* maximum number of beats to be transferred in one RxDMA transaction is 1 */
  #define ETH_DMABMR_RDP_2Beat    ((uint32_t)0x00040000)  /* maximum number of beats to be transferred in one RxDMA transaction is 2 */
  #define ETH_DMABMR_RDP_4Beat    ((uint32_t)0x00080000)  /* maximum number of beats to be transferred in one RxDMA transaction is 4 */
  #define ETH_DMABMR_RDP_8Beat    ((uint32_t)0x00100000)  /* maximum number of beats to be transferred in one RxDMA transaction is 8 */
  #define ETH_DMABMR_RDP_16Beat   ((uint32_t)0x00200000)  /* maximum number of beats to be transferred in one RxDMA transaction is 16 */
  #define ETH_DMABMR_RDP_32Beat   ((uint32_t)0x00400000)  /* maximum number of beats to be transferred in one RxDMA transaction is 32 */
  #define ETH_DMABMR_RDP_4xPBL_4Beat   ((uint32_t)0x01020000)  /* maximum number of beats to be transferred in one RxDMA transaction is 4 */
  #define ETH_DMABMR_RDP_4xPBL_8Beat   ((uint32_t)0x01040000)  /* maximum number of beats to be transferred in one RxDMA transaction is 8 */
  #define ETH_DMABMR_RDP_4xPBL_16Beat  ((uint32_t)0x01080000)  /* maximum number of beats to be transferred in one RxDMA transaction is 16 */
  #define ETH_DMABMR_RDP_4xPBL_32Beat  ((uint32_t)0x01100000)  /* maximum number of beats to be transferred in one RxDMA transaction is 32 */
  #define ETH_DMABMR_RDP_4xPBL_64Beat  ((uint32_t)0x01200000)  /* maximum number of beats to be transferred in one RxDMA transaction is 64 */
  #define ETH_DMABMR_RDP_4xPBL_128Beat ((uint32_t)0x01400000)  /* maximum number of beats to be
```

```
transferred in one RxDMA transaction is 128 */
#define ETH_DMABMR_FB        ((uint32_t)0x00010000)  /* Fixed Burst */
#define ETH_DMABMR_RTPR      ((uint32_t)0x0000C000)  /* Rx Tx priority ratio */
  #define ETH_DMABMR_RTPR_1_1     ((uint32_t)0x00000000)  /* Rx Tx priority ratio */
  #define ETH_DMABMR_RTPR_2_1     ((uint32_t)0x00004000)  /* Rx Tx priority ratio */
  #define ETH_DMABMR_RTPR_3_1     ((uint32_t)0x00008000)  /* Rx Tx priority ratio */
  #define ETH_DMABMR_RTPR_4_1     ((uint32_t)0x0000C000)  /* Rx Tx priority ratio */
#define ETH_DMABMR_PBL    ((uint32_t)0x00003F00)  /* Programmable burst length */
  #define ETH_DMABMR_PBL_1Beat    ((uint32_t)0x00000100)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 1 */
  #define ETH_DMABMR_PBL_2Beat    ((uint32_t)0x00000200)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 2 */
  #define ETH_DMABMR_PBL_4Beat    ((uint32_t)0x00000400)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 4 */
  #define ETH_DMABMR_PBL_8Beat    ((uint32_t)0x00000800)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 8 */
  #define ETH_DMABMR_PBL_16Beat   ((uint32_t)0x00001000)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 16 */
  #define ETH_DMABMR_PBL_32Beat   ((uint32_t)0x00002000)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 32 */
  #define ETH_DMABMR_PBL_4xPBL_4Beat   ((uint32_t)0x01000100)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 4 */
  #define ETH_DMABMR_PBL_4xPBL_8Beat   ((uint32_t)0x01000200)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 8 */
  #define ETH_DMABMR_PBL_4xPBL_16Beat  ((uint32_t)0x01000400)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 16 */
  #define ETH_DMABMR_PBL_4xPBL_32Beat  ((uint32_t)0x01000800)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 32 */
  #define ETH_DMABMR_PBL_4xPBL_64Beat  ((uint32_t)0x01001000)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 64 */
  #define ETH_DMABMR_PBL_4xPBL_128Beat ((uint32_t)0x01002000)  /* maximum number of beats to be transferred in one TxDMA (or both) transaction is 128 */
#define ETH_DMABMR_DSL       ((uint32_t)0x0000007C)  /* Descriptor Skip Length */
#define ETH_DMABMR_DA        ((uint32_t)0x00000002)  /* DMA arbitration scheme */
#define ETH_DMABMR_SR        ((uint32_t)0x00000001)  /* Software reset */

/* Bit definition for Ethernet DMA Transmit Poll Demand Register */
#define ETH_DMATPDR_TPD      ((uint32_t)0xFFFFFFFF)  /* Transmit poll demand */

/* Bit definition for Ethernet DMA Receive Poll Demand Register */
#define ETH_DMARPDR_RPD      ((uint32_t)0xFFFFFFFF)  /* Receive poll demand  */

/* Bit definition for Ethernet DMA Receive Descriptor List Address Register */
#define ETH_DMARDLAR_SRL     ((uint32_t)0xFFFFFFFF)  /* Start of receive list */

/* Bit definition for Ethernet DMA Transmit Descriptor List Address Register */
#define ETH_DMATDLAR_STL     ((uint32_t)0xFFFFFFFF)  /* Start of transmit list */

/* Bit definition for Ethernet DMA Status Register */
#define ETH_DMASR_TSTS       ((uint32_t)0x20000000)  /* Time-stamp trigger status */
#define ETH_DMASR_PMTS       ((uint32_t)0x10000000)  /* PMT status */
#define ETH_DMASR_MMCS       ((uint32_t)0x08000000)  /* MMC status */
#define ETH_DMASR_EBS        ((uint32_t)0x03800000)  /* Error bits status */
  /* combination with EBS[2:0] for GetFlagStatus function */
  #define ETH_DMASR_EBS_DescAccess      ((uint32_t)0x02000000)  /* Error bits 0-data buffer, 1-desc. access */
  #define ETH_DMASR_EBS_ReadTransf      ((uint32_t)0x01000000)  /* Error bits 0-write trnsf, 1-read transfr */
  #define ETH_DMASR_EBS_DataTransfTx    ((uint32_t)0x00800000)  /* Error bits 0-Rx DMA, 1-Tx DMA */
#define ETH_DMASR_TPS         ((uint32_t)0x00700000)  /* Transmit process state */
  #define ETH_DMASR_TPS_Stopped         ((uint32_t)0x00000000)  /* Stopped - Reset or Stop Tx Command issued  */
  #define ETH_DMASR_TPS_Fetching        ((uint32_t)0x00100000)  /* Running - fetching the Tx descriptor */
  #define ETH_DMASR_TPS_Waiting         ((uint32_t)0x00200000)  /* Running - waiting for status */
  #define ETH_DMASR_TPS_Reading         ((uint32_t)0x00300000)  /* Running - reading the data from host memory */
  #define ETH_DMASR_TPS_Suspended       ((uint32_t)0x00600000)  /* Suspended - Tx Descriptor unavailabe */
  #define ETH_DMASR_TPS_Closing         ((uint32_t)0x00700000)  /* Running - closing Rx descriptor */
#define ETH_DMASR_RPS         ((uint32_t)0x000E0000)  /* Receive process state */
  #define ETH_DMASR_RPS_Stopped         ((uint32_t)0x00000000)  /* Stopped - Reset or Stop Rx Command issued */
```

```
  #define ETH_DMASR_RPS_Fetching        ((uint32_t)0x00020000)  /* Running - fetching the Rx descriptor */
  #define ETH_DMASR_RPS_Waiting         ((uint32_t)0x00060000)  /* Running - waiting for packet */
  #define ETH_DMASR_RPS_Suspended       ((uint32_t)0x00080000)  /* Suspended - Rx Descriptor unavailable */
  #define ETH_DMASR_RPS_Closing         ((uint32_t)0x000A0000)  /* Running - closing descriptor */
  #define ETH_DMASR_RPS_Queuing         ((uint32_t)0x000E0000)  /* Running - queuing the recieve frame into host memory */
#define ETH_DMASR_NIS        ((uint32_t)0x00010000)  /* Normal interrupt summary */
#define ETH_DMASR_AIS        ((uint32_t)0x00008000)  /* Abnormal interrupt summary */
#define ETH_DMASR_ERS        ((uint32_t)0x00004000)  /* Early receive status */
#define ETH_DMASR_FBES       ((uint32_t)0x00002000)  /* Fatal bus error status */
#define ETH_DMASR_ETS        ((uint32_t)0x00000400)  /* Early transmit status */
#define ETH_DMASR_RWTS       ((uint32_t)0x00000200)  /* Receive watchdog timeout status */
#define ETH_DMASR_RPSS       ((uint32_t)0x00000100)  /* Receive process stopped status */
#define ETH_DMASR_RBUS       ((uint32_t)0x00000080)  /* Receive buffer unavailable status */
#define ETH_DMASR_RS         ((uint32_t)0x00000040)  /* Receive status */
#define ETH_DMASR_TUS        ((uint32_t)0x00000020)  /* Transmit underflow status */
#define ETH_DMASR_ROS        ((uint32_t)0x00000010)  /* Receive overflow status */
#define ETH_DMASR_TJTS       ((uint32_t)0x00000008)  /* Transmit jabber timeout status */
#define ETH_DMASR_TBUS       ((uint32_t)0x00000004)  /* Transmit buffer unavailable status */
#define ETH_DMASR_TPSS       ((uint32_t)0x00000002)  /* Transmit process stopped status */
#define ETH_DMASR_TS         ((uint32_t)0x00000001)  /* Transmit status */

/* Bit definition for Ethernet DMA Operation Mode Register */
#define ETH_DMAOMR_DTCEFD    ((uint32_t)0x04000000)  /* Disable Dropping of TCP/IP checksum error frames */
#define ETH_DMAOMR_RSF       ((uint32_t)0x02000000)  /* Receive store and forward */
#define ETH_DMAOMR_DFRF      ((uint32_t)0x01000000)  /* Disable flushing of received frames */
#define ETH_DMAOMR_TSF       ((uint32_t)0x00200000)  /* Transmit store and forward */
#define ETH_DMAOMR_FTF       ((uint32_t)0x00100000)  /* Flush transmit FIFO */
#define ETH_DMAOMR_TTC       ((uint32_t)0x0001C000)  /* Transmit threshold control */
  #define ETH_DMAOMR_TTC_64Bytes       ((uint32_t)0x00000000)  /* threshold level of the MTL Transmit FIFO is 64 Bytes */
  #define ETH_DMAOMR_TTC_128Bytes      ((uint32_t)0x00004000)  /* threshold level of the MTL Transmit FIFO is 128 Bytes */
  #define ETH_DMAOMR_TTC_192Bytes      ((uint32_t)0x00008000)  /* threshold level of the MTL Transmit FIFO is 192 Bytes */
  #define ETH_DMAOMR_TTC_256Bytes      ((uint32_t)0x0000C000)  /* threshold level of the MTL Transmit FIFO is 256 Bytes */
  #define ETH_DMAOMR_TTC_40Bytes       ((uint32_t)0x00010000)  /* threshold level of the MTL Transmit FIFO is 40 Bytes */
  #define ETH_DMAOMR_TTC_32Bytes       ((uint32_t)0x00014000)  /* threshold level of the MTL Transmit FIFO is 32 Bytes */
  #define ETH_DMAOMR_TTC_24Bytes       ((uint32_t)0x00018000)  /* threshold level of the MTL Transmit FIFO is 24 Bytes */
  #define ETH_DMAOMR_TTC_16Bytes       ((uint32_t)0x0001C000)  /* threshold level of the MTL Transmit FIFO is 16 Bytes */
#define ETH_DMAOMR_ST        ((uint32_t)0x00002000)  /* Start/stop transmission command */
#define ETH_DMAOMR_FEF       ((uint32_t)0x00000080)  /* Forward error frames */
#define ETH_DMAOMR_FUGF      ((uint32_t)0x00000040)  /* Forward undersized good frames */
#define ETH_DMAOMR_RTC       ((uint32_t)0x00000018)  /* receive threshold control */
  #define ETH_DMAOMR_RTC_64Bytes       ((uint32_t)0x00000000)  /* threshold level of the MTL Receive FIFO is 64 Bytes */
  #define ETH_DMAOMR_RTC_32Bytes       ((uint32_t)0x00000008)  /* threshold level of the MTL Receive FIFO is 32 Bytes */
  #define ETH_DMAOMR_RTC_96Bytes       ((uint32_t)0x00000010)  /* threshold level of the MTL Receive FIFO is 96 Bytes */
  #define ETH_DMAOMR_RTC_128Bytes      ((uint32_t)0x00000018)  /* threshold level of the MTL Receive FIFO is 128 Bytes */
#define ETH_DMAOMR_OSF       ((uint32_t)0x00000004)  /* operate on second frame */
#define ETH_DMAOMR_SR        ((uint32_t)0x00000002)  /* Start/stop receive */

/* Bit definition for Ethernet DMA Interrupt Enable Register */
#define ETH_DMAIER_NISE      ((uint32_t)0x00010000)  /* Normal interrupt summary enable */
#define ETH_DMAIER_AISE      ((uint32_t)0x00008000)  /* Abnormal interrupt summary enable */
#define ETH_DMAIER_ERIE      ((uint32_t)0x00004000)  /* Early receive interrupt enable */
#define ETH_DMAIER_FBEIE     ((uint32_t)0x00002000)  /* Fatal bus error interrupt enable */
#define ETH_DMAIER_ETIE      ((uint32_t)0x00000400)  /* Early transmit interrupt enable */
#define ETH_DMAIER_RWTIE     ((uint32_t)0x00000200)  /* Receive watchdog timeout interrupt enable */
#define ETH_DMAIER_RPSIE     ((uint32_t)0x00000100)  /* Receive process stopped interrupt enable */
#define ETH_DMAIER_RBUIE     ((uint32_t)0x00000080)  /* Receive buffer unavailable interrupt enable */
#define ETH_DMAIER_RIE       ((uint32_t)0x00000040)  /* Receive interrupt enable */
```

```
#define ETH_DMAIER_TUIE      ((uint32_t)0x00000020)  /* Transmit Underflow interrupt enable */
#define ETH_DMAIER_ROIE      ((uint32_t)0x00000010)  /* Receive Overflow interrupt enable */
#define ETH_DMAIER_TJTIE     ((uint32_t)0x00000008)  /* Transmit jabber timeout interrupt enable */
#define ETH_DMAIER_TBUIE     ((uint32_t)0x00000004)  /* Transmit buffer unavailable interrupt enable */
#define ETH_DMAIER_TPSIE     ((uint32_t)0x00000002)  /* Transmit process stopped interrupt enable */
#define ETH_DMAIER_TIE       ((uint32_t)0x00000001)  /* Transmit interrupt enable */

/* Bit definition for Ethernet DMA Missed Frame and Buffer Overflow Counter Register */
#define ETH_DMAMFBOCR_OFOC   ((uint32_t)0x10000000)  /* Overflow bit for FIFO overflow counter */
#define ETH_DMAMFBOCR_MFA    ((uint32_t)0x0FFE0000)  /* Number of frames missed by the application */
#define ETH_DMAMFBOCR_OMFC   ((uint32_t)0x00010000)  /* Overflow bit for missed frame counter */
#define ETH_DMAMFBOCR_MFC    ((uint32_t)0x0000FFFF)  /* Number of frames missed by the controller */

/* Bit definition for Ethernet DMA Current Host Transmit Descriptor Register */
#define ETH_DMACHTDR_HTDAP   ((uint32_t)0xFFFFFFFF)  /* Host transmit descriptor address pointer */

/* Bit definition for Ethernet DMA Current Host Receive Descriptor Register */
#define ETH_DMACHRDR_HRDAP   ((uint32_t)0xFFFFFFFF)  /* Host receive descriptor address pointer */

/* Bit definition for Ethernet DMA Current Host Transmit Buffer Address Register */
#define ETH_DMACHTBAR_HTBAP  ((uint32_t)0xFFFFFFFF)  /* Host transmit buffer address pointer */

/* Bit definition for Ethernet DMA Current Host Receive Buffer Address Register */
#define ETH_DMACHRBAR_HRBAP  ((uint32_t)0xFFFFFFFF)  /* Host receive buffer address pointer */
#endif /* STM32F10X_CL */

/**
  * @}
  */

 /**
  * @}
  */

#ifdef USE_STDPERIPH_DRIVER
  #include "stm32f10x_conf.h"
#endif

/** @addtogroup Exported_macro
  * @{
  */

#define SET_BIT(REG, BIT)     ((REG) |= (BIT))

#define CLEAR_BIT(REG, BIT)   ((REG) &= ~(BIT))

#define READ_BIT(REG, BIT)    ((REG) & (BIT))

#define CLEAR_REG(REG)        ((REG) = (0x0))

#define WRITE_REG(REG, VAL)   ((REG) = (VAL))

#define READ_REG(REG)         ((REG))

#define MODIFY_REG(REG, CLEARMASK, SETMASK)  WRITE_REG((REG), (((READ_REG(REG)) & (~(CLEARMASK))) |
(SETMASK)))

/**
  * @}
  */

#ifdef __cplusplus
}
#endif

#endif /* __STM32F10x_H */

/**
  * @}
  */

  /**
  * @}
```

```
    */

  /****************** (C) COPYRIGHT 2011 STMicroelectronics *****END OF FILE****/

```